
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 12:04:58
Файлов исходного кода: 111
Суммарный объём кода: 1.96 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommen
          # На Ubuntu политика ImageMagick иногда запре
          # разрешаем то, что нужно пайплайну (text -> p
          for policy in /etc/ImageMagick-6/policy.xml /
            if [ -f "$policy" ]; then
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, >
    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, >
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
     <title> Дискретка для тупых – Интерактивный гид по
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target))
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters.length]);

 return (
 <div className="min-h-screen bg-slate-950 text-slate-50"

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
},

```

## `src/components/Navbar.tsx`

### Полное содержимое после изменений

```
````tsx
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C }
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler'
}

export const Navbar: React.FC<NavbarProps> = ({ activeTab
  const [busy, setBusy] = useState(false);

  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };

  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
                <Sparkles className="w-6 h-6" />
              </div>
            <div className="min-w-0 flex-1">
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```

```

+         <button
+             id="tab-compiler-btn"
+             onClick={() => setActiveTab('compiler')}
+             className={`flex-1 lg:flex-none flex items
duration-200 whitespace-nowrap ${
+                 activeTab === 'compiler'
+                     ? 'bg-emerald-600 text-white shadow-lg
+                     : 'text-slate-400 hover:text-white'
+             }`}
+         >
+         <Terminal className="w-4 h-4" /> Python
+     </button>
+     <a
+         id="download-pdf-btn"
+         href="/export/full_code_all_pages.pdf"
+         href={`${import.meta.env.BASE_URL}export/full
+         download="full_code_all_pages.pdf"
+         target="_blank"
+         rel="noreferrer"
@@ -73,7 +84,7 @@ export const Navbar: React.FC<NavbarP
    </a>
    <a
        id="download-txt-btn"
        href="/export/full_code_all_pages.txt"
        href={`${import.meta.env.BASE_URL}export/full
        download="full_code_all_pages.txt"
        target="_blank"
        rel="noreferrer"
    ....

```

`src/components/PythonCompiler.tsx`

Полное содержимое после изменений

```

````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play

```

```
export function PythonCompiler() {
 const [code, setCode] = useState(STARTER_CODE);
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");

 const runCode = useCallback(async () => {
 if (running) return;

 setRunning(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = stdin.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов — код выполняется");

 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не выполнила");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
 } finally {
 setRunning(false);
 }
 }, [code, stdin, runtimeReady]);
```

```


 </div>
 <div className="flex items-center gap-2">
 <button
 type="button"
 onClick={reset}
 className="inline-flex items-center gap-2 text-slate-800 transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 </button>
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : ""}
 {running ? "Запускаю..." : "Запустить"}
 </button>
 </div>
 </div>
 </div>

 <textarea
 value={code}
 onChange={(event) => setCode(event.target.value)}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.key === "Enter") {
 event.preventDefault();
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-800"
 placeholder="Напишите Python-код..."
 />

```

```

<div className="flex gap-2 rounded-xl border border-gray-200 p-4" style={{width: 100%; height: 100%;}}>
 <Info className="w-4 h-4 shrink-0 text-indigo-600 font-medium">
 Ctrl или Cmd + Enter запускает код. Введите код в поле ниже.
 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" style={{width: 100%; height: 100%;}}>
 <Info className="w-4 h-4 shrink-0 text-amber-600 font-medium">
 Первый запуск скачивает примерно несколько килобайт кода.
 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" style={{width: 100%; height: 100%;}}>
 <Info className="w-4 h-4 shrink-0 text-emerald-600 font-medium">
 Код выполняется в браузере и не отправляется на сервер.
 </div>
 </div>
 </div>
</main>
);
}
....

```

### Patch относительно `main`

```

````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и установит библиотеки
+name = "алгоритмы"
+
+def main():
+    print("Привет, мир!")
+    return "Ваш код выполнен успешно."
+
+main()
+`;

```



```

+
+ const runCode = useCallback(async () => {
+   if (running) return;
+
+   setRunning(true);
+   setOutput("");
+   setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+   const stdout: string[] = [];
+   const stderr: string[] = [];
+   const inputLines = stdin.split(/\r?\n/);
+
+   try {
+     const runtime = await getPythonRuntime();
+     setRuntimeReady(true);
+     setRuntimeMessage("Python готов – код выполняется");
+
+     runtime.setStdout({ batched: (message) => stdout
+     runtime.setStderr({ batched: (message) => stderr
+     runtime.setStdin({ stdin: () => (inputLines.length
+     await runtime.runPythonAsync(code);
+
+     const text = [...stdout, ...stderr].join("");
+     setOutput(text || "Готово: программа ничего не выв
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("")
+     setOutput(`${captured}${captured && !captured.en
+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };

```

```

late-800 transition-colors"
+
+       >
+       <RotateCcw className="w-3.5 h-3.5" />
+       </button>
+       <button
+       type="button"
+       onClick={() => void runCode()}
+       disabled={running}
+       className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+       >
+       {running ? <Loader2 className="w-3.5
+       {running ? "Запускаю..." : "Запустить"}
+       </button>
+       </div>
+     </div>
+
+     <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+       if ((event.ctrlKey || event.metaKey) &
+       event.preventDefault();
+       void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+       Ввод для input() • по одной строке на
+     </label>
+     <textarea

```

```
-----
+         </div>
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-eme
+             <span>Код выполняется в браузере и не отпра
+         </div>
+     </div>
+ </div>
+ </main>
+ );
+}
+...
```

`vite.config.ts`

Полное содержимое после изменений

```
```typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
},
},
```

Универсальное пособие • полный исходный код

-----  
КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7  
-----

# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML elements.

-----  
ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2  
-----

# Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу, ты **\*\*строго\*\*** придерживайся готовых шаблонов разметки, описанных ниже.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть описана в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.json``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык и формальный язык).

---

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pdf.mjs",
 "export:shots": "node scripts/export/capture-viz-shots.mjs",
 "verify:unit": "node scripts/verify/live-matrix.test.mjs",
 "verify:code": "node scripts/verify/check-reference.mjs",
 "verify:links": "node scripts/verify/check-links.mjs",
 "verify:dom": "node scripts/verify/run-browser-check.mjs",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:links",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 },
}
```

Универсальное пособие • полный исходный код

```

npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
````
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
````
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается ****на каж**
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док
и коммитит обновлённые `public/export/full\_code\_all\_pages`
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

Аудит контента

```
```bash
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после `<img>`

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```

* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**

```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сложно»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока закрыто

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни интерактива
8. Графы. Компоненты связности	`graph-components`	Нет визуализации
24. Классы сложности, сведение задач	`complexity-classes`	Нет визуализации

## Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---



ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

```
 },
 });
}
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```
export default function App() {
 const liteMode = useMemo(readLiteMode, []);
```

```

 setSidebarMode((m) => {
 const next = m === "wide" ? "compact" : m === "co
 window.localStorage.setItem("sidebarMode", next);
 return next;
 });
 }, []));
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.in
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[i

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
 {/* Сайдбар только на десктопе — на мобильном
 <div
 id="sidebar-root"
 className={`hidden lg:block shrink-0 border
 -all duration-200 print:hidden ${sidebarCol
 " : "lg:w-56"}` }
 >
 {!sidebarCollapsed && (
 <Sidebar selectedChapterId={activeChapterId}
)}
 </div>
 {/* Вкладка-переключатель содержания у левого
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-1
 border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarMode === "wide" ? "Сжать
 ание"}
 title={sidebarMode === "wide" ? "Сжать содер
 }
 >
 {sidebarCollapsed ? <PanelLeftOpen className="
 <span className="text-[10px] font-bold [wri
 </button>

 <main id="main-content" className="flex-1 min
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-b
 <div className="min-w-0">
 <div className="text-[10px] uppercase t
 {activeChapter.category || "Универсальн
 </div>
 <h2 className="text-base sm:text-xl font
 {activeChapter.title}
 </h2>
 </div>
 </div>

```

```
 onHeightChange={setCompilerHeight}
 onClose={() => setCompilerOpen(false)}
 />
))
 </div>
);
}
```

---

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

/** Числовые константы TreeWalker: глобального `NodeFile`
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/g, "\\$&");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}
```

```
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
 color: #a5b4fc;
 border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
 text-decoration: none;
 border-radius: 3px;
 padding: 0 1px;
 cursor: pointer;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}
```



```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```

```
main {
 padding: 0 !important;
 max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
 color: #111827 !important;
 background-color: transparent !important;
 background-image: none !important;
 box-shadow: none !important;
 text-shadow: none !important;
 border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}

article.chapter-body .prose table {
 border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
 page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

---

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
] [k] </p>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины
">i. Любой отрезок накрывается двумя
перация идемпотентна: min,
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрыва
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-700">
 <p class="text-[10px] uppercase tracking-wider">
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Торт-пирамидка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ия, зато красивый, как пирамидка: каждый из
 сех блоков $2^{kx} \times 2^{ky}$), и ответ на запрос
 ой операции нет: крем обратно в торт не запы
 $n(x, x) = x$), и цена $O(1)$ – сто коржей вмес
 <div class="rounded-lg bg-slate-950">
 <div>
 $n \ O(1)$ </div>
 <div>
 = 100 копий матрицы</div>
 </div>
 <p class="text-[12px] text-slate-400">
 шает). Менять элементы тоже нельзя. Сумму ч
 </div>
 </div>
 <p class="text-slate-400 text-sm mt-4">Поря
 ерению) → 2D: построение (уровни <span
 ос (прямоугольник раскладывается на чет
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <p class="text-slate-300 text-sm mb-4">Эти
 за
 они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <p class="text-slate-300 text-sm mb-3">
 цы, у разреженной таблицы – размер, умножен
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 10px; width: fit-content; margin: 10px auto;">P[i][j]</p>  
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 10px; width: fit-content; margin: 10px auto;">ум на его части не вычислишь. Приходится хранить  $\log N \times \log M$  на ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: **лицу** – перекрывающиеся блоки посчитают **лицу** – у минимума нет обратной операции. Стр

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-amber-400">

<div class="bg-rose-950/30 p-5 rounded-lg border">

<ul class="list-disc pl-5 space-y-2 text">

<li><b>Сумма через разреженную таблицу префиксные (билет 5).</b></li>

<li><b>Память.</b>  $N = M = 1000$  даём  $N \times M$  считайте ячейки, потом пишете код.</li>

<li><b>Один log вместо двух.</b> В  $\log N \times \log M$  st[0][0] = 0  
</li>

<li><b>Выход за границу в запросе.</b>  $st[0][0] = 0$  и st[0][0] = 0

<li><b>Нужны обновления.</b> Разреженная таблица (билет 1), а не она.</li>

</ul>

```

 else
 st[r][c][kx][ky] = min(

```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);

```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальную сумму.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму на нескольких сотен на сторону), а запросов очень много.

Вторая половина этого сравнения: [Сумма](#)

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 И
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">S
х». Спуск от корня: узел, который вместе с
скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">M
аг – одно сравнение: чей у у корня больше,
сохраняет порядок x). Итого $O(h)$ сравнений
 </div>
 </div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «У6
```



```
<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
```

Код: вставка, merge, erase (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
```

```
<pre class="bg-slate-950 p-4 rounded">
```

```
def insert(root, key, pri):
```

```
 if root is None:
```

```
 return Node(key, pri)
```

```
 if key <= root.key: # равные x — левый
```

```
 root.left = insert(root.left, key, pri)
```

```
 else:
```

```
 root.right = insert(root.right, key, pri)
```

```
 return root
```

```
for key, pri in pairs_sorted_by_pri_desc: # порядок э
```

```
 root = insert(root, key, pri)</pre>
```

```
<pre class="bg-slate-950 p-4 rounded">
```

```
def merge(a, b): # все x в a < все x в b
```

```
 if not a or not b: return a or b
```

```
 if a.pri > b.pri:
```

```
 a.right = merge(a.right, b); return a
```

```
 b.left = merge(a, b.left); return b
```

```
def erase(root, key): # удалить произвольный элемент
```

```
 if key < root.key: root.left = erase(root.left, key)
```

```
 elif key > root.key: root.right = erase(root.right, key)
```

```
 else: root = merge(root.left, root.right)
```

```
 return root</pre>
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
},
```

```
{
```

```
 id: "splay-tree",
```

```
 title: "4. Splay-дерево",
```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево β (между x и p) – единств
x, и под p.</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

 / \ \
 x y
 / \ \
 α β
 / \ \
 x y
 / \ \
 α β

 x выше
 -----> α
 / \ \
 p r
 / \ \
 β γ

 p выше
 <-----
 / \ \
 p r
 / \ \
 β γ
 </pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
 амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x): 3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x): 3 × Zig-Zig + Zig → высота 8
бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10<
```

```
<p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути. Отсюда амортизация: дорогой за
дом потенциала: потенциал Φ = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x

def find(root, key): # поиск: спуск + сп
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None</pre>
 <p class="text-xs text-slate-400">В
 Одна строка – и вся амортизация.</p>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сумма подматрицы за O(1): 1D и 2D, дв
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 <td class="py-2 pr-3 font-mono">
 </tr>
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-2">
 ышли за границу» – поэтому в 2D почти всегда
 </div>

 <div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-emerald-500">
 <p class="text-[10px] uppercase tracking-wider">
 <p class="font-mono text-emerald-300">
 <p class="text-[11px] text-slate-500">
 </div>
 <div class="bg-slate-900 border border-emerald-500">
 <p class="text-[10px] uppercase tracking-wider">
 <p class="font-mono text-emerald-300">
 <p class="text-[11px] text-slate-500">
 </div>
 <div class="bg-slate-900 border border-emerald-500">
 <p class="text-[10px] uppercase tracking-wider">
 <p class="font-mono text-rose-300">
 <p class="text-[11px] text-slate-500">
 </div>
 </div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-4 border border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb-2">
 о накопилось от угла (0,0) до меня». Хочешь
 у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Тонкая: одна матрица
</div>
 <p class="text-slate-300 text-sm mb
одной матрицы (плюс рамка из нулей). Никаки
прямоугольника от угла. Всё остальное полу
</div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Не умеет: всё, что нельзя вычес
</div>
 <p class="text-slate-300 text-sm mb
максимума или gcd вычитания не существует:
е запросы – это <a class="ticket-link" data
платит она за это памятью <span class="fon
</div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate
 <td class="py-2 pr-3 font-b
 <td class="py-2 pr-3 font-m
 <td class="py-2 pr-3 font-m
 </tr>
 <tr class="border-b border-slate

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 1D (тот же, что выполняет пане
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
 <p class="text-slate-300 text-sm">П
 - часть демонстрации: лента подсвечивает, из
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код: 2D – построение и запрос прямо
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [] # рамка из нуле
for rr in range(n + 1):
 S.append([0] + [0] * m)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
 <p class="text-xl font-mono text-white">
 1. Если граф не планарный, то его нельзя раскрасить 4 цветами.
 2. Если граф планарный, то его можно раскрасить 4 цветами.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Границы не пересекаются в одной точке, поэтому
 можно раскрасить 4 цветами. Всегда можно всего 4
 цвета.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",

```



```

 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">10.1. Алгоритм Косарaju
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Алгоритм Косарaju позволяет разбить орграф на макро-вершины (компоненты сильной связности), инвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте себе карту города, где все улицы имеют одностороннее движение. Если вы попытаетесь проехать по улицам в обоих направлениях, вы обнаружите, что некоторые улицы образуют замкнутые циклы, где вы можете вернуться к началу пути. Эти циклы представляют собой компоненты сильной связности. Алгоритм Косарaju позволяет разбить такую карту на части, где движение всегда происходит в одном направлении по односторонним улицам. Косарaju у на карте.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-2">
 (или вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </h2>

 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </p>
 </div>
 </h3>
 <div class="grid grid-cols-1 gap-6">

```

```

 rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

 </div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 24
 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Пример: класс P (простые задачи) vs класс NP (нестрогие задачи).</p>
 <p class="text-base sm:text-lg font-mono">
 P: класс decision-задач, решаемых за полиномиальное время.
 NP: класс decision-задач с сертификатом.
 </p>
 <p>).

 Сведение (Reduction) A ≤_p B: если A ∈ P, то B ∈ P.
 </p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судоку (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">Пример: сортировка чисел. Класс NP – задачи, требующие проверки ответа (например, правильную сетку (ответ), он БЫСТРО (за класс P) проверить).</p>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
]

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"

 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-blue-300 italic"
 <div class="text-left font-mono text-sm"
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded"
 <p class="text-sm text-blue-300"
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
 <div class="bg-slate-800 p-6 rounded-lg bor"
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Аналогия 1: Путь – из дома в бар,
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>

```

```

 </div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Мосты через DFS Тарьяна: tin/low, усл
 category: "Графы. Продвинутые",
 content: `<section id="bridges-code" class="mb-20 s
 <div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-400">Мосты

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounded">
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400
 </div>
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков
 Единственная ниточка. Порвётся – граф р
 </p>
 </div>
 `

```



```

<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный в
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
}
```

```

 <p><span class="text-emerald-400 font-
 <p><span class="text-rose-400 font-l
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 и K3,3 – д
 Если внутри графа спрятано подра
 – плоской раскладки не будет.
 </p>
 </div>

<div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзамен

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left"
 <thead>
```

```
<tr class="text-slate-500 border-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 pr-3 font-b"
 <th class="py-2 font-bold">
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-800"
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ <
а свободен</td></tr>
```

```
<tr class="border-b border-slate-800"
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>
```

```

 веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: жадная раскраска и проверка
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше $\Delta + 1$ цвета
color = [-1] * n # -1 = ещё не покрашен
for v in order: # порядок влияет на чи
 used = set()
 for to in graph[v]:
 if color[to] != -1:
 used.add(color[to]) # цвета уже покрашены
 c = 0
 while c in used: # минимальный свободны
 c += 1
 color[v] = c
chi = max(color) + 1 # χ , который выдал жад

Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
 if col[start] != -1:
 continue
 col[start] = 0
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
```

```
 Важно: Это работает то-
 еплятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
 <div class="bg-slate-800 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-slate-800
 rder-rose-500 shadow-md">
```

```
 Аналогия: Главный перекресток
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Представь город, где два района соединены
 её перекроют (удалят вершину) — районы будут
 g>хрупкость системы.
```

```
 </p>
```

```
 </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
 <div class="absolute -top-3 left-4 bg-emerald-900
 er border-emerald-500 shadow-md">
```

```
 Телеком: Центральный свитч
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 У тебя несколько компьютеров в одной комнате
 абелем (мостом). Сами свитчи — это точки со-
```

```
 </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
 <summary class="p-4 font-bold text-slate-400 border-
 order-slate-700/50">
```

```
 Душный мод: Код и корень DFS (Скрыто)
```

```
 </summary>
```

```

 <ul class="list-disc list-inside text-sm text-rose-400">
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
 if (!toRemove.includes(c.id)) {
 dedup.set(c.id, c);
 }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
 newChapters.forEach((c) => {
 if (!toRemove.includes(c.id)) dedup.set(c.id, c);
 });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 // Главы без номера билета (введение) стоят ДО билетов
 const numOf = (t: string) => {
 const m = t.match(/(\d+)/);
 return m ? parseInt(m[0], 10) : -1;
 };
 return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
 "segment-trees": [1],
 "sparse-table": [2],
 treap: [3],
 "splay-tree": [4],
 "prefix-sums-2d": [5],
 "graph-dfs-bfs": [6],
 "planarity-euler-formula": [7],

```

```

const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se
 "dijkstra", "bellman-ford", "floyd", "aho-corasick",
 "mst", "string-kmp", "string-z-func", "segment-trees", "s
 "dynamic-programming", "johnson-algo", "graph-articulat
 "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

-----  
ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 207 | РАЗМ

```

/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень)
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера"""
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else None
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)]
 return __safe(v)

```

```

PEP 709: с 3.12 компрехенشنы встроены во фреймворк
на своей строке каждую итерацию. Склеиваем по
if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
Финальные локалы программы (после последней строки)
if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})

```

---

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ

---

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

"Не отсортированный список, а «мешок, из которого  
formal:

"Полное бинарное дерево в массиве: дети узла  $i$  ле

complexity: "вставка/извлечение  $O(\log n)$ , «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за  $O(1)$ . На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"],

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push\_back / pop\_front за  $O(1)$ . Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти  $O(1)$  — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

 short:
 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре",
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набегало от н",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "то"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения Э ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизир",
 "амортизированное", "амортизированного", "амортизир",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи
```

```

{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: " $O(n + m)$ ",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начи",
 "аждой букве.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i$ ",
 complexity: " $O(n)$ ",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин",
 "мемоизация", "мемоизацию", "подзадач", "подзадач",
],
 title: "Динамическое программирование",

```

```

 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас"
 formal:
 "Корректность обычно доказывается «леммой о разрезе»"
 demo: "mst",
 simulator: "mst",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь"
 formal:
 "Связный граф: эйлеров цикл все степени чётные; эйлеров путь все степени нечётные"
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного", "планаризация"],
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе бумаги"
 formal:
 "V - E + F = 2 для связного планарного графа. Курсивом выделены вершины, рёбра и области"
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "всестороннее"
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивают» к корню"
 formal:

```



```

 "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок пересадки выбираем среди всех вершин.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
 formal:
 "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst",
 },
 {
 id: "boruvka",

```

```
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "граф",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двусторонняя улица).",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел. Сжимаем координаты в один порядковый номер.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: " $O(n \log n)$ ",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова д-во"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить). Ожидаемая высота $O(\log n)$.",
 complexity: "ожидаемо $O(\log n)$ ",
}
```

```

 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива

 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строкам"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где

 (Z-функция) и где встречаются слова словаря (бордер-лист).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция — $O(N)$.",
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворотом"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встанет на

 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.

 g и Zig-Zag.",
 complexity: " $O(1)$ ",
 demo: "zig",
 },

```

Универсальное пособие • полный исходный код

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id }))
}).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРӨК: 411 | РАЗМЕР: 1.2 КБ

```

import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Введение. Базовая база: куча, очередь, бинарное
 type: "html",
 category: "Введение",
 content: `
 <section id="intro-content" class="mb-12">
 <div class="flex items-center mb-6 flex-wrap gap-2">
 Введение
 <h2 class="text-3xl font-bold text-white">Базовая база</h2>
 </div>

 <div class="bg-slate-900 border border-indigo-500/40 p-6">
 <p class="text-slate-200 text-sm leading-relaxed">
 <b class="text-indigo-300">Базовая база – это
 дерево как массив, куча, очередь и стек, бинарное
 Кодим по минимуму: там, где за нас уже написали
 </p>
 </div>

 <div class="space-y-8">
```

```
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за $O(\log n)$
```

<p class="text-slate-300 text-sm">Куча – движок, который поддерживает кучу, раз, и список с <span class="font-mono text-sm">heapq</span>.</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4">  
<h3 class="text-xl font-bold text-sky-400 mb-4">Очереди и стеки</h3>  
<p class="text-slate-300 text-sm mb-4"><b>Очереди</b> (BFS) отличаются от стеков (DFS) тем, что первый вышел) отличаются одной операцией снятия элемента</span>: оба конца работают за  $O(1)$ . Обычный список сдвигает весь хвост и стоит  $O(n)$ .</p>

```
<pre class="bg-slate-950 border border-slate-700">from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)</pre>
```

<p class="text-slate-300 text-sm">Вся разница между BFS (волнами), стек – DFS (вглубь).</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4">  
<h3 class="text-xl font-bold text-amber-400 mb-4">Бинарный поиск</h3>  
<p class="text-slate-300 text-sm mb-4">Если массив отсортирован, то можно найти элемент за  $O(\log n)$  с помощью бинарного поиска. Если массив не отсортирован, то нужно сортировать его за  $O(n \log n)$ .</p>

```
<pre class="bg-slate-950 border border-slate-700">from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 – индекс первого элемента, не меньшего 30</pre>
```

<p class="text-slate-300 text-sm">Без сортировки бинарный поиск не работает.</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4">  
<h3 class="text-xl font-bold text-rose-400 mb-4">Сортировка</h3>  
<p class="text-slate-300 text-sm mb-4">Писать сортировку можно по-разному. В Python есть встроенная функция sort, которая сортирует список in-place. Если нужно получить отсортированную копию, то используется sorted().</p>

```

 left(x) = z # z становится левым р
 высоты пересчитываются только у z и x — ост
 </div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-teal-400 mb-4
 <p class="text-slate-300 text-sm mb-4">Оба обхо
 тару». Вся разница в таре: стек уходит вглуби
 ext-emerald-300">0(V + E): каждая верш
 <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank): # tank: список-стек
 seen, tank = {start}, tank([start])
 while tank:
 v = tank.pop() if isinstance(tank, list) else t
 for to in graph[v]:
 if to not in seen:
 seen.add(to); tank.append(to)
 return seen</pre>
 <p class="text-slate-300 text-sm">Дальше эта па
 надстройки над этими восемью строками.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-slate-300 mb-
 <div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 border-indigo-500 shadow-md">
 Аналогия 1: Схема кинозала
 </div>
 <p class="text-slate-300 text-sm">Дерев
 азателей-«usher'ов» не нужно. Место n, дети
 рху вниз и слева направо.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s

```

```
title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 rounded
 <h2 class="text-3xl font-bold text-white">Дейкст
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Позитивное планирование — это когда вы знаете, что вы можете добраться до пункта назначения, но не знаете, как добраться.
 <p class="text-xl font-mono text-white">Позитивное планирование — это когда вы знаете, что вы можете добраться до пункта назначения, но не знаете, как добраться.
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">Позитивное планирование — это когда вы знаете, что вы можете добраться до пункта назначения, но не знаете, как добраться. (нельзя поехать и заработать). Он всегда работает.
 <div id="slot-chapter-image-dijkstra" class="h-400px">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">Ограничения — это когда вы знаете, что вы можете добраться до пункта назначения, но не знаете, как добраться. (нельзя поехать и заработать). Он всегда работает.
 <div id="slot-chapter-image-dijkstra" class="h-400px">
 </div>
 </div>
 </div>
 </div>
`
```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb
еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb
г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12"
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Флойд

```



```

 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы двигаемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка — это переход
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является сло
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
 } else {
 nodes[child].term_link = nodes[nodes[ch
 }

 q.push(child);
 }
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

 options: [
 "Да: минимум и сумма считаются одним приёмом",
 "Нет: блоки перекрываются, а вычесть перекрытие",
 "Да, если заранее хранить суммы в нулевом уровне",
],
 correctIndex: 1,
 explanation: "Два (в 2D – четыре) блока накрывают
 обратная операция, но применить её к перекрытию
 ммы, билет 5."
},
{
 question: "Какие операции подходят разреженной та
 options: [
 "Любые ассоциативные",
 "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
 "Только сумма и произведение"
],
 correctIndex: 1,
 explanation: "Нужно $op(x, x) = x$: тогда дважды на
 приём и уходит в префиксные суммы."
},
{
 question: "Между запросами элементы матрицы меняю
 options: [
 "Разреженную таблицу: она перестраивается за $O(n^2)$ ",
 "Префиксную матрицу: достаточно поправить одну
 "Дерево отрезков: и таблица, и префиксы статичны",
],
 correctIndex: 2,
 explanation: "Обе структуры статичны: изменение э
 ксной. С обновлениями работает дерево отрезков,
 }
},
"prefix-sums-2d": [
 {
 question: "Сколько памяти занимает двумерная преф
 options: [
 "N × M – как исходная матрица, плюс рамка из ну

```

```

 ной операции нет. Для min, max и gcd берут разря
 },
 {
 question: "Изменили один элемент матрицы. Что ста
 options: [
 "Поправится одна клетка за $O(1)$ ",
 "Нужен пересчёт $O(N \cdot M)$: от этой клетки зависит
 "Ничего не станет: таблица остаётся верной"
],
 correctIndex: 1,
 explanation: "Каждая клетка $S[i][j]$ включает в се
 я частые, это задача дерева отрезков (билет 1).
 }
],
"euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечёт
 options: [
 "Есть эйлеров цикл (вернусь домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это
 ётной, заканчиваем в другой."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная
 options: [
 "Потому что Гамильтон — болезнь, и лечить её т
 "Потому что для эйлерова цикла есть простой кри
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критери
 ого критерия. Это NP-полная задача, которую не
 },
 {

```

```

 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
 question: "Планарный граф имеет V=6, E=12. Возможн
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани пр
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с V=
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, $3V - 6 = 9$. $10 > 9$
 }
}

```

```

/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...s
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | n
 const chapterId = useContext(VizChapterContext);
 const [runtime, setRuntime] = useState<VizStateEventD
 chapterId ? latestByChapter.get(chapterId) ?? null
);

 useEffect(() => {
 setRuntime(chapterId ? latestByChapter.get(chapterId
 if (!chapterId) return;
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizStateEventDet
 if (detail?.chapterId === chapterId) {
 setRuntime(detail.variables || detail.line !==
 }
 };
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, ha
 }, [chapterId]);

```

```

 }, [chapterId, currentStep, maxStep, goToStep, selectStep]
 }
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
 typeof value === "number" && Number.isFinite(value) ? value : null;
}

```

```

export const vizString = (value: DebugValue | undefined) => {
 typeof value === "string" ? value : null;
}

```

```

export const vizArray = (value: DebugValue | undefined) => {
 Array.isArray(value) ? value : null;
}

```

```

export const vizRecord = (value: DebugValue | undefined) => {
 value !== null && typeof value === "object" && !Array.isArray(value)
 ? (value as Record<string, DebugValue>)
 : null;
}

```

```

/* — Матрицы из компилятора: СВОЯ размерность, а не дефолтная

```

```

/**
 * Нормализованная матрица, пришедшая из выполненного Рунима
 *
 * Размерность берётся из самих данных: если пользователь
 * ввёл 5 вершин, визуализация рисует 5x5, а не подставляет его
 * 7x7 (раньше ячейки «не той» размерности превращались в 0)
 *
 * Поддерживаются все учебные способы хранения:
 * list[list] d = [[0, 4], [999, 0]]
 * list[dict] d = [{0: 0, 1: 4}, {1: 0}]
 * dict[dict] d = {0: {0: 0, 1: 4}}
 * dict[(i, j)] d = {(0, 1): 4} — ключ после запятой
 */

```

```

export interface LiveMatrix {
 rows: number;
 cols: number;
 /** Значение ячейки; null — таких данных в коде пользователя нет
 at: (i: number, j: number) => number | null;
}

```

```

 if (n === null) return;
 cells.set(`${i}:${j}`, n);
 rows = Math.max(rows, i + 1);
 cols = Math.max(cols, j + 1);
};

const asArray = vizArray(value);
if (asArray) {
 asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
 const inner = vizArray(row);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
 return;
 });
 }
 const record = vizRecord(row);
 if (record) {
 for (const [key, cell] of Object.entries(record)) {
 const j = Number(key);
 if (Number.isInteger(j)) put(i, j, cell);
 }
 }
 });
} else {
 const record = vizRecord(value);
 if (!record) return null;
 for (const [key, cell] of Object.entries(record)) {
 const pair = parsePairKey(key);
 if (pair) {
 put(pair[0], pair[1], cell);
 continue;
 }
 const i = Number(key);
 if (!Number.isInteger(i)) continue;
 const inner = vizArray(cell);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
 continue;
 });
 }
 }
}

```



-----  
/\* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные */
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
 detail: { chapterId, demoId }
 }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
 if (typeof window === "undefined") return () => {};
 const latest = latestDemoByChapter.get(chapterId);
 if (latest !== undefined) cb(latest);
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
 if (detail && detail.chapterId === chapterId) cb(detail.demoId);
 };
 window.addEventListener(DEMO_CHANNEL, handler);
 return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

## Универсальное пособие • полный исходный код

```

tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; листы"},
 { name: "i", role: "индекс элемента массива; его левый и правый дети"},
 { name: "v", role: "текущая вершина дерева отрезка"},
 { name: "l, r", role: "границы отрезка, за которыми лежат левый и правый дети"},
 { name: "m", role: "середина отрезка m = (l + r) // 2"},
],
},
treap: {
 vizTitle: "Декартово дерево: build → split → merge",
 stepNote: "Шаг = один спуск в split/merge/erase (перезапись массива целиком.",
 code: `points = [[45, 8], [3, 6], [6, 5], [2, 4], [1, 3]]
n = len(points) # сколько пар (x; y) – столько вершин
mid = 7 # разрез для split: x <= mid < y
key = 9 # какую точку удаляем в erase
step = 0 # номер шага – листает кадры дерева
height = 0 # высота построенного дерева
root = None

--- тело алгоритма ---
class Node:
 def __init__(s, x, y):
 s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
 """Декартово дерево из отсортированных по x точек:
 pts = sorted(pts, key=lambda p: p[0])
 stack = []
 for x, y in pts:
 cur = Node(x, y)
 last = None
```



```

while x.p:
 p = x.p; g = p.p
 step += 1
 if g is None:
 case = "zig" # родители
 rot(x)
 elif (g.l is p) == (p.l is x):
 case = "zig-zig" # x и p -
 rot(p); rot(x)
 else:
 case = "zig-zag" # змейка
 rot(x); rot(x)
 xk, pk, gk = x.k, p.k, (g.k if g else 0)
 print("шаг", step, "->", case, "| x =", xk, "p = ", pk)
 root = x
 print(" корень теперь", x.k, "| обход:", inorder(root))
 return x

```

# 1) Zig: родитель x – уже корень (дерево 40 -> 20)  
 root = N(40); root.l = N(20); root.l.p = root  
 splay(root.l)

# 2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые  
 root = N(60); root.l = N(40); root.l.p = root; root.l.l = N(20)  
 splay(root.l.l)

# 3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)  
 root = N(60); root.l = N(20); root.l.p = root; root.l.r = N(40)  
 splay(root.l.r),

```

variables: [
 { name: "case", role: "какой случай сработал: \"zig-zig\", \"zig-zag\", \"zig\""},
 { name: "step", role: "номер применения случая – 1, 2, 3"},
 { name: "xk / x", role: "ключ узла, который поднимается"},
 { name: "pk / p", role: "ключ родителя x перед поворотом"},
 { name: "gk / g", role: "ключ деда – нужен в Zig-Zig"},
 { name: "root", role: "корень дерева после расшейки"},
]

```

```

 stepNote: "Шаг = заполнение очередного $P[i] = P[i-1]$
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]` # исходный массив
n = len(a) # длина: префиксов будет n
i = 0 # текущая граница префикса

P = [0] # $P[0] = 0$ – пустой префикс
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)`
variables: [
 { name: "a", role: "исходный массив 1D-демонстрация" },
 { name: "n", role: "длина массива a; префиксов будет n" },
 { name: "P", role: "префиксные суммы: $P[i] = a[0] + \dots + a[i-1]$ " },
 { name: "i", role: "граница префикса: на шаге считаем до i" }
],
},
"dynamic-programming": {
 vizTitle: "Динамическое программирование (мемоизация)",
 stepNote: "Шаг = событие внутри fibМемо(k): вызов, возврат",
 code: `n = 5
memo = {}

def fib(k):
 if k in memo:
 return memo[k]
 if k <= 2:
 return 1
 print(f"считаем fib({k-1}) + fib({k-2})")
 r1 = fib(k - 1)
 r2 = fib(k - 2)
 memo[k] = r1 + r2
 return memo[k]

print("итог:", fib(n))`,
variables: [
 { name: "n", role: "целевой номер числа Фибоначчи" },
 { name: "k", role: "аргумент текущего рекурсивного вызова" },
 { name: "memo", role: "кэш уже посчитанных значений" }
]

```

```

code: `from collections import deque

q = deque(["A"]) # очередь BFS
u = "B" # сосед, найденный из текущ
q.append(u) # enqueue соседа – в конец
front = q[0] # голова очереди: её обрабо
v = q.popleft() # шаг: достали голову и обр
print("front =", front, "| v =", v, "| очередь:", list(
 variables: [
 { name: "front", role: "голова очереди – вершина,
 { name: "v", role: "вершина, извлечённая из очере
 { name: "u", role: "сосед вершины v, которого кла
],
},
"everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",
 stepNote: "Шаг = одна бытовая операция: тарелка на
 code: `from collections import deque

st, q = [], deque() # тарелки (стек) и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0 # счётчик выполненных опера
for op in ("push", "pop", "enqueue", "dequeue"):
 i += 1
print("тарелка:", top, "| первый из очереди:", first, "
 variables: [
 { name: "top / front", role: "верх стопки (стек)
 { name: "i", role: "индекс в массиве кучи; родите
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к неп
 code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
```

## Универсальное пособие • полный исходный код

```

v = -1 # вершина, в которой обход начался
stack = [] # фронт обхода (стек для DFS)
i = 0 # номер шага — двигает по какому-то ребру

--- тело алгоритма ---
for start in range(n):
 if comp[start] != -1:
 continue
 comps += 1 # непокрашенная вершина => новая компонента
 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
 { name: "n", role: "число вершин; в демонстрации — количество вершин в графе" },
 { name: "adj", role: "списки смежности — тот же граф, что и g" },
 { name: "comp", role: "разметка: номер компоненты", range: "-1 ... comps" },
 { name: "used", role: "булев вариант разметки — для BFS" },
 { name: "comps", role: "счётчик найденных компонент" },
 { name: "c", role: "номер компоненты, которую красим сейчас" },
 { name: "v", role: "вершина, где обход находится" },
 { name: "stack", role: "фронт обхода: стек (DFS) или очередь (BFS)" },
 { name: "i / step", role: "номер шага — перематываем кадр" },
],
},
"top-sort": {
```

```
def dfs1(v):
 used[v] = True
 for to in adj[v]:
 if not used[to]:
 dfs1(to)
 order.append(v) # выходим из вершины — она

for v in adj:
 if not used[v]:
 dfs1(v)
 print("DFS из", v, "закончен | order:", order)

транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
 adjT[v] = []
for v in adj:
 for to in adj[v]:
 adjT[to].append(v)
print("обратный граф:", adjT)

фаза 2: идём по order С КОНЦА, DFS по adjT красит одн
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
 v = order[i]
 if v in comp:
 continue
 stack = [v]
 comp[v] = c
 while stack:
 u = stack.pop()
 for to in adjT[u]:
 if to not in comp:
 comp[to] = c
 stack.append(to)
 print("SCC", c, "собрана из", v, ":", comp)
```



```
},
 "bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v,
 code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3 # как на демо

v, to = "B", "G" # шаг: возврат из G
is_bridge = low[to] > tin[v] # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед; ребро (v, to) проверяе
 { name: "tin[v]", role: "время входа в вершину – m
 { name: "low[v]", role: "минимум tin, куда можно ,
 { name: "timer", role: "глобальный счётчик времени
],
 },
 "euler-path-vs-cycle": {
 vizTitle: "Эйлеров маршрут: алгоритм Иера",
 stepNote: "Шаг = переход по неиспользованному ребру
 идти нельзя.",
 code: `n = 5 # вершин: 0...4 (на
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)]
adj = {v: [] for v in range(n)}
for i, (u, v) in enumerate(edges):
 adj[u].append((v, i))
 adj[v].append((u, i))
deg = {v: len(adj[v]) for v in range(n)} # степени ве
odd = [v for v in range(n) if deg[v] % 2 == 1] # нечёт
start = odd[0] if odd else 0 # путь начинаем
used = [False] * len(edges) # какие рёбра :
path = [start] # маршрут: пос.
last = start # где мы сейча
stack = [start] # стек для отк
v = start
step = 0
```

```

V = n # вершин для формулы
E = sum(len(a) for a in adj) // 2 # рёбер: в списке смежн
F = 2 - V + E # граней: из $V - E + F = 2$

--- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "| $V - E + F = 2$ ")
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
 { name: "n", role: "число вершин графа на демонстрации" },
 { name: "adj", role: "списки смежности — тот же граф, что и g" },
 { name: "color", role: "массив цветов вершин (0 = не покрашено)" },
 { name: "v", role: "вершина, которую красим сейчас" },
 { name: "u", role: "сосед вершины v, чей цвет уже известен" },
 { name: "delta", role: "максимальная степень $\Delta = \max \deg(v)$ " },
 { name: "chi", role: "сколько цветов реально понадобится" },
 { name: "i / step", role: "номер шага — перематываем таймер" },
 { name: "V, E, F", role: "для вкладки «Планарность»" },
]
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = старт, извлечение ближайшей вершины",
 code: `import heapq

```

## Универсальное пособие • полный исходный код

```

nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes) # 7 вершин => n - 1 = 6 работ
m = len(E) # 9 рёбер
```

```
dist = {v: INF for v in nodes}
dist["S"] = 0 # старт – База
```

```
--- тело алгоритма ---
```

```
last_changed = ""
for i in range(1, n): # итерация i = 1 ... n-1
 last_changed = ""
 for u, v, w in E: # шаг: релаксация ребра (u, v)
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 last_changed = v
 if last_changed == "":
 print(f"итерация {i}: изменений нет – сошлось для всех вершин")
 break
 print(f"итерация {i}: " + ", ".join(f"{k}={dist[k]}:" for k in nodes))
```

```
контрольный (n-й) проход: улучшение здесь означает отрицательный цикл
cycle_edge = None
for u, v, w in E:
 if dist[u] != INF and dist[u] + w < dist[v]:
 cycle_edge = (u, v, w)
 break
```

```
if cycle_edge:
 print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
```

```
 print("отрицательных циклов нет:", {k: dist[k] for k in nodes})
 variables = [
 { name: "n", role: "число вершин графа – ровно n-1"},
 { name: "m", role: "число рёбер, по которым проходят"},
 { name: "i", role: "номер итерации (после i итераций)"},
 { name: "u, v, w", role: "текущее ребро: откуда, куда"},
 { name: "dist[v]", role: "расстояние; если обновилось"},
],
```

```

 на-Форда" },
 { name: "u, v", role: "ребро (u, v) перевзвешивает",
 { name: "d[v]", role: "расстояние Дейкстры в новом графе"
 },
],
},
mst: {
 vizTitle: "Остовное дерево: Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `n = 9 # вершин: A..I — те, что в графе
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "C"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "D", "F"),
 (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGH I"} # DSU: представителю
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))

```

```

vizTitle: "Z-функция",
stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * n # пустые клетки: значения по умолчанию
z[0] = 0

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`
variables: [
 { name: "n", role: "длина строки s", range: "n = len(s)" },
 { name: "i", role: "позиция, для которой считаем z[i]" },
 { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]" },
 { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[0:]" }
],
},
"aho-corasick": {
 vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат",
 stepNote: "Шаг = один символ текста: state = go(state, c).",
 code: `words = ["HERS", "HIS", "SHE", "HE"] # образцы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние автомата
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в автомате
queue = [] # очередь BFS, к которой добавляем новые вершины
fail = {} # fail[v] = вершина, с которой нужно начать поиск

```

```

 queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
 """Переход автомата: сначала по бору, иначе спускаем
 while v != 0 and (v, ch) not in nxt:
 v = fail[v]
 return nxt.get((v, ch), 0)

for i, c in enumerate(text):
 state = go(state, c)
 v = state
 while v != 0: # собираем все образцы
 if v in term:
 matches.append((i, term[v]))
 v = fail[v]
 print(f"i={i} c='{c}' state={state} найдено={matches}")

print("всего вхождений:", len(matches), matches),
variables: [
 { name: "words", role: "набор образцов – по нему",
 },
 { name: "text", role: "сканируемый текст – подста",
 { name: "state", role: "текущее состояние автомата",
 { name: "i / c", role: "индекс и сам символ текста",
 { name: "nxt", role: "переходы бора: nxt[(v, ch)]",
 { name: "fail", role: "fail[v] – вершина наибольшей",
 { name: "term", role: "в каких вершинах заканчивае",
 { name: "queue", role: "очередь BFS, которой натя",
 { name: "matches", role: "найденные вхождения: (ин",
 { name: "nodes", role: "сколько вершин в боре (ко",
],
},
"complexity-classes": {
 vizTitle: "Классы сложности: рост по n и ops",
 stepNote: "Шаг = одна точка (n, ops): визуализация",
 code: `n = 1 # размер входа

```

```

 },
 intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
 },

 /* — Дополнительные демонстрации внутри страниц (вкл.

 "planarity-euler-formula#greedy": {
 vizTitle: "Жадная покраска вершин ($\chi \leq \Delta + 1$)",
 stepNote: "Шаг = вершина v получает наименьший цвет",
 code: `n = 8 # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3,
delta = max(len(a) for a in adj) # максимальная степень
color = [0] * n # 0 = ещё не покрашена
chi = 0 # сколько цветов понадобилось
v = -1 # вершина, которую красим сейчас
i = 0 # номер шага

--- тело алгоритма ---
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет",
print("chi =", chi, "| delta =", delta, "| chi <= delta
 variables: [
 { name: "color", role: "цвета вершин (0 = не покрашено)",
 { name: "v", role: "текущая вершина", range: "0 ... n-1",
 { name: "delta / chi", role: "максимальная степень / количество цветов",
 { name: "i / step", role: "номер кадра", range: "0 ... n-1"

```

```

],
},
"planarity-euler-formula#edges": {
 vizTitle: "Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$)",
 stepNote: "Шаг = ребро e получает наименьший цвет, <= \Delta",
 code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

--- тело алгоритма ---
for e, (a, b) in enumerate(edges):
 v, u = a, b
 used = set(inc[a].values()) | set(inc[b].values())
 c = 1
 while c in used:
 c += 1
 ecolor[e] = c
 inc[a][e] = c
 inc[b][e] = c
 chi_e = max(chi_e, c)
 print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= \Delta")
variables: [
 { name: "ecolor", role: "цвета рёбер – рёбра на графе", range: "0..delta" },
 { name: "e", role: "номер текущего ребра", range: "0..len(edges)-1" },
 { name: "v, u", role: "концы текущего ребра – подмножества V", range: "0..n-1" },
 { name: "inc", role: "для каждой вершины: какие цвета имеют инцидентные рёбра", range: "0..delta" },
 { name: "chi_e / delta", role: "χ' – сколько цветов", range: "0..delta+1" }
],
},
"graph-components#dfs": {
 vizTitle: "Компоненты связности: DFS-покраска",

```



```

 },
 "graph-components#dsu": {
 vizTitle: "Компоненты связности: DSU по рёбрам",
 stepNote: "Шаг = одно ребро (u, v): union склеивает",
 code: `n = 10 # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n)) # DSU: parent[v] = представи
rank = [0] * n # высота дерева – по ней реш
comps = n # сначала каждая вершина – о
e = -1 # номер обрабатываемого ребр
u, v = -1, -1 # концы текущего ребра
step = 0 # номер шага – двигает покад

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

def union(a, b):
 global comps
 ra, rb = find(a), find(b)
 if ra == rb:
 return False # ребро внутри
 if rank[ra] < rank[rb]:
 ra, rb = rb, ra
 parent[rb] = ra
 if rank[ra] == rank[rb]:
 rank[ra] += 1
 comps -= 1
 return True

for e, (a, b) in enumerate(edges):
 u, v = a, b
 step = e + 1
 union(a, b)
print("компонент:", comps)

```

```

k = 3 # размер клики, которую ищем

--- тело алгоритма ---
Сведение 3-SAT <=p КЛИКА: вершина = {номер дизъюнкта,
ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
 print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима <=> в графе есть клика размера k")
variables: [
 { name: "step", role: "номер кадра сведения (0..6)" },
 { name: "cls", role: "класс задачи A, которую сводим к КЛИКА" },
 { name: "n", role: "размер входа x; после сведения к КЛИКА" },
 { name: "k", role: "размер клики, которую ищет рекурсия" },
],
},
"complexity-classes#verify": {
 vizTitle: "Верификатор 3-SAT: проверка сертификата",
 stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения в формулу",
 code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
cert = [1, 0, 1, 0] # сертификат: x1=1, x2=0, x3=1
k = 0 # номер проверяемого дизъюнкта
ok = True # вердикт верификатора

--- тело алгоритма ---
for k, cl in enumerate(clauses):
 hit = False
 for l in cl:
 val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
 if val == 1:
 hit = True
 ok = ok and hit
 print("дизъюнкт", k + 1, cl, "выполнен" if hit else "не выполнен")
print("сертификат", "принят" if ok else "отвергнут")`,
 variables: [
 { name: "cert", role: "сертификат – назначение переменных" },
],
},

```

```

edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представител
rank = {v: 0 for v in "ABCDEFGHI"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе 51)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое рассматриваем

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
 variables: [
 { name: "i", role: "номер текущего ребра – перебираем"},
 { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
 { name: "parent", role: "DSU-массив: цвета компонентов"},
 { name: "mst", role: "рёбра, уже взятые в остов", "51"},
 { name: "total", role: "вес остова (51 на этом графе)"},
],
},
"mst#prim": {
 vizTitle: "Остовное дерево: Прим (рост из вершины E)",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

```

```

vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)"
stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРХОВА"
code: `n = 9 # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C", "D"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "F", "G"),
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"} # DSU: корень компоненты
cheapest = {} # корень компоненты -> самое дешёвое ребро
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
phase = 0 # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None # ребро-кандидат

--- тело алгоритма ---
def find(x):
 while comp[x] != x:
 comp[x] = comp[comp[x]]
 x = comp[x]
 return x

while len(mst) < n - 1:
 phase += 1
 cheapest = {}
 for (w, a, b) in edges: # КАЖДАЯ компонента ОДНОВЕРХОВА
 ra, rb = find(a), find(b)
 if ra == rb:
 continue
 u, v = a, b
 if ra not in cheapest or w < cheapest[ra][0]:
 cheapest[ra] = (w, a, b)
 if rb not in cheapest or w < cheapest[rb][0]:
 cheapest[rb] = (w, a, b)
 if not cheapest:
 break # граф несвязен
 for (w, a, b) in cheapest.values():
 ra, rb = find(a), find(b)
 if ra == rb:
 continue # две компоненты

```

```

 { name: "k", role: "уровень: блоки $2^k \times 2^k$ (шаг
 { name: "st2[{r,c,k,k}]", role: "минимум квадратн
 },
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваем
 { name: "kx, ky", role: "уровень самого крупного
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
 S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
 variables: [
 { name: "i, j", role: "текущая ячейка таблицы S (

```

```

const keep: string[] = [];
for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
 keep.push(line);
 }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
 keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, sync: Sync) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 const init = extractInit(sync.code);
 return `${head}\n${init}\n`;
}

```

=====

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

-----

ФАЙЛ 26/111: src/data/tickets/ticket1\_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМЕР: 1.5 КБ

-----

```
import { Chapter } from '../../types';
```

```

span class="font-mono text-emerald-300">[0, 1
тям <span class="font-mono text-emerald-300
– это одиночные элементы массива. В узле л
чество единиц, И/ИЛИ – любая операция, кото
>).</p>

```

```

<p class="text-slate-300 text-sm mb-4">Отсю
а каждом уровне встречает один из трёх случ
не пересекается – возвращаем нейтральный э
склеиваем. Частичных пересечений на уровне
одного элемента – спуск к листу и пересчёт

```

```

<div class="grid grid-cols-1 sm:grid-cols-3
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 <div class="border border-emerald-500 shadow-md">

```

Аналогия 1: Корпоративная пре

```

 </div>
 <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v])</div>
свой баланс. Это и есть отложенное обнов
</div>

```

```
<p class="text-slate-500 text-xs mt-3">
 кивают в строгом порядке: сначала присваива
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
за $O(1)$, но строятся один раз и умирают при
ми.</p>
```

```
 <p class="text-slate-400 text-sm mb-1">
идемпотентные запросы (min, gcd, И) по ста
 <p class="text-slate-400 text-sm mb-1">
n ячеек (не $2n!$) – иначе на $n = 6$ поймать п
n ... $2n-1$] и n – степень двойки (или использ
 <p class="text-slate-400 text-sm">•
ть лень во всё дерево – это $O(n)$ и весь смы
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
-2 py-0.5 rounded text-xs font-mono text-em
ся один раз), запрос на отрезке <code class=
-slate-800"> $O(\log n)$ </code>, одиночное обно
-300 border border-slate-800"> $O(\log n)$ </code>
s font-mono text-emerald-300 border border-
xt-xs font-mono text-emerald-300 border bord
ounded text-xs font-mono text-emerald-300 b
```

```
<p class="text-slate-500 text-xs mt-3 leading
 в редакторах и базах, сжатие координат + «
оить», дерево отрезков по дереву (HLD), пер
</p>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

Код: построение и запрос суммы (C

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```



```

 y[4 * MAXN]; // «обещание»: прибавит

void push(int v, int l, int r) {
 if (lazy[v] == 0) return;
 int m = (l + r) / 2;
 tree[2 * v] += lazy[v] * (m - l + 1); // п
 lazy[2 * v] += lazy[v];
 tree[2 * v + 1] += lazy[v] * (r - m); // и
 lazy[2 * v + 1] += lazy[v];
 lazy[v] = 0; // о
}

void add(int v, int l, int r, int L, int R, int x) { /
 if (r < L || R < l) return;
 if (L <= l && r <= R) {
 tree[v] += x * (r - l + 1);
 lazy[v] += x; // д
 return;
 }
 push(v, l, r); // о
 int m = (l + r) / 2;
 add(2 * v, l, m, L, R, x);
 add(2 * v + 1, m + 1, r, L, R, x);
 tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

<p class="text-slate-400 text-xs">Пр  
 г накопиться. Забыл push – получил правильн  
 </div>  
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">  
 Код: итеративное дерево (2n, снизу)  
 </summary>  
 <div class="p-5 text-sm text-slate-300  
 <pre class="bg-slate-950 p-4 rounde  
 # для удобств

```
</section>
```

```
`
 },
];
```

---

ФАЙЛ 27/111: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТР0К: 1194 | РАЗМ:

---

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [
 {
```

```
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Дейк
ершины
требование к графу – неотрицательные вес
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Ограничения: минус ломает жадно.
 </div>
 <p class="text-slate-300 text-sm mb-4">
 s→a = 2
 b→a = -2. Вернётся к началу вытащит 3 - 2 = 1 придёт Джонсона (билет 17).</p>
 </div>
 </div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500">
 <h3 class="text-xl font-bold text-blue-400">Демонстрация

 <p class="text-slate-300 text-sm mb-4">Демонстрация (D, Дом 2 E, ТЦ F, Метро G) и 11 односторонних ширины (u → v): переменные prev, done, dist ают картинку напрямую.</p>

 <ol class="list-decimal pl-5 space-y-2 text-slate-300">
 Инициализация. ∞; в кучу кладём prev пуст.
 Извлечь минимум. Достаём пару du. Если done – пара устарела.
 Зафиксировать. dist меняется.
 Релаксировать рёбра. Для каж. u: если dist[v] > dist[u] + weight(u, v) (dist[v] = dist[u] + weight(u, v)).
 Повторять, пока куча не пуста.


```



```

<h3 class="text-xl font-bold text-emerald-400">
 <div class="overflow-x-auto my-4">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200">
priority_queue (C++)</td><td class="py-2 pr-3 font-mon">
й выбор</td></tr>
 <tr class="border-b border-slate-200">
ease-key</td><td class="py-2 pr-3 font-mon">
очереди</td></tr>
 <tr class="border-b border-slate-200">
поиск минимума</td><td class="py-2 pr-3 font-mon">
</tr>
 <tr class="border-b border-slate-200">
><td class="py-2 pr-3 font-mon">0(V + E)</td></tr>
 <tr class="align-top"><td class="font-mon">0(E + V · C)</td><td class="py-2">ц
 </tbody>
 </table>
 </div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
INF = float("inf")

def dijkstra(graph, start):

```

```

print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
 <p class="text-slate-400 text-xs">П
 же вершины, потому что читает ваши <span c
nt-mono">done.</p>
 </div>
</details>

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">> Код: C++ (priority
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <queue>;
using namespace std;
const long long INF = 4e18; // не INT_MAX:

int n;
vector<vector<pair<int,int>>>> g;
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 priority_queue<pair<long long,int>;,
 vector<pair<long long,int>>&
 greater<pair<long long,int>>;>
 pq.push({0, s});
 while (!pq.empty()) {
 auto [du, u] = pq.top(); pq.pop();
 if (du > d[u]) continue; // устарев
 for (auto [v, w] : g[u]) {
 if (du + w < d[v]) {
 d[v] = du + w;
 p[v] = u;
 pq.push({d[v], v});
 }
 }
 }
}

```

```

 . Память – матрица
 </div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-amber-400

 <div class="bg-rose-950/30 p-5 rounded-lg b
 <p class="text-sm font-bold text-rose-3
 <ul class="list-disc pl-5 space-y-2 tex
 Отрицательные веса. Алгор
 ксация вершины необратима. Нужен Форд–Беллм
 Отрицательные циклы. При
 ечное число. Сначала проверьте граф Фордом–
 Ленивое удаление. Без пр
 одна вершина обрабатывается несколько раз:
 Переполнение INF. <span
 уходит в отрицательное число и «улу
 "font-mono">u.
 Ориентированность. Демо-
 ом графе каждое ребро добавляется в оба спи
 Дейкстра ≠ BFS. BFS – ча
 С разными весами FIFO ломается, нужна куча
 «Одна пара». Алгоритм сч
 ий выход при извлечении цели (билет 17), но

 </div>

 <p class="text-slate-300 text-sm mt-4 mb-4"
 ов ценой скорости <span class="font-mono te
 font-mono text-emerald-300"> $O(V^3)$; б
 после перевзвешивания потенциалов). Предст
 </div>
 </div>
</section>`
 },
 {

```

```

 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 <div class="border border-emerald-500 shadow-md">
 Аналогия 1: Слухи в деревне
 </div>
 <p class="text-slate-300 text-sm mb
 день» она проходит ровно одно звено
 азах, нужно четыре дня. Отсюда и предел <sp
 деревне из V жителей просто не бывает. Порядо
 </div>

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Отрицательный цикл: временная п
 </div>
 <p class="text-slate-300 text-sm mb
 меньше: <span class="font-mono text-emerald
 и ответ « $-\infty$ ». Контрольная итерация – это де
 .</p>
 </div>
 </div>
 </div>

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400

```



вееет» на 2.</p>

```
<table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 pr-2 font">
 <th class="py-1.5 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-
-2 font-mono">0</td><td class="py-2 pr-2 font
td></tr>
 <tr class="border-b border-slate-
-2 font-mono">-2</td><td class="py-2 pr-2 font
/td></tr>
 <tr class="border-b border-slate-
-2 font-mono">-6</td><td class="py-2 pr-2 font
</td></tr>
 <tr class="border-b border-slate-
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>
 <tr class="align-top"><td class=
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
</tbody>
```

```
</table>
```

```
<p class="text-slate-400 text-xs mt-3">
остижимые из цикла вершины (A, C, D, E, F).
х, до которых из него есть путь.</p>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
```

```

 <tbody class="text-slate-300">
 <tr class="border-b border-
2">одна вершина → все</td><td class="py-2 p
 <tr class="border-b border-
font-mono">0((V+E) log V)</td><td class="p
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">нет</td><td c
 /tr>

 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">молча ошибётся
emerald-300">обнаруживает {d[i][i] < 0}<
 <tr class="border-b border-
ass="py-2 pr-2">список смежности</td><td cl
 <tr class="align-top"><td c
φ, веса ≥ 0</td><td class="py-2 pr-2">минус
 </tbody>
 </table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">» Код: Форд–Беллман
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te

def bellman_ford(n, edges, s):
 """edges = [(u, v, w), ...] — ориентированные рёбра
 dist = [INF] * n
 prev = [-1] * n
 dist[s] = 0
 changed_vertex = -1

 for it in range(n):
 # n-1 рабочих

```

```
dist2, _, cyc2 = bellman_ford(len(nodes), E + [{idx["D": i, "w": 1} for i in cyc2])
print([nodes[i] for i in cyc2]) # ['D', 'A', 'B', 'C']
print([dist2[i] for i in range(len(nodes))]) # все вершины имеют вес 0, кроме 'D' с INF
```

`<p class="text-slate-400 text-xs">Примечание: вершина с <span class="font-mono">INF</span> означает, что вершина не достижима из источника <span class="font-mono">float('inf')</span>`

`<details class="bg-slate-900/40 rounded-lg p-4 border border-slate-700/50"><summary class="p-4 font-bold text-slate-300">Код: C++ (структура)</summary><div class="p-5 text-sm text-slate-300">`

```
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">#include <iostream>
using namespace std;
const long long INF = 4e18;
```

```
struct Edge { int u, v; long long w; };
```

```
int n; // число вершин
vector<Edge> edges; // плоский список рёбер
vector<long long> d;
vector<int> p;
```

`// возвращает вершину, улучшенную на «лишней» итерации,`

```
int bellman_ford(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 int x = -1;
 for (int it = 0; it < n; ++it) { // n-1 итераций
 x = -1;
 for (const auto &e : edges) {
 if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
 d[e.v] = d[e.u] + e.w;
 p[e.v] = e.u;
 x = e.v;
 }
 }
 }
 return x;
}
```

```
def spfa(n, adj, s):
 """adj[u] = [(v, w), ...]. Релаксируем только «живые» вершины.
 INF = float("inf")
 dist = [INF] * n
 in_queue = [False] * n
 counter = [0] * n # сколько раз вершина попала в очередь
 dist[s] = 0
 q = deque([s])
 in_queue[s] = True
 while q:
 u = q.popleft()
 in_queue[u] = False
 for v, w in adj[u]:
 if dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 if not in_queue[v]:
 q.append(v)
 in_queue[v] = True
 counter[v] += 1
 if counter[v] >= n:
 return None # отрицательный цикл
 return dist
```

- Ранний выход. Если з
- рафах экономит кратно.
- SPFA (Shortest Path Faster Algorithm) — алгоритм нахождения кратчайших путей, у которых только что улучшился dist[v], но худший случай остаётся dist[v] из рёбер) SPFA ловит TL. Критерий цикла — количество итераций, против течения — за O(V + E) итераций даётся финальные dist[v].
- DAG. Если граф ациклический, то SPFA работает за O(V + E).

```

type: "html",
category: "Графы. Пути",
content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <p class="text-slate-300 text-sm mb-4">Флой,
 ext-emerald-300"> $0(V^3)$ и <span class="
 (но не с отрицательными циклами) и пишется
 an> размера $V \times V$ </
 вующих рёбер, <span class="font-mono text-em
 n> в остальных клетках.</p>

 <div class="bg-slate-800 p-6 rounded-lg border
 <div class="absolute -top-3 left-4 bg-slate-700
 border-emerald-500 shadow-md">
 Суть фокуса (по шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл
 ass="font-mono text-emerald-300"> k т
 , если в качестве промежуточных
 , ..., $k\}$?». Каждый шаг добавляет в ра
 </p>
 <ul class="text-sm text-slate-300 space-y-2">
 Шаг $k = -1$ (старт): разрё

```

```
border-rose-500 shadow-md">
```

Аналогия 2: Почему k обязан быть

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
s="font-mono">k один раз и бо
садку не соберётся. На демо-графе этой стра
ono text-emerald-300">d[2][1] = ∞, х
пользует «поздние» посредники). Проверка пер
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md">
```

```
<h3 class="text-xl font-bold text-blue-400">Аналогия 2: Почему k обязан быть
```

```
<p class="text-slate-300 text-sm mb-4">Демо-график функции d[i][j] для
вершины 0..6, (i, j), которая улучшает
ляторе переменные j
class="font-mono text-emerald-300">j
напрямую: измените j
```

```
<div class="bg-slate-800/70 p-5 rounded-lg border-rose-500 shadow-md">
```

```
<p class="text-sm font-bold text-white">Аналогия 2: Почему k обязан быть
```

```
<table class="w-full text-xs text-left">
```

```
<thead>
```

```
<tr class="text-slate-500 border-rose-500">
```

```
<th class="py-1.5 pr-2 font-mono">i</th>
```

```
<th class="py-1.5 pr-2 font-mono">j</th>
```

```
<th class="py-1.5 font-bold">d[i][j]</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-700">
-2 font-mono">0</td><td class="py-2">в вершине 0</td><td class="py-2">0</td></tr>
```

```
<tr class="border-b border-slate-700">
-2 font-mono">4</td><td class="py-2">открываем граф</td><td class="py-2">4</td></tr>
```

```
<tr class="border-b border-slate-700">
```

```

 </table>
 </div>
 </div>

 <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-800">
 <h3 class="text-xl font-bold text-indigo-400">Динамика
 <div class="bg-slate-900/40 rounded-lg p-4 border-1 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">Формальная динамика
 <div class="p-5 text-sm text-slate-300">
 <p>Обозначим $d[i][j]$, все промежуточные вершины
 i и $d[i][j]$ и $d[i][j]$
 <p>База: $d[i][j]$
 i
 <p>Переход: путь из $d[i][j]$
 {0...k} либо не использует
 [i][j], – либо использует $d[i][j]$
 <p>Почему хватает одной матрицы.
 <p>Следствие: если в графе есть
 d[k][k] < 0 и рассуждение
 </div>
 </div>
 </div>

 <div class="bg-slate-900/40 rounded-lg p-4 border-1 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">Эксперимент: пере
 <div class="p-5 text-sm text-slate-300">
 <p>Матрица
 <pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
M = [[0,4,INF,2,INF,INF,INF],

```

-----  
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-white">

```
def floyd_warshall(n, edges, directed=True):
 d = [[INF] * n for _ in range(n)]
 nxt = [[-1] * n for _ in range(n)]
 for i in range(n):
 d[i][i] = 0 # диагональ об

 for u, v, w in edges:
 if w < d[u][v]: # кратчайше
 d[u][v] = w
 nxt[u][v] = v
 if not directed and w < d[v][u]:
 d[v][u] = w
 nxt[v][u] = u

 for k in range(n): # k — ВНЕШНИЙ
 for i in range(n):
 if d[i][k] == INF:
 continue
 for j in range(n):
 if d[k][j] == INF:
 continue
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
 nxt[i][j] = nxt[i][k]

 negative = [i for i in range(n) if d[i][i] < 0]
 return d, nxt, negative

def path(nxt, i, j):
 if nxt[i][j] == -1:
 return None # j недостижим
 p = [i]
 while p[-1] != j:
 p.append(nxt[p[-1]][j])
 return p
```



```

bool has_negative_cycle() {
 for (int i = 0; i < n; ++i) if (d[i][i] < 0)
 return false;
}

long long diameter() { // сама
 long long res = 0;
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)
 if (d[i][j] < INF) res = max(res, d[i][j]);
 return res;
}

int center() { // верш
 int best = -1; long long bestEcc = INF;
 for (int i = 0; i < n; ++i) {
 long long ecc = 0;
 for (int j = 0; j < n; ++j) {
 if (d[i][j] == INF) { ecc = INF; break; }
 ecc = max(ecc, d[i][j]);
 }
 if (ecc < bestEcc) { bestEcc = ecc; best = i; }
 }
 return best;
}

```

</div>  
 </details>

<details class="bg-slate-900/40 rounded-lg p-4">  
 <summary class="p-4 font-bold text-slate-500">Та же динамика для  
 -b border-slate-700/50">> Та же динамика для  
 <div class="p-5 text-sm text-slate-300">  
 <table class="w-full text-xs text-slate-500">  
 <thead>  
 <tr class="text-slate-500 border-b">  
 <th class="py-1.5 pr-2">v  
 <th class="py-1.5 pr-2">v  
 <th class="py-1.5 font-b

```

 Параллельные рёбра. Во в
no">d[u][v] стартует с худшего веса.
 Отрицательные циклы. Форм
ых с циклом, ответ
t-mono">d[i][j] = -∞, если существуе
 d[i][k] < ∞
 Недостижимость ≠ 0. В ор
таких пар из 49). Выводить 999 или 0 как «р
 Память. <span class="font
ачений (≈100 МБ на
источка берите Дейкстру.
 Сам путь. Матрица nxt[i][j] (как

 </div>

 <p class="text-slate-300 text-sm mt-4">С
от одного источника и дешевле, если все пары н
n class="font-mono">V Дейкстр быстрее
). Алгоритм Джонсона (билет 17) комб
/span> Дейкстр на перевзвешенном графе. Тра
ции достижимость проверяется по DAG.</p>
 </div>
</div>
</section>`
 },
 {
 id: "johnson-algo",
 title: "17. Графы. Поиск кратчайшего пути. Ускорени
 type: "html",
 description:
 "Джонсон (перевзвешивание потенциалов), встречный
 category: "Графы. Пути",
 content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-

```

```

 </div>
 </div>

 <div class="overflow-x-auto my-6">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">разрешает Дейкс
 <td class="py-2">нет отрицательных
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">не обходим весь граф ра
 <td class="py-2">нужен путь до одной цели</td>
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">две маленькие волн
 <td class="py-2">е вдвое меньше раскрытий</td>
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">раскрывает вершины в сторону цели
 <td class="py-2">-2">есть допустимая эвристика $h(v)$ </td></tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">убирает кучу целиком</td><td class="py-2">
 только 0 и 1</td></tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">куча → массив корзины
 <td class="py-2">С небольшое</td></tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">decrease-key за $O(1)$
 <td class="py-2">теоретически; на практике двоичная/d-ар
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3">не релаксируем все рё
 <td class="py-2">, на практике быстро</td><td class="py-2">o

```

</div>

<h3 class="text-lg font-bold text-white mt-3">  
<p class="text-slate-300 text-sm mb-3">Если  
азад из  $t$  (по перевёрнутым рёбрам). Волны р  
: в «широком» графе (дорожная сеть) это эконо  
огда волны «встретились» в одной вершине; на  
огда <span class="font-mono text-emerald-300">  
более короткий путь.</p>

<h3 class="text-lg font-bold text-white mt-3">  
<p class="text-slate-300 text-sm mb-3">Дейк  
по <span class="font-mono"> $g(v)$ </span> (уже  
( $v$ ) =  $g(v) + h(v)$ </span>, где  $h(v)$  – оценка  
а требования: <b>допустимость</b> ( $h(v) \leq$  и  
монотонность/согласованность</b> ( $h(u) \leq w(u$   
вырождается ровно в Дейкстру – это хороший

<h3 class="text-lg font-bold text-white mt-3">  
<p class="text-slate-300 text-sm mb-3">Если  
кучу заменяют <b>деком</b>: ребро веса 0 к  
новременно живут максимум два уровня), поэто  
>. Обобщение – <b>алгоритм Дила (корзины)</b>  
 $E + V \cdot C$ ). Оба приёма – стандартный ответ на

<h3 class="text-lg font-bold text-white mt-3">  
<p class="text-slate-300 text-sm mb-3">Форд  
оторых расстояние только что уменьшилось</b>  
ческого прогона, но асимптотика та же,  $O(V \cdot E)$   
пользуют осторожно, с эвристиками SLF/LLL).  
) уменьшает высоту и на практике быстрее, к  
</p>

<h3 class="text-lg font-bold text-white mt-3">  
<p class="text-slate-300 text-sm mb-3">Нави  
в <b>предобработку</b>. Сначала – <b>сжатие</b>  
индексами  $0 \dots N$ , и уже по ним строят граф и се  
стояния от нескольких «ориентиров» до всех в

```

1) потенциалы: Форд–Беллман от фиктивной вершины
h = [0] * (n + 1) # из s все рёбра
for _ in range(n): # V итераций: V-
 changed = False
 for u, v, w in edges:
 if h[u] + w < h[v]:
 h[v] = h[u] + w
 changed = True
 if not changed:
 break
else:
 return None # отрицательный

2) перевзвешивание: все $w' \geq 0$
g = [[] for _ in range(n)]
for u, v, w in edges:
 g[u].append((v, w + h[u] - h[v]))

3) Дейкстра от каждой вершины + обратный перевод
d = [[INF] * n for _ in range(n)]
for s in range(n):
 d[s][s] = 0
 heap = [(0, s)]
 while heap:
 dist, v = heapq.heappop(heap)
 if dist > d[s][v]:
 continue
 for to, w in g[v]:
 if d[s][to] > dist + w:
 d[s][to] = dist + w
 heapq.heappush(heap, (d[s][to], to))
 for v in range(n):
 if d[s][v] < INF:
 d[s][v] = d[s][v] - h[s] + h[v] # o
return d</pre>
</div>
</details>

```

```

heap = [(h(s), 0, s)] # (f, g, v)
closed = set()
while heap:
 f, gv, v = heapq.heappop(heap)
 if v == t:
 return gv # ранний выход: 1
 if v in closed:
 continue
 closed.add(v)
 for to, w in g[v]:
 nd = gv + w
 if nd < gdist.get(to, INF):
 gdist[to] = nd
 heapq.heappush(heap, (nd + h(to), nd, to))
return INF

```

# --- 0-1 BFS: веса 0 и 1, дека вместо кучи ---

```

from collections import deque
def bfs01(g, s, n): # g[v] = [(to, w)]
 INF = float("inf")
 d = [INF] * n; d[s] = 0
 dq = deque([s])
 while dq:
 v = dq.popleft()
 for to, w in g[v]:
 if d[v] + w < d[to]:
 d[to] = d[v] + w
 (dq.appendleft(to) if w == 0 else dq.append(to))
 return d

```

<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Вопросы, которые любят задавать (и ответы на них)
 </summary>
 <div class="p-5 text-sm text-slate-300">

```

export const tickets18to20Mst: Chapter[] = [
 {
 id: "mst",
 title: "18–20. Графы. Остовное дерево: Краскал, Прим",
 type: "html",
 description:
 "Три способа построить минимальное остовное дерево",
 category: "Графы. Остовные",
 content: `
<section id="mst" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 он связан. MST – такое остовное дерево, у
 </div>

 <p class="text-slate-300 text-sm mb-4">Все
 реза (cut property): разрежем вершины н
 MST. Алгоритмы отличаются только тем, к
 Прим – «выросшее дерево против остального г

 <p class="text-slate-300 text-sm mb-4">Втор
 никогда не входит в MST. Именно его исполь
 бразовавшемся цикле (рёбра-то мы берём по в

 <div class="overflow-x-auto my-6">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 borde
 `
 }
]

```

Аналогия к билету 18 • Краск

</div>

<p class="text-slate-300 text-sm">С  
». По всей карте растут независимые кусочки  
дерево. Порядок рёбер важен, порядок вершин  
</div>

<div class="bg-slate-800 p-6 rounded-lg  
<div class="absolute -top-3 left-4  
rder border-emerald-500 shadow-md">

✦ Аналогия к билету 19 • Прим:

</div>

<p class="text-slate-300 text-sm">Пр  
а захватывает ближайший непосещённый город.  
ва» против «снаружи».</p>  
</div>

<div class="bg-slate-800 p-6 rounded-lg  
<div class="absolute -top-3 left-4  
r border-rose-500 shadow-md">

Аналогия к билету 20 • Боровк

</div>

<p class="text-slate-300 text-sm">С  
айшему чужому племени: к вечеру компонент в  
<b> $\log_2 V$ </b> дней образуется единая федер  
</div>

</div>

<div class="bg-slate-900/40 rounded-lg bord  
<p class="text-slate-300 text-sm mb-2">  
<p class="text-slate-400 text-sm mb-1">  
сстояние от одной вершины до каждой; MST ми  
куче другой: у Прима – вес ребра к дереву,  
тся.</p>

<p class="text-slate-400 text-sm mb-1">  
вных весах деревьев может быть несколько, н

<p class="text-slate-400 text-sm mb-1">  
дом вместо почти-константного find/union.</p>



```

 if (unite(uv.first, uv.second)) { // не замыка
 cost += w;
 if (++taken == n - 1) break; // V - 1 ребр
 }
}
}

```

</div>  
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">

Код 19: Прим с кучей (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300  
 <pre class="bg-slate-950 p-4 rounde

heapq

```

INF = float("inf")
key = [INF] * n # минимальный вес ребра, ведущ
used = [False] * n
key[0] = 0
heap = [(0, 0)] # (вес ребра, вершина) ← вот
total = 0

```

```

while heap:
 w, v = heapq.heappop(heap)
 if used[v]:
 continue # устаревшая запись в куче
 used[v] = True
 total += w
 for to, weight in graph[v]:
 if not used[to] and weight < key[to]:
 key[to] = weight
 heapq.heappush(heap, (weight, to))
print(total)

```

<p class="text-slate-400 text-xs">С  
 (накапливаем путь от старта), здесь <span  
 а — другая задача.</p>

```
 <p class="text-slate-400 text-xs">3
ну), значит фаз не больше $\log_2 V$.</p>
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg p-4 font-bold text-slate-300 border-slate-700/50">
 <summary>
 Вопросы, которые любят задавать (и
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>• Почему жадность здесь работает?
меном ребра в предполагаемом MST.</p>
 <p>• Чем Прим отличается от Дейкстры?
ного веса против дерева кратчайших путей).</p>
 <p>• Какой алгоритм лучше на плотном графе?
 <p>• Зачем нужна Борувка, если есть Крускал?
независимо, поэтому её любят на GPU и в расчётах.
 <p>• Что если граф несвязный? – все
 </div>
 <p>• Как найти второе по величине ребро на образовавшемся цикле (LCA + максимум)?
 </div>
</details>
</div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 29/111: src/data/tickets/ticket21\_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 444 | РАЗМЕР: 1.2 КБ

---

```
import { Chapter } from '../../../types';
```

```

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #f0f0f0; border-radius: 10px; padding: 10px;">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md" style="border: 1px solid #f0f0f0; border-radius: 10px; padding: 10px;">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb-0" style="color: #f0f0f0; font-size: 0.8em; margin-bottom: 0;">
 Для LLM, которая стримит токены
 на каждом шаге. Если мы частично совпали с запятой
 какого места этого слова мы можем продолжить
 </p>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg" style="background-color: #f0f0f0; border-radius: 10px; padding: 10px;">
 <summary class="p-4 font-bold text-slate-300" style="color: #f0f0f0; padding: 10px;">
 Пример вычисления (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300" style="padding: 10px; color: #f0f0f0; font-size: 0.8em;">
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2" style="list-style-type: none; padding-left: 20px;">
 a: Префиксов нет. <code>0</code>
 ab: Из начала ("a") не совпало

 abc: Опять мимо. <code>0</code>
 abca: 0! Начало ("a") не совпало

 abcab: Начало ("ab") не совпало

 abcabc: Начало ("abc") не совпало

 abcabcd: «d» всё испробовано (не последняя буква). Длина 3: abc против "abc"

 </div>
</details>

```

```

 <p class="text-slate-400 text-sm">•
5] = 4, период 6 - 4 = 2 («ab»).</p>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: поиск подстроки КМР (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
fix_function(s):
n = len(s)
pi = [0] * n
for i in range(1, n):
 j = pi[i - 1]
 while j > 0 and s[i] != s[j]:
 j = pi[j - 1] # откат по уже п
 if s[i] == s[j]:
 j += 1
 pi[i] = j
return pi

def kmp_search(text, pattern):
 m = len(pattern)
 pi = prefix_function(pattern) # O(m) один раз
 j = 0
 found = []
 for i, ch in enumerate(text): # O(n) по тексту
 while j > 0 and ch != pattern[j]:
 j = pi[j - 1]
 if ch == pattern[j]:
 j += 1
 if j == m:
 found.append(i - m + 1) # позиция вхожде
 j = pi[j - 1] # ищем дальше, в
 return found

тот же результат одной префикс-функцией: п от pattern

```

### Аналогия 1: Линейка-шаблон

```
</div>
<p class="text-slate-300 text-sm mb-4">
 Тут мы берём всю строку и «прикидываю
 я начну читать строку отсюда, насколько длин
 Это самый быстрый способ найти индекс
 >, считаешь Z-функцию, и там, где <code>Z[i]
</p>
</div>
```

### <!-- Аналогия 2 -->

```
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
border-rose-500 shadow-md">
```

### Аналогия 2: LLM Самокопираст

```
</div>
<p class="text-slate-300 text-sm mb-4">
 В LLM Z-функция может применяться к строке
 [i]</code> (где <i>i</i> указывает назад на индекс
 сама себя.
</p>
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg p-6">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
```

### Пример вычисления (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300">
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2">
 <code>Z[0]</code> – не считаем, так как Z[0] = 0
 <code>Z[1]</code> (bcabcd): Z[1] = 0
 <code>Z[2]</code> (cabcd): Z[2] = 0
 <code>Z[3]</code> (abcd): Z[3] = 0
```

```
<h3 class="text-xl font-bold text-blue-400 mb-4">

<div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Тр
</div>
<p class="text-slate-300 text-sm">Позволяет найти
 го один проход.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 ет — мы падаем по суффиксной ссылке ("
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
 </div>

<details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 ou
 r-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-
 <pre class="bg-slate-950 p-4 rounded overfl
```

```
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
ат). Сведение A $\leq$ B – преобразовать вход A
.</p>
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Клас
машина Тьюринга) с выбранным бюджетом ресурс
про худший случай, а не про «в средн
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Судoku (Р против I

```
</div>
```

```
<p class="text-slate-300 text-sm">К
ий путь, MST). Класс NP – когда реша
проверишь, что там нет нарушений правил. Во
и решить?». Открыт с 1971 года и стоит мил.
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-purple-500 shadow-md">
```

Аналогия 2: Переводчик (сведе

```
</div>
```

```
<p class="text-slate-300 text-sm">С
ский (сведение за полином) и друг-англичанин
Отсюда главный инструмент: если к B сводитс
ь не стоит.</p>
```

```
</div>
```

```
</div>
```

```
<h3 class="text-lg font-bold text-white mt-4">
<p class="text-slate-300 text-sm mb-3">Изве
ения в цепочке доказываается теоремой об иер
<div class="bg-slate-950 border border-slate-
 <p class="font-mono text-sm sm:text-bas
```

```
<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Стандартная задача — получить сертификат полиномиального размера и проверить, что
— «все вершины по разу и каждая соседняя пара вершин образует
ь полиномиальное сведение $A \leq_p B$. Тогда B — полиномиально разрешима.
<p class="text-slate-300 text-sm mb-3">Теорема 1.1. Задача SAT NP-полна. Доказывается «в лоб»: выводится универсальная
ево формулой размера $\text{poly}(n)$, где переменная x принимает значение 0 или 1.
ильна принятию входа. Дальше всё катится цепочкой полиномиальных
гамильтонов цикл $\rightarrow$ TSP.</p>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <ul class="list-disc list-inside text-slate-300">
 Точный перебор с отсечениями (meet-in-the-middle).
 Приближённые алгоритмы: $\text{set cover} - \ln n$.
 Эвристики и метаэвристики: ke .
 Параметризованная сложность $\text{n})$.
 Специальные классы входов: полиномиальными (независимое множество на д...
 Готовые решатели: SAT/ILP-решатели, реализующие свой
воего алгоритма.

</div>
```

```
<div class="bg-slate-900/40 rounded-lg border">
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
 NP: любая задача из P автоматически в NP. Проблема P-полна (задача
 <p class="text-slate-400 text-sm mb-1">
 постановки — неразрешима, зато NP-трудна). NP-трудная задача —
 <p class="text-slate-400 text-sm mb-1">
 стной трудной A к B ($A \leq_p B$), а не наоборот.
```



```

 ci, li = verts[i]; cj, lj = verts[j]
 if ci == cj: # из од
 continue
 if li.lstrip("-") == lj.lstrip("-") and (li
 continue # х и н
 edges.append((i, j))
 return verts, edges, len(clauses) # клика

```

# Проверка задачи из P для контраста: двудольность = 2-р

```

def is_bipartite(n, graph):
 from collections import deque
 col = [-1] * n
 for s in range(n):
 if col[s] != -1:
 continue
 col[s] = 0
 dq = deque([s])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1
 dq.append(to)
 elif col[to] == col[v]:
 return False
 return True

```

<p class="text-slate-400 text-xs">0  
 ТОВЫЙ ОТВЕТ, НО НЕ НАХОДИТЬ ЕГО; ДЛЯ ЗАДАЧИ

</div>

</details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">

Вопросы, которые любят задавать (

</summary>

<div class="p-5 text-sm text-slate-300

<p>• Дайте определение NP через вер

```

</div>
<div class="space-y-8">
 {/* База представлений */}
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-slate-300"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-slate-300 italic"
 <p class="text-xl font-mono text-white"
 + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb
 глупо, но проверка "знает ли Вася Петю" мгл
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb
 т. Оптимально для почти всех задач.</p>
 </div>
 </div>
 </div>
 </div>

 {/* DFS vs BFS */}
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400"

 <div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l

```

```
<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
```

Код обходов C++ (Основа)

```
</summary>
```

```
<div class="grid grid-cols-1 md:grid-col-
```

```
<pre class="bg-slate-950 p-3 rounded-
```

```
// DFS (Рекурсия с возвратом)
```

```
vector<bool> vis;
```

```
vector<vector<int>>> adj;
```

```
void dfs(int v) {
```

```
 vis[v] = true;
```

```
 for (int u : adj[v]) {
```

```
 if (!vis[u]) dfs(u);
```

```
 }
```

```
}</pre>
```

```
<pre class="bg-slate-950 p-3 rounded-
```

```
// BFS (Очередь FIFO)
```

```
queue<int> q;
```

```
q.push(start_node);
```

```
vis[start_node] = true;
```

```
while (!q.empty()) {
```

```
 int v = q.front(); q.pop();
```

```
 for (int u : adj[v]) {
```

```
 if (!vis[u]) {
```

```
 vis[u] = true;
```

```
 q.push(u);
```

```
 }
```

```
 }
```

```
}</pre>
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
},
```

```
{
```

```
class="font-mono">10 ≤ 3(5) − 6 = 9 −
 2. K3,3 (полный двудольный 3×3
ss="font-mono">9 ≤ 12 − формально пр
2V − 4: 9 ≤
</p>
</div>
```

```
<p>Теорема Куратовского (1930) / Вагн
фа, являющегося подразделением (subdivis
</p>
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-purple-400
<div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-purple-300 italic
 <p class="text-xl font-mono text-white"
</div>
```

```
<div class="space-y-3 text-slate-300 text-s
 <p>Сложность задачи раскраски:</p>
 <ul class="list-disc list-inside space-y-1
 k = 2 (2-раскрашиваемость):
) обычным BFS/DFS обходом.
 k ≥ 3 (k-colorability):


```

```
 <p class="mt-3">Теоремы о раскраске:
 <ul class="list-disc list-inside space-y-1
 Жадная раскраска: дает в
максимальная степень вершины.
 Теорема Брукса: <span cl
ных графов Kn и нечётных циклов C2k+1.
 Теорема о 4 красках: Люб
no text-emerald-300">χ(G) ≤ 4).

```

```

 border border-emerald-500 shadow-md">

```

Аналогия 1: Острова

```

 </div>

```

```

 <p class="text-slate-300 text-sm">Т
ались серые (неисследованные) зоны? Прыгаеш
и компонент. Краска – это $comp[v]$, прыжок ч

```

```

 </div>

```

```

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border border-emerald-500 shadow-md">

```

Аналогия 2: Группы друзей (DSF)

```

 </div>

```

```

 <p class="text-slate-300 text-sm">К
омпании в одну (union). Кто представитель к
биты на компании, и вопрос «в одной ли комп
рут, только если концы в разных компонентах

```

```

 </div>

```

```

</div>

```

```

<h3 class="text-lg font-bold text-white mt-

```

```

<div class="overflow-x-auto">

```

```

 <table class="w-full text-xs text-left

```

```

 <thead>

```

```

 <tr class="text-slate-500 borde

```

```

 <th class="py-2 pr-3 font-b

```

```

 <th class="py-2 pr-3 font-b

```

```

 <th class="py-2 pr-3 font-b

```

```

 <th class="py-2 font-bold">

```

```

 </tr>

```

```

 </thead>

```

```

 <tbody class="text-slate-300">

```

```

 <tr class="border-b border-slate

```

```

 /td><td class="py-2 pr-3">забыли про стрелки

```

```

 у графу</td><td class="py-2 font-mono"> $0(V +$

```

```

 <tr class="border-b border-slate

```

```

 (SCC)</td><td class="py-2 pr-3">из любой в

```

```

 ><td class="py-2 font-mono"> $0(V + E)$ </td></

```

```

 <tr class="align-top"><td class=

```

еская сортировка – это расписание, которое  
p>

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Ограничения: Циклы
</div>
<p class="text-slate-300 text-sm mb
получить опыт, нужна работа". Алгоритм выявл
</div>
</div>
```

```
<div class="bg-rose-950/40 p-4 rounded-lg b
 <p class="text-rose-300 font-bold text-
 <p class="text-slate-300 text-xs leadin
 Обычный массив <span class="font-mono
ли в графе есть ориентированный цикл, код с
 Правильно: используем раскраск

* <span class="font-mono text-sla

* <span class="font-mono text-amb

* <span class="font-mono text-eme

Если во время обхода из серы верши
и гарантированный цикл!
 </p>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
```

Корректный код C++ с детекцией ци

```
</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
```

```
vector<int> adj[MAXN];
int color[MAXN]; // 0=белый, 1=серый (в стеке), 2=чёрный
vector<int> ans;
bool has cycle = false;
```

```
description: "Разбиение орграфа на макро-вершины. А
category: "Графы. Продвинутое",
content: `
```

```
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-emerald-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Компоненты
и <i>ориентированного</i> графа, в котором
 </div>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">
 «Это просто цикл?» – Нет!
</p>
```

```
<details class="bg-slate-900/40 rounded-lg border border-slate-700">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
```

Код: Алгоритм Косарайю (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе
```

```
 Разворачиваем этот список (и получаем топологическую
о не топологическая сортировка самого графа
AG конденсации.
```

`p_edge_id` (индекс ребра, по которому пришли)

**Код поиска мостов C++ (Скрыто)**

```
// Для мультиграфа с edge_id:
vector<pair<int, int>> adj[MAXN]; // {to, edge_id}
int tin[MAXN], low[MAXN], timer;
```

```
void dfs_bridges(int v, int p_edge_id = -1) {
 tin[v] = low[v] = ++timer;
 for (auto [to, edge_id] : adj[v]) {
 if (edge_id == p_edge_id) continue; // Пропускаем обратное ребро
 if (tin[to]) {
 low[v] = min(low[v], tin[to]); // Обратное ребро
 } else {
 dfs_bridges(to, edge_id);
 low[v] = min(low[v], low[to]);
 if (low[to] > tin[v]) {
 // Ребро (v, to) с номером edge_id является мостом
 }
 }
 }
}
```

```
},
{
 id: "graph-articulation",
```



```

 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
void dfs_cutpoints(int v, int p = -1) {
 tin[v] = low[v] = ++timer;
 int children = 0;
 for (int to : adj[v]) {
 if (to == p) continue;
 if (tin[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs_cutpoints(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1) {
 is_cutpoint[v] = true;
 }
 ++children;
 }
 }
 if (p == -1 && children > 1) {
 is_cutpoint[v] = true;
 }
}
}</pre>
 </div>
 </details>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров путь и цикл",
 type: "html",
 description: "Условия существования и построение Эй.
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r

```

→ рекурсивный переход. При возврате (на вы-  
ты: `<span class="font-mono text-emerald-300`

```
<details class="bg-slate-900/40 rounded-lg p-4 font-bold text-slate-300 border-slate-700/50">
```

Код построения Эйлера цикла C++

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded
```

```
vector<unordered_multiset<int>>> adj;
vector<int> path;
```

```
void find_euler(int v) {
 while (!adj[v].empty()) {
 int u = *adj[v].begin();
 adj[v].erase(adj[v].find(u));
 adj[u].erase(adj[u].find(v)); // Убираем ребро
 find_euler(u);
 }
```

```
 path.push_back(v); // Добавляем в путь на выходе!
```

```
</pre>
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
 },
```

```
];
```

---

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

---

```
// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);
edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 useVizStepSync(currentStepIndex, setCurrentStepIndex,
 const v = vizNumber(vars.v);
 if (v === null) return null;
 return steps.map((step, index) => ({ step, index }));
 });
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);
```

```

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
}
};

for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),

```

```

 {isPlaying ? {
 <Pause className="w-4 h-4 ml-1" />
 } : {
 <Play className="w-4 h-4 ml-1" />
 }}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить обход"
 title="Сбросить обход"
 className="flex items-center justify-center ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-3 g
 <div className="lg:col-span-2 relative h-96 lg:l
 <svg className="w-full h-full" viewBox="0 0 8
 {/* Edges */}
 {edges.map((edge, i) => {
 const u = nodes.find((n) => n.id === edge
 const v = nodes.find((n) => n.id === edge

 const isBridge =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);

 return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}

```

```

 strokeWidth="3"
 className="transition-all duration-300"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events-none"
 >
 {node.label}
 </text>

 {isVisited && (
 <>
 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}

```

```

 <div className="flex items-center gap-2 to
 <div className="w-8 h-1 bg-rose-500 bor
 Мост
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 fon
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => {
 <div
 key={idx}
 className={`text-xs p-2 rounded $
xt-slate-500`} `}
 >
 {step.desc}
 </div>
 })}
 </div>
 </div>
 <div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 roun
 <div
 className="bg-rose-500 h-full transition
 style={{
 width: `${currentStepIndex / Math.ma
 }}
 />
 </div>
 </div>
 </div>
</div>

```

```

 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
 } else {
 return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
 }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};

const adjList = getAdjList(edges);

const codeLines = [

```



```

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}.
 updated ? ` Релаксация успешна! dist[${edge.u}] = ${dist[edge.u]},
 },
 activeCodeLine: updated ? 6 : 5,
));
 }

 if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: ` Ранний выход после итерации ${iter}.
 activeCodeLine: 10
 });
 break;
 }
}

if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
 let cycleFound = false;
 for (const edge of edges) {
 if (dist[edge.u] !== 999 && dist[edge.u] + edge.w < dist[edge.v]) {
 cycleFound = true;
 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} → ${edge.v}
 activeCodeLine: 9,
 });
 break;
 }
 }
}

if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null, distances: { ...dist },
 log: ` Завершение алгоритма.
 activeCodeLine: 11
 });
}

```

```

 log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveI}
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 <div className="flex flex-wrap items-center justify-center"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-blue-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-blue-300">Полный
 </div>
 </div>

 <div className="flex items-center gap-3">
 <div className="bg-slate-900 p-1 rounded-lg border"
 <button onClick={() => setHasCycle(false)} className="
 bg-blue-600 text-white" : 'text-slate-400'
 <button onClick={() => setHasCycle(true)} className="
 r gap-1 ${hasCycle ? 'bg-rose-600 text-white'
 </div>
 <button onClick={() => { setIsPlaying(false);
 TEXT : "Вернуться к первому шагу"} className="
 bled:cursor-not-allowed text-white px-3 py-1
 <button onClick={() => setIsPlaying(!isPlaying
 нную демонстрацию"} className={`flex items-center
 sabled:opacity-40 disabled:cursor-not-allowed
 }</button>
 <button onClick={() => { setIsPlaying(false);
 ; }} disabled={compilerLinked} title={compiler
 gap-1 bg-blue-600 hover:bg-blue-500 disabled:backg
 ansition-colors">Шар</button>
 </div>
 </div>
 </div>

```

```

const isCyclePart = hasCycle && ((edge.u === uNode.id && edge.v === 'D'));

let strokeColor = '#475569';
let marker = 'url(#arrow-bf-normal)';
if (isChecking) { strokeColor = '#f59e0b'; }
else if (currentStep.hasNegativeCycle && isChecking) { strokeColor = '#f59e0b'; }

return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={currentStep.hasNegativeCycle && isCyclePart ? 4 : 2} marker={marker} />
 <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)} stroke={strokeColor} strokeDash={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#475569'} />
 <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} text={currentStep.hasNegativeCycle && isCyclePart ? 'D' : ''} fill={strokeColor} fontSize="11" fontWeight="bold" textAnchor="middle" />
 </g>
);
}}}

{nodes.map((node) => {
 const dist = currentStep.distances[node.id];
 const isCheckingNode = currentStep.checkingNode === node.id;

 return (
 <g key={node.id} className="transition-appear">
 <circle cx={node.x} cy={node.y} r={isCheckingNode ? '#7dd3fc' : '#64748b'} strokeWidth={2} />
 <text x={node.x} y={node.y + 5} fill={strokeColor} />
 <text x={node.x} y={node.y - 28} fill={strokeColor} font-weight="bold" textAnchor="middle" className="label" />
 <text x={node.x} y={node.y + 35} fill={strokeColor} />
 </g>
);
})}

```

```

 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge
 return (
 <div key={n.id} className={`p-2 rounded
: 'border-slate-700/60 bg-slate-800/40'}`}>
 <div className="text-xs text-slate
 <div className={`font-mono font-bo
v>
 </div>
);
 }}}}
 </div>
 </div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg p
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5
e === item.line ? 'bg-blue-900/80 text-amber
 <span className="text-slate-600 w-5 s
 <span className="whitespace-pre flex-
 </div>
 }}}}
 </div>
</div>
</div>
</div>
</div>
);
}

```

```
// Init
steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0)."});

queue.push(0);
visited.add(0);
steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по"});

while (queue.length > 0) {
 steps.push({
 activeLine: 4,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: `Очередь не пуста, элементов: ${queue.length}`});

 const curr = queue.shift()!;
 steps.push({
 activeLine: 5,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Достаем из очереди (${curr}).`});
```

```

 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
 });

 return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const step = STEPS[stepIdx];

 const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
 }, []);

 // @ts-expect-error зарезервировано под кнопку «назад»
 const handlePrev = useCallback(() => {
 setStepIdx(prev => Math.max(prev - 1, 0));
 }, []);

 useEffect(() => {
 if (!isPlaying) return;
 if (stepIdx >= STEPS.length - 1) {
 setIsPlaying(false);
 return;
 }
 const timer = setTimeout(handleNext, 1200);
 return () => clearTimeout(timer);
 }, [isPlaying, stepIdx, handleNext]);

 return (
 <div className="bg-slate-900 border-2 border-slate-500" >
 <div className="flex flex-col md:flex-row items-center" >

```

```

 { /* Рёбра */
 { EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n1)
 return (
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
 className={`transition-all duration-500`}
 >
)
 })
 }
)
}

```

```

 { /* Вершины */
 { NODES.map(n => {
 const isCurrent = step.currentNode === n.id
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n.id)

 let fill = '#1e293b'; // slate-800
 let stroke = '#475569'; // slate-600
 let scale = 1;
 void scale;

 if (isCurrent) {
 fill = '#3b82f6'; // blue-500
 stroke = '#60a5fa'; // blue-400
 scale = 1.3;
 } else if (isInQueue) {
 fill = '#059669'; // emerald-600
 stroke = '#34d399'; // emerald-400
 scale = 1.1;
 } else if (isVisited) {
 fill = '#0f172a'; // slate-900
 stroke = '#3b82f6'; // blue-500
 }
 })
 }
)
}

```

```

 {
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); vi' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmp' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in grap' },
 { line: 7, code: ' if neighbor not in' },
 { line: 8, code: ' visited.add(ne' },
 { line: 9, code: ' queue.push(nei' },
].map((cl) => {
 const isActive = step.activeLine === cl
 return (
 <div key={cl.line} className={`px-2 p
d border-l-2 border-blue-400 -ml-0.5` : 'te
 <span className="opacity-40 w-6 inl
 {c
 </div>
);
 })}
 </div>
 </div>

 <div className="bg-slate-800/80 border border
 <div className="text-sm font-bold text-eme
 Состояние Очереди (Queue):
 <span className="text-xs text-slate-500
 </div>
 <div className="flex items-center gap-2 mi
 {step.queue.length === 0 ? (
 <span className="text-slate-500 text-x
) : (
 step.queue.map((qId, i) => (
 <span key={`_${qId}-${i}`} className=
enter justify-center rounded-md shadow-lg s
 {qId}

))
)
)
 }
)
}

```



```
floyd: {
 src: floydImg,
 alt: 'Все ко всем Флойд',
 caption: ' Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
},
};

export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2 border-${info.borderColor}>

 </div>
 <p className={`text-center text-xs mt-2 font-bold text-${info.captionColor}>
 {info.caption}
 </p>
 </div>
);
}
```

---

ФАЙЛ 35/111: src/components/ChapterNav.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import type { Chapter } from "../types";
```

```
interface Props {
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
```

```

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-white transition-all"
 >
 <div className="flex items-center gap-1.5 text-indigo-400 mb-1">
 <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая тема
 </div>
 <div className="text-sm font-bold text-slate-700">
 {prev.title}
 </div>
 </button>
) : (
 <div className="hidden sm:block" />
)}

 {next && (
 <button
 onClick={() => onGo(next.id)}
 className="group text-right p-4 rounded-xl bg-white transition-all sm:col-start-2"
 >
 <div className="flex items-center justify-end gap-1.5 text-indigo-400 mb-1">
 Следующая тема <ChevronRight className="w-3.5 h-3.5" />
 </div>
 <div className="text-sm font-bold text-slate-700">
 {next.title}
 </div>
 </button>
)}
 </nav>
);
};

```

---

```

if (!list || list.length === 0) return null;

const q = list[idx];
const answered = picked !== null;
const correct = answered && picked === q.correctIndex

const pick = (i: number) => {
 if (answered) return;
 setPicked(i);
 if (i === q.correctIndex) setScore((s) => s + 1);
};

const next = () => {
 if (idx + 1 < list.length) {
 setIdx(idx + 1);
 setPicked(null);
 } else {
 setFinished(true);
 }
};

const restart = () => {
 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
};

return (
 <section aria-label="Блиц по теме" className="mt-8" >
 <div className="bg-slate-700/50 p-5 rounded-xl border" >
 <div className="flex items-center justify-between" >
 <h3 className="flex items-center gap-2 text-brown-400" >
 <Zap className="w-4 h-4 text-indigo-400" />
 Блиц: {list.length} вопроса по этой теме
 </h3>

 {finished ? `Итого ${score}/${list.length}` : ''}

 </div>
 </div>
 </section>
)

```

```

 return (
 <button
 key={i}
 type="button"
 onClick={() => pick(i)}
 disabled={answered}
 className={`w-full text-left flex items-center justify-between transition-colors disabled:cursor-default ${tone}`}
 >
 {answered && isRight && <CheckCircle />
 {answered && chosen && !isRight && <Close />
 {option}
 }
 </button>
);
 })}
 </div>

 {answered && (
 <div role="status" className="mt-3 rounded p-4">
 <p className={`text-xs font-bold mb-1 ${correct ? "text-emerald-500" : "text-red-500"} }>
 {correct ? "Верно" : "Неверно"}
 </p>
 <p className="text-[13px] leading-relaxed">
 {feedback}
 </p>
 </div>
)}

 <div className="mt-3 flex justify-end">
 <button
 type="button"
 onClick={next}
 disabled={!answered}
 className="inline-flex items-center gap-2 font-medium text-indigo-200 transition-colors hover:text-indigo-300"
 >
 {idx + 1 < list.length ? "Следующий вопрос" : "Завершить"}
 <ArrowRight className="w-3.5 h-3.5" />
 </button>
 </div>
)}
 </div>
)

```

```

 onOpenCompiler?: () => void;
 }

const isTouch = () => typeof window !== "undefined" &&

export const ChapterView: React.FC<Props> = ({ chapter,
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");

 useTermHints(contentRef, [chapter.id]);
 /**
 * Упоминания «билет 15» становятся переходами на стр. 15
 * Разметка идёт ПОСЛЕ терминов: подсказки глоссария
 */
 useTicketLinks(contentRef, [chapter.id], { selfId: chapter.id });

 const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
 }, []);

 const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(anchor), 1000);
 }, [cancelHide]);

 useEffect(() => {
 setAnchor(null);
 setSaving("idle");
 }, [chapter.id]);

 /**
 * Печать/«сохранить в PDF» этой темы: раскрываем все
 * чтобы на бумагу попал весь текст (доказательства,

```

```

 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect()
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
 // ссылка на другой билет: переходим внутри приложения
 const link = (e.target as HTMLElement).closest<HTMLLinkElement>
 if (link) {
 e.preventDefault();
 onGo(link.dataset.goto ?? "");
 return;
 }
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
};

```

```

 еры"
 className="flex items-center gap-1.5 text-x
 300 hover:text-white hover:border-sky-500 t
 >
 <FileText className="w-3.5 h-3.5" /> Текст

</div>

<div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.con
/>

<p className="mt-6 flex items-start gap-2 text-
ed-lg px-3 py-2 print:hidden">
 <MousePointerClick className="w-4 h-4 shrink-1

 Подчёркнутые термины – интерактивные: навед
 объяснение на пальцах, мини-анимацию и кноп

</p>

{viz && (
 <section id="chapter-viz" className="mt-8 scr
 <div className="flex items-center justify-b
 <h3 className="flex items-center gap-2 te
 <Sparkles className="w-4 h-4 text-indig
 Демонстрация: {viz.title}
 </h3>
 <div className="flex items-center gap-2">
 <button
 onClick={() => onOpenSimulator(chapte

```

```

 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
 />
</>
);
};

```

ФАЙЛ 38/111: src/components/ComplexityViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, Rotate } from 'react-feather';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizStep } from 'viz';

```

```
/**
```

```
 * Билет 24 – «Классы сложности, сведение задач».
```

```
 *
```

```
 * Четыре вкладки:
```

```
 * 1. «Иерархия» — $P \subseteq NP \subseteq PSPACE \subseteq EXP$ и NP-полные
```

```
 * 2. «Сведение» — $A \leq_p B$ по шагам: преобразовал вход
```

```
 * 3. «Верификатор» — проверка сертификата для 3-SAT по
```

```
 * 4. «Рост» — живой график по переменным n и ops
```

```
 * сколько операций насчитал твой алгоритм
```

```
 * похоже (полином или экспонента).
```

```
 *
```

```
 * Синхронизация с компилятором (PAGE_SYNC["complexity-"
```

```
 * cls — имя класса (подсветит нужный овал), step/i —
```

```
 * k — номер проверяемого дизъюнкта, cert/assign — сер
```

```
 * n и ops — точки на графике роста, ok/ans — вердикт
```

```
 */
```

```
type Tab = "hierarchy" | "reduce" | "verify" | "growth"
```

```
const TABS: { id: Tab; label: string; icon: React.React
```



Универсальное пособие • полный исходный код

```

-полна.", boxes: ["A", "x", "f", "y", "B", "ans"],
];
```

```
const BOX_LABEL: Record<string, { t: string; c: string }> =
 A: { t: "задача A\n(NP-полная)", c: "#f43f5e" },
 x: { t: "вход x\n $|x| = n$ ", c: "#64748b" },
 f: { t: "f: сведение\n $0(poly(n))$ ", c: "#f59e0b" },
 y: { t: "вход y\n $|y| = poly(n)$ ", c: "#64748b" },
 B: { t: "решатель B\n(гипотеза: быстрый)", c: "#6366f1" },
 ans: { t: "ответ\n $A(x) = B(f(x))$ ", c: "#10b981" },
};
```

```
/* ----- Верификатор -----
```

```
/** 3-SAT формула: 4 дизъюнкта по 3 литерала, переменные
```

```
const CLAUSES: [number, number, number][] = [
 [1, 2, -3],
 [-1, 3, 4],
 [2, -3, -4],
 [-2, 1, 3],
];
```

```
/** Сертификат, который проверяем: $x = (1, 0, 1, 0)$ — он
```

```
const CERT = [1, 0, 1, 0];
```

```
const VERIFY_STEPS = CLAUSES.map((cl, i) => ({
 title: `Дизъюнкт $\{i + 1\}$: $\{cl.map(l => (l > 0 ?$
 note: `Подставляем сертификат $x = \{CERT.join(", ")$
 .map(l => $(l > 0 ? \neg x\{l\} = \{CERT[l - 1]\}$ ` : $\neg x\{$
 .join(", ")}. $\{cl.some(l => (l > 0 ? CERT[l - 1]$
 : "Все литералы ложны — дизъюнкт НЕ выполнен.")`
 ok: $cl.some(l => (l > 0 ? CERT[l - 1] === 1 : CERT[$
})));
```

```
/* ----- Рост -----
```

```
interface Sample { n: number; ops: number }
```

```
const GROWTH_LINES = [
```

```
}, [playing, maxStep]));
```

```
/* живые переменные компилятора */
```

```
const liveStep = vars ? (vizNumber(vars.step) ?? vizNumber(0)) : null;
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);
```

```
const liveCls = vars ? vizString(vars.cls) : null;
const liveK = vars ? vizNumber(vars.k) : null;
const liveCert = vars ? vizArray(vars.cert) ?? vizArray(0) : null;
const liveOk = vars ? (vars.ok === undefined ? null : vizBoolean(vars.ok)) : null;
const liveN = vars ? vizNumber(vars.n) : null;
const liveOps = vars ? (vizNumber(vars.ops) ?? vizNumber(0)) : null;
```

```
/** Копим точки (n, ops): код пользователя крутит цикл
```

```
useEffect(() => {
 if (liveN === null || liveOps === null) return;
 if (!Number.isFinite(liveN) || !Number.isFinite(liveOps)) return;
 setSamples((prev) => {
 const last = prev[prev.length - 1];
 if (last && last.n === liveN && last.ops === liveOps) return prev;
 const withoutN = prev.filter((s) => s.n !== liveN);
 return [...withoutN, { n: liveN, ops: liveOps }];
 });
}, [liveN, liveOps]);
```

```
/** Оценка показателя роста по последним точкам (лог-среднее)
```

```
const verdict = useMemo(() => {
 const pts = samples.filter((s) => s.ops > 0 && s.n > 0);
 if (pts.length < 3) return null;
 const xs = pts.map((p) => Math.log(p.n));
 const ys = pts.map((p) => Math.log(p.ops));
 const mx = xs.reduce((a, b) => a + b, 0) / xs.length;
 const my = ys.reduce((a, b) => a + b, 0) / ys.length;
 let num = 0;
 let den = 0;
```

```

const reduceFrame = REDUCE_STEPS[Math.min(step, REDUCE_STEPS.length)];
const verifyIdx = Math.min(step, VERIFY_STEPS.length);
const verifyDone = verifyIdx >= VERIFY_STEPS.length;
const allOk = VERIFY_STEPS.every((s) => s.ok);

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-1" >
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-purple-600 text-white" >
 <Boxes className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold" >
 <p className="text-xs text-purple-300">P · I
 </p>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-4" >
 {TABS.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={"flex items-center gap-1.5 px-3 py-2 bg-purple-600 text-white" : "bg-slate-900/60 text-slate-300"}
 >
 {t.icon} {t.label}
 </button>
))}
 </div>

 {(tab === "reduce" || tab === "verify") && (
 <div className="flex flex-wrap items-center gap-2" >
 <button onClick={() => { setStep(0); setPlaying(true); }}
 className="text-sm text-slate-300 hover:text-white"><RotateCcw className="h-4 w-4" />
 <button onClick={() => { setPlaying(false); setStep(0); }}
 className="text-sm lg bg-slate-900 border border-slate-700 text-slate-300">

```

```

 <text x={250} y={352} textAnchor="middle"
 <text x={250} y={368} textAnchor="middle"
 рхии времени)</text>
 </svg>
 </div>
 <p className="text-xs text-slate-400 leading-
 <p className="text-[11px] text-slate-500">
 Подсказка: выполни в компиляторе <code clas
 </p>
</div>
})

```

```

{ /* _____ СВЕДЕНИЕ _____
{tab === "reduce" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <div className="flex flex-wrap items-center
 {reduceFrame.bboxes.map((b, i) => {
 const meta = BOX_LABEL[b];
 const active = i === reduceFrame.active
 return (
 <React.Fragment key={b + i}>
 <div
 className="rounded-lg border px-3
 style={{
 borderColor: active ? meta.c :
 background: active ? meta.c + "
 opacity: active ? 1 : 0.45,
 boxShadow: active ? `0 0 18px $
 minWidth: 96,
 }}
 >
 <span className="text-[11px] font
 </div>
 {i < reduceFrame.bboxes.length - 1 &w
 </React.Fragment>
);
 })}
)}
)}
}

```

```

 <span className="font-mono text-xs
span>
 <span className="font-mono text-[10
 ")}
 <span className={"text-[11px] font-l
0")}>
 {checked ? (st.ok ? "✓ истинно" :

 </div>
);
 }
 </div>
 {(verifyDone || liveOk !== null) && (
 <div className={"mt-3 rounded-lg border p
/30 text-emerald-300" : "border-rose-600/40
 {(liveOk ?? allOk) ? "Сертификат принят
вергнут: хотя бы один дизъюнкт ложен."}
 </div>
)}
</div>
<div className="bg-slate-900/70 border border
 <p className="text-sm font-bold text-white
 <p className="text-xs text-slate-400 leading
определение NP: короткий сертификат + быстр
 </div>
</div>
)}

{/* _____ ПОСТ _____
{tab === "growth" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <div className="flex flex-wrap items-center
 <span className="text-[10px] uppercase tr
 <span className="font-mono text-xs bg-sla
 ?? "-">
 <span className="font-mono text-xs bg-sla
e0ps ?? "-">

```

```

 </p>
 })
 </div>
 <div className={"rounded-xl border p-3.5 " +
 /60"}>
 <p className="text-sm font-bold text-white" >
 {verdict ? `Похоже на ${verdict.name}` :
 </p>
 <p className="text-xs text-slate-400 leading-5" >
 {verdict
 ? `Наклон в лог-лог координатах $\approx$ ${verdict.slopeTail.toFixed(2)} – кривая загибается
 : "Как только в коде появятся n и ops,
 e с n)."}
 </p>
 </div>
 </div>
 })
}

{/* живые переменные */}
<div className="mt-3 flex flex-wrap items-center" >

 {vars ? (
 ["n", "ops", "cls", "k", "cert", "ok"].map((name) => {
 vars[name] === undefined ? null : (

 {name} = {JSON.stringify(vars[name]).slice(0, 10)}

)
 })
) : (

 открой панель Python: переменные <code class="font-mono text-slate-300">k</code>, <code className="font-mono text-slate-300">n</code>
 управляют этой демонстрацией

)}
 </div>

```

```

 { line: " }", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: "}", highlight: "normal" },
];

```

```

interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

```

```

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
]

```

```

{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа A (уже посещён)! Обратное ребро
 activeCodeLine: [7,8,9]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6

```



```

 },
];

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length - 1);
 const rawV = vizString(vars.v) ?? (vizNumber(vars.v) ?? null);
 const rawTo = vizString(vars.to) ?? (vizNumber(vars.to) ?? null);
 const toId = (value: string | null) => {
 if (value === null) return null;
 const byLabel = GRAPH_NODES.find((node) => node.label === value);
 return byLabel ?? (Number.isFinite(Number(value)) ? Number(value) : null);
 };
 const v = toId(rawV);
 const to = toId(rawTo);
 if (v === null) return null;
 const candidates = DFS_STEPS
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => step.currentNode === v && (step.to === to || step.to === null));
 return candidates.at(-1)?.index ?? null;
});
const [dfsAuto, setDfsAuto] = useState(false);
const dfsTimer = useRef<NodeJS.Timeout | null>(null);

useEffect(() => {
 if (dfsAuto) {
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false); }
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer.current); };
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

```

```

const u = GRAPH_NODES.find(n => n.id === e.u);
const v = GRAPH_NODES.find(n => n.id === e.v);
const bridge = step.bridges.includes(e.key);
const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.activeEdge[1]===e.v) ||
 (step.activeEdge[0]===e.v && step.activeEdge[1]===e.u)
);
const activeBack = step.backEdge && (
 (step.backEdge[0]===e.u && step.backEdge[1]===e.v) ||
 (step.backEdge[0]===e.v && step.backEdge[1]===e.u)
);
let sc = "#475569", sw = 2, dash = "none";
if (bridge) { sc = "#eab308"; sw = 4; }
else if (activeTree) { sc = "#10b981"; sw = 2; }
else if (activeBack) { sc = "#f43f5e"; sw = 2; }
else if (step.visitedIndices.includes(e.u) || step.visitedIndices.includes(e.v)) { sc = "#444"; sw = 2; }
return <line key={e.key} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={sc} strokeWidth={sw} strokeDash={dash}
 className="transition-all duration-500"/>
}}}
{step.currentNode !== null && step.currentNode.id === e.u ?
 const n = GRAPH_NODES.find(x => x.id === e.v);
 if (!n) return null;
 return <circle cx={n.x} cy={n.y} r={18} fill={cur} stroke={st}/>
 >;
}}())
{GRAPH_NODES.map(n => {
 const cur = step.currentNode === n.id;
 const vis = step.visitedIndices.includes(n.id);
 const st = step.stack.includes(n.id);
 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b" : "#444"}
 stroke={cur ? "#fff" : st ? "#34d399" : "#444"}
 strokeWidth={2.5}
 className="transition-all duration-300"/>
 <text x={n.x} y={n.y+3} textAnchor="middle">{n.label}</text>
 </g>
 >;
})}

```

```

 </pre>
 </div>
 </div>

 { /* Stack */}
 <div className="bg-slate-900/50 rounded-xl border border-slate-800" >
 <div className="text-[10px] font-bold text-slate-400" >
 Стек DFS
 {step.stack.length}
 </div>
 <div className="flex flex-col-reverse gap-1" >
 {step.stack.length === 0
 ? <div className="text-[10px] text-slate-400" >
 Стек пуст
 </div>
 : step.stack.map((id, i) => {
 <div key={id} className={`px-2 py-0.5 text-[10px]`}>
 {i === step.stack.length - 1
 ? "bg-emerald-600/20 text-emerald-400"
 : "bg-slate-900 text-slate-400"}
 Вершина {GRAPH_NODES.find(n=>n.id === id).name}
 </div>
 })
 }
 </div>
 </div>

 { /* Log */}
 <div className="bg-indigo-950/10 border border-indigo-800" >
 <p className="text-[11px] text-slate-300 leading-tight" >
 <div className="mt-2 pt-2 border-t border-slate-800" >
 Мосты:

 {step.bridges.length > 0 ? step.bridges.map(b => b.id) : ""}

 </div>
 </div>
 </div>
 </div>
</div>

```

```
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];
```

```
const adjList: { [key: string]: { v: string; w: number } } = {};
nodes.forEach(n => (adjList[n.id] = []));
edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
});
```

```
const codeLines = [
 { line: 1, text: 'function dijkstra(graph, start):' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0' },
 { line: 3, text: ' pq = PriorityQueue(); pq.push(start, 0)' },
 { line: 4, text: ' while not pq.empty():' },
 { line: 5, text: ' d, u = pq.pop() // Извлечение минимума' },
 { line: 6, text: ' if d > dist[u]: continue' },
 { line: 7, text: ' for v, weight in graph.neighbors(u):' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' dist[v] = dist[u] + weight' },
 { line: 10, text: ' pq.push(v, dist[v])' },
 { line: 11, text: ' return dist // Все кратчайшие пути' },
];
```

```
const [steps, setSteps] = useState<Step[]>([]);
```

```

 for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Смотрим соседа ${v}. Текущее расстояние:
 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Улучшение (Релаксация)! Новое кратчайшее
 расстояние: ${dist[v]}.`,
 activeCodeLine: 9,
 });
 }
 }
 }
}

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Дейкстра всё доставил! Все возможные кратчайшие
 расстояния найдены.`,
 activeCodeLine: 11,
});

setSteps(simulationSteps);
}, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 currentStepIndex++;
 }, 1000);
 }
 return () => {
 if (timer) clearTimeout(timer);
 };
}, [isPlaying, currentStepIndex]);

```

```

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полн
 </p>
 </div>
 </div>

 <div className="flex flex-wrap items-center justify-center">
 <button
 onClick={() => { setIsPlaying(false); setCurrentStepIndex(0); }}
 disabled={compilerLinked}
 title={compilerLinked ? HINT_TEXT : "Вернуть к началу"}
 className="flex items-center gap-1 bg-slate-700 text-white px-3 py-2 rounded"
 >

 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 disabled={compilerLinked}
 title={compilerLinked ? HINT_TEXT : "Прогресс"}
 className={`flex items-center gap-1 px-4 py-2 rounded ${
 disabled ? 'cursor-not-allowed' : 'bg-amber-600 hover:bg-amber-500'
 }`}
 >
 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); if (currentStepIndex > 0) { setCurrentStepIndex(currentStepIndex - 1); } }}
 disabled={compilerLinked || currentStepIndex === 0}
 title={compilerLinked ? HINT_TEXT : "Следующий"}
 className="flex items-center gap-1 bg-indigo-600 text-white px-3 py-2 rounded"
 >

 </button>
 </div>

```



```

 return (
 <div key={u} className={`p-3 rounded-lg
 isCurrentNode ? 'bg-emerald-900/60' :
 'text-slate-300'
 }`>
 <div className="flex items-center gap-2">
 <span className={`px-2 py-0.5 rounded-3xl
 text-slate-300'}`>
 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2">
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep === idx
 return (
 <span key={idx} className={`px-2 py-0.5 rounded-3xl
 isCheckingThis ? 'bg-amber-900/60 text-slate-300'
 }`>
 {edge.v}

 {edge.w}

);
)
 })}
 </div>
 </div>
);
 })}
 </div>
 </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>

```



```

 </div>

 {/* Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60" >
 <div>
 <h4 className="text-lg font-bold text-slate-100">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4" >
 {codeLines.map((item) => {
 const isActive = currentStep.activeCodeLine === item.line
 return (
 <div key={item.line} className={`py-1 px-2
 ${isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-400'}>
 {item.line}
 {item.code}
 {isActive &&

 }
 </div>
);
 })}
 </div>
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 41/111: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord } from '...';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from '...';

```

```

 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано`,
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }

 if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр`,
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
 }

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibМемо(${n - 1})`,
 codeLine: 4,
 memoState: { ...memoMap }
 });

 const res1 = simulateFib(n - 1);

```

```

 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]));

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
const liveMemo = vizRecord(vars?.memo);
const compilerLinked = liveK !== null || !!liveMemo;
const memoState = liveMemo
 ? Object.fromEntries(Object.entries(liveMemo).flatMap(
 ([key, value]) => {
 const number = vizNumber(value);
 return number === null ? [] : [[Number(key), number]];
 }
))
 : baseStep?.memoState || {};
const currentStep: DPStep | null = baseStep && compilerLinked
 ? {
 ...baseStep,
 n: liveK ?? baseStep.n,
 memoState,
 desc: `Компилятор: k=${liveK ?? "-"}; таблица м...
 }
 : baseStep;

return (
 <div className="bg-slate-900 rounded-2xl border border-slate-800" >
 /* Header & Controls */
 <div className="flex flex-col lg:flex-row lg:items-center" >
 <div>
 <div className="flex items-center gap-3 mb-2" >
 <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text-white" >
 </div>
 <p className="text-sm text-slate-400" >
 Смотри, как таблица DP кэширует результаты

```

```

 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:bg-amber-600 text-slate-950'
 : 'bg-emerald-600 text-white hover:bg-emerald-700 text-white'
)` }
 >
 {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); setStepsIndex(0); }}
 disabled={currentStepIndex === steps.length - 1}
 className="p-2 text-slate-400 hover:text-slate-500"
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
</div>
</div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded-lg" >
 <h4 className="text-xs font-bold uppercase tracking-wide" >DP Table View
 <div className="flex flex-wrap items-center gap-2" >
 {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
 const isCached = num in memoState || num <= 2;
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return (
 <div
 key={num}
 className={`flex flex-col items-center justify-center gap-2 p-2
 ${isCurrent ? 'bg-emerald-950 border-emerald-500' : 'bg-slate-900 border-emerald-500/50'}
 ${isCached ? 'border-slate-800' : 'border-slate-800'}
 ` >
 {num}
 {val}
 </div>
);
 })}
 </div>
</div>

```

```

 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg-white'
 : 'border-transparent text-slate-400 hover:bg-slate-100'
 }`
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
 </div>

```

```

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify-between"
 {activeTab === 'table' && (
 <div className="bg-slate-950 p-6 rounded-2xl border border-slate-800"
 <h5 className="font-bold text-white text-sm"
 {steps.length} шагов

 </h5>
 <p className="text-sm text-slate-400">
 В классической рекурсии для N={targetN} необходимо вычислить
 вычисленное значение в таблице, мы моментами не можем
 </p>
 <div className="p-4 bg-slate-900 rounded-xl border border-slate-800"
 Всего шагов симуляции: {steps.length}
 {steps[targetN]}

 </div>
 </div>
)}

```

```

{activeTab === 'code' && (
 <div className="bg-slate-950 rounded-2xl border border-slate-800"
 <div className="bg-slate-900 px-6 py-3 border border-slate-800"
 {code}
 {steps.length}
 </div>
)}

```



```

 return;
 }
 const key = ekey(last, vId);
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощает");
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 if (nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
 } else {
 setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С
 не цикл.`);
 }
 setC1Done(true);
 } else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $
 }
};

```

/\* — что говорит компилятор — \*/

```

const live = useMemo(() => {
 if (!vars) return null;
 const rawPath = vizArray(vars.path) ?? vizArray(var
 const path = (rawPath ?? []).
 .map((x) => (typeof x === "number" ? x : vizNumbe
 .filter((x): x is number => x !== null && Number.
 const usedRaw = vizArray(vars.used) ?? vizArray(var
 const usedFlags = (usedRaw ?? []).map((x) => Boolean
 const degRec = vizRecord(vars.deg) ?? vizRecord(var
 const deg = degRec
 ? C1_NODES.map((n) => {
 const val = degRec[String(n.id)] ?? degRec[n.

```

```

: c1Msg;

const verdict =
 odd.length === 0
 ? { text: "0 нечётных → ЭЙЛЕРОВ ЦИКЛ есть (старт и финиш в одной)" }
 : odd.length === 2
 ? { text: `2 нечётные (${odd.map((id) => C1_NODES[id].label).join(" и ")}, в другой`, cls: "bg-amber-950/40 border-amber-950/40" }
 : { text: `${odd.length} нечётных → эйлерова маршрута нет` };

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 все рёбра ровно по одному разу</h4>
 <button onClick={resetC1} className="flex items-center justify-center gap-2 border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> Сброс
 </button>
 </div>

 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)] border border-transparent">
 <svg className="w-full h-[260px] z-10 relative">
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!;
 const v = C1_NODES.find(n => n.id === e.v)!;
 const key = ekey(e.u, e.v);
 const used = visitedKeys.includes(key);
 // номер этого ребра в маршруте – подписываем
 const order = path.length > 1 ? (() => {
 for (let s = 0; s + 1 < path.length; s++)
 if (path[s] === u && path[s + 1] === v) return s + 1;
 return null;
 })() : null;
 const isLastEdge = order !== null && order === C1_EDGES.length - i;
 return <g key={i}>

```



```

 : "bg-slate-900/50 border-slate-700 text-slate-500"
 }` }>
 {msg}
 </div>

 { /* Критерий Эйлера */ }
 <div className={`p-3 rounded-xl border text-xs font-mono`} >
 Критерий Эйлера (для связного графа): { verdict }

 Степени: { C1_NODES.map((n) => `${n.label}=${n.degree}`) }
 две нечётны. Алгоритм Иера строит маршрут за

 </div>

 <div className="flex flex-wrap items-center gap-2" >

 /> из компилятора:
 {live ? (
 <>

 {live.path.map((id) => C1_NODES[id].label).join(' ')}

 ойдено: {live.keys.length}/{C1_EDGES.length}

 {odd.length}
 {live.step !== null &&
 x-2 py-1">step = {live.step}}
 </>
) : (

 открой панель Python и запусти код: массив C1_NODES —
 том графе, а <code className="font-mono text-[11px] text-slate-500" >
 есчитают критерий

)}
 </div>
</div>
);

```

```

const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

const adjList: { [key: number]: { v: number; w: number } } = {
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
};

const codeLines = [
 { line: 1, text: 'function floyd_warshall(matrix, V' },
 { line: 2, text: ' dist = copy(matrix)' },
 { line: 3, text: ' for k from 0 to V - 1: // Пос' },
 { line: 4, text: ' for i from 0 to V - 1:' },
 { line: 5, text: ' for j from 0 to V - 1' },
 { line: 6, text: ' if dist[i][k] + d' },
 { line: 7, text: ' dist[i][j] = ' },
 { line: 8, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist = initialMatrix.map(row => [...row]);

 simulationSteps.push({
 k: -1, i: -1, j: -1, matrix: dist.map(r => [...r])
 });
});

```

```

 setCurrentStepIndex(0);
 setIsPlaying(false);
 }, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
/**
 * Матрица из кода пользователя – СВОЕЙ размерности.
 * протискивались в демо-сетку 7x7: при n < 7 появлялись нули
 * при n > 7 компонент падал на nodeNames[i].split(..)
 */
const liveSource = pickLiveVar(vars, ["d", "dist", "dist2"]);
const liveMat = liveMatrix(liveSource?.value);
const compilerLinked = liveK !== null || liveI !== null || liveJ !== null;

const rows = liveMat?.rows ?? baseStep.matrix.length;
const cols = liveMat?.cols ?? (baseStep.matrix[0]?.length ?? 0);
const shownMatrix: (number | null)[][] = liveMat
 ? Array.from({ length: rows }, (_, i) => Array.from({ length: cols }, (_, j) => liveMat[i][j] || baseStep.matrix[i][j]));
const demoSize = baseStep.matrix.length;
const sizeMismatch = compilerLinked && liveMat && (liveMat.length !== demoSize);

```

```

 <button onClick={() => { setIsPlaying(false);
 ; }} disabled={compilerLinked} title={compilerLinked}
 id-600 text-white rounded-lg text-sm disabled="disabled"
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 m-6" >
 { /* Граф */ }
 <div className="w-full lg:w-2/3 bg-indigo-950/20 p-6
 justify-center min-h-[400px]" >
 <div className="absolute top-3 left-3 bg-indigo-950/20
 font-bold shadow-sm" >
 {currentStep.k >= 0 && currentStep.k < 7 ? currentStep.k : 0}
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-fw-normal" markerWidth="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 <marker id="arrow-fw-active" markerWidth="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#818cf8" />
 </defs>
 {Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id === u);
 const vNode = nodesSvg.find((n) => n.id === edge.v);
 const isTarget = currentStep.i === u && currentStep.k === 0;
 const isVia = (currentStep.i === u && currentStep.k > 0);
 let strokeColor = '#475569'; let markerId = 'arrow-fw-normal';
 if (isTarget) { strokeColor = '#10b981'; markerId = 'arrow-fw-active'; }
 else if (isVia) { strokeColor = '#818cf8'; markerId = 'arrow-fw-via'; }
 return (
 <g key={` ${u} - ${edge.v} `}>
 <line x1={uNode.x} y1={uNode.y} x2=vNode.x
 y2=vNode.y stroke={strokeColor} />
 <markerEnd={markerId} strokeDasharray={isTarget ? [5, 5]} : [] />
 </g>
)
 })
 })
 </svg>
 </div>
 </div>

```

```

 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap
 isI ? 'bg-amber-900/40 border-amber-
 isK ? 'bg-indigo-900/40 border-ind
 `}>
 <span className="font-bold text-whit
e-400">→
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== c
 const isCur = currentStep.j === edge.v);
 return (
 <span key={idx} className={`px-2
dow' : isVia ? 'bg-indigo-500 text-white bo
 {edge.v} ({edge.w})

)
 })}
 </div>
);
 })}
 </div>
 </div>
 </div>

<div className="flex flex-col lg:flex-row gap-6">
 /* Матрица расстояний */
 <div className="w-full lg:w-1/2 bg-rose-950/10
 <div className="flex justify-between items-center
 <h4 className="text-lg font-bold text-rose-1
 </div>
 {sizeMismatch ? (
 <p className="mb-3 rounded-lg border border
 {sizeMismatch}

```

```
 {/* Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60" >
 <div>
 <h4 className="text-lg font-bold text-slate-100">
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5 px-4 ${
 item.line === currentLine ? 'bg-blue-900/80 text-amber-400' : 'text-slate-400'
 }`} >
 {item.line}
 {item.code}
 </div>
))}
 </div>
 </div>
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 44/111: src/components/GraphComponentsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useContext, useEffect, useLayoutEffect, useState } from 'react';
import { ChevronLeft, ChevronRight, Play, Pause, RotateCcw } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from './VizChapterContext';
```

```
/**
```

```
 * Билет 8 – «Графы. Компоненты связности».
```

```
 *
```

```
 * Две вкладки:
```

```
 * 1. «Обход» – DFS/BFS-покраска: внешний цикл по непокрашенным узлам,
 * каждый новый запуск = +1 к счётчику компонент.
```

```
 * 2. «DSU» – рёбра приходят по одному, union склеивает компоненты,
 * comps уменьшается.
```

```

const ADJ: number[][] = NODES.map(() => []);
EDGES.forEach(([u, v]) => {
 ADJ[u].push(v);
 ADJ[v].push(u);
});
ADJ.forEach((a) => a.sort((x, y) => x - y));

/** Палитра компонент. */
const COLORS = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];

interface DfsFrame {
 title: string;
 note: string;
 comp: number[]; // 0 = не покрашена, иначе нет
 cur: number | null; // текущая вершина
 frontier: number[]; // стек/очередь
 comps: number; // счётчик компонент
 edge: [number, number] | null;
}

/** Покадровая раскладка DFS-обхода: ровно тот порядок,
function buildDfsFrames(): DfsFrame[] {
 const frames: DfsFrame[] = [];
 const comp = new Array(NODES.length).fill(0);
 const push = (cur: number | null, frontier: number[],
 => {
 frames.push({ title, note, comp: [...comp], cur, frontier });
 });
 let c = 0;
 push(null, [], c, "Старт: все вершины серые", "comp = 0, все вершины дой непокрашенной.");
 for (let start = 0; start < NODES.length; start++) {
 if (comp[start] !== 0) continue;
 c += 1;
 const stack = [start];
 comp[start] = c;
 push(start, stack, c, `Новый запуск: c = ${c}, стартовое ребро: ${stack[0]} - ${stack[1]}`);
 }
 return frames;
}

```

```

frames.push({ title, note, parent: [...parent], comps
push("Старт: каждая вершина сама по себе", `parent[v]
 .`, null, false);
EDGES.forEach(([u, v], i) => {
 const ru = find(u);
 const rv = find(v);
 if (ru === rv) {
 push(`Ребро ${i}: (${u}, ${v}) – корни совпали`,
 кл, union ничего не меняет, comps остается ${com
 return;
 }
 parent[rv] = ru;
 comps -= 1;
 push(`Ребро ${i}: union(${u}, ${v})`, `find(${u}) =
 ент стало ${comps}.`, [u, v], true);
});
push(`Готово: comps = ${comps}`, `Все ${EDGES.length}
 связности.`, null, false);
return frames;
}

```

```

const DFS_FRAMES = buildDfsFrames();
const DSU_FRAMES = buildDsuFrames();

```

```

/** Корень DSU-множества с полным сжатием пути. */
const dsuRoot = (parent: number[], v: number): number => {
 let cur = v;
 const seen = new Set<number>();
 while (parent[cur] !== cur && !seen.has(cur)) {
 seen.add(cur);
 cur = parent[cur];
 }
 return cur;
};

```

```

export const GraphComponentsViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();

```



```

 if (!arr) return null;
 return arr.slice(0, NODES.length).map((v) => (typeo
}, [vars]));
/** parent / p из кода → DSU-множества по корням. */
const liveParent = useMemo(() => {
 const arr = vars ? (vizArray(vars.parent) ?? vizArr
 if (!arr || arr.length < NODES.length) return null;
 return arr.slice(0, NODES.length).map((v) => vizNum
}, [vars]);
const liveCur = vars ? (vizNumber(vars.v) ?? vizNumbe
const liveComps = vars ? (vizNumber(vars.comps) ?? vi
const liveFrontier = useMemo(() => {
 if (!vars) return null;
 const arr = vizArray(vars.stack) ?? vizArray(vars.d
 if (!arr) return null;
 return arr.map((v) => vizNumber(v)).filter((v): v i
}, [vars]);
/** Шаг из кода: i / e / step — двигает покадровую де
const liveStep = vars ? (vizNumber(vars.step) ?? vizN

useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep)
 goTo(liveStep);
}, [liveStep, goTo]);

/* — какая разметка сейчас в силе: код пользователя
/** Массив parent, который сейчас рисуем: из кода пол
const parentNow = liveParent ?? (mode === "dsu" ? fra
/** Номер множества по parent[]: корни нумеруются в п
const setOf = useCallback(
 (parent: number[], id: number): number => {
 const roots = Array.from(new Set(parent.map((_, i
 return roots.indexOf(dsuRoot(parent, id)) + 1;
 },
 []
);

const compOf = (id: number): number => {

```

```

 onClick={() => switchMode("dsu")}
 className={"flex items-center gap-1.5 px-3 py-1.5 rounded-lg bg-slate-900 text-white shadow" : "text-slate-400 hover:text-white"}
 >
 <Network className="w-3.5 h-3.5" /> DSU (переключатель)
 </button>
 </div>
 </div>

 {/* панель управления */}
 <div className="flex flex-wrap items-center gap-2" >
 <button onClick={() => { setStep(0); setPlaying(false); goToStep(0); }} >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button onClick={() => { setPlaying(false); goToStep(-1); }} >
 <ChevronLeft className="w-4 h-4" />
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1.5 rounded-lg bg-slate-900 border border-slate-700 text-slate-400 hover:text-white transition-colors"
 >
 {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
 </button>
 <button onClick={() => { setPlaying(false); goToStep(1); }} >
 <ChevronRight className="w-4 h-4" />
 </button>

 компонент: {comps}

 </div>

 {/* поле */}

```

```

 }}}
 { /* стек/очередь и DSU-таблица справа */ }
 <g>
 <text x={470} y={24} textAnchor="end" fill=
 {mode === "dfs" ? "стек DFS" : "parent[]"}
 </text>
 {mode === "dfs"
 ? frontier.slice(-8).map((v, i) => (
 <g key={i}>
 <rect x={440} y={32 + i * 20} width=
 <text x={453} y={44 + i * 20} textA
 </g>
))
 : (parentNow ?? []).slice(0, 10).map((pv,
 const color = COLORS[(setOf(parentNow
 return (
 <g key={i}>
 <rect x={432} y={32 + i * 15} wid
 <text x={436} y={41.5 + i * 15} f
 </g>
);
))}
 </g>
 </svg>
 </div>

 { /* подпись кадра */ }
 <div className="mt-3 bg-slate-900/70 border borde
 <p className="text-sm font-bold text-white mb-1
 {runtime && vars && <Terminal className="w-3.
 {title}
 </p>
 <p className="text-xs text-slate-400 leading-re
 </div>

 { /* живые переменные из компилятора */ }
 <div className="mt-3 flex flex-wrap items-center
 <span className="text-[10px] uppercase tracking

```

```

 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];

 const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
 };

 type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
 };

 function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, j

 function dfs(v: string) {

```

```

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `04
 tack: [...q], visitedNodes: [...Array.from(
 }
 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `04
 om(vis)] });

 return steps;
}

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("df
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);

 const steps = useMemo(() => mode === 'dfs' ? genera
 useVizStepSync(stepIdx, setStepIdx, steps.length -
 const v = vizString(vars.v);
 const structure = vizArray(vars.stack) ?? vizArra
 const visited = vizArray(vars.visited);
 if (v === null && !structure && !visited) return
 const candidates = steps
 .map(({step, index}) => ({ step, index }))
 .filter(({ step }) =>
 (v === null || step.node === v) &&
 (!structure || step.queueOrStack?.length ===
 (!visited || step.visitedNodes?.length === vi
));
 return candidates.at(-1)?.index ?? null;
 });
}

```



```

text-emerald-200'}
 ${idx === (mode === 'dfs'
0_10px_rgba(245,158,11,0.2)]]' : ''}
 `}>
 Узел: {item}
 {idx === (mode === 'dfs'
 <span className="ml-1
)}
 </div>
)}
 {(!step.queueOrStack || step.queue
 <div className="text-slate-5
)}
</div>
</div>
</div>

<div className="flex flex-col sm:flex-row gap-2
 {/* Controls */}
 <div className="flex bg-slate-900 border
 <button onClick={() => {setAutoPlay
="⏮️ Назад" title="⏮️ Назад" className="p-2
0 disabled:opacity-30 disabled:background-color:
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={() => setAutoPlay(
justify-center min-w-[80px] ${mode === 'dfs'
>
 {autoPlay ? <><Pause size={16}
 </button>
 <button onClick={() => {setAutoPlay
= steps.length - 1} aria-label="⏮️ Вперед"
00 rounded-lg disabled:opacity-30 disabled:background-color:
 <SkipForward strokeWidth={3} size={16}
 </button>
 <button onClick={() => {setAutoPlay
ame="p-2 text-slate-500 hover:text-rose-400
 <RefreshCw strokeWidth={3} size={16}
 </button>
 </div>
</div>

```

```

const a = arr.map(x => ({ ...x }));
let idx = a.length - 1;
while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
}
return a;
}

function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);

```



```

const liveI = vizNumber(runtime?.variables?.i);
const visualHeap: Patient[] = liveHeap
 ? liveHeap.map((value, index) => {
 const tuple = vizArray(value);
 const priority = vizNumber(tuple?.[0]) ?? vizNumber(1);
 return {
 id: `python-${index}`,
 name: tuple ? String(tuple[1] ?? value) : String(index),
 emoji: " ",
 symptom: `индекс ${index}`,
 priority,
 animState: "idle",
 };
 })
 : heap;

const [customName, setCustomName] = useState('');
const [customPriority, setCustomPriority] = useState(1);
const [log, setLog] = useState<string>('');
const [shakeEmpty, setShake] = useState(false);
const [busy, setBusy] = useState(false);
const [healingPatient, setHealing] = useState<Patient>({});

const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

// --- INSERT: Новый пациент поступает в приемный покой
const insertPatient = useCallback((name: string, priority: number) => {
 if (busy || heap.length >= 15) {
 addLog('⚠ Все палаты переполнены! Дождитесь выписки');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);

 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const finalName = name || preset.name.split(' ')[0];
 const finalEmoji = preset.emoji;
 const finalSymptom = preset.symptom;

```

```

// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);

 addLog(`СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);

 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'win' } : x));

 setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, animState: 'exit' } : x));

 setTimeout(() => {
 // Run binary heap extraction
 setHeap(prev => {
 if (prev.length === 0) return [];
 const a = prev.map(x => ({ ...x }));
 a[0] = { ...a[a.length - 1] };
 a.pop();
 return siftDown(a).map(x => ({ ...x, animState: 'siftDown' }));
 });

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animState: 'healed' } : null);
 addLog(`Успех! Пациент ${topPatient.name} выжил!`);
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 400);
 }, 400);
 }, 400);
 }, 400);
}

```

```

 });
 }
 return res;
});

return (
 <div className="flex flex-col gap-6">
 { /* МНЕМОНИКА / ПРАВИЛО */ }
 <div className="bg-emerald-950/40 border border-emerald-500">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 text-center">
 Priority Queue</h3>
 <p className="text-xs text-slate-300 mt-1 leading-relaxed">
 В реанимации не важно, кто пришел раньше, а кто в
 настоящее время (максимальный приоритет).
 Куча (Heap) автоматически перестраивает папку,
 оказавшаяся на самом вершине дерева (в корне).
 </p>
 </div>
 </div>

 { /* УПРАВЛЕНИЕ */ }
 <div className="flex flex-col lg:flex-row items-stretch gap-6">
 { /* Добавить случайного */ }
 <button
 onClick={handleRandomInsert}
 disabled={busy || heap.length >= 15}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-emerald-500 to-emerald-700
 opacity-40 disabled:cursor-not-allowed text-emerald-400 font-medium
 border border-emerald-500 rounded-lg transition-all cursor-pointer focus:outline-none"
 >
 Привезти по Скорой (INSERT)
 </button>

 { /* Добавить кастомного */ }
 <form onSubmit={handleCustomInsert} className="flex-1 min-w-[150px] bg-gradient-to-r from-emerald-500 to-emerald-700
 opacity-40 disabled:cursor-not-allowed text-emerald-400 font-medium
 border border-emerald-500 rounded-lg transition-all cursor-pointer focus:outline-none"
 >
 <input type="text" value="" placeholder="Введите текст для добавления в кучу" />
 <button type="submit" className="flex-1 min-w-[150px] bg-gradient-to-r from-emerald-500 to-emerald-700
 opacity-40 disabled:cursor-not-allowed text-emerald-400 font-medium
 border border-emerald-500 rounded-lg transition-all cursor-pointer focus:outline-none"
 >
 Добавить
 </button>
 </form>
 </div>
 </div>
);

```

```

 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 {/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
 <div className="grid grid-cols-1 lg:grid-cols-12" >

 {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
 <div className={`lg:col-span-8 bg-slate-900 rounded-2xl
 -[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`>
 <div className="absolute top-4 left-4 text-[14px] font-medium">
 Сортировка по тяжести (Двоичная куча / Max-Heap)
 </div>

 <div className="relative w-full h-full min-h-[400px]">
 <svg className="absolute inset-0 w-full h-full" >
 {edges}
 </svg>

 {visualHeap.map((patient, idx) => {
 const pos = nodePos(idx);
 const isRoot = idx === 0;
 const isCompilerNode = liveI === idx;
 return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2
 ${isRoot ? 'z-10' : ''}
 ${patient.animState === 'in' ? 'animate-in' : ''}
 `>

```

```

<div className="flex-1 flex flex-col justify-
 {healingPatient ? (
 <div className="flex flex-col items-cente
 <div className="relative">
 <span className="text-6xl block anim-l
 <span className="absolute -top-4 -right
 <span className="absolute -bottom-2 -
 </div>
 <div>
 <h4 className="text-white font-black
 Пациент: {healingPatient.name
 </h4>
 <p className="text-emerald-400 text-x
 Состояние: Стабилизация ({healingPa
 </p>
 <div className="flex justify-center g
 <span className="w-2 h-2 rounded-fu
 <Heart className="w-4 h-4 text-rose
 </div>
 </div>
 </div>
) : (
 <div className="text-center text-slate-60

 <p className="text-xs font-bold font-mo
 <p className="text-[10px] mt-1">Ждём са
 </div>
)}
</div>

<div className="w-full h-3 bg-gradient-to-r f
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-

```

```

/** Наведение на карточку ставит анимацию на паузу – мож
export const DemoPauseContext = createContext(false);
```

```
export function useLoop(steps: number, ms = 900) {
 const [step, setStep] = useState(0);
 const paused = useContext(DemoPauseContext);
 const period = Math.round(ms * SPEED);

 useEffect(() => {
 if (steps <= 1 || paused) return;
 const t = setInterval(() => setStep((s) => (s + 1) *
 return () => clearInterval(t);
 }, [steps, period, paused]);

 return step;
}
```

```
export const C = {
 idle: "#334155",
 idleStroke: "#475569",
 text: "#e2e8f0",
 dim: "#94a3b8",
 active: "#6366f1",
 done: "#10b981",
 warn: "#f59e0b",
 bad: "#f43f5e",
 edge: "#475569",
};
```

```
export const Svg: React.FC<{ children: React.ReactNode;
 const paused = useContext(DemoPauseContext);
 return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-right">
```

```

x2={b[0]}
y2={b[1]}
stroke={color}
strokeWidth={width}
strokeDasharray={dashed ? "4 3" : undefined}
style={{ transition: MOVE }}
/>
);

```

ФАЙЛ 48/111: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7
 const pos: Record<string, [number, number]> = { i: [4, 9], j: [3, 7] };
 const viaOpen = step >= 1;
 const relaxed = step >= 2;
 return (
 <Svg
 caption={
 step === 0
 ? "k ещё запрещена: путь i → j = 9"
 : step === 1
 ? "Разрешаем пересадку через k: 3 + 4 = 7"
 : "7 < 9 → D[i][j] = 7"
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad : C.done} />
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done : C.bad} />
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done : C.bad} />
 </Svg>
);
}

```

```

 <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}
 >>
 >>
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={ci} />
 })}
 </Svg>
);
};

```

/\*\* Что такое граф: вершины, рёбра, направление и вес.

```

export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 100], B: [150, 100], C: [250, 100] };
 const edges: [string, string, number][] = [
 ["A", "B", 1],
 ["B", "C", 1],
 ["C", "A", 1],
];
 return (
 <Svg>
 <caption>{
 step === 0
 ? "Вершины – объекты (города, слова, состояния)"
 : step === 1
 ? "Рёбра – связи между ними"
 : "Вес ребра – цена перехода"
 }
 >
 {edges.map(([u, v, w], i) => {
 const a = pos[u];
 const b = pos[v];
 return (
 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.active : C.inactive} />
 {step >= 2 && (
 <text>
 x={(a[0] + b[0]) / 2}
 y={(a[1] + b[1]) / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 </text>
)}
 </g>
)
 })}
)
}

```



```

 {v}
 </text>
 </g>
)))

 {step >= 1 &&
 sorted.map(({v, i}) => (
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} height={16}>
 <text x={53 + i * 52} y={77} textAnchor="middle">
 {v}
 </text>
 </g>
)))

 {step >= 2 &&
 raw.map(({v, i}) => (
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} width={boxW} height={16}>
 <text x={startX + i * (boxW + 4) + boxW / 2} y={113}>
 {sorted.indexOf(v)}
 </text>
 </g>
)))
 </Svg>
);
};

/* ----- Splay: три случая поворота -----

type Pt = [number, number];

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, string][];
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);

```

```

 />
);

 /** Zig-Zig – x и p с одной стороны, крутим сначала вер
 export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions=[["Линия g → p → x («бамбук»)", "Поворот В
 frames=[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112]
]}
 />
);

 /** Zig-Zag – x и p с разных сторон, крутим сначала ниж
 export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions=[["Зигзаг: g влево, x вправо", "Поворот НИ
 frames=[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112]
 { pos: { x: [130, 30], p: [80, 100], g: [180, 100]
]}
 />
);

```

ФАЙЛ 49/111: src/components/hints/demosGraph.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],

```

```

 <Node key={k} x={x} y={y} label={k} fill={k ===
 }}}
 </Svg>
);
};

export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"],
 ["носки", "штаны"],
 ["штаны", "боты"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, n.x]));
 return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."} `}
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed[i]} />
))}
 {nodes.map((n) => {
 const idx = placed.indexOf(n.id);
 return (
 <g key={n.id}>
 <rect x={n.x - 34} y={n.y - 12} width={68} height={24} />
 <text x={n.x} y={n.y} textAnchor="middle" dx={
 {idx >= 0 && -30}
 } dy={
 {idx >= 0 && -10}
 }>
 {n.id}
 </text>
 <text x={n.x + 30} y={n.y - 12} textAnchor="left">
 {placed[idx]}
 </text>
 </g>
);
 })}
);
 });
}
<text x={224} y={62} textAnchor="middle" fontSize={16}>
 Порядок:
</text>
<text x={224} y={74} textAnchor="middle" fontSize={16}>
 {placed.join(" → ")}
</text>

```

```

 <Node key={k} x={x} y={y} label={k} fill={cut ?
 }}}
 </Svg>
);
};

```

```

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [["A", "B"], ["A",
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные
 {edges.map(([u, v], i) => {cut && (u === "C" || v
 {Object.entries(pos).map(([k, [x, y]]) =>
 cut && k === "C" ? (
 <circle key={k} cx={x} cy={y} r={12} fill="no
) : (
 <Node key={k} x={x} y={y} label={k} fill={k =
 } />
)
)}
 </Svg>
);
};

```

```

export const MstDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);

```

ФАЙЛ 50/111: src/components/hints/demosStruct.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], [4, 8, 9, 12, 2], [4, 2, 9, 12, 8]];
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68], [130, 112], [80, 156]];
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока не окажется на вершине">
 <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b={pos[2]} />
 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b={pos[2]} />
 {heap.map((v, i) => (
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label={v} />
))}
 </Svg>
);
};
```

```
export const StackDemo: React.FC = () => {
 const states = [["A"], ["A", "B"], ["A", "B", "C"], ["A", "B", "C", "D"], ["A", "B", "C", "D", "E"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 const popping = step >= 3;
 return (
 <Svg caption={popping ? "поп: забираем ВЕРХНИЮЮ тарелку" : "загружаем тарелку"}>
 <rect x={90} y={22} width={80} height={96} rx={6} />
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={26} />
 <text x={110} y={100 - i * 26} />
 <fill={i === items.length - 1 ? (popping ? "red" : "blue") : "white"} />
 </g>
))}
 </Svg>
);
};
```

```

const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#f44336"];
const root = (i: number): number => (p[i] === i ? i : root(p[i]));
return (
 <Svg caption="find = «кто твой староста», union = 0"
 {p.map((par, i) =>
 par !== i ? {
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]}
 fill="none" stroke={rootColor[root(i)]} stroke-width={2}
 } : null
) : null
)}
 {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
 </Svg>
);
};

```

```

const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "•", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },
];

```

```

export const TrieDemo: React.FC = () => {
 const step = useLoop(TRIE_NODES.length + 1, 700);
 return (
 <Svg caption="Путь от корня по буквам = слово; общий путь = префикс"
 {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} />
 })}
 {TRIE_NODES.map((n) => <Node key={n.id} x={n.x} y={n.y} />)}
 </Svg>
);
};

```

```

export const TrieBfsDemo: React.FC = () => {

```

```

)))
 </Svg>
);
};

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
 return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых"
 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n.w}
 fill={n.lvl === step ? C.active : C.idle} stroke={C.idle} />
 <text x={n.x} y={n.y} textAnchor="middle" dom
 fill={n.lvl === step ? C.active : C.idle} />
 </g>
)))}
 </Svg>
);
};
};
```

```

export const PrefixSumDemo: React.FC = () => {
 const a = [3, 1, 4, 1, 5, 9];
 const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
 const step = useLoop(4, 1200);
 const l = 1, r = 3;
 const show = step >= 1;
 return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] - P[${l}]` :
 "о префиксные суммы P"}>

```

```

)))
 </Svg>
);
};

export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => (
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={30} height={10}
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done}
))}
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} strokeDash=[4, 4]}
 <text x={130} y={20} textAnchor="middle" fontSize={14}
 </Svg>
);
};

export const NpDemo: React.FC = () => {
 const step = useLoop(2, 1400);
 return (
 <Svg caption={step === 0 ? "Найти решение – долго (N^2)" : "Решение найдено"}
 <ellipse cx={130} cy={64} rx={112} ry={48} fill={C.done} stroke={C.done} strokeDash=[4, 4]}
 <ellipse cx={92} cy={64} rx={52} ry={32} fill="#333" stroke={C.done} strokeDash=[4, 4]}
 <text x={92} y={64} textAnchor="middle" dominantBaseline="middle"
 <text x={206} y={34} textAnchor="middle" fontSize={14}
 <circle cx={196} cy={84} r={13} fill={step === 1 ? C.done : C.wait}
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle"
 ?"></text>
 <text x={130} y={124} textAnchor="middle" fontSize={14}
 </Svg>
);
};

```



```

 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
 fill={idx === step - 1 ? C.active : fill} stroke={fill}
 {filled && {
 <text x={57 + c * 38} y={36 + r * 32} text={C[idx]}
 fontSize={10} fontWeight="700" fill="black" />
 }}
 </g>
 </>
);
 })
})
<text x={130} y={122} textAnchor="middle" fontSize={14}
 каждая клетка считается один раз и переиспользуется />
</text>
</Svg>
);
};

```

ФАЙЛ 51/111: src/components/hints/MiniDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from "react";
import { DemoPauseContext } from "../demoKit";
import { ArticulationDemo, BfsDemo, BridgeDemo, DfsDemo,
 import {
 AmortizedDemo, DsuDemo, HeapDemo, NpDemo, PrefixSumDemo,
 SparseTableDemo, StackDemo, SuffixLinkDemo, TrieBfsDemo,
 } from "../demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBfsDemo,
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "../demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"

```

```
export const MINI_DEMO_KINDS = Object.keys(REGISTRY);

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};
```

---

ФАЙЛ 52/111: src/components/hints/SimulatorModal.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useEffect } from "react";
import { createPortal } from "react-dom";
import { X } from "lucide-react";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../../data/vizStepBu
import { VizLiveStatus } from "../VizLiveStatus";

interface Props {
 vizId: string | null;
 onClose: () => void;
}

/** Модальное окно с полноценным симулятором – вызывает
```

```
 {/* Тот же id главы: развёрнутая демонстрация
 <VizChapterContext.Provider value={vizId}>
 <Component />
 </VizChapterContext.Provider>
 </div>
 </div>
 </div>,
 document.body
);
};
```

---

ФАЙЛ 53/111: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useLayoutEffect, useRef, use
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";
```

```
export interface HintAnchor {
 termId: string;
 rect: DOMRect;
}
```

```
interface Props {
 anchor: HintAnchor | null;
 onClose: () => void;
 onOpenSimulator: (vizId: string) => void;
 onCardEnter: () => void;
 onCardLeave: () => void;
}
```

```
const CARD_W = 320;
```

```

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo-500" >
 <div className="flex items-start gap-2 px-3 pt-3" >
 <h4 className="text-sm font-bold text-indigo-400" >
 {entry.term}
 </h4>
 <button
 onClick={onClose}
 className="text-slate-500 hover:text-white"
 aria-label="Заккрыть подсказку"
 >
 <X className="w-4 h-4" />
 </button>
 </div>
 <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
 <p className="text-[13px] leading-relaxed text-slate-300" >
 {entry.demo && <MiniDemo kind={entry.demo} />
 {entry.formal && (

```

```

const [step, setStep] = useState(0);
const MAX_STEP = 7;
useVizStepSync(step, setStep, MAX_STEP, (vars) => {
 const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
 return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
});

const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },
];

const isNodeVisible = (id: string) => {
 if (id === 'S' && (step === 0 || step === 7)) return true;
 return false;
};

const getPotential = (id: string) => {
 if (step >= 2 && step < 7) {
 if (id === 'S') return 0;
 if (id === 'A') return 0;
 if (id === 'B') return -2;
 if (id === 'C') return -1;
 }
 return null;
};

```

```

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
 };

 const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0})`;
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${ -2})`;
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${ -1})`;
 return e.newWeight;
 }
 return e.weight;
 };

 const isEdgeTextHighlighted = (e: any) => {
 if (e.id === 'AB' && step === 4) return true;
 if (e.id === 'BC' && step === 5) return true;
 if (e.id === 'CA' && step === 6) return true;
 return false;
 };

 const isEdgeTextEmerald = (e: any) => {
 if (e.id === 'AB' && step > 4) return true;
 if (e.id === 'BC' && step > 5) return true;
 if (e.id === 'CA' && step > 6) return true;
 };

```

```

 Старый вес:
 Вычисляем: $-2 + 0 - (-2) =$ <span cla
 </div>
);
 case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$ <span c
 </div>
);
 case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$ <span cla
 </div>
);
 case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S
 Все новые веса ребер в графе стали
 При этом старые кратчайшие пути оста
 </div>
);
 default: return "";
}
};

return (
 <div className="bg-slate-800 p-4 rounded-xl border">
 <h4 className="text-sm md:text-base font-bo">

 Визуализация: Перевзвешивание алгоритмом
 </h4>

 <div className="flex flex-col md:flex-row g

```

```

const textHigh = isEdgeTextt
const textEmer = isEdgeTextt

return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 className={`str
 markerEnd={getM
 />

 <text
 x={midX} y={midY}
 textAnchor="mid
 dominantBaseline
 className={`tex
 ${textHigh
 textEmer
 isS ? 'fi
 `}>
 {getEdgeContent
 </text>
</g>
)
}})

{/* Nodes */}
{nodes.map(n => {
 if (!isNodeVisible(n.id)) r
 const pot = getPotential(n.

 return (
 <g key={n.id} className=
 <circle
 cx={n.x} cy={n.y}
 className={`str
 ${n.id === 'S'
 ? 'fill-[#2

```



```

 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 text-white font-medium
 -slate-400 transition-colors" title="Шаг на
 <Undo strokeWidth={3} size={24} />
 </button>
 <button onClick={() => setStep(2)}
 className={`flex-1 font-bold text-white transition-colors
 ${step === MAX_STEP
 ? 'bg-emerald-600/50'
 : 'bg-indigo-600 hover:bg-indigo-500'}
 {step === MAX_STEP ? "Готово" : "Следующий шаг"}
 {step !== MAX_STEP && <Skip strokeWidth={3} size={24} />
 }
 </button>
 </div>

 { /* Progress Dots */ }
 <div className="flex justify-center gap-2" >
 {Array.from({length: MAX_STEP + 1}, (_, i) => (
 <div key={i} className={`w-10 h-10 rounded-full
 ${i < step ? 'bg-indigo-900' : 'bg-slate-400'}
 </div>
))}
 </div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 55/111: src/components/KosarajuViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...";
import {
 Play,

```

```

 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode === v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 // Generate step-by-step Kosaraju steps
 const generatedSteps: AlgoStep[] = [];
 const visited = new Set<number>();
 const order: number[] = [];
 const sccMap: Record<number, number> = {};

 const recordStep = (
 phase: AlgoStep["phase"],
 desc: string,
 currentNode: number | null,
 transposed: boolean,
 currentSccId: number,
) => {
 generatedSteps.push({
 phase,

```

```

 dfs1(to);
 }
}

order.push(u);
recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Запи
 u,
 false,
 -1,
);
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

const topoOrder = [...order].reverse();
recordStep(
 "transpose",
 `Первый проход завершен! Стек выхода в топологиче
 ебра.` ,
 null,
 true,
 -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

```

```

 sccId,
);
}
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент,
 null,
 true,
 sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,
 order: [],
 transposed: false,
 sccMap: {},
 currentSccId: -1,
 activeEdges: initialEdges,
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1500);
 } else {
 setIsPlaying(false);
 }
}, []);

```

```

 onClick={reset}
 aria-label="Сбросить обход"
 title="Сбросить обход"
 className="flex items-center justify-center"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 { /* Playfield SVG */ }
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 { /* Markers for directed arrows */ }
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#66BB6A" />
 </marker>
 <marker
 id="arrow-active"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1E8449" />
 </marker>
 </defs>
 </svg>
 </div>
</div>

```

```

 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 :

return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 markerEnd={`url(#${markerId})`}
 className="transition-all duration-300"
 />
);
}}}

{/* Nodes */}
{initialNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited =
 step.visited.has(node.id) ||
 (step.phase === "dfs1" && step.currentNode === node.id);
 const sccId = step.sccMap[node.id];

 let classes = "fill-slate-800 stroke-slate-800";
 if (sccId) {

```

```

 SCC #{sccId}
 </text>
 })
 </g>
);
 }}}
</svg>

{step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
)}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-4
 Порядок обхода и стек
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-xs text-white leading-re
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-a
 {/* Top-Sort output / Order stack represent
 <div>
 <span className="text-[10px] text-slate-5
 СТЕК ВЫХОДА (DFS 1 ORDER):

 <div className="flex flex-wrap gap-1 bg-s
 {step.order.length === 0 ? (
 <span className="text-slate-600 text-

```

```

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-800 px-2 py-1"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 px-2 py-1"
 >
 Вперед
 </button>
 </div>
 </div>
 </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 56/111: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useCallback, useContext, useEffect, useLayoutEffect, useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizStepBus } from '../data/vizStepBus';

interface Vertex {

```



```

{ id: '3-4', u: 3, v: 4, weight: 3, status: 'pending'
{ id: '1-2', u: 1, v: 2, weight: 4, status: 'pending'
{ id: '0-3', u: 0, v: 3, weight: 5, status: 'pending'
{ id: '1-4', u: 1, v: 4, weight: 6, status: 'pending'
{ id: '4-5', u: 4, v: 5, weight: 7, status: 'pending'
{ id: '3-6', u: 3, v: 6, weight: 8, status: 'pending'
{ id: '2-5', u: 2, v: 5, weight: 9, status: 'pending'
{ id: '6-7', u: 6, v: 7, weight: 10, status: 'pending'
{ id: '4-7', u: 4, v: 7, weight: 11, status: 'pending'
{ id: '7-8', u: 7, v: 8, weight: 12, status: 'pending'
{ id: '5-8', u: 5, v: 8, weight: 13, status: 'pending'
];
```

```
export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)
 const [heapQueue, setHeapQueue] = useState<string[]>(
 const [activeCheatTab, setActiveCheatTab] = useState<

 const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Предста
 овы начинать!',
 type: 'info',
 },
]));
```

```
// --- KRUSKAL STEPS ---
```

```
const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верши
```

```

 type: 'info'
 }, ...prev]);
 },
 // Step 6: Include B-C
 () => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный цв
 type: 'success'
 }, ...prev]));
 },
 // Step 7: Inspect A-D (5)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю – мы перекрашиваем всё в один цвет!"',
 type: 'warning'
 }, ...prev]));
 },
 // Step 8: Include A-D (Merge to purple)
 () => {
 setVertices(prev => prev.map(v => v.color === 'blu
 setEdges(prev => prev.map(e => e.status === 'inclu
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий клан
 type: 'success'
 }, ...prev]));
 },
 // Step 9: Inspect B-E (6) - Cycle!
 () => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',

```

```

 type: 'info'
 }, ...prev]);
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.',
 type: 'success'
 }, ...prev]);
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]);
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]);
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветн
 type: 'info'
 }, ...prev]);
},

```

```

 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: ' Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фи
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
];

// --- PRIM STEPS ---
const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5
 setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вер
 description: 'Мы выбрали вершину E как стартовую
 ритетную очередь (Heap): D-E(3), B-E(6), E-F(
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Pop D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро
 description: 'Плесень тянется по ребру D-E к вер
 type: 'info'
 }, ...prev]));
 },
 // Step 3: Include D-E, add D's edges to heap

```

```

 type: 'info'
 }, ...prev]);
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-E(6) добавлено в кучу.',
 type: 'success'
 }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]);
},
// Step 9: Include B-C, add C's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (9)']);
 setLogs(prev => [{
 title: 'Успех: Вершина C захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {

```

```
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]));
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]));
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
```

```

 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
},
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой',
 type: 'info'
 }, ...prev]));
},
// Step 23: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setHeapQueue(['F-I (13 - цикл)']));
 setLogs(prev => [{
 title: ' Успех: Плесень захватила весь граф
 description: 'Все 9 вершин поглощены. Итоговый
 иной – мы разрастались из одной точки с помо
 type: 'success'
 }, ...prev]));
},
]);

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 }
];

```

```

// Step 3: Five-Year Plan 2 (Inspecting bridge between two kolkhozes)
() => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F}
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мост
 description: 'Теперь Красный и Синий колхозы од
 -D с весом 5. Остальные мосты B-E(6) и C-F(9)
 type: 'warning'
 }, ...prev]));
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
 setVertices(prev => prev.map(v => ({ ...v, color:
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? { .
));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский
 description: 'Оба колхоза объединились по мосту
 type: 'success'
 }, ...prev]));
},
// Step 5: Check remaining edges & mark as rejected
() => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шаг
 description: 'Все лишние ребра (B-E, E-H, C-F,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }, ...prev]));
}
];

```



```

 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 setLogs(introLog(algo));
};

/* ----- связь с Python-компилятором -----
const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;

// активная вкладка = подрежим страницы: код в панели
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, activeAlgo);
}, [chapterId, activeAlgo]);

/** Перемотка на кадр n: сброс и повтор первых n шагов
const goToStep = useCallback((n: number) => {
 const target = Math.max(0, Math.min(currentSteps.length, n));
 setVertices(initialVertices);
 setEdges(initialEdges);
 setHeapQueue([]);
 setLogs(introLog(activeAlgo));
 for (let i = 0; i < target; i++) currentSteps[i]();
 setStepIndex(target);
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [activeAlgo, currentSteps]);

/** Номер кадра из переменных кода: i / step / число
const selectByVariables = useCallback<VizStepSelector>((v) => {
 const i = vizNumber(v.i) ?? vizNumber(v.step);
 if (i !== null) return i + 1;
 const mst = vizArray(v.mst) ?? vizArray(v.answer) ?? vizArray(v.steps);
 if (mst) return mst.length;
 const taken = vizArray(v.taken) ?? vizArray(v.used);
 if (taken) return Math.max(0, taken.length - 1);
 return null;
});

```

```

 }
 return null;
};

const PALETTE = ['red', 'blue', 'purple', 'mold'];
/** Цвет по корне компоненты: одинаковый корень → один
const colorOfComp = (root: string | undefined): string {
 if (!root) return 'none';
 let h = 0;
 for (let i = 0; i < root.length; i++) h = (h * 31 +
 return PALETTE[h % PALETTE.length];
};

/** Всё, что код пользователя рассказал про граф. */
const live = useМемо(() => {
 if (!vars) return null;
 const parentRec = vizRecord(vars.parent) ?? vizReco
 const parentArr = vizArray(vars.parent) ?? vizArray
 /** Метка множества вершины, как её записал код (бу
 const rawComp = new Map<number, string>();
 /** То же, но только если метка – вершина графа: по
 const parentIdx = new Map<number, number>();
 const feed = (id: number, val: unknown) => {
 const label = String(typeof val === 'string' ? va
 rawComp.set(id, label);
 const inner = typeof val === 'string' ? val : typ
 const pid = idOf(inner);
 if (pid !== null) parentIdx.set(id, pid);
 };
 if (parentRec) {
 Object.entries(parentRec).forEach(([k, val]) => {
 const id = idOf(k);
 if (id === null) return;
 feed(id, val);
 });
 } else if (parentArr) {
 parentArr.forEach((val, id) => {
 if (id > 8) return;

```

```

 const any = compOfVertex.size > 0 || takenEdges.length
 (curU !== null && curV !== null) || heapEdges.length
 return any ? { compOfVertex, takenEdges, takenVertices } : {
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [vars]);

```

```

/** Рёбра, которые рисуем: код пользователя важнее ка
const effEdges: Edge[] = useMemo(() => {
 if (!live) return edges;
 const key = (a: number, b: number) => `${Math.min(a, b)}-${Math.max(a, b)}`;
 const takenKey = new Set(live.takenEdges.map((e) => key(e.u, e.v)));
 const heapKey = new Set(live.heapEdges.map((e) => key(e.u, e.v)));
 const cheapKey = new Set(live.cheapestEdges.map((e) => key(e.u, e.v)));
 return edges.map((e) => {
 const k = key(e.u, e.v);
 const isCur = (live.curU === e.u && live.curV === e.v);
 if (isCur) return { ...e, status: 'inspecting' as const };
 if (takenKey.has(k)) return { ...e, status: 'included' as const };
 if (heapKey.has(k)) return { ...e, status: 'in_heap' as const };
 if (cheapKey.has(k)) return { ...e, status: 'inspected' as const };
 return { ...e, status: 'pending' as const };
 });
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [edges, live]);

```

```

const effVertices: Vertex[] = useMemo(() => {
 if (!live) return vertices;
 return vertices.map((v) => {
 const root = live.compOfVertex.get(v.id);
 if (root !== undefined) return { ...v, color: colorOf(root) };
 if (live.takenVertices.includes(v.id)) return { ...v, color: 'taken' };
 if (live.curU === v.id || live.curV === v.id) return { ...v, color: 'cur' };
 return { ...v, color: 'none' };
 });
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [vertices, live]);

```

```

const getColorClass = (color: string, id: number) => {

```

```


 <h3 className="text-2xl font-bold text-wh
 </div>
 <p className="text-slate-400 text-sm">
 Наблюдай за работой Краскала (Раскраска),
 </p>
</div>

{/* Algorithm Tabs */}
<div className="flex flex-wrap items-center g
 <div className="flex items-center bg-slate-
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-
 : 'text-slate-400 hover:text-slate-
)
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow
 : 'text-slate-400 hover:text-slate-
)
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button

```

```

 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex}
 </button>
</div>
</div>
</div>

{/* Что компилятор сообщил графу прямо сейчас */}
<div className="mb-6 flex flex-wrap items-center" >

 <Terminal className="w-3.5 h-3.5" /> из компилятора:

 {live ? (
 <>
 {live.curU !== null && live.curV !== null && (

 ребро {String.fromCharCode(65 + live.curU)}

) : ''}

)}
 {live.takenEdges.length > 0 && (

 в осто́ве: {live.takenEdges.map((e) => e)}

)}
 {live.compOfVertex.size > 0 && (

 компонент: {new Set(Array.from(live.compOfVertex))}

)}
 {live.takenVertices.length > 0 && (

 дерево выросло: {live.takenVertices.map((v) => v)}

)}
 {live.heapEdges.length > 0 && (

 в куче: {live.heapEdges.length}

)}
)}

```



```

 });
 }}}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" style={effVertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: vertex.x + "%",
 className={"absolute -translate-x-1/2 -translate-y-1/2"},
font-weight: 'bold', text-align: 'center', border: '2px solid black', transition: 'all 0.3s ease'}
id)}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'prim' ? (

 MST построено с помощью {activeAlgo}

) : null}
 </div>
)}}}
</div>

{stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom-6 shadow-xl z-20" style={{ background: 'white', padding: '10px', border: '1px solid black' }}>
 <p className={"font-weight: 'bold', text-align: 'center' + (activeAlgo === 'kruskal' ? ' : text-rose-300' : ' : text-slate-300')}>
 MST построено с помощью {activeAlgo}
 </p>
 <p className="text-slate-300 text-xs mt-2" style={{ text-align: 'center' }}>
 Общий вес минимального остова: 51. Всего ребер: 5.
 </p>
 </div>
)}
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl border-1px border-slate-800 p-10" style={{ background: 'black', color: 'white', padding: '10px' }}>

```

```

 : 'bg-slate-900/60 border-slate-700'
 }
 >
 <div className="flex items-center gap-2">
 {log.type === 'success' && <CheckCircle className="text-emerald-500" />}
 {log.type === 'warning' && <HelpCircle className="text-amber-500" />}
 {log.type === 'error' && <XCircle className="text-red-500" />}
 {log.type === 'info' && <Play className="text-blue-500" />}
 <h5 className="font-bold text-sm text-slate-300">{log.message}</h5>
 </div>
 <p className="text-xs text-slate-300">{log.timestamp}</p>
 </div>
)))
</div>
</div>
</div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-slate-700 p-2">
 <div className="flex flex-col sm:flex-row items-center gap-2">
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-400" />
 <h4 className="text-xl font-bold text-white">Cheat Sheet</h4>
 </div>
 <div className="flex bg-slate-950 p-1 rounded">
 <button
 onClick={() => setActiveCheatTab('kruskal')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 (activeCheatTab === 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300")
 }
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={
 "px-3 py-1.5 rounded text-xs font-medium" +
 (activeCheatTab === 'prim' ? "bg-emerald-600 text-white" : "bg-slate-800 text-slate-300")
 }
 >
 Прима (Билет 19)
 </button>
 </div>
 </div>
</div>

```



```

<div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
 <Code className="w-3.5 h-3.5 text-ind
 </p>
 <pre className="text-xs text-emerald-400
 80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]

def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False

2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
 if union(u, v):
 mst.append((u, v, w))`}
 </pre>
 </div>
</div>
</div>
))

{activeCheatTab === 'prim' && (
 <div className="grid grid-cols-1 lg:grid-cols
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
 <Clock className="w-3.5 h-3.5 text-em
 </p>

```

```

 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}
```

```

 </pre>
</div>
</div>
</div>
}}
```

```

{activeCheatTab === 'boruvka' && {
 <div className="grid grid-cols-1 lg:grid-cols-2">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-rose-500"/>
 </p>
 <p className="text-2xl font-black text-white">Время
 <p className="text-slate-400 text-xs mt-2">
 p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-rose-500"/>
 </p>
 <p className="text-2xl font-black text-white">Награда
 <p className="text-slate-400 text-xs mt-2">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 ении:</p>
 <p className="text-xs text-slate-300 leading-relaxed">
 , организуя массовое распараллеленное слияние
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-rose-500"/>

```

## Универсальное пособие • полный исходный код

```

import { PAGE_SYNC } from "../data/vizSync";
import { linkifyTicketsHtml } from "../utils/ticketLink";
import { quizzes } from "../data/quizzes";

/**
 * «Лёгкая» текстовая версия пособия.
 *
 * Открывается по адресу `?lite=1` (весь список тем) или
 * (одна тема). Здесь НЕТ навигации, сайдбара, интерактивных
 * панели компилятора – только сам текст тем, чтобы:
 * 1) любую страницу можно было распечатать/сохранить
 * 2) внешний парсер (ридер, «версия для слабовидящих»,
 * LLM-индексатор) получал чистый текст без служебных
 *
 * Все спойлеры <details> раскрыты, визуализация описания
 * показывает, какие у неё вкладки и какие переменные кода
 */

const LIGHT_CSS = `
:root { color-scheme: light; }
body { background: #ffffff !important; color: #111827; }
.lite-wrap { max-width: 46rem; margin: 0 auto; padding: 1.6; }
.lite-wrap h1 { font-size: 1.75rem; font-weight: 700; }
.lite-wrap h2 { font-size: 1.35rem; font-weight: 700; color: #111827; }
.lite-wrap h3 { font-size: 1.1rem; font-weight: 700; }
.lite-wrap h4 { font-size: 1rem; font-weight: 700; margin: 0; }
.lite-wrap p, .lite-wrap li { color: #1f2937; font-size: 1.1rem; }
.lite-wrap a { color: #1d4ed8; }
.lite-meta { color: #4b5563; font-size: .85rem; margin: 0; }
.lite-topics { font-size: .8rem; color: #374151; margin: 0; font-family: sans-serif; }
.lite-viz { margin: 1.25rem 0; padding: .8rem 1rem; border: 1px solid #ccc; }
.lite-viz h4 { margin: 0 0 .35rem; font-size: .95rem; }
.lite-viz ul { margin: .35rem 0 0 1.1rem; padding: 0; }
.lite-viz code, .lite-wrap code { font-family: ui-monospace; padding: 0 .25rem; }

```

```

const list = useMemo(() => {
 if (!topicId) return chapters;
 const one = chapters.filter((c) => c.id === topicId);
 return one.length ? one : chapters;
}, [topicId]);

return (
 <div className="lite-wrap">
 <style dangerouslySetInnerHTML={{ __html: LIGHT_CSS }} />

 <h1>Универсальное пособие: алгоритмы и структуры</h1>
 <p className="lite-meta">
 Билеты 1–24 • текстовая версия (без навигации, без картинок)
 {topicId ? `одна тема: ${list[0]?.title} ?? ${list[0]?.description}` : ''}
 полная интерактивная версия
 </p>

 {!topicId && (
 <nav className="lite-toc">
 <h3>Содержание</h3>

 {list.map((c) => (
 <li key={c.id}>

 {c.title} — билет {c.chapterTopics[0].length}

))}

 </nav>
)}

 {list.map((chapter) => {
 const viz = getViz(chapter.id);
 const topics = chapterTopics[chapter.id] ?? [];
 const syncKeys = Object.keys(PAGE_SYNC).filter((key) => {
 return (
 <article key={chapter.id} id={chapter.id}>

```

```

 });
 }
 }
 <p className="lite-meta" style={{ margin: 0; padding: 0; }}>
 В интерактивной версии эта демонстрация переменных перерисовывают её напрямую
 </p>
 </div>
)}

{quizzes[chapter.id] ?? []}.length > 0 &&
 <div className="lite-quiz">
 <h3>Блиц: вопросы по теме</h3>

 {quizzes[chapter.id] ?? []}.map((q, i) => (
 <li key={i}>
 <p>
 {q.question}
 </p>

 {q.options.map((option, j) => (
 <li key={j}>
 {j === q.correctIndex ? Правильно : Неправильно}

))}

 <p className="lite-meta">{q.explanation}

))}

 </div>
)}
</article>
);
}}}
</div>
);
};

```

```
/**
 * Таблица «ваших» расстояний, когда имена вершин в коде
 *
 * Схема демонстрации нарисована на фиксированном наборе
 * S...F у Форда–Беллмана). Если в коде пользователя верши
 * (0...6, v1...v9), картинка физически не может их показат
 * просто пропадали, и казалось, что визуализация живёт
 * выводятся рядом, а подсказка объясняет, как связать
 */
export function LiveDistTable({ dist, prev, done, demoIds }) {
 if (!dist) return null;

 const keys = Object.keys(dist);
 if (keys.length === 0) return null;

 const demoSet = new Set(demoIds);
 const unknown = keys.filter((key) => !demoSet.has(key));
 // Все вершины совпали со схемой – таблица не нужна,
 if (unknown.length === 0) return null;

 const doneSet = new Set((done ?? []).map((value) => S + value));
 const shown = keys.slice(0, 64);

 return (
 <div className="mb-6 rounded-xl border border-amber-200 p-4">
 <p className="text-[11px] leading-relaxed text-amber-700">
 {title} В вашем коде вершины{" "}

 {unknown.slice(0, 8).join(", ")}
 {unknown.length > 8 ? " ..." : ""}

 , а на схеме демо-графа нарисованы{" "}
 {demoIds.join(", ")}
 показаны отдельной таблицей. Чтобы они управлялись
 (кнопка-«глаз» в панели Python подставляет его
 </p>
 <div className="mt-2.5 flex flex-wrap gap-1.5">
```

```

import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
 {
 img: dijkstraImg,
 alt: 'Фриказоид (Декстер Дуглас / Дейкстра)',
 title: '✂ Фриказоид (Декстер Дуглас)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'неотрицательными',
 highlightColor: 'text-emerald-400',
 rest: 'весами ($w \geq 0$). Подсунь отрицательное ребро',
 complexity: ' $O((V+E) \log V)$ ',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый — зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы за $V-1$ и',
 complexity: ' $O(V \cdot E)$ ',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
 alt: 'Все-ко-Все Флойд',
 title: ' Все ко Все (Флойд)',
 desc: 'Матрица + три цикла (',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
```

```
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы – показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedCh
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : ""
 return () => {
 document.body.style.overflow = "";
 };
 }, [open]);

 useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escap
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, []);

 return (
 <>
 <div className="lg:hidden sticky top-[57px] z-40"
 <button
 onClick={() => setOpen(true)}
 className="w-full flex items-center gap-2.5 p-
 aria-label="Открыть содержание"
```



```
 </>
);
};
```

---

ФАЙЛ 61/111: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { Sparkles, FileDown, FileText, Code2, Loader2, }
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
 /** Открыта ли боковая панель компилятора. */
 compilerOpen: boolean;
 onToggleCompiler: () => void;
}
```

```
/**
 * Шапка приложения.
 *
 * Кнопка «Скачать PDF» всегда на самом видном месте во
 */
```

```
export const Navbar: React.FC<NavbarProps> = ({ compilerOpen,
 const [busy, setBusy] = useState(false);
```

```
 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };
};
```

```
>
 <Terminal className="w-4 h-4" /> Python
</button>

<div id="download-menu" className="flex items
 <a
 id="download-guide-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/
 download="guide_lite.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex-1 sm:flex-none flex items
 bg-indigo-600 hover:bg-indigo-500 text-white
 n-200 whitespace-nowrap"
 title="Скачать PDF пособия: компактный код"
 >
 <FileDown className="w-4 h-4 text-indigo-600" />

<details className="relative shrink-0">
 <summary
 className="flex items-center justify-center
 bg-slate-700/60 border border-slate-600 cu
 title="Ещё форматы: PDF с кодом, TXT и .py"
 >
 Ещё
 </summary>
 <div className="absolute right-0 top-full
 5 shadow-xl shadow-black/50 min-w-[170px]">
 <a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}exp
 download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center gap-2 px-2
 rald-600/20 border border-emerald-500/30 tr
 title="Полный PDF-дамп исходного кода"
```

```
import React, { useCallback, useContext, useEffect, useLayoutEffect } from "react";
import { ChevronLeft, ChevronRight, Play, Pause, RotateCcw } from "react-icons/bs";
import { PlanarityDemo } from "../PlanarityDemo";
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from "../VizChapterContext";
```

```
/**
 * Билет 7 – «Графы. Планарные. Покраска».
 *
 * Четыре вкладки:
 * 1. «Планарность» – K5 и K3,3 с подсвеченными пересечениями.
 * 2. «Жадная покраска» – $\chi \leq \Delta + 1$: вершины красятся по порядку, каждой
 * достаётся наименьший цвет, не занятый соседями;
 * 3. «Двудольность» – проверка 2-раскрашиваемости BFS. Чётный цикл = двумя
 * цветами, нечётный цикл = двумя красками не обойтись;
 * 4. «Покраска рёбер» – $\chi' \leq \Delta + 1$ (Визинг), а для двудольных $\chi' = \Delta$
 * (теорема Кёнига).
 *
 * Два графа на выбор: «основной» (8 вершин, $\Delta = 3$, есть цикл К5),
 * и «двудольный» (6 вершин, $\Delta = 3$, $\chi = 2$, $\chi' = 3$).
 *
 * Синхронизация с Python-компилятором (PAGE_SYNC["planarity"]):
 * color/col – массив цветов вершин (красит кружки на панели);
 * esolor/es – массив цветов рёбер, v – текущая вершина;
 * u – сосед, chi/k – число цветов, delta/D – максимальная степень;
 * i/step – номер кадра.
 */
```

```
type Tab = "planar" | "greedy" | "bipartite" | "edges";
type GraphKey = "main" | "bip";
```

```
interface Graph {
 key: GraphKey;
 label: string;
 n: number;
 edges: [number, number][];
```

```

}

interface Frame {
 title: string;
 note: string;
 color: number[]; // 0 = не покрашена
 ecolor?: number[]; // 0 = не покрашено
 v: number | null;
 u: number | null;
 e: number | null;
 used?: number[];
 bad?: [number, number] | null;
}

/** Жадная покраска вершин: $\chi \leq \Delta + 1$. */
function greedyFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const color = new Array(g.n).fill(0);
 const delta = Math.max(...g.adj.map((a) => a.length));
 frames.push({
 title: "Старт: ни одна вершина не покрашена",
 note: `color = [0]·n. Порядок обхода – просто по номерам. Максимально уложится в  $\Delta + 1 = \{\text{delta} + 1\}$  цветов: у вершины v соседей не более  $\Delta$ .`,
 color: [...color], v: null, u: null, e: null,
 });
 for (let v = 0; v < g.n; v++) {
 const used = Array.from(new Set(g.adj[v].filter((x) => color[x] !== 0)));
 let c = 1;
 while (used.includes(c)) c += 1;
 color[v] = c;
 const deg = g.adj[v].length;
 frames.push({
 title: `Вершина ${g.names[v]}: берём цвет ${c}`,
 note: used.length
 ? `Соседи уже заняли цвета ${used.join(", ")}. У вершины ${g.names[v]} ещё нет покрашенных соседей. Всегда хватит ( $\chi \leq \Delta + 1$ ).`
 : `У вершины ${g.names[v]} ещё нет покрашенных соседей.`,
 color: [...color], v, u: null, e: null, used,
 });
 }
}

```

```

 color[u] = 3 - color[v];
 queue.push(u);
 frames.push({
 title: `${g.names[v]} → ${g.names[u]}: цвет
 note: `Сосед не покрашен: color[${g.names[u]
 x}).join(", ") || "пуста"}]`,
 color: [...color], v, u, e: null,
 });
 } else if (color[u] === color[v]) {
 conflict = [v, u];
 frames.push({
 title: `Конфликт: ${g.names[v]} и ${g.names
 note: `Рёбро (${g.names[v]}, ${g.names[u]})
 покрасить: $\chi \geq 3$.`,
 color: [...color], v, u, e: null, bad: conf
 });
 }
}
}
}
frames.push({
 title: conflict ? "Граф НЕ двудольный" : "Граф двуд
 note: conflict
 ? `BFS упёрся в нечётный цикл (${g.names[conflict
 краски — их и даёт жадный алгоритм на вкладке «
 : "Все вершины разбиты на две доли, рёбер внутри
 праведлива теорема Кёнига: $\chi' = \Delta$ (рёберная пок
 color: [...color], v: null, u: null, e: null, bad:
 });
 return frames;
}

```

```

/** Жадная покраска рёбер: $\Delta \leq \chi' \leq \Delta + 1$ (Визинг), для
function edgeFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const ecolr = new Array(g.edges.length).fill(0);
 const inc: Map<number, number>[] = Array.from({ length
 const delta = Math.max(...g.adj.map((a) => a.length))

```

```

 { id: "planar", label: "1 · Планарность: K5 и K3,3",
 { id: "greedy", label: "2 · Покраска вершин ($\chi \leq \Delta+1$)",
 { id: "bipartite", label: "3 · Двудольность (2 краски)",
 { id: "edges", label: "4 · Покраска рёбер (Визинг, Кёниг)",
];

```

```

export const PlanarColoringViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const [tab, setTab] = useState<Tab>("greedy");
 const [graphKey, setGraphKey] = useState<GraphKey>("m");
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);

```

```

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

```

```

 const graph = useMemo(() => makeGraph(graphKey), [graphKey]);
 const frames = useMemo(() => {
 if (tab === "greedy") return greedyFrames(graph);
 if (tab === "bipartite") return bipartiteFrames(graph);
 if (tab === "edges") return edgeFrames(graph);
 return [] as Frame[];
 }, [tab, graph]);
 const maxStep = Math.max(0, frames.length - 1);

```

```

 useEffect(() => {
 setStep(0);
 setPlaying(false);
 }, [tab, graphKey]);

```

```

 useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {

```

---

```
const usedEcolors = Array.from(new Set(graph.edges.map
```

```
const go = useCallback((d: number) => {
 setPlaying(false);
 setStep((s) => Math.max(0, Math.min(maxStep, s + d)
}, [maxStep]));
```

```
if (tab === "planar") {
 return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-
 <Header />
 <Tabs tab={tab} setTab={setTab} />
 <PlanarityDemo />
 <p className="text-[11px] text-slate-500 mt-2">
 Формула Эйлера $V - E + F = 2$ и оценка $E \leq 3V$
 </p>
 </div>
);
}
```

```
return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-
 <Header />
 <Tabs tab={tab} setTab={setTab} />

 <div className="flex flex-wrap items-center gap-2
 <span className="text-[10px] uppercase tracking
 {[["main", "bip"] as GraphKey[]].map((k) => (
 <button
 key={k}
 onClick={() => setGraphKey(k)}
 className={"px-3 py-1.5 rounded-lg text-[11px]
 "bg-slate-900/60 text-slate-400 hover:text-
 >
 {makeGraph(k).label}
 </button>
)))}
 </div>
```

```

 strokeDasharray={ec === 0 && !isBad
 className="transition-all duration-
 opacity={isCur || isBad ? 1 : 0.85}
 />
 {tab === "edges" && ec > 0 && (
 <text x={{graph.pos[a].x + graph.po
 ={colorOf(ec)} fontSize="10" fontWeight="bo
)}
 </g>
);
 }}}
 {graph.pos.map((p, id) => {
 const c = nodeColor(id);
 const isCur = id === curV;
 const isNbr = id === curU;
 return (
 <g key={id}>
 <circle cx={p.x} cy={p.y} r={isCur ?
 "#38bdf8" : c > 0 ? colorOf(c) : "#475569"}
 <text x={p.x} y={p.y + 4} textAnchor=
 ames[id]}</text>
 {c > 0 && tab !== "edges" && <text x=
 old">UBER {c}</text>}
 {tab === "edges" && <text x={p.x} y={
 }</text>}
 </g>
);
 }}}
</svg>
</div>

<div className="lg:col-span-2 space-y-3">
 <div className="bg-slate-900/70 border border
 <p className="text-sm font-bold text-white
 <p className="text-xs text-slate-400 leadin
 </div>
 <div className="bg-slate-950 border border-sl
 <p className="text-[10px] uppercase trackin

```



```


)}
 </div>
 </div>
);
};

const Header: React.FC = () => {
 <div className="flex flex-wrap items-center justify-b
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-fuchsia-600 text-white r
 <Palette className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold tex
 <p className="text-xs text-fuchsia-300">K5/K3,3
 </div>
 </div>
 </div>
);

const Tabs: React.FC<{ tab: Tab; setTab: (t: Tab) => vo
 <div className="flex flex-wrap gap-2 mb-4">
 {TABS.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={"flex items-center gap-1.5 px-3 py-2
 -600 text-white" : "bg-slate-900/60 text-slat
 >
 {t.icon} {t.label}
 </button>
 })}
 </div>
);

export default PlanarColoringViz;

```

```

 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
 stroke={xing ? "#f43f5e" : "#4f46e5"} />);
 }
 const labels = ["A","B","C","D","E"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20 z-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border">

```

```

 const labels = ["\u04141", "\u04142", "\u04143"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#ccc">
 <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 }
 }
</svg>
<div className="absolute bottom-2 left-2 right-2 top-2 z-20">
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

<div className="bg-amber-950/40 border border-amber-500 p-10">
 ▲ Запомни: K5 и K3,3 – два кита непланарности.
 гомеоморфный K5 или K3,3 – он непланарен. Теорема Кёрши.
</div>
</div>
);
}

```

ФАЙЛ 64/111: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import {
 ArrowRightLeft,
 Check,
 CheckCircle2,
 ChevronDown,
 ChevronLeft,
 ChevronRight,
 ChevronUp,
 Circle,
 CircleX,
 CircleX2,
 CircleX4,
 CircleX8,
 CircleX16,
 CircleX32,
 CircleX64,
 CircleX128,
 CircleX256,
 CircleX512,
 CircleX1024,
 CircleX2048,
 CircleX4096,
 CircleX8192,
 CircleX16384,
 CircleX32768,
 CircleX65536,
 CircleX131072,
 CircleX262144,
 CircleX524288,
 CircleX1048576,
 CircleX2097152,
 CircleX4194304,
 CircleX8388608,
 CircleX16777216,
 CircleX33554432,
 CircleX67108864,
 CircleX134217728,
 CircleX268435456,
 CircleX536870912,
 CircleX1073741824,
 CircleX2147483648,
 CircleX4294967296,
 CircleX8589934592,
 CircleX17179869184,
 CircleX34359738368,
 CircleX68719476736,
 CircleX137438953472,
 CircleX274877906944,
 CircleX549755813888,
 CircleX1099511627776,
 CircleX2199023255552,
 CircleX4398046511104,
 CircleX8796093022208,
 CircleX17592186044416,
 CircleX35184372088832,
 CircleX70368744177664,
 CircleX140737488355328,
 CircleX281474976710656,
 CircleX562949953421312,
 CircleX1125899906842624,
 CircleX2251799813685248,
 CircleX4503599627370496,
 CircleX9007199254740992,
 CircleX18014398509481984,
 CircleX36028797018963968,
 CircleX72057594037927936,
 CircleX144115188075855872,
 CircleX288230376151711744,
 CircleX576460752303423488,
 CircleX1152921504606846976,
 CircleX2305843009213693952,
 CircleX4611686018427387904,
 CircleX9223372036854775808,
 CircleX18446744073709551616,
 CircleX36893488147419103232,
 CircleX73786976294838206464,
 CircleX147573952589676412928,
 CircleX295147905179352825856,
 CircleX590295810358705651712,
 CircleX1180591620717411303424,
 CircleX2361183241434822606848,
 CircleX4722366482869645213696,
 CircleX9444732965739290427392,
 CircleX18889465931478580854784,
 CircleX37778931862957161709568,
 CircleX75557863725914323419136,
 CircleX151115727451828646838272,
 CircleX302231454903657293676544,
 CircleX604462909807314587353088,
 CircleX1208925819614629174706176,
 CircleX2417851639229258349412352,
 CircleX4835703278458516698824704,
 CircleX9671406556917033397649408,
 CircleX19342813113834066795298816,
 CircleX38685626227668133590597632,
 CircleX77371252455336267181195264,
 CircleX154742504910672534362390528,
 CircleX309485009821345068724781056,
 CircleX618970019642690137449562112,
 CircleX1237940039285380274899124224,
 CircleX2475880078570760549798248448,
 CircleX4951760157141521099596496896,
 CircleX9903520314283042199192993792,
 CircleX19807040628566084398385987584,
 CircleX39614081257132168796771975168,
 CircleX79228162514264337593543950336,
 CircleX158456325028528675187087900672,
 CircleX316912650057057350374175801344,
 CircleX633825300114114700748351602688,
 CircleX1267650600228229401496703205376,
 CircleX2535301200456458802993406410752,
 CircleX5070602400912917605986812821504,
 CircleX10141204801825835211973625643008,
 CircleX20282409603651670423947251286016,
 CircleX40564819207303340847894502572032,
 CircleX81129638414606681695789005144064,
 CircleX162259276829213363391578010288128,
 CircleX324518553658426726783156020576256,
 CircleX649037107316853453566312041152512,
 CircleX1298074214633706907132624082305024,
 CircleX2596148429267413814265248164610048,
 CircleX5192296858534827628530496329220096,
 CircleX10384593717069655257060992658440192,
 CircleX20769187434139310514121985316880384,
 CircleX41538374868278621028243970633760768,
 CircleX83076749736557242056487941267521536,
 CircleX166153499473114484112975882535043072,
 CircleX332306998946228968225951765070086144,
 CircleX664613997892457936451903530140172288,
 CircleX1329227995784915872903807060280344576,
 CircleX2658455991569831745807614120560689152,
 CircleX5316911983139663491615228241121378304,
 CircleX10633823966279326983230456482242756608,
 CircleX21267647932558653966460912964485513216,
 CircleX42535295865117307932921825928971026432,
 CircleX85070591730234615865843651857942052864,
 CircleX170141183460469231731687303715884105728,
 CircleX340282366920938463463374607431768211456,
 CircleX680564733841876926926749214863536422912,
 CircleX1361129467683753853853498429727072845824,
 CircleX2722258935367507707706996859454145691648,
 CircleX5444517870735015415413993718908291383296,
 CircleX10889035741470030830827987437816582766592,
 CircleX21778071482940061661655974875633165533184,
 CircleX43556142965880123323311949751266331066368,
 CircleX87112285931760246646623899502532662132736,
 CircleX174224571863520493293247799005065324265472,
 CircleX348449143727040986586495598010130648530944,
 CircleX696898287454081973172991196020261297061888,
 CircleX1393796574908163946345982392040522594123776,
 CircleX2787593149816327892691964784081045188247552,
 CircleX5575186299632655785383929568162090376495104,
 CircleX11150372599265311570767859136324180752990208,
 CircleX22300745198530623141535718272648361505980416,
 CircleX44601490397061246283071436545296723011960832,
 CircleX89202980794122492566142873090593446023921664,
 CircleX178405961588244985132285746181186892047843328,
 CircleX356811923176489970264571492362373784095686656,
 CircleX713623846352979940529142984724747568191373312,
 CircleX1427247692705959881058285969449495136382746624,
 CircleX2854495385411919762116571938898990272765493248,
 CircleX5708990770823839524233143877797980545530986496,
 CircleX11417981541647679048466287755595961091061972992,
 CircleX22835963083295358096932575511191922182123945984,
 CircleX45671926166590716193865151022383844364247891968,
 CircleX91343852333181432387730302044767688728495783936,
 CircleX182687704666362864775460604089535377456991567872,
 CircleX365375409332725729550921208179070754913983135744,
 CircleX730750818665451459101842416358141509827966271488,
 CircleX1461501637330902918203684832716283019655932542976,
 CircleX2923003274661805836407369665432566039311865085952,
 CircleX5846006549323611672814739330865132078623730171904,
 CircleX11692013098647223345629478661730264157247460343808,
 CircleX23384026197294446691258957323460528314494920687616,
 CircleX46768052394588893382517914646921056628989841375232,
 CircleX93536104789177786765035829293842113257979682750464,
 CircleX187072209578355573530071658587684226515959365500928,
 CircleX374144419156711147060143317175368453031918731001856,
 CircleX748288838313422294120286634350736906063837462003712,
 CircleX1496577676626844588240573268701473812127674924007424,
 CircleX2993155353253689176481146537402947624255349848014848,
 CircleX5986310706507378352962293074805895248510699696029696,
 CircleX11972621413014756705924586149611790497021399392059392,
 CircleX23945242826029513411849172299223580994042798784118784,
 CircleX47890485652059026823698344598447161988085597568237568,
 CircleX95780971304118053647396689196894323976171195136475136,
 CircleX191561942608236107294793378393788647952342390272950272,
 CircleX383123885216472214589586756787577295904684780545900544,
 CircleX766247770432944429179173513575154591809369561091801088,
 CircleX1532495540865888858358347027150309183618739122183602176,
 CircleX3064991081731777716716694054300618367237478244367204352,
 CircleX6129982163463555433433388108601236734474956488734408704,
 CircleX12259964326927110866866776217202473468949912977468817408,
 CircleX24519928653854221733733552434404946937899825954937634816,
 CircleX49039857307708443467467104868809893875799651909875269632,
 CircleX98079714615416886934934209737619787751599303819750539264,
 CircleX196159429230833773869868419475239575503198607639501078528,
 CircleX392318858461667547739736838950479151006397215279002157056,
 CircleX784637716923335095479473677900958302012794430558004314112,
 CircleX1569275433846670190958947355801916604025588861116008628224,
 CircleX3138550867693340381917894711603833208051177722232017256448,
 CircleX6277101735386680763835789423207666416102355444464034512896,
 CircleX12554203470773361527671578846415332832204710888928069025792,
 CircleX25108406941546723055343157692830665664409421777856138051584,
 CircleX50216813883093446110686315385661331328818843555712276103168,
 CircleX100433627766186892221372630771322662657637687111424552206336,
 CircleX200867255532373784442745261542645325315275374222849104412672,
 CircleX401734511064747568885490523085290650630550748445698208825344,
 CircleX803469022129495137770981046170581301261101496891396417650688,
 CircleX1606938044258990275541962092341162602522202993782792835301376,
 CircleX3213876088517980551083924184682325205044405987565585670602752,
 CircleX6427752177035961102167848369364650410088811975131171341205504,
 CircleX12855504354071922204335696738729300820177623950262342682411008,
 CircleX25711008708143844408671393477458601640355247900524685364822016,
 CircleX51422017416287688817342786954917203280710495801049370729644032,
 CircleX102844034832575377634685573909834406561420991602098741459288064,
 CircleX205688069665150755269371147819668813122841983204197482918576128,
 CircleX411376139330301510538742295639337626245683966408394965837152256,
 CircleX822752278660603021077484591278675252491367932816789931674304512,
 CircleX1645504557321206042154969182557350504982735865633579863348609024,
 CircleX3291009114642412084309938365114701009965471731267159726697218048,
 CircleX6582018229284824168619876730229402019930943462534319453394436096,
 CircleX13164036458569648337239753460458804039861886925068638906788872192,
 CircleX26328072917139296674479506920917608079723773850137277813577744384,
 CircleX52656145834278593348959013841835216159447547700274555627155488768,
 CircleX105312291668557186697918027683670432318895095400549111254310977536,
 CircleX210624583337114373395836055367340864637790190801098222508621955072,
 CircleX421249166674228746791672110734681729275580381602196445017243910144,
 CircleX842498333348457493583344221469363458551160763204392890034487820288,
 CircleX1684996666696914987166688442938726917102321526408785780068975640576,
 CircleX3369993333393829974333376885877453834204643052817571560137951281152,
 CircleX6739986666787659948666753771754907668409286105635143120275902562304,
 CircleX13479973333575319897333507543509815336818572211270286240551805124608,
 CircleX26959946667150639794667015087019630673637144422540572481103610249216,
 CircleX53919893334301279589334030174039261347274288845081144962207220498432,
 CircleX107839786668602559178668060348078522694548577690162289924414440996864,
 CircleX215679573337205118357336120696157045389097155380324579848828881993728,
 CircleX431359146674410236714672241392314090778194310760649159697657763987456,
 CircleX862718293348820473429344482784628181556388621521298319395315527974912,
 CircleX1725436586697640946858688965569256363112777243042596638790631055949824,
 CircleX3450873173395281893717377931138512726225554486085193277581262111899648,
 CircleX6901746346790563787434755862277025452451108972170386555162524223799296,
 CircleX13803492693581127574869511724554050904902217944340773110325048447598592,
 CircleX27606985387162255149739023449108101809804435888681546220650096895197184,
 CircleX55213970774324510299478046898216203619608871777363092441300193790394368,
 CircleX110427941548649020598956093796432407239217743554726184882600387580788736,
 CircleX220855883097298041197912187592864814478435487109452369765200775161577472,
 CircleX441711766194596082395824375185729628956870974218904739530401550323154944,
 CircleX883423532389192164791648750371459257913741948437809479060803100646309888,
 CircleX1766847064778384329583297500742918515827483896875618958121606201292619776,
 CircleX3533694129556768659166595001485837031654967793751237916243212402585239552,
 CircleX7067388259113537318333190002971674063309935587502475832486424805170479104,
 CircleX14134776518227074636666380005943348126619871175004951664972849610340958208,
 CircleX28269553036454149273332760011886696253239742350009903329945699220681916416,
 CircleX56539106072908298546665520023773392506479484700019806659891398441363832832,
 CircleX113078212145816597093331040047546785012958969400039613319782796882727665664,
 CircleX226156424291633194186662080095093570025917938800079226639565593765455331328,
 CircleX452312848583266388373324160190187140051835877600158453279131187530910662656,
 CircleX904625697166532776746648320380374280103671755200316906558262375061821325312,
 CircleX1809251394333065553493296640760748560207343510400633813116524750123642650624,
 CircleX3618502788666131106986593281521497120414687020801267626233049500247285301248,
 CircleX7237005577332262213973186563042994240829374041602535252466099000494570602496,
 CircleX14474011154664524427946373126085988481658748083205070504932198000989141204992,
 CircleX28948022309329048855892746252171976963317496166410141009864396001978282409984,
 CircleX57896044618658097711785492504343953926634992332820282019728792003956564819968,
 CircleX115792089237316195423570985008687907853269984665640564039457584007913129639936,
 CircleX231584178474632390847141970017375815706539969331281128078915168015826259279872,
 CircleX463168356949264781694283940034751631413079938662562256157830336031652518559744,
 CircleX926336713898529563388567880069503262826159877325124512315660672063305037119488,
 CircleX1852673427797059126777135760139006525652319754650249024631321344126610074238976,
 CircleX3705346855594118253554271520278013051304639509300498049262642688253220148477952,
 CircleX741069371118823650710
```

```
interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Размер панели (px) управляется родителем и сохраняется */
 width?: number;
 height?: number | null;
 onWidthChange?: (w: number) => void;
 onHeightChange?: (h: number) => void;
 /** Перейти к странице пособия (посмотреть визуализацию) */
 /** Кнопка «открыть страницу в пособии» – не нужна, если есть */
 onOpenGuide?: () => void;
 /** Скрыть панель. */
 onClose: () => void;
}

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

/**
 * Подмена интерпретатора для проверок: DOM-харнес (jsdom)
 * из node_modules) и CI-скриншоты кладут сюда готовый
 * на CDN. В браузере у пользователя поле не определено
 */
type PythonRuntimeProvider = PyodideRuntime | (() => Promise<PyodideRuntime>);

declare global {
```

```
/* -----
/* Шаблон синхронизации: короткая шапка + минимальный
/* скелет, шаг-в-шаг с подсветкой визуализации (см. da
/* -----
```

```
/* -----
/* Шпаргалка Python: команды сортировки, циклы, структ
/* «Вставить» кладёт сниппет в редактор под курсор.
/* -----
```

```
interface Snippet {
 label: string;
 tip: string;
 code: string;
}
```

```
const SNIPPETS: { group: string; items: Snippet[] }[] =
 [
 {
 group: "Сортировка",
 items: [
 {
 label: "arr.sort() / sorted()",
 tip: "In-place сортировка по возрастанию и нова
 code: 'arr = [5, 2, 8, 1]\narr.sort()
) # копия по убыванию\n',
 },
 {
 label: "sorted(key=...)",
 tip: "Сортировка по ключу: рёбра по весу, пары
 code: 'edges = [(3, "A", "B"), (1, "B", "C"), (3
 es)\n',
 },
],
 },
 {
 group: "Циклы",
 items: [

```

```

 label: "Матрица n × m",
 tip: "Таблица расстояний: dist[i][j], бесконечны",
 code: 'n, m = 3, 4\ndist = [[float("inf")] * m for i in range(n)]',
 },
 {
 label: "Множество visited",
 tip: "Посещённые вершины: in/add за O(1).",
 code: 'visited = set()\nvisited.add("A")\nprint("A" in visited)',
 },
],
},
{
 group: "Вывод",
 items: [
 {
 label: "f-строка",
 tip: "Подставить значения переменных шага в строку",
 code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}] = {dist[i][j]}")',
 },
],
},
];

```

/\*\*

\* Переживает скрывание панели: код пользователя не теряет

\*

\* Хранилище — ПО СТРАНИЦАМ. Раньше слот был один, и код

\* восстанавливался на новой странице: редактор показывал

\* и визуализация жили значениями чужого кода.

\*/

const savedEditors = new Map<string, { code: string; baseId: string }>();

export function PythonCompiler({ chapterId, chapterTitle, chapterTitleProps }) {

/\*\* Активная вкладка демонстрации страницы (у sparse-массива)

const [demoId, setDemoId] = useState<string | null>(null);

const sync = useMemo(() => getPageSync(chapterId, demoId), [chapterId, demoId]);

const syncVarBases = useMemo(() => syncVarBases(chapterId, demoId), [chapterId, demoId]);

```

const editorRef = useRef<HTMLTextAreaElement>(null);
/**
 * Панель жива? Заккрытие (крестик, «глаз» в шапке, см.
 * компонент, но `runPythonAsync` к тому моменту может
 * этого флага доехавший до конца прогон публиковал с
 * после закрытия: панель исчезла, а визуализация про
 * кода, которого на экране больше нет.
 */
const aliveRef = useRef(true);
useEffect(() => {
 aliveRef.current = true;
 return () => {
 aliveRef.current = false;
 };
}, []);

// Код хранится по страницам, поэтому при открытии па
// бывает: рассинхрон возможен только после смены вкл
const [desyncedFrom, setDesyncedFrom] = useState<string>('');
const prevTplRef = useRef({ base: initTemplate, full: fullTemplate });

/** Активная трасса. input нужен, чтобы изменение std
const [debug, setDebug] = useState<{ result: DebugResult }>({ result: 'none' });
/** Esc оставляет неизменённый текст в редакторе, пока
const [editHold, setEditHold] = useState<string | null>(null);
/** Автопроход включается пользователем; первый шаг по
const [playing, setPlaying] = useState(false);
const [playbackSpeed, setPlaybackSpeed] = useState<number>(1);
 if (typeof window === "undefined") return 1;
 const saved = Number(window.localStorage.getItem(PLAYBACK_SPEED_KEY));
 return PLAYBACK_SPEEDS.includes(saved as (typeof PLAYBACK_SPEEDS)[0]) ? saved : 1;
});
const listRef = useRef<HTMLDivElement>(null);

useEffect(() => {
 window.localStorage.setItem(PLAYBACK_SPEED_KEY, String(playbackSpeed));
}, [playbackSpeed]);

```

```

 }, [initTemplate, template]));

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать – бу
 }
}, [code]);

/* — Пошаговый отладчик —————

/** Связь больше не зависит от эталонных строк: совпа
const hasViz = sync.variables.length > 0;

/**
 * Снимок ПОСЛЕ выделенной строки. Контейнеры берём и
 * (там уже виден append/st[i][j]=...), но индексы цикл
 * события: после st[i][j]= Python уже мог увеличить
 */
const snapshotAt = useCallback((result: DebugResult,
 const step = result.steps[idx];
 const next = result.steps[idx + 1];
 const afterLocals = next?.locals ?? result.finalLoca
 const afterValues = next?.values ?? result.finalVal
 const beforeValues = step?.values ?? {};
 const line = step?.line ? source.split("\n")[step.l
 // Нас интересуют только простые скалярные цели (`i
 // `i +=`). Контейнеры и так всегда берутся из afte
 // внутри print(f"i={i}") не будет ошибочно принят
 const assignment = line.match(
 /^\\s*([A-Za-z_][A-Za-z0-9_]*(?:\\s*,\\s*[A-Za-z_][A
);
 const assigned = new Set(assignment?.[1].split(",")

 const variables = { ...afterValues };

```



```

const goDebug = useCallback(
 (nextIdx: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

const stepBy = useCallback(
 (delta: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

const stepTo = useCallback((idx: number) => goDebug(i

const stopDebug = useCallback(() => {
 setPlaying(false);
 setEditHold(code);
 setDebug(null);
 clearVizState(chapterId);
}, [chapterId, code]);

// Базовый интервал 650 мс; множитель скорости хранитс
useEffect(() => {
 if (!playing || !debug) return;
 if (debug.idx >= debug.result.steps.length - 1) {
 setPlaying(false);
 return;
 }

```

```

setRuntimeReady(true);
runtime.setStdout({ batched: (message) => stdout.print(message) });
runtime.setStderr({ batched: (message) => stderr.print(message) });
runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });

const raw = await runtime.runPythonAsync(buildDebugPanel);
// Панель могли закрыть, пока Python считал: выходим
if (!aliveRef.current) return;
const result = JSON.parse(String(raw)) as DebugResult;
const printed = clipOutput([...stdout, ...stderr]);

if (result.steps.length === 0) {
 // Шагов нет (только комментарии, синтаксическая ошибка)
 // заменяется пустым листингом, а визуализация не работает
 // показывает собственный встроенный сценарий (сценарий по умолчанию)
 setDebug({ result, idx: 0, src: source, input, output: printed });
 clearVizState(chapterId);
 setOutputOpen(true);
 setOutput(
 result.error
 ? `Ошибка:\n${result.error}`
 : printed || "В коде нет исполняемых строк"
);
 setRuntimeMessage(
 result.error
 ? "Код не выполнен — редактор остался открытым"
 : "Нет шагов трассы: связь с визуализацией потеряна"
);
} else {
 setDebug({ result, idx: 0, src: source, input, output: printed });
 setPlaying(autoPlay && result.steps.length > 1);
 // Программа что-то напечатала — значит, вывод работает
 // результат своего же print() панель не имеет возможности показать
 if (printed.trim() !== "") setOutputOpen(true);
 setOutput(
 (printed || "Программа ничего не вывела — трассировка не работает")
 ? (result.error ? `Ошибка (выполнение прервано)` : "Программа завершилась")
 : ""
);
}

```

```

 e.preventDefault();
 setPlaying(false);
 stepBy(1);
 }
 if (e.key === "ArrowLeft") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(-1);
 }
 if (e.key === " ") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying((v) => !v);
 }
 if (e.key === "Escape") stopDebug();
};
// capture-фаза: чтобы глобальный обработчик навига
window.addEventListener("keydown", onKey, true);
return () => window.removeEventListener("keydown",
}, [debug, stepBy, stopDebug]);

// держим текущую строку листинга в видимой области
/**
 * Панель показывает листинг трассы, только если шаги
 * Пустой код или синтаксическая ошибка больше не пря
 * раньше на их месте возникал пустой листинг без воз
 * «исчезала» и кнопка, и само поле ввода.
 */
const showTrace = !!debug && debug.result.steps.length
/** Трасса построена по старому тексту: пользователь
const stale = !!debug && debug.src !== code;
/** Листинг трассы можно свернуть, но редактор от это
const [traceOpen, setTraceOpen] = useState(true);
const curDebug = debug ? { step: debug.result.steps[d
const curDebugLine = curDebug?.step?.line;
useEffect(() => {
 if (curDebugLine === undefined || !listRef.current)

```





```

 }
 >
 <button
 type="button"
 onClick={() => {
 const next = !refOn;
 setRefOn(next);
 // «Убрать эталон» — значит убрать код
 // выполняется, поэтому демонстрация бо
 // значений и честно показывает встроенн
 // странице скажет об этом явно).
 setCode(next ? template : "");
 setDesyncedFrom(null);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 clearVizState(chapterId);
 setTraceRequest((n) => n + 1);
 }}
 className={`p-1.5 rounded-lg transition-c
 refOn
 ? "bg-indigo-500/30 text-white ring-1
 : "text-indigo-300 hover:text-white h
 `}
 aria-label={refOn ? "Убрать эталонный код
ктор"}
 aria-pressed={refOn}
 >
 {refOn ? <Eye className="w-4 h-4" /> : <E
 </button>
</Tooltip>
<label
 className="ml-1 inline-flex items-center gap
 der border-slate-700 hover:border-emerald-5
 title="Скорость автоматического проигрывани
 >
 {running ? <Loader2 className="w-3.5 h-3.5
ld-400" />}

```

```

 side="bottom"
 content={
 hasViz
 ? "Связь идёт по именам ниже: реально
а не сравнивается."
 : "На выбранной странице нет интерактив
 }
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex item
 hasViz ? "text-emerald-400" : "text-slate-400"
 }`
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет визуализации"}

 </Tooltip>
 </div>
</div>

{stripTab === "vars" ? (
 <div className="px-3 pb-2.5 space-y-2 max-h-300">
 <div className="flex items-center gap-2 min-w-300">
 {onOpenGuide ? (
 <Tooltip side="bottom" content="Открыть руководство">
 <button
 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center justify-center gap-2
e-700 text-slate-300 hover:text-white hover:underline">
 {chapterName}
 </button>
 </Tooltip>
) : (
 <span className="inline-flex items-center justify-center gap-2
text-slate-300 max-w-full">

```

```

 tabIndex={0}
 className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emerald-300
 >
 {v.name}
 </code>
 </Tooltip>
)))
 {sync.stepNote && (
 <Tooltip side="bottom" content={`War
 <span
 tabIndex={0}
 className="cursor-help inline-flex
text-[10px] text-slate-400 hover:text-white
 >
 <ArrowRightLeft className="w-3 h-3
 <span className="truncate max-w-5

 </Tooltip>
)}
</div>
)}
</div>
) : (
 <div className="px-3 pb-2.5 max-h-36 overflow
 {SNIPPETS.map(({gItem}) => (
 <div key={gItem.group}>
 <div className="text-[9px] font-bold up
 <div className="flex flex-wrap gap-1.5"
 {gItem.items.map(({s}) => (
 <Tooltip key={s.label} side="bottom
 <button
 type="button"
 onClick={() => insertSnippet(s.
 className="inline-flex items-center
- [10.5px] text-indigo-300 hover:text-white
 title=""
 >

```



```

 onChange={(event) => {
 setEditHold(null);
 setRefOn(false);
 setCode(event.target.value);
 }}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
 event.preventDefault();
 void debugRun();
 }
 event.stopPropagation();
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="flex-[2] min-h-[110px] w-full resize-none focus:ring-2 focus:ring-inset focus:ring-blue-500"
 placeholder="Напишите Python-код... (пустой редактор)"
 />

 {showTrace && debug && (
 <div className={`flex flex-col border-t border-l border-r border-b`}
 /* панель управления шагами */
 <div className="shrink-0 flex flex-wrap items-center justify-between"
 <Tooltip content={playing ? `Пауза автопроигрывателя` : `В начало трассы`}
 <button
 type="button"
 onClick={runOrTogglePlayback}
 className={`mx-0.5 inline-flex items-center justify-center rounded-md border border-transparent text-sm font-medium hover:bg-amber-500/25`} : "text-emerald-500"
 aria-label={playing ? `Пауза автопроигрывателя` : `В начало трассы`}
 >
 {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}

 </button>
 </Tooltip>
 <Tooltip content="В начало трассы">

```

```

 <Tooltip content={`Выполнение остановлено`}>

 лимит

 </Tooltip>
)}
 <Tooltip content={hasViz ? "Визуализация : st... Эталонный текст не обязателен – пишите нет имён, связанных с демонстрацией; Python" : ""}>
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center border-emerald-500/40 bg-emerald-500/10 text-emerald-500`}
 >
 {hasViz ? <Link2 className="w-3 h-3" href="#">
 {hasViz ? "имена → демо" : "свободный"}

 </Tooltip>
 <Tooltip content={traceOpen ? "Свернуть листинг и переменные текущего шага."}>
 <button
 type="button"
 aria-label={traceOpen ? "Свернуть листинг" : "Показать листинг"}
 aria-expanded={traceOpen}
 onClick={() => setTraceOpen((v) => !v)}
 className="ml-auto inline-flex shrink-0 rounded px-1.5 py-0.5 text-emerald-500 font-bold text-slate-300 transition-colors"
 >
 {traceOpen ? <ChevronDown className="w-3 h-3" /> : <ChevronUp className="w-3 h-3" />}
 листинг
 </button>
 </Tooltip>
 <Tooltip content="Остановить прогон (Esc) на показывает встроенное демо. Автозапуск не работает.">
 <button
 type="button"
 aria-label="Остановить прогон трассы"

```

```

 { /* панель переменных – «Variables», как в
 <div className="shrink-0 max-h-[34%] overflow-hidden" >
 <div className="flex items-center gap-1.5" >
 <Variable className="w-3 h-3 text-indigo-500" />
 Переменные на этом шаге

 </div>
 {debug.result.error && (
 <pre className="mt-1 whitespace-pre-wrap" />
)}
 {debugOrdered.length === 0 && !debug.result.error && (
 <div className="mt-1 text-[11px] text-slate-500" />
)}
 </div>
)}
 <div className="mt-1 flex flex-wrap gap-1" >
 {debugOrdered.map(([name, value]) => {
 const changed = debugChanged[name];
 const synced = syncVarSet.has(name);
 return (
 <span
 key={name}
 className={`inline-flex items-baseline gap-1`}
 changed={changed}
 synced={synced}
 >
 {value}
 {changed ? "border-amber-500/50 bg-amber-500/10" :
 synced ? "border-emerald-500/40 bg-emerald-500/10" :
 "border-slate-700 bg-slate-500/10"}

 {name}

)
 })}
 </div>
);
}}}

```

```


 <Terminal className="w-3.5 h-3.5 text-emerald-500">
 {outputOpen}

 {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500">
 {outputOpen}
 }

 </button>
 {outputOpen && (
 <pre
 data-testid="compiler-output"
 className="flex-1 min-h-[70px] overflow-auto text-slate-500 font-mono"
 style={{}}
 >
 {output}
 </pre>
)}
 <div className="px-3 py-1.5 border-t border-slate-200">
 </div>
 </aside>
);
}

```

ФАЙЛ 65/111: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray, vizString } from '../viz-runtime';

```

```

interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
}

```

```

 emoji: " ",
 color: "from-emerald-500/20 to-emerald-600/20 b",
 bubbleText: `q[${index}] из Python`,
 animState: "idle",
)))
: queue;

```

```

const [servedHistory, setServedHistory] = useState<st
const [isServing, setIsServing] = useState(fa
const [shakeEmpty, setShake] = useState(fa
const [log, setLog] = useState<st

```

```

const addLog = (msg: string) => setLog(prev => [msg,

```

```

// ENQUEUE: Встать в конец очереди (хвост / Tail)

```

```

const enqueue = useCallback(() => {
 if (isServing) return;
 if (queue.length >= 6) {
 addLog('⚠ Хвост очереди уже на улице, повару тяже
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
}

```

```

const preset = PRESETS[counter % PRESETS.length];
counter++;

```

```

const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
};

```

```

setQueue(prev => [...prev, newGuest]);
addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj

```

```

setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.

```

```

 { id: 'r1', name: 'Студент Вася', emoji: '🍴', color: 'red', text: 'Я

 т голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: '👨‍💻', color: 'blue', text: 'Я

 апишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'green', text: 'Я

 без сметаны, плиз.', animState: 'in' },
]));
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'out' })), 500);
 });

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-indigo-500 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-tight">
 В очереди за супом всё честно. Кто первый встал, тот съел.

 Новые гости встают строго в конец очереди.

 Здесь нет обхода и блата — только строгий порядок.
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-indigo-500 to-indigo-600

 text-white font-medium rounded-md opacity-40 disabled:cursor-not-allowed transition-all

 hover:scale-95 transition-all cursor-pointer">
 Добавить в очередь
 </button>
 </div>
 </div>
);
}

```

```

<div className="flex flex-col items-center" style={{width: 1000px; height: 1000px; background-color: #f0f0f0; border: 1px solid #ccc; padding: 10px; position: relative; margin: 0 auto;}}
 <div className="relative" style={{width: 100%; height: 100%; position: relative;}}
 <div className="absolute top-10 left-10" style={{width: 100px; height: 100px; background-color: #333; color: white; display: flex; align-items: center; justify-content: center; font-size: 24px; font-weight: bold; line-height: 1;}}
 <div className="w-1.5 h-6 bg-slate-100" style={{width: 100%; height: 100%; background-color: #e0e0e0; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 <div className="w-2 h-4 bg-slate-400" style={{width: 100%; height: 100%; background-color: #808080; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 </div>
 <span className="text-5xl block animate-pulse" style={{width: 100%; height: 100%; background-color: #fff; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 </div>
 <div className="text-center" style={{width: 100%; height: 100%; background-color: #fff; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 <span className="text-3xl block" style={{width: 100%; height: 100%; background-color: #fff; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 <span className="text-[10px] font-bold font-mono tracking-wider" style={{width: 100%; height: 100%; background-color: #fff; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 ПОВАР

 </div>
 </div>
 </div>
 </div>
 </div>

```

{/\* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА \*/}

```

<div className="flex flex-col items-center" style={{width: 1000px; height: 1000px; background-color: #f0f0f0; border: 1px solid #ccc; padding: 10px; position: relative; margin: 0 auto;}}
 <span className="text-2xl animate-pulse" style={{width: 100%; height: 100%; background-color: #fff; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 <span className="text-[8px] font-mono" style={{width: 100%; height: 100%; background-color: #fff; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 </div>
 </div>

```

{/\* СПИСОК ОЧЕРЕДИ \*/}

```

<div className="flex-1 flex items-end gap-4" style={{width: 1000px; height: 1000px; background-color: #f0f0f0; border: 1px solid #ccc; padding: 10px; position: relative; margin: 0 auto;}}
 {displayQueue.length === 0 ? (
 <div className="flex-1 flex flex-col justify-end" style={{width: 100%; height: 100%; background-color: #fff; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 <span className="text-4xl mb-1" style={{width: 100%; height: 100%; background-color: #fff; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 <p className="text-xs font-bold font-mono" style={{width: 100%; height: 100%; background-color: #fff; border: 1px solid #ccc; position: relative; margin: 0 auto;}}
 <p className="text-[10px]">Все сыты
 </p>
 </div>
) : (
 displayQueue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === displayQueue.length - 1;
 return (
 <div
 key={customer.id}

```

```

 </div>
);
 })
 })
</div>

</div>
</div>

{ /* Пол кухни */ }
<div className="w-full h-3 bg-gradient-to-r f
</div>

{ /* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */ }
<div className="md:col-span-3 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Сытые гости (Архив FIFO)
 </div>

 <div className="absolute top-4 right-4 flex i
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pu
 СЫТЫ
 </div>

 <div className="flex-1 flex flex-col justify-
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col item

 <p className="text-xs font-bold font-mo
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-1.5 w-fr
 {servedHistory.map((item, idx) => (
 <div
 key={idx}
 className="w-full py-1.5 px-3 round

```



```

import { Terminal } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime,
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
 y?: number;
}
```

```
export const SalmonAutomatonWidget: React.FC = () => {
 const [words, setWords] = useState<string[]>(['CAR',
 const [newWord, setNewWord] = useState('');
 const [nodes, setNodes] = useState<{ [key: number]: T
 const [simulationStep, setSimulationStep] = useState<
 const [bfsQueue, setBfsQueue] = useState<number[]>([
 const [currentNode, setCurrentNode] = useState<number
 const [speechText, setSpeechText] = useState<string>('
 'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
 буль!')
 });
 const [isTalking, setIsTalking] = useState<boolean>(fa
 const [isBlinking, setIsBlinking] = useState<boolean>

 /* — связь с Python-компилятором (PAGE_SYNC["aho-cor
 words/patterns – набор образцов: бор ПЕРЕСТРАИВАЕТС
 state – текущая вершина автомата, step – шаг BFS,
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
```

```

 if (liveQueue) {
 const q = liveQueue.map((x) => (typeof x === "number" ? x : undefined));
 if (q.length) setBfsQueue(q);
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [vars]));

const compilerLinked = Boolean(liveWordsSig || liveState);

// Регулярное моргание рыбы
useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
}, []);

// Эффект разговора при смене текста
useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false), 1000);
 return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {} }
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 newNodes[curr].children[c] = {
 id: count, char: c, parent: curr, children: {}
 };
 count++;
 }
 curr = newNodes[curr].children[c].id;
 }
 });
};

```

```

 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * 2 : rightX;
 if (leftX === -1) currentX++;
 }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД)
 S, чтобы расставить fail-ссылки!`
);
};

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове S`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
};

```

```

 return;
}

const r = bfsQueue[0];
const newQueue = bfsQueue.slice(1);
const updatedNodes = { ...nodes };
const rNode = updatedNodes[r];
const childrenIds = Object.values(rNode.children);

let logs = `Рассматриваем вершину #${r} (${rNode.char})`;

for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
 }

 logs += `Для #${u} ('${char}') fail = #${failTarget}`;
}

if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переходим к следующему';
}

setNodes(updatedNodes);
setBfsQueue(newQueue);
setCurrentNode(r);
setSpeechText(logs);

```

```

 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_10px]_black' : ''
 }`}
 >
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45" fill="#e91e63" />
 <path d="M 40 85 Q 100 55 160 85 Q 100 115 40 85" fill="#e91e63" />

 {/* Хвост */}
 <path
 d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
 fill="#be123c"
 className="origin-[35px_100px] animate-pulse"
 />

 {/* Плавники */}
 <path d="M 90 55 L 70 30 L 110 55 Z" fill="#f87171" />
 <path d="M 90 145 L 70 170 L 110 145 Z" fill="#f87171" />
 <path
 d="M 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate-pulse"
 />

 {/* Глаз (с морганием) */}
 <circle cx="145" cy="85" r="12" fill="#fff" />
 {isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85" stroke="black" />
) : (
 <circle cx="148" cy="85" r="6" fill="#fff" />
 <circle cx="151" cy="82" r="2" fill="#fff" />
)}

 {/* Рот (анимация разговора) */}
 {isTalking ? (
 <path d="M 165 105 Q 155 120 165 125 Q 175 105 165 100" fill="#fff" />
) : (
 <path d="M 165 105 Q 155 120 165 125 Q 175 105 165 100" fill="#e91e63" />
)}

```

```

 оен!'}
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-4 py-2 shadow-lg shadow-emerald-900/40 transition-colors"
 >
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-500 text-white px-4 py-2 shadow-lg shadow-blue-900/40 transition-colors"
 >
 Шаг BFS

 {currentStep}

 </button>
)}
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-500 text-white px-4 py-2 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузить
 </button>
)}
 </div>
 </div>
</div>
</div>

{/* SVG Canvas for Automaton Tree */}

```

```

 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x) / 2 - 10}
 y={({parent.y + node.y) / 2}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
</g>
);
 }}}

 {/* Draw Fail Links */}
 {simulationStep > 0 && Object.values(nodes).length > 0} {
 if (node.id === 0 || node.fail === 0) return;
 // n draw all! But maybe wait, we can just draw the first one
 // Wait, all nodes get fail link. Let's draw them all!
 if (step >= 1 and computed it)
 // To avoid too many lines, let's draw only the first one
 const target = nodes[node.fail];
 if (!target || !node.x || !node.y || !target.x || !target.y) return;

 // Simple bezier curve for fail link
 const isCurrent = currentNode === node.id;

 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y - 10} Q ${node.x + 10} ${node.y - 16} `}

```

```

 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && {
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
 }}
 </g>
);
 }}}
</svg>
);
}}({})
</div>

```

```

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4" style={style}>
 <div className="bg-slate-900/50 p-5 rounded-xl" style={style}>
 <div>
 <h5 className="text-white font-bold mb-3" style={style}>
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">

```



```

 { /* Таблица Вершин и суффиксных ссылок */ }
 <div className="lg:col-span-2 bg-slate-900/50 p-4">
 <h5 className="text-white font-bold mb-3 flex items-center">

 Таблица переходов и fail-ссылок

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto border border-slate-700">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-slate-700">
 <th className="py-2 px-3">ID Вершины</th>
 <th className="py-2 px-3">Символ</th>
 <th className="py-2 px-3">Родитель</th>
 <th className="py-2 px-3">fail-ссылка</th>
 <th className="py-2 px-3">term_link</th>
 <th className="py-2 px-3">Дети</th>
 <th className="py-2 px-3">Слова (dict)</th>
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800">
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node.id
 const isInQueue = bfsQueue.includes(node.id)

 return (
 <tr>
 <td>{node.id}</td>
 <td>{node.symbol}</td>
 <td>{node.parent}</td>
 <td>{node.fail}</td>
 <td>{node.term_link}</td>
 <td>{node.children}</td>
 <td>{node.words}</td>
 </tr>
)
 })}
 </tbody>
 </table>
 </div>
 </div>
)
}

```

```

) : {
 <span className="bg-rose-950
0 transition-colors cursor-help" title={`Ск
 #{node.term_link}

 }}
 </td>
 <td className="py-2.5 px-3 text-s
 {Object.keys(node.children).len
 ? '0'
 : Object.entries(node.children
 .map([c, id]) => `${c}→#
 .join(', '))}
 </td>
 <td className="py-2.5 px-3">
 {node.words.length === 0 ? {
 <span className="text-slate-6
) : {
 <span className="bg-emerald-9
 {node.words.join(', ')}

 }}
 </td>
</tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>

```

```

{/* что сейчас сообщил компилятор */}
<div className="mt-4 flex flex-wrap items-center
 <span className="text-[10px] uppercase tracking
 <Terminal className="w-3.5 h-3.5" /> из компи

 {compilerLinked ? {

```

## Универсальное пособие • полный исходный код

```

 treeState: Record<number, number>; // node index
 highlightNodes: number[]; // current
 mergeChildren?: [number, number]; // pair be
 arrayHighlight?: [number, number]; // [L, R]
 result?: number; // partial
}
```

// ===== PURE LOGIC: build tree =====

```
function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}
```

// ===== STEP GENERATORS =====

// 1. Build steps (recursive top-down)

```
function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
```

```
function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 2, treeState: tree, highlightNodes: [v]
 });
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 4, treeState: tree, highlightNodes: [v]
 });
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]`,
 codeLine: 6, treeState: tree, highlightNodes: [v]
 });
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 const res = left + right;
 steps.push({ id: id++, desc: `Возврат в узел ${v}:`,
 codeLine: 8, treeState: tree, highlightNodes: [v],
 rangeChildren: [2 * v, 2 * v + 1], arrayHighlight: [2 * v, 2 * v + 1]
 });
 return res;
}

const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}]`,
 codeLine: 10, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: ans
});
return steps;
}
```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```
function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 // Build iterative tree: leaves at [n..2n-1]
 const tree: Record<number, number> = {};
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

 const steps: Step[] = [];
 let id = 0;
```

```
function buildVisualTree(treeState: Record<number, number>): string {
 if (l === r) return { v, l, r, val: treeState[v] ?? null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m),
 right: buildVisualTree(treeState, m + 1, r) };
}
```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
];
```

```
const QUERY_TD_CODE = [
 "query(v, l, r, ql, qr) {",
 " if (ql > r || qr < l) return 0;",
 " // -----",
 " if (ql <= l && r <= qr) return tree[v];",
 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
];
```

```
const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " }"
];
```

```

const { step, idx, setIdx, playing, setPlaying, speed
const n = arr.length;

const vTree = useMemo(() => buildVisualTree(step.tree

const renderNode = useCallback((nd: VNode): React.Rea
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const has = nd.val !== null;

 let cls = 'bg-slate-800 border-slate-700 text-slate
 if (isHigh) cls = `${clr.ring} text-white ring-2 sc
 else if (isChild) cls = `${clr.childRing} text-ambe
 else if (has) cls = `${clr.filled} text-slate-200`;

 return (
 <div key={nd.v} className="flex flex-col items-ce
 <div className={`flex flex-col items-center jus
 ${cls}`}>
 <span className="text-[7px] opacity-60 font-m
 [{nd.l
 <span className="text-xs font-extrabold font-r
 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full l
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
 </div>
)}
 </div>
);
}, [step, clr]);

return (
 <div className={`rounded-2xl border overflow-hidden

```

```


 </div>
);
 }}}
</pre>

```

```

 { /* Description + result */
 <div className="px-3 py-2.5 bg-slate-900/60 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-900/60" >

 {step.desc}
 {step.result !== undefined && (

 {step.result} : color === 'emerald' ? 'text-emerald-300' : 'text-slate-400'

)}
 </div>

```

```

 { /* Controls */
 <div className="px-3 py-2.5 bg-slate-950 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-950" >
 <div className="flex items-center bg-slate-900/60 border-t-1 border-l-1 border-r-1 border-b-1 border-slate-900/60" >
 <button onClick={() => { setPlaying(false); setSpeed(1200); }} title="Оборот" ><RotateCcw className="w-3.5 h-3.5" /></button>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }} title="Шаг назад" className="p-1.5 text-slate-400" >Шаг назад</button>
 <button onClick={() => setPlaying(!playing)} title={playing ? 'Пауза' : 'Воспроизведение'} >{playing ? <Pause className="w-3 h-3" /> : <Play className="w-3 h-3" />}</button>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }} title="Шаг вперед" className="p-1.5 text-slate-400" >Шаг вперед</button>
 <button onClick={() => { setPlaying(false); setSpeed(1200); }} title="Вперед" ><ChevronRight className="w-3.5 h-3.5" /></button>
 </div>
 <div className="flex items-center" >
 <select value={speed} onChange={e => setSpeed(Number.parseInt(e.target.value))} >
 <option value={1200}>Медл.</option>
 <option value={600}>Сред.</option>
 <option value={300}>Быстр.</option>
 </select>
 </div>
 </div>

```

```

const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.floor(Math.random() * 10) + 1);
 setArr(a);
 setCustomInput(a.join(', '));
};

const buildSteps = useMemo(() => genBuildSteps(arr), [arr]);
const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr), [arr]);
const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr), [arr]);

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border border-indigo-500" >
 {/* ---- TOP BAR: array editor + query range ---- */}
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 {/* Array input */}
 <form onSubmit={handleApply} className="flex flex-col">
 <div className="flex items-center justify-between">
 <label className="text-[10px] font-bold uppercase">Array</label>
 <button type="button" onClick={randomize}>Refresh</button>
 </div>
 <div className="flex gap-2">
 <input
 type="text"
 value={customInput}
 onChange={e => setCustomInput(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-600 text-white px-4 py-2">Применить</button>
 </div>
 </form>
 </div>
 </div>
 </div>
);

```



```

 </div>
 </div>
 <div className="text-[10px] text-slate-500" style={style}>
 Запрос: $\text{sum}(A[\{qL\}..\{qR\}]) = \{\text{arr.slice}(qL, qR)\}$
 </div>
 </div>
 </div>
</div>

```

```

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1" style={style}>
 <button onClick={() => setView('build')} className="border border-indigo-500 text-slate-200" style={style}>
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} className="border border-emerald-500 text-slate-200" style={style}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} className="border border-blue-500 text-slate-200" style={style}>
 <Info className="w-3.5 h-3.5" /> Почему / Спра
 </button>
</div>

```

```

{/* ---- TAB CONTENT ---- */}
{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=""
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 >

```

```

<div className="bg-slate-900 p-4 rounded-lg" style={{width: 100%; height: 100%;}}
 <div className="text-slate-400">Корень: L
 <div className="text-slate-300">mid = l(0
 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правый:
 h - l) >> l)} эл.)</div>
 <div className="text-slate-500 mt-2">Всего
 } внутренних.</div>
</div>
</div>

```

```

{/* Comparison cards */}
<div className="grid grid-cols-1 xl:grid-cols-2" style={{width: 100%; height: 100%;}}
 <div className="bg-slate-950 p-5 rounded-xl" style={{width: 100%; height: 100%;}}
 <div className="absolute -top-3 left-4 bg-white border border-emerald-500 shadow-md" style={{width: 100%; height: 100%;}}
 Запрос Сверху-вниз (Рекурсия)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Начинаем с корня. Если отрезок узла полн
 наче спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between border-bottom border-slate-700" style={{width: 100%; height: 100%;}}
 0{
 <div className="flex justify-between border-bottom border-slate-700" style={{width: 100%; height: 100%;}}
 0{
 <div className="flex justify-between" style={{width: 100%; height: 100%;}}
 ✓ легко</div>
 </div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl" style={{width: 100%; height: 100%;}}
 <div className="absolute -top-3 left-4 bg-white border border-amber-500 shadow-md" style={{width: 100%; height: 100%;}}
 Запрос Снизу-вверх (Итерация)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Стартуем с двух указателей (l и r) на л
 </p>
 </div>
 </div>
 </div>

```

```
interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели */
 onNavigate?: () => void;
 /** Узкий режим: номер билета и название в одну строку */
 compact?: boolean;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, compact }) => {
 const [query, setQuery] = useState("");

 /**
 * Группы – это разделы пособия (SECTIONS), а не случаи.
 * связанные темы стоят рядом, порядок разделов не важен.
 * страница встретила первую.
 */
 const groups = useMemo(() => {
 const q = query.trim().toLowerCase();
 const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
 return SECTIONS.map((section) => ({
 section,
 items: filtered.filter((c) => sectionOf(c.id).id === section.id),
 })).filter((g) => g.items.length > 0);
 }, [query]);

 return (
 <aside className="w-full bg-slate-900/80 lg:bg-transparent" >
 <div className={compact ? "p-2 pb-1.5 shrink-0" : "p-2 pb-1.5"} >
 {compact && (
 <h3 className="text-[10px] font-bold text-slate-400" >
 <Compass className="w-3.5 h-3.5 text-indigo-400" />
 </h3>
)}
 <h3 className={compact ? "text-xs font-bold text-slate-400" : "text-sm font-bold text-slate-400"} >
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 </div>
 </aside>
);
};
```

```

const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
// «18–20. Графы. Основное дерево...» → 3
const num = chapter.title.match(/^(\d+)(?:\s+)?/)[1]
const label = num ? chapter.title.slice(0, num) : ''
return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavior: 'smooth' })
 }}
 title={chapter.title}
 aria-current={isSelected ? "page" : undefined}
 className={`w-full text-left rounded-md
 ${isSelected ? "bg-indigo-600/15 border-indigo-600/15" : "bg-slate-800/40 border-transparent"}
 `}
 >

 {num ? num[1] : "•"}

 {label}
 {hasViz && (
 <span className="ml-1.5 inline-block h-4 w-4" style={{
 background: 'linear-gradient(to right, transparent 49%, #000000 49%, #000000 51%, #ffffff 51%, #ffffff 53%)'
 }} />
 ая визуализация" />
)}

 {!compact && (
 <ChevronRight
 className={`w-4 h-4 shrink-0 mt-px
 ${isSelected ? "text-indigo-400" : "text-slate-600"}
 `}
 />
)}
 </button>
)

```

```
interface Plate {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Person {
 id: string;
 name: string;
 icon: string;
}
```

```
interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}
```

```
export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' |

 // Stack state
 const [stack, setStack] = useState<Plate[]>([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
 { id: '3', name: 'Блюдец от торта', icon: ' ' }],
));
 const [popMessage, setPopMessage] = useState<string |

 // Queue state
 const [queue, setQueue] = useState<Person[]>([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
));
 const [dequeueMessage, setDequeueMessage] = useState<

 // Heap state
```

```

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюде от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥣' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🧑' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setDequeueMessage(` Встал в конец очереди: ${newPerson.name} ${newPerson.icon}`);
};

const servePerson = () => {
 if (queue.length === 0) {
 setDequeueMessage("⚠ Очередь пуста! Суп ждет новы");
 return;
 }
 const firstPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(` Выдан суп первому в очереди (F${firstPerson.name} ${firstPerson.icon})`);
};

const resetQueue = () => {

```

```

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-800 p-3">
 <div className="border-b border-slate-800 pb-6 mb-2">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text">
 Интерактивный симулятор структур данных
 </h2>
 <p className="text-slate-400 text-sm mt-2">
 Проверь работу принципов LIFO, FIFO и Приоритетной очереди
 </p>
 </div>

 { /* Tabs */ }
 <div className="grid grid-cols-1 md:grid-cols-3 gap-3">
 <button
 onClick={() => setActiveTab('stack')}
 className={`flex items-center justify-between gap-2 px-4 py-2
 ${activeTab === 'stack'
 ? 'bg-blue-600/20 border-blue-500 text-blue-400'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
 `}
 >
 <div className="flex items-center gap-3">
 <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'text-blue-400' : 'text-slate-400'}
 <div className="text-left">

```

```

 <Award className={`w-5 h-5 ${activeTab ===
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">Приор
 </div>
 </div>
 <span className="text-xs bg-emerald-500/20 te
 Beam Search

 </button>
 </div>

 { /* STACK TAB */ }
 {activeTab === 'stack' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция стопки тарелок
 <span className="text-xs font-mono bg-b
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Новые тарелки ставятся наверх. Мыть мож
 </p>
 </div>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPlate}
 className="flex-1 md:flex-none flex ite
 rounded-xl text-sm font-bold shadow-lg tra
 >
 <Plus className="w-4 h-4" /> Поставить
 </button>
 <button
 onClick={popPlate}
 className="flex-1 md:flex-none flex ite
 border-blue-500/40 px-4 py-2.5 rounded-xl

```



```

 {
 /* Reversed map so top of stack is visible */
 {stack.slice().reverse().map(({plate, index}) => {
 const isTop = index === 0;
 return {
 <div
 key={plate.id}
 className={`w-full flex items-center justify-center
 ${isTop
 ? 'bg-gradient-to-r from-blue-500 to-blue-100'
 : 'bg-slate-900/90 border-slate-800'}
 `}
 >
 <div className="flex items-center justify-center" >
 {plate.name}

 {isTop ? 'Готово к помывке' : 'Положил последней → помыл первой (LIFO)'}

 </div>
 </div>
);
 })}
 </div>
 </div>
</div>
)}

/* QUEUE TAB */

```

```
</div>
```

```
{dequeueMessage && {
 <div className="bg-indigo-950/60 border border-
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 round
 {dequeueMessage}
 </div>
}}
```

```
{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очер
 <p className="text-xs mt-1">Добавьте ро
 </div>
 } : {
 <div className="w-full flex items-center
 <div className="flex flex-col items-cen
300">

 <span className="text-xs font-bold font
 </div>
```

```
<div className="text-indigo-500 flex it
 <ArrowRight className="w-6 h-6 animat
</div>
```

```
{queue.map((person, index) => {
 const isFirst = index === 0;
 return {
 <div
 key={person.id}
 className={`flex flex-col items-c
 isFirst
```

```

 { /* HEAP TAB */
 {activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
 x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>
 </div>
 <div className="flex items-center gap-3 w-f
 <button
 onClick={popHeap}
 className="flex-1 md:flex-none flex ite
 py-2.5 rounded-xl text-sm font-bold shadow-
 >
 Взять кандидата с макс. вероятностью
 </button>
 <button
 onClick={resetHeap}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
 tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>

 { /* Add Form */
 <form onSubmit={addHypothesis} className="bg-
 s-12 gap-4 items-end">
 <div className="md:col-span-6">

```

```

 <div className="bg-emerald-950/60 border bo
 gap-3 animate-fade-in">
 <span className="inline-block w-2 h-2 rou
 {heapMessage}
 </div>
 </div>
)}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
 >
 {sortedHeap.length === 0 ? (
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча
 <p className="text-xs mt-1">Добавьте ги
 </div>
) : (
 <div className="w-full max-w-2xl space-y-
 <div className="flex items-center justifi
 КАНДИДАТ / ГИПОТЕЗА
 ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)
 </div>

 {sortedHeap.map(({item, index}) => {
 const isTop = index === 0;
 const percent = (item.probability * 10
 return (
 <div
 key={item.id}
 className={`flex items-center justifi
 isTop
 ? 'bg-gradient-to-r from-emera
00/10 scale-[1.01]`
 : 'bg-slate-900/80 border-sla
 }`}
 >
 <div className="flex items-center
 <span className={`flex items-ce

```

```
 </div>
 </div>
 </div>
)}
</div>
);
};
```

---

ФАЙЛ 70/111: src/components/SplayPlayground.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useContext, useEffect, useState } from "react";
import { VizChapterContext, emitVizDemo } from "../data";
import { SplayRotationsViz } from "../SplayRotationsViz";
import SplayWalkViz from "../SplayWalkViz";
import { SplayRotationSandbox } from "../SplayRotationSandbox";
```

```
/**
```

```
 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
 * (прицел → поворот, призраки старых позиций, обход)
 * 2. «Большое дерево» – те же случаи внутри настоящего дерева
 * поворот – кадры «прицел → отстёгиваем среднее положение»
 * 3. «Собери сам» – песочница: подними узел кликами, сбрось
 */
```

```
type Tab = "cases" | "big" | "play";
```

```
export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({ defaultTab }) => {
 const [tab, setTab] = useState<Tab>(defaultTab);
 const chapterId = useContext(VizChapterContext);
```

```
 // вкладка = подрежим страницы: панель Python подставлена
```

```
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);
```

```
 <p className="text-xs text-slate-500 mt-3">
 Порядок поворотов в песочнице сверяется с пра
 </p>
 })
 </div>
);
};
```

```
export default SplayPlayground;
```

---

ФАЙЛ 71/111: src/components/SplayRotationSandbox.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useCallback, useMemo, useState } from "
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, U
```

```
/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерев
 * поднять отмеченный узел в корень. Клик по узлу = оди
 * Порядок поворотов выбираешь сам; правильный ли он по
 * только после того, как узел окажется наверху (или по
 */
```

```
export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}
```

```
const leaf = (key: number): TNode => ({ key, left: null
```

```
const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t
```

```

 const n4: TNode = { key: 4, left: leaf(3), right:
 return { key: 8, left: n4, right: leaf(9) };
 },
},
];

/* ----- операции над деревом -----

export const findPath = (root: TNode | null, key: number): TNode[] => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 if (gg.right === p) gg.right = x;
}

```

```

 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
 };
 walk(root, 0);

 const cols = Math.max(1, order);
 const rows = Math.max(1, Math.max(...nodes.map((n) =>
 const colW = 46;
 const rowH = 62;
 return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + co
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
 });
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.l
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; co
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 }
);
}

```



```

 поворотов выбираешь сам, разбор покажем в к
 </p>
</div>
<button
 onClick={() => reset((presetIdx + 1) % PRESETS.length)}
 className="flex items-center gap-1.5 px-3 py-1.5 rounded-md text-sm font-medium transition-colors shrink-0"
>
 <Dices className="w-3.5 h-3.5" /> Другое дерево
</button>
</div>

<div className="flex gap-2 mb-3 overflow-x-auto" >
 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-sm font-medium transition-colors shrink-0 ${
 i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-indigo-600"
 }`}
 >
 {p.name}
 </button>
 })}
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800 p-4" >
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420) }}
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 })}
 </svg>
</div>

```

```

 </text>
 </g>
);
 }}}
</svg>
</div>

<div className="flex flex-wrap items-center gap-2"
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-2
 -white disabled:opacity-40 text-xs font-bol
 >
 <Undo2 className="w-3.5 h-3.5" /> Отменить
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-2
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-2
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-
 : "bg-slate-800 border-slate-700 text-sla
 }`}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5"
 {showAnswer ? "Скрыть разбор" : "Сдаться / по
 </button>

 <span className="text-[11px] text-slate-500 ml-
 поворотов: {moves.length} • минимум: {minimal


```

```

 }}
 </div>
 }}
 </div>
);
};

```

ФАЙЛ 72/111: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useEffect, useMemo, useRef, useState } from 'react';
import { Gauge, Pause, Play, RotateCcw, Terminal } from 'react-d3';
import { useVizRuntime, vizNumber, vizString } from 'viz-runtime';

```

```
/**
```

```
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag.
```

```
 *
```

```
 * Что сделано ради понятности:
```

```
 * * у узлов настоящие числа (20 / 40 / 60), а не абстрактные;
```

```
 * * сразу видно, что дерево остаётся деревом поиска;
```

```
 * * под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ;
```

```
 * * это и есть доказательство, что поворот ничего не меняет;
```

```
 * * кадр «прицел» ничего не двигает, только подсвечивает;
```

```
 * * на кадре «результат» остаются призраки – пунктирные линии;
```

```
 * * места и стрелки «откуда → куда»;
```

```
 * * по умолчанию анимация НЕ крутится сама: пользователь управляет;
```

```
 * * идёт в своём темпе. Автопрокрутка включается кнопкой.
```

```
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
```

```
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
```

```
 pos: Pos;
```

```
 edges: [NodeId, NodeId][];
```

```

when: "родитель х — уже корень, деда нет",
rotations: "1 поворот",
keys: { x: 20, p: 40 },
inorder: ["A", "20", "B", "40", "C"],
ranges: "A < 20 · B между 20 и 40 · C > 40",
frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду",
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 — 20",
 text: "Ничего пока не двигается. Смотри на подсве",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 — корень",
 text: "40 стало правым сыном двадцатки. Поддерев
 ева от 40 — это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

```

/\* ----- Zig-Zig -----

```

const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
 edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"

```

```

 moved: ["p", "g", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот же",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
 title: "Стало: 20 в корне, ветка x подтянулась",
 text: "Сравни с первым кадром: А было на глубине
 ась. Splay не «уменьшает высоту вообще», он под
 сумме запросов (амортизация).",
 moved: ["x", "p", "A", "B"],
 ms: RESULT_MS,
 },
],
};

```

```

/* ----- Zig-Zag -----
const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
 key: "zig-zag",

```

```

 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",
 moved: ["x", "p", "g", "C"],
 ms: RESULT_MS,
 },
],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

const COLOR: Record<string, { fill: string; stroke: str
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
 g: { fill: "#f59e0b", stroke: "#fcd34d" },
 sub: { fill: "#1e293b", stroke: "#475569" },
};

```

```

const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x – ищем его",
 p: "p – родитель",
 g: "g – дед",
};

```

```
const isSubtree = (id: NodeId) => id === "A" || id ===
```

```

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? [])

 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f

```

```

const hot = Boolean(focus && focus.includes(a))
return (
 <line
 key={` ${a} - ${b} - ${i} `}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
}})

```

```

{Object.keys(pos) as NodeId[]}.map((id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id))
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke="black" />}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 />
 </g>
)
}

```

```

useEffect(() => {
 if (!liveCase && liveStep === null) return;
 const norm = (liveCase ?? "").trim().toLowerCase();
 const idx = CASE_BY_NAME[norm] ?? caseIdx;
 const sig = `${idx}:${liveStep ?? ""}`;
 if (sig === lastSig.current) return;
 const first = lastSig.current === null;
 lastSig.current = sig;
 setPlaying(false);
 if (first || idx !== caseIdx) {
 setCaseIdx(idx); // новый случай – начинаем его
 setFrame(0);
 } else {
 // тот же случай, следующий шаг кода – пролистывае
 setFrame((f) => Math.min(CASES[idx].frames.length, f + 1));
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [liveCase, liveStep]);

const active = CASES[caseIdx];
const total = active.frames.length;
const idx = Math.min(frame, total - 1);
const current = active.frames[idx];
const prevPos = active.frames[Math.max(idx - 1, 0)].pos;
const duration = useMemo(() => Math.round(current.ms - prevPos), [current, prevPos]);
const last = idx === total - 1;

useEffect(() => {
 setFrame(0);
}, [caseIdx]);

useEffect(() => {
 if (!playing) return;
 const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
 return () => clearTimeout(t);
}, [playing, frame, duration, total]);

return (

```



```

 <p className="text-[13px] text-slate-400 leading-4">
 </div>

```

```

<div className="bg-slate-900 rounded-xl border border-slate-800">
 <Tree frame={current} prev={prevPos} keys={activeKeys}>

```

```

 {/* обход слева направо – одинаковый на всех кадрах */}
 <div className="mt-2 pt-2 border-t border-slate-800">
 <div className="flex items-center gap-1.5 flex-wrap">
 Обход
 {active.inorder.map((t, i) => (
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-mono text-sm
 /\d/.test(t) ? "bg-emerald-500/15 text-emerald-400" : ""
 }`}
 >
 {t}

))}

 </div>
 <p className="text-[11px] text-slate-500 mt-1">
 </div>
 </div>

```

```

 {/* индикатор автопрокрутки */}
 <div className="h-1 w-full bg-slate-800 rounded-f">
 <div
 key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
 className="h-full bg-indigo-500"
 style={
 playing
 ? { animation: `demoProgress ${duration}ms linear 1` }
 : { width: "100%", background: "#334155" }
 }
 </div>
 </div>

```

```

 }` }
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "0.5"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-3
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto"
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : (
 <span key={id} className="flex items-center"
 <span className="w-3 h-3 rounded-full"
 {id === "x" ? "bg-red" : id === "p" ? "bg-blue" : "bg-green"}
 />

)
)}
 </div>
</div>

{/* что сейчас сообщил компилятор */}
<div className="mt-3 pt-3 border-t border-slate-800"
 <span className="text-[10px] uppercase tracking-wider"
 <Terminal className="w-3.5 h-3.5" /> из компилятора

 {liveCase || liveX !== null || liveStep !== null ? (
 <div>
 {liveCase && (
 <span className="text-[11px] font-mono bg-slate-800 text-white"
 case = {liveCase}

)}
)}
)}

```

```

* cut — отстёгиваем среднее поддереву β (пунктир, по
* rot — поворот: зелёным подсвечены рёбра, которых н
* Рядом с логом всегда висит врезка «Механика одного п
* На каждом кадре — живая автопроверка in-order (сорти
*
* Чистые функции (splaySteps, rotateUp, bstInsert, col
*/
```

```
export interface SN {
 key: number;
 left: SN | null;
 right: SN | null;
 parent: SN | null;
}
```

```
export function cloneWithParents(t: SN | null): SN | null {
 if (!t) return null;
 const n: SN = { key: t.key, left: null, right: null, parent: null };
 const l = t.left ? cloneWithParents(t.left) : null;
 const r = t.right ? cloneWithParents(t.right) : null;
 if (l) {
 n.left = l;
 l.parent = n;
 }
 if (r) {
 n.right = r;
 r.parent = n;
 }
 return n;
}
```

```
export function bstInsert(root: SN | null, key: number): SN | null {
 if (!root) return { key, left: null, right: null, parent: null };
 const r = cloneWithParents(root)!;
 let cur: SN = r;
 for (;;) {
 if (key < cur.key) {
 if (!cur.left) {
 cur.left = { key, left: null, right: null, parent: cur };
 return cur.left;
 }
 cur = cur.left;
 } else {
 if (!cur.right) {
 cur.right = { key, left: null, right: null, parent: cur };
 return cur.right;
 }
 cur = cur.right;
 }
 }
}
```

```

const es: string[] = [];
const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
};
walk(t);
return es.sort();
}

const diffEdges = (before: string[], after: string[]):
const rm = before.filter((e) => !after.includes(e));
const ad = after.filter((e) => !before.includes(e));
const parts: string[] = [];
for (const a of ad) {
 const [p, c] = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}`);
 else parts.push(`поддерево ${c} переехало к ${p}`);
}
return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
 kind: SplayStepKind;
 root: SN | null;
 aimEdge?: [number, number];
 /** Кадр cut: ребро, по которому отстёгивается среднее */
 cutEdge?: [number, number];
 cutLabel?: string;
 /** Куда β переедет: сторона у верхнего узла. */
 cutDest?: string;
 pathKeys?: number[];
 caseLabel?: string;
 note: string;
 rotCount: number;

```

```

 });

 // --- подъём ---
 let rotCount = 0;
 let target: SN = x as SN;
 const fixRoot = () => {
 while (root && root.parent) root = root.parent;
 };
 /** Один поворот = три кадра: aim → cut → rot. */
 const emitRotation = (child: SN, label: string, aimEdge: string) => {
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge,
 caseLabel: label,
 note: aimNote,
 rotCount,
 });
 const p = child.parent!;
 const mid = p.left === child ? child.right : child.left;
 steps.push({
 kind: "cut",
 root: cloneWithParents(root)!,
 cutEdge: [child.key, mid ? mid.key : child.key],
 cutLabel: mid ? `β = поддереву ${mid.key}` : "β пусто",
 cutDest: mid ? (p.left === child ? "справа у p" : "слева у p") : "никуда",
 caseLabel: label,
 note: mid
 ? `Отстёгиваем среднее поддерево β = ${mid.key}
 и у них общий — после поворота переедет ${p.left === child ? "справа у p" : "слева у p"}
 : `Среднего поддерева нет (β пусто): ${child.key}
 rotCount,
 });
 const before = collectEdges(root);
 rotateUp(child);
 fixRoot();
 rotCount += 1;
 const diff = diffEdges(before, collectEdges(root));
 };

```

```

 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zig",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key}
);
} else {
 const side = `${g.key} → ${pIsLeft ? "влево" : "вправо"}
 emitRotation(
 target,
 "Zig-Zag",
 [p.key, target.key],
 `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: п
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Второе ребро: ${target.parent.key}—${target.key}
 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zag",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key}
);
}
}

```

```

steps.push({
 kind: "done",
 root: cloneWithParents(root)!,
 note: ` ${target.key} в корне за ${rotCount} ${rotCount}
 rotCount,

```

```
];
```

```
/* ----- врезка «механика одного по
```

```
const MechanismInset: React.FC<{ step: SplayStep | null
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
```

```
 <div className="bg-slate-950/80 rounded-lg border b
```

```
 <h4 className="text-xs font-bold text-slate-300 m
```

```
 <p className="text-[10px] text-slate-500 mb-2">
```

```
 Как вправление вывиха: одна кость ходит вокруг ,
```

```
 </p>
```

```
 <svg viewBox="0 0 340 190" className="w-full">
```

```
 <g transform="translate(10,6)">
```

```
 <text y="10" fill="#94a3b8" fontSize="11" fon
```

```
 <line x1="55" y1="46" x2="30" y2="86" stroke=
```

```
 <line x1="55" y1="46" x2="80" y2="86" stroke=
```

```
 <line x1="30" y1="86" x2="12" y2="126" stroke
```

```
 <line
```

```
 x1="30" y1="86" x2="48" y2="126"
```

```
 stroke={isCut ? "#f59e0b" : "#64748b"}
```

```
 strokeWidth={isCut ? 3 : 2}
```

```
 strokeDasharray={isCut ? "5,3" : "none"}
```

```
 />
```

```
 <circle cx="55" cy="46" r="15" fill="#1e293b"
```

```
 <text x="55" y="50" textAnchor="middle" fill=
```

```
 <circle cx="30" cy="86" r="15" fill="#78350f"
```

```
 <text x="30" y="90" textAnchor="middle" fill=
```

```
 <circle cx="12" cy="126" r="13" fill="#0f172a"
```

```
 <text x="12" y="130" textAnchor="middle" fill=
```

```
 <circle
```

```
 cx="48" cy="126" r="13"
```

```
 fill={isCut ? "#78350f" : "#0f172a"}
```

```
 stroke={isCut ? "#f59e0b" : "#64748b"}
```

```
 strokeWidth={isCut ? 2.5 : 1.5}
```

```
 strokeDasharray={isCut ? "4,3" : "none"}
```

```
 />
```

```

 </g>
 </svg>
 <p className="text-[10px] text-slate-500 leading-5">
 Обход слева-направо α x β p y не меняется никогда,
 числа поворотов. Единственный, кто меняет хозяина — это α .
 </p>
 </div>
);
};

/* ----- рендер ----- */

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
 slot += 1;
 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>(null);
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [input, setInput] = useState("");
 const [treeVers, setTreeVers] = useState(0);
 const baseRef = useRef<SN>(initial);

```



```

 }, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.w);
const autoH = Math.max(300, ...nodes.map((n) => n.py);

const dBefore = useMemo(() => (steps ? depthOf(steps[steps.length - 1]) : 0));
const dNow = steps ? depthOf(viewRoot, steps[steps.length - 1]) : 0;
const hBefore = useMemo(() => (steps ? heightOf(steps[steps.length - 1]) : 0));
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
 if (!steps || !step || idx <= 0) return null;
 return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);
const movedCount = useMemo(() => {
 if (!prevEdgeSet) return 0;
 return edges.filter((e) => !prevEdgeSet.has(`${e.a}-${e.b}`)).length;
}, [prevEdgeSet, edges]);

const ino0 = useMemo(() => (steps ? inorderKeys(steps[steps.length - 1].root).join(",") : ""));
const inoNow = inorderKeys(viewRoot).join(",");
const checkOk = !steps || ino0 === inoNow;

const doInsert = () => {
 const v = parseInt(input, 10);
 if (Number.isNaN(v)) return;
 const withNew = bstInsert(baseRef.current, v);
 baseRef.current = withNew;
 setTreeVers((t) => t + 1);
 runKey(v, true);
 setInput("");
};

const stepColor = (k: number): string => {
 if (!step) return "#0f172a";
 if (step.kind === "search" && step.pathKeys?.includes(step.a)) return "#f1f3f4";
 if ((step.kind === "aim" || step.kind === "cut") && step.a === step.b) return "#f1f3f4";
 return "#0f172a";
};

```

```

 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current);
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
}
placeholder="ключ + Enter"
className="w-28 bg-slate-950 border border-
s:border-indigo-500"
/>
<button
 onClick={() => {
 baseRef.current = buildInitial();
 setSteps(null);
 setIdx(0);
 setPlaying(false);
 setTreeVers((v) => v + 1);
 }}
 className="px-3 py-2 rounded-lg text-xs fon
>
 Исходное
</button>
<button
 onClick={() => {
 const shuffled = [...INITIAL_KEYS].sort(() =>
 let r: SN | null = null;
 for (const k of shuffled) r = bstInsert(r, k);
 baseRef.current = r!;
 setSteps(null);
 setIdx(0);
 setPlaying(false);
 setTreeVers((v) => v + 1);
 }}
 className="px-3 py-2 rounded-lg text-xs fon
>
 Перемешать

```

```

<div className="absolute top-3 left-3 bg-indigo-500 text-white font-bold">
 Кликни по любому узлу – splay покадрово
</div>
))
<svg viewBox={`0 0 ${autoW} ${autoH}`} className="w-full h-full"
 {edges.map((e) => {
 const a = pos.get(e.a);
 const b = pos.get(e.b);
 if (!a || !b) return null;
 const isAim = step?.aimEdge && step.aimEdge === e.key;
 const isCutEdge =
 step?.kind === "cut" && step.cutEdge && step.cutEdge[1] === e.key;
 const isMoved = !isAim && !isCutEdge && preStep?.kind === "cut";
 return (
 <g key={`e-${e.a}-${e.b}`}>
 <line
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={isAim ? "#f59e0b" : isCutEdge ? "#f00" : "#ccc"}
 strokeWidth={isAim ? 4 : isCutEdge ? 2 : 1}
 strokeDasharray={isAim ? "6,4" : isCutEdge ? [4, 4] : []}
 className="transition-all duration-500"
 />
 {isCutEdge && (
 <text x={(a.px + b.px) / 2 + 14} y={(a.py + b.py) / 2}
 className="transition-all duration-500"
 {step?.cutLabel} → {step?.cutDest}
 </text>
)}
 </g>
);
 })}
);
)))
{nodes.map((n) => {
 const aimE = step?.aimEdge ?? (step?.kind === "aim" ? step.aimEdge : null);
 const isAimNode = aimE?.includes(n.key);
 return (
 <g

```

```

 </p>
 <p className="min-h-[40px] flex items-center"
 {step ? step.note : "Кликни по узлу или в
 ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? "text-rose-300"}
 {checkOk ? " " : " "} проверка кадра: 0
 {inorderKeys(viewRoot).length}
 {movedCount > 0 ? ` · переехало рёбер: `
 : ""}
 </p>
 }}
 </div>
 <div className="w-full lg:w-[300px] shrink-0"
 <MechanismInset step={step} />
 </div>
</div>
</div>

{/* Счётчики + код */}
<div className="flex flex-col lg:flex-row gap-6 m-6"
 <div className="w-full lg:w-1/2 bg-rose-950/10"
 <h4 className="text-sm font-bold text-rose-300"
 {steps ? (
 <table className="w-full text-sm">
 <tbody className="font-mono">
 <tr className="border-b border-slate-800">
 <td className="py-2 text-slate-400">р.
 <td className="py-2 text-right text-slate-400">
 </tr>
 <tr className="border-b border-slate-800">
 <td className="py-2 text-slate-400">в.
 <td className="py-2 text-right text-slate-400">
 </tr>
 <tr>
 <td className="py-2 text-slate-400">н.
 <td className="py-2 text-right text-slate-400">
 </tr>
 </tbody>
 </table>
) : ""}
 </div>
 </div>

```

```
import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';
```

```
interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}
```

```
const PLATE_PRESETS = [
 { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pasta' },
 { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-pizza' },
 { name: 'Блюдец от торта', emoji: '🍰', color: 'from-cake' },
 { name: 'Тарелка от тако', emoji: '🌮', color: 'from-taco' },
 { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-sushi' },
 { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'from-pancake' },
];
```

```
let counter = 0;
```

```
export const StackViz: React.FC = () => {
 const [dirtyStack, setDirtyStack] = useState<Plate[]>(
 [
 { id: 'p1', name: 'Тарелка из-под спагетти', emoji: '🍝', animState: 'idle' },
 { id: 'p2', name: 'Тарелка из-под пиццы', emoji: '🍕', animState: 'idle' },
 { id: 'p3', name: 'Блюдец от торта', emoji: '🍰', animState: 'idle' },
]
);
```

```
 const runtime = useVizRuntime();
```

```

 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
 }, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(` POP: Начинаем мыть верхнюю тарелку с ${top

// Step 1: Start washing animation (sponge and soap
setDirtyStack(prev => prev.map(p => p.id === topPla
setShowSoap(true);

setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and
 const cleanPlate: Plate = {
 ...topPlate,
 isDirty: false,
 animState: 'clean',
 };

 setCleanRack(prev => [cleanPlate, ...prev].slice(0,

```

```
</div>
```

```
{/* УПРАВЛЕНИЕ */}
```

```
<div className="flex items-center gap-3 flex-wrap
```

```
 <button
```

```
 onClick={pushPlate}
```

```
 disabled={isWashing}
```

```
 className="flex-1 min-w-[150px] bg-gradient-to
```

```
 acy-40 disabled:cursor-not-allowed text-w
```

```
 ale-95 transition-all cursor-pointer flex i
```

```
 >
```

```
 Положить грязную (PUSH)
```

```
 </button>
```

```
 <button
```

```
 onClick={popPlate}
```

```
 disabled={isWashing || dirtyStack.length === 0}
```

```
 className="flex-1 min-w-[150px] bg-gradient-to
```

```
 opacity-40 disabled:cursor-not-allowed text
```

```
 ve:scale-95 transition-all cursor-pointer f
```

```
 >
```

```
 Помыть верхнюю (POP)
```

```
 </button>
```

```
 <button
```

```
 onClick={reset}
```

```
 className="px-5 py-3.5 rounded-2xl bg-slate-800
```

```
 r-pointer border border-slate-700"
```

```
 >
```

```
 Сброс
```

```
 </button>
```

```
</div>
```

```
{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
```

```
<div className="grid grid-cols-1 md:grid-cols-12
```

```
 {/* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */}
```

```
 <div className="md:col-span-7 bg-slate-900 roun
```

```

 >

 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 </div>

 /* Тарелки (отрисовываем в обратном порядке)
 {displayStack.slice().reverse().map((plate, idx) => {
 const idx = displayStack.length - 1 - idx;
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded-md
 shadow-md select-none transition-all duration-200
 ${plate.color}
 ${plate.animState === 'in' ? 'animate-in' : ''}
 ${plate.animState === 'washing' ? 'animate-wash' : ''}
 ${isTop ? 'ring-4 ring-blue-500/50' : ''}
 `}
 >
 {plate.name}

 {isTop && (

 LIFO

)}

 </div>
);
 })}
 </div>
)}

```



```

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-slate-500 to-slate-900">
 </div>
 </div>

 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="text-[10px] font-mono text-slate-400">
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`}
 >
 {idx === 0 ? LOG : null}
 {entry}
 </div>
))}
 </div>
 </div>
);
};

```

---

ФАЙЛ 75/111: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import { useState, useEffect, useRef, useMemo } from "react";
import { useVizRuntime, vizArray, vizNumber, vizString } from "viz-runtime";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

```

```

function generateKmpSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const pi = new Array(n).fill(0);
 for (let i = 1; i < n; i++) {
 let j = pi[i - 1];
 while (s[i] !== s[j] && j > 0) j = pi[j - 1];
 if (s[i] === s[j]) j++;
 pi[i] = j;
 }
}

```

```

const n = s.length;
const z = new Array(n).fill(0);
const steps: any[] = [];

let l = 0, r = 0;
steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$
 i` });

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource
 падают! Окно растёт.` });
 z[i]++;
 }

 if (i + z[i] < n) {
 steps.push({ i, l, r, zTarget: z[i], zSource
 [i]}}' разные. Стоп.` });
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Упё
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксирыва
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 steps.push({ i, l, r, z: [...z], desc: ` 0

```

```

useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const baseStep = steps[stepIdx] || steps[0];
const vars = runtime?.variables;
const liveValues = vizArray(mode === "kmp" ? vars?.l
const hasLiveValues = !!vars && Object.prototype.ha
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveL = vizNumber(vars?.l);
const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveVal
const step = compilerLinked
 ? {
 ...baseStep,
 i: liveI ?? baseStep.i,
 j: liveJ ?? baseStep.j,
 l: liveL ?? baseStep.l,
 r: liveR ?? baseStep.r,
 [mode === "kmp" ? "pi" : "z"]: hasLiveValues
 desc: `Компилятор: i=${liveI ?? "-"}${mode ===
 }
 : baseStep;

```

```

let bgColor = "bg-slate-900";
let textColor = isSeparator ? "text-white" : "text-slate-900";

// KMP Logic
if (mode === "kmp") {
 if (step.highlight?.include(index)) {
 bgColor = "border-amber-900/40";
 border = "border-amber-900/40";
 }
}

// Z Logic
if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900/40";
 border = "border-emerald-900/40";
 }
 if (step.zTarget === index) {
 border = "border-amber-900/40";
 bgColor = step.match === " " ? "bg-slate-900/40" : "bg-emerald-900/40";
 }
}

return (
 <div key={index} className={
 {/* L/R markers for Z */
 {mode === "z" && step.l <= index && index <= step.r
 ? <div className="absolute left-0 right-0 top-0 bottom-0 border border-emerald-900/40" />
 : ""
 }
 {mode === "z" && step.zTarget === index
 ? <div className="absolute left-0 right-0 top-0 bottom-0 border border-amber-900/40" />
 : ""
 }
 }
 {/* Index */}
 {step.match}
)

```

```

 {mode === "z" && step.z}
 <div className="abs
 <span className
 </div>
 }}
 {mode === "z" && step.z}
 <div className="abs
 <span className
 </div>
 }}

 {/* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 }}
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 }}
 </div>
)
 }}}
 </div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled
-400 hover:text-white hover:bg-slate-800 ro
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={playPause} classNa
x items-center justify-center min-w-[80px]"
 {autoplay ? <><Pause size={16}
 </button>
 <button onClick={nextStep} disabled

```

```
 <div className="bg-slate-800 px-4 py-2 b
late-400 uppercase">
 Код C++ ({mode === 'kmp' ? 'КМП
 </div>
 <pre className="p-4 text-xs font-mono te
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}`}

 </pre>
 </div>
 </div>
);
}
```

---

ФАЙЛ 76/111: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from "react";
```

```
interface TooltipProps {
```

```
 /** Содержимое подсказки (можно JSX). */
```

```
 content: React.ReactNode;
```

```
 /** Элемент, на который наводят курсор / фокус. */
```

```


);
 };
};
```

---

ФАЙЛ 77/111: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";
```

```
interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}
```

```
interface TEdge {
 u: number;
 v: number;
}
```

```
const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
];
```

```

const fullyProcessed = new Set<number>();
const topoList: number[] = [];
const dfsStack: number[] = [];

const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
};

recordStep(
 "Начало топологической сортировки. Используем кла",
 null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} ("${dagNodes[u].name}`,
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in
 recordStep(

```



```

 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
 };

 useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1) {
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
 } else {
 setIsPlaying(false);
 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
 };

 const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

```

```

 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
</marker>
<marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
>
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
</marker>
</defs>

{/* Edges */}
{dagEdges.map((edge, idx) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v));

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#3341
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-ar
 className="transition-all duration-30

```

```

 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-500"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 ({node.label}) {node.name.split(" ").join(" ")}
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
}
}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

```

```

 Черные (готово)

 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 text-white"
 >
 Вперед
 </button>
 </div>
</div>
</div>
</div>
</div>
);
};

```

```

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];

/** Порядок ввода (как в черновике) – намеренно НЕ отсортирован */
export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 12, grade: 0 },
];

// (3,5) дубль ключа/приоритета – уберём: ввод без повторов
export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p, i) => !p.key || i === 0);

const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

/* ----- чистые алгоритмы (экспортируются для тестов) ----- */

export function insert(root: TNode | null, p: Pair): TNode | null {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade };
 if (p.key <= root.key) root.left = insert(root.left, p);
 else root.right = insert(root.right, p);
 return root;
}

export function buildTree(ps: Pair[]): TNode | null {
 if (ps.length === 0) return null;
 const sorted = sortedByY(ps);
 const root = insert(null, sorted[0]);
 for (let i = 1; i < sorted.length; i++) {
 root = insert(root, sorted[i]);
 }
 return root;
}

```

```

 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, note: `Соединение целиком.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id, note: `Выборим его правое поддеревом с (${b.key}, ${b.grade})` });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id, note: `Выборим (${a.key}, ${a.grade}) с его левым поддеревом.` });
 b.left = go(a, b.left);
 return b;
 };
 return { root: go(a, b), trace };
}

export function eraseNode(root: TNode | null, key: number): TNode | null {
 const trace: { atId: NodeId | null; note: string }[] = [];
 const go = (t: TNode | null): TNode | null => {
 if (!t) return null;
 if (key < t.key) {
 trace.push({ atId: t.id, note: `${key} < ${t.key}` });
 t.left = go(t.left);
 return t;
 }
 if (key > t.key) {
 trace.push({ atId: t.id, note: `${key} > ${t.key}` });
 t.right = go(t.right);
 return t;
 }
 trace.push({ atId: t.id, note: `Нашли (${t.key}, ${t.grade})` });
 const m = mergeTree(t.left, t.right);
 trace.push(...m.trace.map((s) => ({ atId: s.chosenId, note: s.note })));
 return m.root;
 };
}

```

```

 walk(root, 0);
 return pos;
};

const treeEdges = (root: TNode | null): { from: NodeId;
const es: { from: NodeId; to: NodeId }[] = [];
const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id });
 if (n.right) { es.push({ from: n.id, to: n.right.id });
};
walk(root);
return es;
};

interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
 nodeStroke?: (id: NodeId) => string;
 edgeColor?: (from: NodeId, to: NodeId) => string;
 edgeDash?: (from: NodeId, to: NodeId) => boolean;
 pulseId?: NodeId | null;
 subtitle?: string;
}

const TreeView: React.FC<TreeViewProps> = ({ root, width
const pos = useMemo(() => layoutTree(root), [root]);
const nodes = useMemo(() => [...pos.entries()].map(([
const edges = useMemo(() => treeEdges(root), [root]);
const autoW = Math.max(420, ...nodes.map((n) => n.px +
const autoH = Math.max(260, ...nodes.map((n) => n.py +
const w = width ?? autoW;
const h = height ?? autoH;
const byId = (id: NodeId) => nodes.find((n) => n.id ===
return (
 <div>
```

```

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 60) / 10);
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H - 24) / 8);

interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;
 badEdge?: { a: Pair; b: Pair } | null;
 onPointDown?: (p: Pair) => void;
 onPointUp?: (p: Pair) => void;
 onMove?: (e: React.PointerEvent<SVGSVGElement>) => void;
}

const Plane: React.FC<PlaneProps> = ({ points, drawnEdges, pending,
 }) => {
 const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.grade) });
 return (
 <svg
 viewBox={`0 0 ${PL_W} ${PL_H}`}
 className={`w-full ${interactive ? "touch-none select-none" : ""}`}
 onPointerMove={onMove}
 style={interactive ? { cursor: "crosshair" } : undefined}
 >
 <defs>
 <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1"/>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1"/>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(8)} stroke="black" stroke-width="1"/>
 </svg>
);
};

```



```

 <circle cx={PX(p.key)} cy={PY(p.grade)} r={
 stroke={isSel ? "#a5b4fc" : done ? "#38bdf8"
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} f
 ">{p.key}</text>
 </g>
 });
 }}}

 {pending && (
 <g className="transition-all duration-500">
 <circle cx={PX(pending.key)} cy={PY(pending.g
 >
 <text x={PX(pending.key)} y={PY(pending.grade
 ly="monospace">{pending.key}</text>
 </g>
)}
</svg>
);
};

```

/\* ===== ИГРА: соединение мышкой =====

export type GEdge = { a: Pair; b: Pair }; // a – родите

```

export const CANON_GAME: GEdge[] = (() => {
 const es: GEdge[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ a: { key: n.key, grade: n.g
 if (n.right) { es.push({ a: { key: n.key, grade: n.
 };
 walk(buildTree(POINTS));
 return es;
})();

```

const sameP = (p: Pair, q: Pair) => p.key === q.key && p

const gpid = (p: Pair) => `\${p.key}:\${p.grade}`;

```

const acyclic = !edges.some((e) => reaches(edges, e.a)
const bst = forestIsBst(edges, all);
const roots = all.filter((p) => !edges.some((e) => gp
const complete = edges.length === all.length - 1 && r
return { heap, oneParent, acyclic, bst, complete, win
}

```

/\*\* Попытка хода: почему нельзя — или ОК. \*/

```

export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
 if (sameP(a, b)) return { ok: false, reason: "Это одна точка" };
 if (a.grade === b.grade)
 return { ok: false, reason: "«!» равны — «кто выше?»" };
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpId(child)))
 return { ok: false, reason: `У точки ${child.key};
 линию». ` };
 if (reaches(edges, parent, child))
 return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
 const next = [...edges, { a: parent, b: child }];
 if (!forestIsBst(next, POINTS))
 return { ok: false, reason: "Линия вниз по у допустима" };
 return { ok: true };
}

```

/\* ===== РЕЖИМ BUILD ===== \*/

```

type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

```

```

const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } {
 const pairs: Pair[] = [];
 const errors: string[] = [];
 const seen = new Set<string>();
 const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
 for (const chunk of chunks) {
 const nums = chunk.split(/\sxx*/+).filter(Boolea

```

```

 if (x.grade === y.grade) return { ok: false, reason:
 " };
 const [parent, child] = x.grade > y.grade ? [x, y] :
 if (edges.some((e) => pid(e.b) === pid(child)))
 return { ok: false, reason: `у точки ${child.key};
 // цикл: родитель уже сидит в компоненте ребёнка (реб
 const childComp = new Set<string>([pid(child)]);
 let grew = true;
 while (grew) {
 grew = false;
 for (const e of edges) {
 if (childComp.has(pid(e.b)) && !childComp.has(pid
 childComp.add(pid(e.a));
 grew = true;
 }
 }
 }
 if (childComp.has(pid(parent))) return { ok: false, r
 return {
 ok: true,
 edge: { a: parent, b: child },
 reason:
 x.grade > y.grade ? undefined : "линия в treap вс
 };
 }
}

```

```

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected:
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true,
 const heapOk = edges.every((e) => e.a.grade > e.b.gra
 const parentCount = new Map<string, number>();
 for (const e of edges) parentCount.set(pid(e.b), (par
 const singleParent = [...parentCount.values()].every(
 // связность: компоненты через union-find
 const comp = new Map<string, string>(points.map((p) =>

```

```

 if (sorted[0].key === sorted[1].key || sorted[0].key === sorted[1].key) {
 bstOk = false; // двое детей с равным x или с равным y
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяем, куда вставить
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
};
walk(roots[0]);
if (seen.size !== n) bstOk = false;
for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
}
return { heapOk, singleParent, connected, countOk, bstOk };
}

```

```

/** Дерево из проведённых линий – для показа результата
export function edgesToTree(edges: GameEdge[]): TNode | null {
 const parentOf = new Map<string, GameEdge>();
 for (const e of edges) parentOf.set(pid(e.b), e);
 const roots = edges.filter((e) => !parentOf.has(pid(e.a)));
 if (!roots.length && edges.length) return null;
 const nodes = new Map<string, TNode>();
 const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 grade: p.grade,
 left: null,
 right: null,
 });
 const rootPair = roots.length ? roots[0].a : edges[0].a;
 const root = make(rootPair);
 for (const e of edges) {
 const p = make(e);
 if (e.a === rootPair) root.left = p;
 else if (e.b === rootPair) root.right = p;
 else {
 const parent = nodes.get(pid(e.a));
 if (parent) {
 if (e.a < e.b) parent.left = p;
 else parent.right = p;
 }
 nodes.set(pid(e.b), p);
 }
 }
 return root;
}

```

```

const [sortCost, setSortCost] = useState<string | null>(null);

const parsed = useMemo(() => parsePairs(raw), [raw]);
const userPairs = parsed.errors.length ? [] : parsed.pairs;

const n = userPairs.length;

const userTree = useMemo(() => (userPairs.length >= 1 ?
const canonEdges = useMemo(() => {
 const es: { a: Pair; b: Pair }[] = [];
 const walk = (node: TNode | null) => {
 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, group: node.group, left: node.left }, b: node.left }); }
 if (node.right) { es.push({ a: { key: node.key, group: node.group, right: node.right }, b: node.right }); }
 };
 walk(userTree);
 return es;
}, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges].filter(e => e.a.group !== e.b.group));

useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return; }
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.length : leftRightEdges.length;
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 900);
 return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, canonEdges, leftRightEdges]);

```

```

const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 setBadEdge({ a: x, b: y });
 window.setTimeout(() => setBadEdge(null), 900);
 }
};

const pointDown = (p: Pair) => {
 if (phase !== "game") return;
 if (!sel) {
 setSel(p);
 setGameMsg(`Выбрана (${p.key}; ${p.grade}) — теперь`);
 } else if (sel.key === p.key && sel.grade === p.grade) {
 setSel(null);
 setGameMsg(null);
 } else {
 attemptConnect(sel, p);
 setSel(null);
 }
};

const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
};

const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (!sel) return;
 const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
 setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100,
 y: ((e.clientY - rect.top) / rect.height) * 100 });
};

```

```

 кто раньше в списке (сортировка ста
 {parsed.equalX && <div>✂ Есть равные x
 de.key → left).</div>}
 </div>
 })
</div>

{/* Координатная плоскость – НАВЕРХУ */}
<div className="bg-emerald-950/20 rounded-xl p-4" >
 <div className="flex flex-wrap items-center justify-center" >
 <h4 className="text-base font-bold text-emerald-400" >
 <div className="flex items-center gap-2" >
 {phase === "sort" ? (
 <>

 {["quicksort", "counting"] as const }.map((a) =>
 <button
 key={a}
 onClick={() => { setAlgo(a); setSort(a);
Math.round(n * Math.log2(n)) : 0} сравнений
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-white
600 text-white" : "bg-slate-700 text-slate-400`} >
 {a === "quicksort" ? "quicksort (ср
 </button>
)}}
 </>
) : (
 <>

 <button
 onClick={() => startConnect("y-desc")}
 className={`px-3 py-1.5 rounded-lg text-white
bg-emerald-600 text-white" : "bg-slate-700 text-slate-400`} >
 по y ↓ (алгоритм)
 </button>

```

```

 <> quicksort: счётчик сравнений: <b cl
{comparisons} ($n \cdot \log_2 n$).</>
) : (
 <> counting: точки в корзины по значен
, потому что у – маленькие целые.</>
)
) : (
 <> Отсортировано ({sortCost ?? "способ не
Игру </>
)
) : phase === "game" ? (
 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказа
ево: линии вниз по у, обход по х отсортирова
) : gameMsg ? (
 <> Δ {gameMsg}</>
) : (
 <> Кликни точку-родителя (выше по «!»), за
)
) : connDone ? (
 <> Проведено {connEdges.length} линий в по
: точки прибиты, дерево одно. А теперь
) : (
 <> Линий проведено: {connIdx} из {connEdgesA
)}
</div>
</div>

{/* Чек-лист инвариантов */}
<div className={`rounded-xl p-4 border transition
er-slate-700 bg-slate-900/40`} `>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {phase === "game" ? (
 <>
 {verdict.hearOk ? " " : " "} каждое р
 {verdict.singleParent ? " " : " "} у
 {verdict.connected ? " " : " "} все т

```



```

 })
 </div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SPLIT_KEYS = [2, 3, 6, 7, 9, 12, 45];

const SplitMode: React.FC = () => {
 const [x0, setX0] = useState(7);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const fullTree = useMemo(() => buildTree(POINTS), [])
 const split = useMemo(() => splitTree(buildTree(POINTS), live), [live]);
 const steps = split.trace;
 const idx = Math.min(stepIdx, steps.length - 1);

 // mid из кода выбирает разрез, step листает кадры сплит-деревьев
 useEffect(() => {
 if (live.mid === null || !SPLIT_KEYS.includes(live.mid)) {
 setX0(live.mid);
 setStepIdx(0);
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.mid]);
 useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.step, steps.length]);
 const done = idx >= steps.length - 1;

 const inL = new Set<NodeId>();
 const inR = new Set<NodeId>();
 steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? inL.add(s.id) : inR.add(s.id)));

 const lTree = split.l;
 const rTree = split.r;

```

```

 <h5 className="text-sm font-bold text-rose-500">
 <TreeView root={rTree} width={480} height={300}>
 </div>
 </div>
})

<div className="text-xs text-slate-400 bg-slate-900 p-4">
 Как это работает на самом деле: идём с
 деревом), отдаётся вместе с веткой – д
</div>
</div>
);
};

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const { l, r } = useMemo(() => splitTree(buildTree(P0, P1)), [P0, P1]);
 const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)), [l, r]);
 const steps = merged.trace;

 // step из кода листает кадры слияния
 useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.step, steps.length]);
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;
 const cur = steps[idx];

 return (
 <div className="space-y-4">
 <div className="flex items-center gap-3">
 <button onClick={() => setStepIdx(Math.max(0, idx - 1))}
 className="bg-slate-700 text-white disabled:opacity-30">

```

```
};
```

```
/* ===== РЕЖИМ ERASE ===== */
```

```
const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const fullTree = useMemo(() => buildTree(POINTS), [])
 const res = useMemo(() => eraseNode(buildTree(POINTS)
 const steps = res.trace;

 // key из кода выбирает удаляемую точку, step листает
 useEffect(() => {
 if (live.key === null || !POINTS.some((p) => p.key === key))
 setKey(live.key);
 setStepIdx(0);
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.key]);
 useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.step, steps.length]);
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;

 const pathIds = useMemo(() => {
 const ids: NodeId[] = [];
 for (const s of steps.slice(0, idx + 1)) if (s.atId !== null)
 return ids;
 }, [steps, idx]);
 const pathSet = new Set(pathIds);

 return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Удалить
 </div>
 </div>
);
};
```

```

 </div>
 </div>
);
 };
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
 const full = useMemo(() => buildTree(POINTS), []);
 // Честная слотовая разметка treap: индексы как в куче
 const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null)
 const walk = (n: TNode | null, i: number) => {
 if (!n || i >= 16) return;
 arr[i] = n.id;
 walk(n.left, 2 * i);
 walk(n.right, 2 * i + 1);
 };
 walk(full, 1);
 return arr;
 }, [full]);
 const nodeCount = slots.filter(Boolean).length;

 return (
 <div className="space-y-4">
 <div className="flex gap-2">
 <button onClick={() => setPick("heap")} className="
 ext-white" : "bg-slate-700 text-slate-300"`>
 <button onClick={() => setPick("treap")} className="
 text-white" : "bg-slate-700 text-slate-300">
 </div>

 {pick === "heap" ? (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300"

```

```

 Грабля «дети в массиве по $2k/2k+1$ »: форм
 ивым: слои дырявые, и «адрес» ребёнка посчита
 </div>
</div>
);
};

```

/\* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) ===== \*/

```

const SearchMode: React.FC = () => {
 // Подобрено так, что бинарный поиск ГАРАНТИРОВАННО пр
 // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежи
 const shuffled: Pair[] = [
 { key: 12, grade: 0 },
 { key: 3, grade: 6 },
 { key: 45, grade: 8 },
 { key: 9, grade: 3 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 7, grade: 2 },
 { key: 1, grade: -4 },
];
 const target = 7;
 const sortedByKey = [...shuffled].sort((a, b) => a.key
 const [stage, setStage] = useState<0 | 1 | 2>(0); //
 const [bidx, setBidx] = useState(0);

```

// шаги бинарного поиска на отсортированном массиве д

```

const bsSteps = useMemo(() => {
 const steps: { lo: number; hi: number; mid: number;
 let lo = 0, hi = sortedByKey.length - 1;
 while (lo <= hi) {
 const mid = Math.floor((lo + hi) / 2);
 if (sortedByKey[mid].key === target) { steps.push(
 нашли за ${steps.length + 1} сравнение(й).` });
 if (sortedByKey[mid].key < target) { steps.push({
 } – вся левая половина выброшена.` }); lo = mid
 else { steps.push({ lo, hi, mid, found: false, no

```

```

 <div className="text-slate-400 text-[10px]
 </div>
);
 }}}
</div>

<div className="text-xs text-slate-300 bg-slate-900"
 {stage === 0 && <>Массив пар не отсортирован
 а беспорядок.</>}</div>
 {stage === 1 && <> Бинарный поиск посмотрел на
 ю половину... в которой лежала цель. На
 реставил: массив как был в беспорядке, та
 {stage === 2 && bidx < bsSteps.length - 1 && <>
 {stage === 2 && bidx >= bsSteps.length - 1 && <>
 сравнения. Но чтобы поиск вообще заработал,
 сравнения на всех. «Отсортирую бинарни
</div>

<div className="text-xs text-slate-400 bg-slate-900"
 Есть ли разница, чем сортировать? Сравн
 няя граница, каждое сравнение даёт 1 бит. Но
 ля маленьких целых у (вкладка «Собрать», кноп
 задать порядок соединения.
</div>
</div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
 const [edges, setEdges] = useState<GEdge[]>([]);
 const [sel, setSel] = useState<Pair | null>(null);
 const [hover, setHover] = useState<Pair | null>(null);
 const [msg, setMsg] = useState<string>("Кликни точку-
 ");

 const v = gameVerdict(edges, POINTS);

```

```

<div className="bg-emerald-950/20 rounded-xl p-4" style={style} >
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} className="w-full h-full" >
 <defs>
 <pattern id="gm-grid" width="14" height="14">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1" />
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="black" />
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1" />
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PL_H - 16} stroke="black" stroke-width="1" />

 {edges.map((e, i) => (
 <line
 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}
 x2={PX(e.b.key)} y2={PY(e.b.grade)}
 stroke={i === edges.length - 1 ? "#34d399" : "black"}
 strokeWidth={i === edges.length - 1 ? 3.5 : 1}
 className="transition-all duration-300"
 />
))}

 {POINTS.map((p) => {
 const isSel = sel && sameP(sel, p);
 const isHover = hover && sameP(hover, p) && !isSel;
 return (
 <g key={`gp-${p.key}:${p.grade}`} onClick={onClick}
 l)) className="cursor-pointer">
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={12} fill="black" />
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={12} fill={
 "#a5b4fc" : isHover ? "#94a3b8" : "#38bdf8"
 } stroke="black" stroke-width="2" />
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} fill="white" font-size="10"
 space">{p.key}</text>
 <text x={PX(p.key)} y={PY(p.grade) + 24} fill="white" font-size="10"
 }</text>
 </g>
);
 })}
 </svg>

```

```

const vars = runtime?.variables;

const pairsText = useMemo(() => {
 if (!vars) return null;
 const rawPts = vizArray(vars.points) ?? vizArray(va
 if (!rawPts || rawPts.length === 0) return null;
 const parts: string[] = [];
 rawPts.forEach((it) => {
 if (Array.isArray(it) && it.length >= 2) {
 const x = vizNumber(it[0]);
 const y = vizNumber(it[1]);
 if (x !== null && y !== null) parts.push(`${x}
 } else if (typeof it === "string") {
 const m = it.match(/-?\d+(?:\.\d+)?/g);
 if (m && m.length >= 2) parts.push(`${m[0]} ${m
 }
 });
 return parts.length ? parts.join(", ") : null;
}, [vars]);

const mid = vars ? vizNumber(vars.mid) : null;
const key = vars ? vizNumber(vars.key) : null;
const step = vars ? vizNumber(vars.step) ?? vizNumber
const height = vars ? vizNumber(vars.height) ?? vizNum
const n = vars ? vizNumber(vars.n) : null;
return { vars, pairsText, mid, key, step, height, n }
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "lay

const TABS: { id: Tab; label: string }[] = [
 { id: "build", label: "Собрать" },
 { id: "split", label: "Split ∞ " },
 { id: "merge", label: "Merge" },
 { id: "erase", label: "Erase" },
 { id: "layers", label: "Миф: $2k/2k+1$ " },

```



```

 }}}
</div>

/* что сейчас сообщил компилятор */
<div className="mb-4 flex flex-wrap items-center">

 <Terminal className="w-3.5 h-3.5" /> из компилятора

 {live.vars ? (
 <>
 {live.pairsText &&
 points = {live.pairsText}
 }
 {live.n !== null &&
 n = {live.n}
 }
 {live.height !== null &&
 height = {live.height}
 }
 {live.mid !== null &&
 mid = {live.mid}
 }
 {live.key !== null &&
 key = {live.key}
 }
 {live.step !== null &&
 step = {live.step}
 }
 </>
) : (

 открой панель Python: переменные <code class="text-emerald-300">mid</code>, <code class="text-emerald-300">for</code>, <code class="text-emerald-300">step</code> управляют этой демонстрацией

)}
</div>

{tab === "build" && <BuildMode />}
{tab === "split" && <SplitMode />}
{tab === "merge" && <MergeMode />}
{tab === "erase" && <EraseMode />}
{tab === "layers" && <LayersMode />}
{tab === "search" && <SearchMode />}

```

```
const matrixSource = pickLiveVar(vars, MATRIX_NAMES);
const matrix = liveMatrix(matrixSource?.value);

const detail = matrix
 ? `${matrixSource?.name} ${matrix.sizeLabel}`
 : names.length
 ? names.slice(0, 6).join(", ") + (names.length > 6 ? "..." : "")
 : undefined;

return (
 <VizSourceBadge
 linked={linked}
 awaiting={!linked && !!runtime}
 line={runtime?.line}
 detail={detail}
 demoTitle={demoTitle}
 className={className}
 />
);
}
```

---

ФАЙЛ 80/111: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
```

```

 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы — куча сама держит лучшую",
 Component: HeapViz,
 },
 "segment-trees": {
 title: "Дерево отрезков",
 hint: "Запросы и обновления на отрезке за $O(\log n)$.",
 Component: SegmentTreeVisualizer,
 },
 "sparse-table": {
 title: "Разреженная таблица (1D и 2D)",
 hint: "Наведите курсор на любую ячейку — подсветится",
 Component: SparseTableViz,
 },
 treap: {
 title: "Treap: плоскость → дерево → Split/Merge/Era",
 hint: "7 вкладок: Собрать (сортировка + соединение) →

 обери дерево мышкой, судья-инварианты объясняет к",
 Component: TreapBuildViz,
 },
 "prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число — увидите, из чего оно слож",
 Component: PrefixSumViz,
 },

```

```

 title: "Остовное дерево: Краскал, Прима, Борувка",
 hint: "Три алгоритма (билеты 18–20) на одном графе. Переменные parent/mst/taken/u/v/w перекрашивают граф.",
 Component: KruskalSimulator,
 },
 "graph-components": {
 title: "Компоненты связности: обход и DSU",
 hint: "Две вкладки: покраска обходом (каждый новый компонент из вашего кода красят вершины.",
 Component: GraphComponentsViz,
 },
 "complexity-classes": {
 title: "Классы сложности и сведения",
 hint: "Иерархия $P \subseteq NP \subseteq PSPACE \subseteq EXP$, сведение $A \leq B$ ",
 Component: ComplexityViz,
 },
 "string-kmp": { title: "Префикс-функция (КМП)", Component: () },
 "string-z-func": { title: "Z-функция", Component: () },
 "aho-corasick": {
 title: "Ахо–Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => (
 <div className="space-y-10">
 <WaterfallAnimationWidget />
 <SalmonAutomatonWidget />
 </div>
),
 },
 intro: {
 title: "Интерактивный симулятор базовых структур: стек, очередь, куча",
 hint: "Проверь LIFO, FIFO и приоритетную очередь на практике.",
 Component: Simulator,
 },
 "everyday-basics": {
 title: "Бытовой тренажёр: стек, очередь, куча",
 hint: "Тарелки, очередь в столовой и мешок гипотез",
 Component: Simulator,
 },

```

```

*
* Без него демонстрация молча проигрывала собственный
* когда редактор Python пуст или в нём другие имена/ра
* так, будто компилятор «исполняет какой-то свой код».
* «из компилятора» или «встроенное демо».
*/
export function VizSourceBadge({ linked, awaiting = false }) {
 const tip = linked
 ? `Значения берутся из выполненного Python-кода${demoTitle}
 : awaiting
 ? `Панель Python выполнила код и стоит на строке
 нажмите «шаг вперёд» (→) или «в конец трассы» —
 лизация их не увидит: список имён этой темы есть
 : "Сейчас показана встроенная демонстрация компонента
 визуализация переключится на ваши переменные.";

 return (
 <div className={`flex flex-wrap items-center gap-1.5`} >
 <Tooltip side="bottom" content={tip}>
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center gap-1.5`}
 linked
 ? "border-emerald-500/50 bg-emerald-500/10"
 : awaiting
 ? "border-sky-500/50 bg-sky-500/10 text-slate-800"
 : "border-slate-600 bg-slate-800/80 text-slate-800"
 >

 </div>
 {linked ? <Cpu className="h-3 w-3" /> : awaiting}
 {linked ? "данные из компилятора" : awaiting}
 {linked && detail ? : ""}
 </div>
)
}

{!linked && !awaiting && demoTitle ? (

 Встроенная демонстрация

) : ""}

```

```

 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 }

const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0,
 // Бетка H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1,
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2,
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3,
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4,
 // Бетка H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2,
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3,
 // Бетка S -> H -> E
 7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1,
 8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2,
 9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3,
 = 2 (HE)
 };

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: { H: 8 },
 8: { E: 9 },
 9: {},
};

export const WaterfallAnimationWidget: React.FC = () =>
```

```

 if (liveState !== null && liveState >= 0 && WATERFA
 if (liveI !== null && liveI >= 0) setCurrentIndex(M
 ToScan).length));
 if (liveMatchesSig) {
 const parsed = (JSON.parse(liveMatchesSig) as unk
 .map((it) => {
 if (Array.isArray(it) && it.length >= 2) {
 const idx = vizNumber(it[0]);
 const word = vizString(it[1]);
 if (idx !== null && word) return { index: i
 }
 if (typeof it === "string") {
 const m = it.match(/(\d+)\D+([A-Za-zA-Яа-я]
 if (m) return { index: Number(m[1]), word:
 }
 return null;
 })
 .filter((x): x is { index: number; word: string
 if (parsed.length) setFoundMatches(parsed);
 }
 // eslint-disable-next-line react-hooks/exhaustive-
}, [vars]));

```

```

const compilerLinked = Boolean(liveText || (liveState

```

```

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===

```

```

const [textToScan, setTextToScan] = useState<string>(
const [currentIndex, setCurrentIndex] = useState<numb
const [currentNode, setCurrentNode] = useState<numbe
const [isSearchDone, setIsSearchDone] = useState<bool
const [searchLog, setSearchLog] = useState<string>(
 'Режим поиска: Введите текст и жмите «Следующий шаг
);

```

```

const [jumpPath, setJumpPath] = useState<{ from: numb
const [foundMatches, setFoundMatches] = useState<{ in
const [activeSearchCodeLine, setActiveSearchCodeLine]

```

```

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===

```

```

 _NODES[nextNode].label})). `;
 setJumpPath({ from: curr, to: nextNode, type: 'no
 curr = nextNode;
 setActiveSearchCodeLine(3);
} else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char]
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Л
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${orig
 рывок по fail-ссылке в #${curr} (${WATERFALL_N
 . `;
 setJumpPath({ from: originalCurr, to: nextNode,
 }
 curr = nextNode;
}

const currentMatches: { index: number; word: string
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) :
 currentMatches.push({ index: currentInd
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
}

```





```

 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. Роль
#7 (S). fail[HERS] = #7 (S). Автомат полностью обучен',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ? currentTrainInfo : null;
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNodeId];

```

```

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border-1px border-slate-700">
 {/* Шапка и переключатель режимов */}
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white">
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400">
 Визуализация пошагового построения всех функций
 </p>
 </div>
 </div>
 </div>

 {/* Переключатель режимов */}
 <div className="bg-slate-900 p-1.5 rounded-xl border-1px border-slate-800">
 <button
 onClick={() => setActiveMode('training')}

```

```

 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 p-2`}
 >
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font-medium'
 : 'bg-slate-800 text-slate-400'
 }
 >
 {step}
 {idx === trainStep &&
 </button>
)}}
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => Math.max(0, prev - 1))}
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg-slate-600 transition-all"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => Math.min(TRAINING_STEPS_INFO.length - 1, prev + 1))}
 disabled={trainStep === TRAINING_STEPS_INFO.length - 1}
 className="flex-1 bg-indigo-600 hover:bg-indigo-500 transition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.length - 1 ? 'Последний шаг' : 'Далее'}
 </button>
 </div>
 </div>
)

```

```

 </div>
 <div className={`p-1.5 rounded flex items-center justify-between border-blue-500 text-blue-300 font-bold'
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
 root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold mb-2">
 <Lightbulb className="w-4 h-4 text-amber-400"/>
 </h5>
 <div className="bg-slate-800 p-4 rounded-lg w-full min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
</div>
</div>
</div>
) : (
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3">
 <div className="bg-slate-900/60 p-5 rounded-x">
 <div>
 <h5 className="text-white font-bold mb-2">
 ⚡ Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4">
 Введи текст, чтобы лосось просканировал
 </p>
 <input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."

```

```

 >
 {currentIndex === 0 ? 'Начать пла
 <ArrowRight className="w-4 h-4" />
 </button>
</div>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60" >
 <div>
 <h5 className="text-white font-bold mb-3" >
 ❗ Текущее действие в коде
 </h5>
 <div className="bg-slate-950 p-4 rounded-1" >
 <div className={`p-1.5 rounded flex item
border-blue-500 text-blue-300 font-bold` :
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && И
инициализация}
 </div>
 <div className={`p-1.5 rounded flex item
border-rose-500 text-rose-300 font-bold an
 2. while (curr !== root && !trie
 {activeSearchCodeLine === 2 && П
рыжок по fail!}
 </div>
 <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
 3. curr = trie[curr].get(char)
 {activeSearchCodeLine === 3 && П
ем по течению}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
 4. if (is_terminal[curr]) report
 {activeSearchCodeLine === 4 && П
овпадение!}
 </div>
 </div>
 </div>

```

```

 </div>
 </div>

 { /* Само SVG дерево водопада */ }
 <div className="w-full overflow-x-auto custom-s<
 <svg viewBox="0 0 800 520" className="w-full

 { /* Горизонтальные линии и подписи уровней
 {[
 { depth: 0, y: 60, label: 'Depth 0 (Корень
 { depth: 1, y: 160, label: 'Depth 1 (H, S
 { depth: 2, y: 260, label: 'Depth 2 (HE, I
 { depth: 3, y: 360, label: 'Depth 3 (HER,
 { depth: 4, y: 460, label: 'Depth 4 (HERS
]}.map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14
 />
 <text x="32" y={level.y + 4} fill="#94a
 {level.label}
 </text>
 </g>
 }}}

 { /* Рендер прямых переходов (потоков воды)
 {Object.entries(TRIE_TRANSITIONS).map(([from
 const fromNode = WATERFALL_NODES[Number(f
 return Object.entries(children).map(([cha

```



```

r={isCurrent || isTargetChild ? '24' : ''}
fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : ''}
stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : ''}
strokeWidth={isCurrent || isTargetChild ? 1 : 0}
className="transition-all duration-300"
/>

```

```

{/* Текст внутри вершины */}

```

```

<text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ? 'white' : 'black'}
 fontSize={isCurrent || isTargetChild ? 12 : 10}
 fontWeight="bold"
 textAnchor="middle"
>
 {node.label}
</text>

```

```

{/* Метка искомой вершины в режиме обхода */}

```

```

{isTargetChild && (
 <g transform={`translate(${node.x}, ${node.y})`}
 <rect x="-35" y="-12" width="70" height="24" fill={fill} stroke={stroke} strokeWidth={strokeWidth}
 <text x="0" y="2" fontSize="10" fill="white"
 ИЩЕМ FAIL
 </text>
 </g>
)}

```

```

{/* Индикатор словарных слов */}

```

```

{hasWords && (
 <g transform={`translate(${node.x + 10}, ${node.y + 10})`}
 <circle cx="0" cy="0" r="9" fill={fill} stroke={stroke} strokeWidth={strokeWidth}
 <text x="0" y="3" fontSize="10" fill="white"
 *
 </text>
 </g>
)}

```



```

 Улов лосося (Найденные слова
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic">
 Пока ничего не поймали. Запустите шаги сирены
 </p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border border-emerald-500 p-2 text-white text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]">
 <CheckCircle2 className="w-4 h-4 text-emerald-500" />
 {match.word}

 Индекс {match.index}

 </div>
 </div>
))}
 </div>
)}
</div>
)}

/* что сейчас сообщил компилятор */
<div className="mt-6 flex flex-wrap items-center">

 <Terminal className="w-3.5 h-3.5" /> из компилятора

 {compilerLinked ? (
 <div>
 {liveText &&
 >text = {liveText}
 }
 {liveI !== null &&
 py-1">i = {liveI}{liveText ? ` (${liveText}` : ''}
 }
 {liveState !== null && (

```

```

 vizNumber,
 VizChapterContext,
 } from "../../data/vizStepBus";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка — наведение на число:
 * * в 1D наводим на P[i] — подсвечивается кусок массива
 * * в 1D наводим на a[i] — видно, в какие префиксы он
 * * в 2D наводим на ячейку — подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];

const cellBase =
 "flex items-center justify-center rounded-md font-monospace" +
 "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 const chapterId = useContext(VizChapterContext);

 // Вкладка 2D = своё демо (свой скелет): компилятор prefixsum
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode === "2d" ? "2d" : "1d");
 }, [chapterId, mode]);

 return (
 <div className="w-full bg-slate-950 rounded-2xl border" />
)
}

```

```

const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, 'liveP');
const compilerLinked = liveI !== null || hasLiveP;
// i/P управляют ячейками напрямую; индекс Python-строки
const shownDriven = compilerLinked
 ? liveI !== null
 ? Math.max(0, Math.min(A1.length, Math.trunc(liveI / 52)))
 : Math.max(0, Math.min(A1.length, (liveP?.length / 52)))
 : null;

const covered =
 hover?.kind === "pref"
 ? { from: 0, to: hover.i - 1 }
 : hover?.kind === "a"
 ? { from: hover.i, to: hover.i }
 : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 /* индексы */
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={` ${cellBase} !h-5`}>
 {i}
 </div>
))}
 </div>

 /* массив a */
 <div className="flex gap-1.5 items-center mb-1">
 <div className="w-[46px] shrink-0 text-right">
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.from && i <= covered.to;
 const inRange = i >= range.l && i <= range.r;
 const isDrivenAdd = shownDriven !== null && i === shownDriven;
 })}
 </div>
 </div>
 </div>
 </div>
)
)

```

```

onMouseEnter={() => setHover({ kind:
onMouseLeave={() => setHover(null)}
className={` ${cellBase} cursor-help $
 isDrivenCur
 ? "bg-emerald-600 text-white ring
 : blank
 ? "bg-slate-950 text-slate-600
 : masked
 ? "bg-slate-800/50 text-slate-700
 : active
 ? "bg-emerald-500 text-white
 : isR
 ? "bg-emerald-700 text-white
 : isL
 ? "bg-rose-700 text-white
 : "bg-slate-900 text-slate-300
 }`}
title={blank || masked ? `P[${i}] - n
>
{blank || masked ? "." : String(shown
</div>
);
 }}}
</div>
</div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border
 {shownDriven !== null && !hover ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">Компилятор
 i =
 <span className="font-mono text-emerald-300
 {hasLiveP && <> = <span className="font-mono
 Пустой
 </p>
) : hover?.kind === "pref" ? (

```

```

 <p className="font-mono text-sm sm:text-base text-emerald-400">

$$\sum_{i=a[l]..a[r]} P[r + 1]$$

 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за $O(1)$ по
 </p>
 </div>
</div>
);
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveI = vizNumber(vars?.i);
 const liveJ = vizNumber(vars?.j);

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array<number>().length);
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1];
 }
 }
 return p;
 }, [n, m]);

 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
 const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

 /**
 * Матрицы из кода пользователя рисуются В СВОЕЙ размерности

```

```

const r2 = vizNumber(vars?.r2);
const c2 = vizNumber(vars?.c2);
if (r1 === null || c1 === null || r2 === null || c2
 setRect({
 r1: Math.max(0, Math.min(srcRows - 1, Math.trunc(
 c1: Math.max(0, Math.min(srcCols - 1, Math.trunc(
 r2: Math.max(0, Math.min(srcRows - 1, Math.trunc(
 c2: Math.max(0, Math.min(srcCols - 1, Math.trunc(
 }));
}, [vars, srcRows, srcCols]));

// Границы запроса держим внутри реальной сетки (у по
const r1 = Math.max(0, Math.min(rect.r1, prefRows - 1
const c1 = Math.max(0, Math.min(rect.c1, prefCols - 1
const r2 = Math.max(r1, Math.min(rect.r2, prefRows -
const c2 = Math.max(c1, Math.min(rect.c2, prefCols -

const D = cellAt(r1, c1);
const B = cellAt(r1, c2 + 1);
const Cc = cellAt(r2 + 1, c1);
const Aa = cellAt(r2 + 1, c2 + 1);
const total = Aa !== null && B !== null && Cc !== nul
const fmt = (v: number | null) => (v === null ? "-" :

const cornerOf = (i: number, j: number) => {
 if (i === r2 + 1 && j === c2 + 1) return "A";
 if (i === r1 && j === c2 + 1) return "B";
 if (i === r2 + 1 && j === c1) return "C";
 if (i === r1 && j === c1) return "D";
 return null;
};

return (
 <div className="space-y-5">
 {sizeMismatch ? (
 <p className="rounded-lg border border-amber-500
 {sizeMismatch}
 </p>

```

```

<div className="text-xs text-slate-400 mb-2">
 Префиксные суммы {liveS ? `${livePrefSource
</div>
<div className="inline-block">
 {grid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const blank = v === null || v === unde
 const shownValue = v;
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hov
 const isCompilerCell = liveI === i &&
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border
 return {
 <div
 key={j}
 onMouseEnter={() => setHover({ i,
 onMouseLeave={() => setHover(null
 className={`${cellBase} cursor-he
 isCompilerCell
 ? "bg-emerald-600 text-white
 : isHover
 ? "bg-amber-500 text-white
 : blank
 ? "bg-slate-950 text-slat
 : cornerColor
 }`}
 title={hasLiveS ? `${livePrefSour
 >
 {blank ? "." : String(shownValue)}
 {corner && !isHover && (

```

```
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>
</div>
</div>
);
};
```

---

ФАЙЛ 84/111: src/components/visualizers/SparseTableViz.  
КАТЕГОРИЯ: Визуализаторы (вложенные) | СТР0К: 899 | РАЗМ: 1000

---

```
import React, { useContext, useEffect, useRef, useState } from "react";
import {
 emitVizDemo,
 liveMatrix,
 useVizRuntime,
 vizArray,
 vizNumber,
 vizRecord,
 VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;
```



```

 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx-1][ky],
 ST2D[r + (1 << (kx-1))][c][kx-1][ky]
);
 } else {
 ST2D[r][c][kx][ky] = Math.min(
 ST2D[r][c][kx][ky-1],
 ST2D[r][c + (1 << (ky-1))][kx][ky-1]
);
 }
 }
 }
}

```

```

export const SparseTableViz: React.FC = () => {
 // Эта страница посвящена именно двумерной таблице: 1D
 // вкладкой, но больше не подменяет 2D-визуализацию и
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d_build2">("1d");
 const chapterId = useContext(VizChapterContext);

 // Каждая вкладка = своё демо с собственным скелетом:
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab === "2d_build" ? "2d_build2" : tab);
 }, [chapterId, tab]);

 return (
 <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
 <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
 <button
 onClick={() => setTab("1d")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white : "bg-slate-800 text-slate-400"
 >
 1. Сворачивание 1D
 </button>
 </div>
 </div>
);
}

```

```

const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'liveSource');
const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'liveTable');
const compilerLinked = liveI !== null || liveJ !== null;
const shownSource = hasLiveSource
 ? liveSource?.length
 ? liveSource
 : Array.from({ length: N }, () => null)
 : arr1D;
const liveTarget =
 liveI !== null && liveJ !== null && Number.isInteger(liveI) && Number.isInteger(liveJ)
 ? { i: liveI, j: liveJ }
 : null;

/** Размерность таблицы пользователя: сетка демо 8x4,
const liveTable = liveMatrix(runtimeVars?.st);
const dimMismatch = liveTable
 ? liveTable.rows !== N || liveTable.cols !== LOG
 ? `Демо-сетка построена для массива из ${N} элементов. Таблица ниже рисует ячейки демо-сетки: значения элементов в панели переменных компилятора.`
 : null
 : hasLiveSource && liveSource && liveSource.length
 ? `Демо-массив из ${N} элементов, а в вашем коде используется демо-размера ${N}x${LOG}.`
 : null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;

```

```

 </button>
 </div>
 </div>

 <div className="bg-slate-900 rounded-xl border bo
 <div className="text-[11px] uppercase tracking-t
 <div className="flex gap-1 sm:gap-1.5 overflow-:
 {shownSource.map((v, idx) => {
 const inCover = !!covered && idx >= covered
 const display = v === undefined || v === nu
 ? "."
 : typeof v === "object"
 ? JSON.stringify(v)
 : String(v);
 return (
 <div key={idx} className="flex flex-col i
 <div
 className={`flex items-center justify
w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
 inCover ? "bg-indigo-500 text-white
 `}
 >
 {display}
 </div>
 <span className="text-[9px] text-slate-1
 </div>
)};
)}}
 </div>
 <div className="mt-2 text-xs min-h-[36px]">
 {hover && covered ? (
 <p className="text-slate-300">
 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {compilerLin
 j}}
 {" "}
 – это минимум на отрезке{" "}


```

```

 { /* Matrix */ }
 { Array.from({ length: N }).map((_, i) => (
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mono" style={fontStyle} >
 {i}
 </div>
 { Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;
 const liveRow = vizArray(liveRows?.[i]);
 const liveValue = liveRow?.[j];
 // Уровень j=0 – это исходный a[i], от которого берется
 // очищаемого рабочего массива и не должен быть вычислен
 const sourceValue = j === 0 ? liveSource[i] : liveValue;
 const runtimeValue = liveValue ?? sourceValue;
 const runtimeHasValue = runtimeValue != null;
 const manualVisible =
 isValid &&
 (j === 0 || computeSteps.findIndex((s) => s.i === i) < j);
 const isVisible = compilerLinked ? runtimeHasValue : manualVisible;

 let isActive = !!liveTarget && liveTarget.i === i &&
 !compilerLinked;
 let isSrc1 = false;
 let isSrc2 = false;

 if (compilerLinked && liveTarget && liveTarget.i === i &&
 !compilerLinked) {
 isSrc1 = i === liveTarget.i && j === 0;
 isSrc2 = i === liveTarget.i + (1 << (j - 1)) && j > 0;
 } else if (!compilerLinked && step > 0) {
 const currentStep = computeSteps[step - 1];
 isActive = currentStep.i === i && currentStep.j === j;
 if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 = true;
 if (currentStep.i + (1 << (currentStep.j - j)) === i) isSrc2 = true;
 }
 }
 return (
 <div>
 {runtimeValue}
 </div>
);
 }) }
 </React.Fragment>
) }
);
}

// Готовые вычисленные уровни демо не

```

```

 ? "bg-slate-950 to-transparent"
 : "bg-transparent"
)` }
 title={compilerLinked ? `st[${id}]` : ""}
 >
 {displayValue}
 </motion.div>
)}

 {/* Arrows visualization */}
 {(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 << 2) : 40,
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc1 ? 40 : 0} 0 0`}
 fill="none"
 stroke={isSrc1 ? "#10b981" : "#10b981"}
 strokeWidth="3"
 />
 </motion.svg>
)}
</div>
);
}})
</React.Fragment>
)}}
</div>
</div>

```

```

 </p>
);
 }}()
) : (
 <p className="text-slate-500 italic">
 Нажмите далее, чтобы увидеть, как блоки сво
 </p>
)}
</div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: number}>({ r: 0, c: 0 });
 const [locked, setLocked] = useState<{ r: number; c: number}>({ r: 0, c: 0 });
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveK = vizNumber(vars?.k);
 /**
 * Шаг, закреплённый кликом по вкладке. Без него live
 * перетирал выбор пользователя и вкладки «Шаг 1/2/3»
 * клик есть, а матрица не меняется. Закрепление живё
 */
 const [pinnedK, setPinnedK] = useState<number | null>(null);
 const prevLiveK = useRef<number | null>({ current: liveK });
 useEffect(() => {
 if (liveK !== prevLiveK.current) {
 prevLiveK.current = liveK;
 setPinnedK(null);
 }
 }, [liveK]);
 const liveR = vizNumber(vars?.r);
 const liveC = vizNumber(vars?.c);
 const liveTable = vizRecord(vars?.st2);
 const hasLiveSource = !!vars && Object.prototype.hasOwnProperty.call(vars, 'source');
 const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'table');

```

```

</p>
{dimMismatch2D ? {
 <p className="mb-4 max-w-[520px] rounded-lg b
 {dimMismatch2D}
 </p>
} : null}

<div className="flex bg-slate-950 p-1 rounded-l
 {[0, 1, 2, 3].map(step => {
 <button
 key={step}
 onClick={() => { setK(step); setPinnedK(s
 className={`px-4 py-2 rounded-md text-sm
 'text-slate-400 hover:text-white hover:bg-
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${s
 </button>
 })}
 </div>
{pinnedK !== null && compilerLinked && {
 <p className="-mt-6 mb-6 text-[11px] text-sla
 Шаг закреплён вручную: когда <span classNam
 демонстрация вернётся к шагу кода.
 </p>
})}

<div className="flex flex-col items-center gap-
 scrollbar">
 {[...Array(shownK + 1)].map((_, i) => shownK
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
 const isCurr = step === shownK;

 return {
 <React.Fragment key={step}>
 {idx > 0 && <div className="text-slate
 <div className={`flex flex-col items-c
 <h4 className={`font-bold mb-3 flex

```

```

=====
== c ? 'ring-[3px] ring-white ring-offset-2
 : `bg-indigo-500 text-whi
&& locked.c === c ? 'ring-[3px] ring-white
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (shownK
 const isTop = r < activeCel
 const isLeft = c < activeCe

 if (isTop && isLeft) {
 highlightClass = 'bg-ros
md:scale-[1.08]';
 } else if (isTop && !isLeft
 highlightClass = 'bg-blur
] md:scale-[1.08]';
 } else if (!isTop && isLeft
 highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
 } else {
 highlightClass = 'bg-amb
3)] md:scale-[1.08]';
 }
 zIndex = 'z-10';
 }
 }

 return (
 <div
 key={idx}
 onMouseEnter={() => { if(is
 onMouseLeave={() => { if(is
 onClick={() => {
 if(isCurr) {
 if(locked && locked.
 else setLocked({r, c
 }
 }}
 className={`flex items-centr

```



```

 <div className="bg-[#0a0fle] rounded
 /

 </div>
 <p className="mt-6 italic text-indigo
 <Play size={14} className="fill-in
a.
 </p>
 </>
)}
</div>
) : (
 <div className="flex flex-col gap-4 animat
 <div className="bg-[#0a0fle] rounded-xl
ate-300 shadow-inner overflow-x-auto">
 // Те
 int<
="text-amber-300">1}

 // Кв
 ST</
 <span className="text-rose
e="text-amber-300">1], <sp
 <span className="text-blue
ext-amber-300">1][k-<span className=
pan>

 <span className="text-emerg
="text-amber-300">1][k-<span classNa
span>

 <span className="text-ambe
white">c + pL][k-<span className="te
ame="text-slate-500">// Правый Низ<b
 {'}}'));
 </div>

 {activeCell ? (
 <div className="bg-slate-950 p-5 round
.2)] mt-2 transition-all">
 <p className="text-slate-300 mb-3 t

```

```

 </div>
 </>
)
 }}({})
</div>

 <div className="pt-3 mt-3 border-t border-l border-r border-b"
 > <span className="text-white"
0">{ST2D[activeCell.r][activeCell.c][shownK
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L}
 </div>
)}
</div>
)}
</div>
</div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);
 const runtime = useVizRuntime();

```

```

// Координаты запроса – те же захардкоженные имена,
useEffect(() => {
 const vars = runtime?.variables;
 const nr1 = vizNumber(vars?.r1);

```

```

 <div className="flex flex-col items-center"
 <div className="grid grid-cols-[repeat(
 active shadow-inner w-max max-w-full">
 {Array.from({length: 64}).map((_, i) => {
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r <= r2 && c >= c1 && c <= c2;

 return (
 <div key={idx} className={`text-${
 y-center rounded text-[clamp(9px,2.4vw,12px)]
 0 border-rose-500/50 border scale-105 z-10`
 {val2D[r][c]}
 </div>
)
 })}
 </div>
 <div
 className="absolute border-[3px solid #f08080]
 ow-[0_0_15px_rgba(244,63,94,0.3)] border-dashed
 style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) + 'px)',
 width: 'calc(' + (W/8*100) + 'px)',
 }}
 />

```

```

 {/* Показываем 4 угла в самой матрице */}
 <div className="absolute pointer-events-none
 shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
 lc(${(Ly / 8) * 100}% - 16px)`, height: `calc(
 <div className="absolute pointer-events-none
 shadow-[0_0_10px_#3b82f6]" style={{ top: `calc(
 width: `calc(${(Ly / 8) * 100}% - 16px)`, height: `calc(
 <div className="absolute pointer-events-none

```

```

 <p className="text-sm font-sans text-slate-500">
 Чтобы покрыть весь прямоугольник, мы
 Name="font-mono bg-[#0a0f1e] px-1 rounded b
 роса. (Цветные прямоугольники слева).
 </p>

 <div className="bg-[#0a0f1e] rounded-xl
 text-slate-300 shadow-inner overflow-x-auto
 <div className="absolute right-3 top-3
 // 1.
 int<
 ext-amber-300">1}; <span className="
 int<
 ext-amber-300">1}; <span className="
 int<
 "text-slate-500">// Выс: {Lx}

 int<
 "text-slate-500">// Шир: {Ly}
<b

 // 2.
 int<

 В запроса)

 r1][<span className="text-white bg-r

 во, чтобы правый край уперся в c2)<b

 r1][<span className="text-white bg-b

 px, чтобы нижний край уперся в r2)<b

 nded">r2 - Lx + 1][<span className="

 и влево от c2)


```

```

 {badge && <div className=
r-slate-700 text-white px-1 font-bold rounded
 </div>
)
 }}}}
 </div>
</div>
</div>

 <div className="mt-auto bg-slate-950 font
)]">
 <div className="text-center font-bold
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg bord
 <span className="text-slate-400 font

 <span className="text-blue-400 m
n>
 <span className="text-emerald-40
span>
 <span className="text-amber-400 r
 <span className="text-slate-4
 <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>

 </div>
</div>
)
}

```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

```

 if (wtfContainer) wtfContainer.classList
 }
}

```

```

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 document.querySelectorAll('[id^="theme-"]')
 btn.classList.remove('outline', 'outline
 });
}

```

```

if (theme === 'dark') document.getElementById('
if (theme === 'light') document.getElementById('
if (theme === 'terminal') document.getElementById('
);

```

```

document.getElementById('btn-theme-dark').c
t-white transition-colors' : 'px-3 py-1 rou
n-colors';
document.getElementById('btn-theme-light').c
ext-white transition-colors' : 'px-3 py-1 r
ion-colors';
document.getElementById('btn-theme-notebook
-600 text-white transition-colors' : 'px-3 p
ransition-colors';

```

```

}

```

```

// Initialize theme and mode
setTheme('dark');

```

```

// Setup intersection observer for TOC highligh
document.addEventListener('DOMContentLoaded', (
 const mobileMenuBtn = document.getElementById('
 const mobileDrawer = document.getElementById('
 const closeDrawerBtn = document.getElementById('

```

```

 });
 });

 const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
 };

 const observer = new IntersectionObserver((
 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link lo
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href
 link.classList.add('bg-
 } else {
 link.classList.remove('
 }
 });
 }
 }));
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
});
</script>
</body>
</html>

```

```
/* Global formula scroll settings */
.formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
}
.formula-scroll::-webkit-scrollbar {
 display: none;
}

mjx-container {
 max-width: 100% !important;
}
mjx-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
}
mjx-container[display="true"]::-webkit-scrollbar {
 display: none;
}

@media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-size: 10px; }
 .uno-card::before, .uno-card::after { font-size: 10px; }
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; height: 28px !important; font-size: 4px; }
}
```



```

body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-color:
body.theme-light .bg-slate-700, body.theme-light
 important; }
body.theme-light .text-slate-200, body.theme-light
body.theme-light .text-slate-400 { color: #475568; }
body.theme-light .border-slate-600, body.theme-light
body.theme-light .border-slate-500 { border-color:
body.theme-light #top-controls { background-color:
}

body.theme-notebook {
 background-color: #f8e8e8 !important; /* light pink */
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient(45deg,
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook
 .bg-slate-700 {
 background-color: rgba(255, 255, 255, 0.7);
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook
 .text-slate-400 { color: #3b2818; }
body.theme-notebook .text-slate-400 { color: #546e7a; }
body.theme-notebook .border-slate-600, body.theme-notebook
 .border-slate-500 { border-color: #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "Monospace"; }
body.theme-notebook #top-controls { background-color: #f8e8e8; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1); }
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {

```

```

 over:text-white hover:bg-slate-700 transition
 </div>
 </div>
 </div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d"
 </marker>
 <marker id="arrow-orange" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#f97316"
 </marker>
 <marker id="arrow-pink" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b6"
 </marker>
 <marker id="arrow-cyan" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf"
 </marker>
 <marker id="arrow-indigo" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8"
 </marker>
 <marker id="arrow-lime" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63f"
 </marker>
 <marker id="arrow-green" markerWidth="6" ma
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade88"
 </marker>

 <!-- ЛЕГО БЛОКИ -->
 <g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <g id="lego-red">

```



```


4. Степени и корни

 </summary>
 <div class="pl-6 space-y-2 text-amber-500">

 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-fuchsia-500 pl-3 font-bold text-fuchsia-300">
5. Группа корней из единицы

 </summary>
 <div class="pl-6 space-y-2 text-fuchsia-500">

 </div>
</details>

 <details class="group">
 <summary class="list-none cursor-pointer text-lime-500 pl-3 font-bold text-lime-300">
6. Делимость, НОД и Евклид

 </summary>
 <div class="pl-6 space-y-2 text-lime-500">
```

```


 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-emerald-green-500 pl-3 font-bold text-green-400">
 9. Кольцо многочленов

</summary>
 <div class="pl-6 space-y-2 text-emerald-green-500">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Гильберта

</summary>
 <div class="pl-6 space-y-2 text-emerald-green-500">

 </div>
</details>

<details class="group">
```

```

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q13-1" class="blo
 <a href="#q13-2" class="blo
 <a href="#q13-3" class="blo
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-14" clas
der-fuchsia-500 pl-3 font-bold text-fuchsia
 14. Кратность корня. Ко

 </summary>
 <div class="pl-6 mt-2 space-y-2
 <a href="#q14-1" class="blo
 <a href="#q14-2" class="blo
 <a href="#q14-3" class="blo
 deg)
 <a href="#q14-4" class="blo
 ition-colors border border-rose-500/30"> K
 </div>
 </details>

 <details class="group">
 <summary class="list-none curso
 <a href="#question-15" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критер

 </summary>
 <div class="pl-6 mt-2 space-y-2
 <a href="#q15-1" class="blo
 <a href="#q15-2" class="blo
 <a href="#q15-3" class="blo
 <a href="#q15-4" class="blo
 </div>
 </details>

```

```
sia-500 pl-3 text-slate-400">
 <span class="font-bold text-fuch

```

```
 <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m
 ▲ Ликбез: Алгоритмы выживания

```

```
</nav>
```

```
<script>
```

```
 document.querySelectorAll('details.'
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelectorA
 if (other !== detail
 other.removeAtt
 }
 });
 }
 });
 });
```

```
 // Scroll spy logic
 document.addEventListener('DOMConten
 const sections = document.query
 ="q5-"], div[id^="q6-"], div[id^="q7-"], di
 v[id^="q13-"], div[id^="q14-"], div[id^="q1
 let isScrolling = false;
```

```
 document.querySelectorAll('.gro
 link.addEventListener('clie
 isScrolling = true;
 setTimeout(() => isScro
 });
 });
```

```
 const observer = new Intersection
```

```
 });
 });

 function fallbackModal(contentRaw,
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';
 modal.style.backgroundColor = 'black';
 modal.style.zIndex = '99999';
 modal.style.display = 'flex';
 modal.style.flexDirection = 'column';
 modal.style.alignItems = 'center';
 modal.style.justifyContent = 'center';

 const box = document.createElement('div');
 box.style.width = '80%';
 box.style.height = '80%';
 box.style.backgroundColor = '#1a1a1a';
 box.style.padding = '20px';
 box.style.borderRadius = '10px';
 box.style.display = 'flex';
 box.style.flexDirection = 'column';
 box.style.color = '#fff';

 const header = document.createElement('div');
 header.innerText = 'Экспорт в PDF';
 header.style.marginBottom = '10px';

 const subHeader = document.createElement('div');
 subHeader.innerText = 'Из-за не
ние веб-студии в новой вкладке (через меню)';
 subHeader.style.marginBottom = '10px';
 subHeader.style.color = '#f87171';
 subHeader.style.fontSize = '0.9em';
```



```

 closeBtn.style.border = 'none';
 closeBtn.style.borderRadius = '10px';
 closeBtn.style.cursor = 'pointer';
 closeBtn.onclick = () => modal.close();

 box.appendChild(header);
 box.appendChild(subHeader);
 if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(copyBtn);
 }
 btnContainer.appendChild(closeBtn);
 box.appendChild(btnContainer);
 modal.appendChild(box);
 document.body.appendChild(modal);
 }
}

```

```

function downloadMD() {
 let clone = document.querySelector('#md-content').cloneNode(true);

 // Smart markdown parsing based on DOM
 // Smart markdown parsing based on DOM
 clone.querySelectorAll('h2').forEach(el => {
 clone.querySelectorAll('h3').forEach(el => {
 clone.querySelectorAll('.analog').forEach(el => {
 clone.querySelectorAll('li').forEach(el => {
 clone.querySelectorAll('svg').forEach(el => {
 let caption = '';
 if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
 caption = el.nextElementSibling.alt;
 }
 el.textContent = '\n\n[Изображение: ' + caption + ']';
 });
 });
 });
 });
 });
 clone.querySelectorAll('.img-caption').forEach(el => {
 let text = clone.innerText;
 text = text.replace(/\n{3,}/g, '\n\n');
 });
}

```



```

 }
 </script>

 <!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
 <div class="mt-8 flex flex-col gap-4 bo
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-
 <button onclick="downloadWord()
ont-bold flex items-center justify-center g
 Word
 </button>
 <a href="algebra.pdf" target="_l
-sm font-bold flex items-center justify-cen
 PDF

 <button onclick="downloadMD()"
-bold flex items-center justify-center gap-
 Markdown
 </button>
 <button onclick="downloadHTML()
t-sm font-bold flex items-center justify-ce
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-
 <div class="flex items-center g
 <button id="theme-dark" onc
justify-center text-amber-300 hover:text-wh
line-amber-500" title="Темная тема"> </butt
 <button id="theme-light" on
r justify-center text-amber-50 hover:text-wh
">* </button>

```

```

 14.
 15.
 17.
 18.
 19.
 20.
 21.
 22.

</div>
</header>

<!-- ===== WTF CONTAINER =====>
<div id="wtf-container" class="hidden">
 <h2 class="text-3xl font-black text-cen

 <div class="overflow-x-auto w-full pb-4">
 <div class="grid grid-cols-3 gap-4">
 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slat

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
 <div class="text-xs text-orange
 <div class="font-bold text-slat

 <a href="#question-2" onclick="setM

```

```

-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-11" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 5: Корни / Уравнения -->
 <div class="hidden md:block"></div>
 <a href="#question-13" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->

```

```
 <a href="#question-22" onclick="setf
-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsia
 <div class="font-bold text-slate

```

```
 <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ
 <div class="col-span-1 md:col-span-3
border-b border-red-500/30 pb-2">Кластер 4:
```

```
 <a href="#proof-algorithms" onclick="
:col-span-3 bg-red-900/20 p-4 border-l-4 bo
 <div class="text-xs text-red-500
 <div class="font-bold text-slate

 </div>
</div>
```

```
 <!-- ЛИКБЕЗ SECTION INSERTED HERE -->
 <section id="proof-algorithms" class="m
 <div class="flex items-center mb-6">
 <div class="bg-red-500/20 p-3 r
 <svg class="w-8 h-8 text-red
rg/2000/svg">
 <path stroke-linecap="r
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border
 </div>
```

```
 <div class="prose prose-invert max-w
 <p class="text-lg">
 Хватит тупить. Доказательств
 есь для того, чтобы жать на кнопки. Вот жес
 </p>
```

```
 <h2 class="text-2xl font-bold b
```

счет ограничений (размер остатка) докажи, что

- 

**Индукция**

**Индукция**

Просто перепиши формулу с буквой  $k$ .

усок  $k$ , замени его на жареного карася из

**Двойная делимость**

**Двойная делимость**

вверх по уравнениям).

рху вниз, показывая, что  $\Delta r_i$

х. Твой  $d$  больше.

**Алгоритмы экзекуции**

```
<li class="bg-slate-800/50" data-bbox="541 95 1000 285">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 <code class="text-x
 </div>

<li class="bg-slate-800/50" data-bbox="541 311 1000 521">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 x двух, потом последних.

 <code class="text-x
 </div>

<li class="bg-slate-800/50" data-bbox="541 547 1000 757">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 есть сложение — есть ноль. Если есть умноже
 <code class="text-x
 </div>

<li class="bg-slate-800/50" data-bbox="541 783 1000 995">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm">
 авь их драться. Умножение всегда врывается
 <code class="text-x
 </div>

```



```
* Выгрузка главы (или всего пособия) в самостоятельный
*
* Файл получается автономным: стили защиты внутрь, инт
* его можно открыть с флешки, отправить в мессенджер и
* Живые React-визуализации в статику не переносятся —
* подставляется заметка со ссылкой на онлайн-версию, а
* которые в приложении всплывают по наведению, собираю
* в конце документа.
*/
```

```
/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText));
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен — пропускаем */
 }
 }
 }
 return parts.join("\n");
}
```

```
const EXTRA_CSS = `
/* — правки для автономного файла — */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; paddi
.export-head { border-bottom: 1px solid #1e293b; paddin
```

```

} {
 const host = document.createElement("div");
 host.innerHTML = html;

 try {
 annotateTerms(host);
 } catch {
 /* если браузер не осилил регулярку — выгружаем текст */
 }

 /*
 * Упоминания «билет 15» становятся якорями на главу 15
 * только для книги, где все главы на месте: в файле book.html
 * вёл бы в никуда, а битые ссылки — ровно то, от чего мы
 * хотим избавиться
 */
 if (opts.ticketAnchors) {
 try {
 annotateTicketLinks(host, { selfId: opts.selfId });
 host.querySelectorAll<HTMLAnchorElement>("a[data-goto]").forEach(
 a => {
 a.setAttribute("href", `#ch-${a.dataset.goto}`);
 a.removeAttribute("data-goto");
 }
);
 } catch {
 /* ссылки — улучшение, а не обязательная часть файла */
 }
 }

 const ids: string[] = [];
 host.querySelectorAll<HTMLElement>(".term-hint").forEach(
 el => {
 const id = el.dataset.termId;
 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = el.textContent ?? "";
 el.replaceWith(link);
 }
);
}

```



```

const figures: string[] = [];
for (const m of mine.slice(0, 4)) {
 const res = await fetch(`${base}export/viz-shots/`);
 if (!res.ok) continue;
 const blob = await res.blob();
 if (blob.size > 3_000_000) continue;
 const data = await new Promise<string>((resolve,
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
));
 figures.push(
 `<figure class="export-fig">${m.name ? `Режим «${escapeHtml(m.name)}»` : `Режим «Неизвестно»`}`
);
}
return figures.length
 ? `<div class="export-figs"><h3 class="export-figs">Визуализация`
 : "";
} catch {
 return "";
}
}

function triggerDownload(html: string, filename: string) {
 const blob = new Blob([html], { type: "text/html;charset=utf-8" });
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
 document.body.appendChild(a);
 a.click();
 a.remove();
 setTimeout(() => URL.revokeObjectURL(url), 2000);
}

/** Имя файла: «04-splay-derevo.html» — без кириллицы и

```



```
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;
```

```
/** Внутри чего ссылки не ставим: код, уже существующие
const SKIP_SELECTOR = "code, pre, script, style, a, but
```

```
/** Одна страница на диапазон? «билеты 18–20» → mst, а
function chapterForRange(from: number, to: number): string {
 let found: string | undefined;
 for (let n = from; n <= to; n += 1) {
 const id = ticketToChapter.get(n);
 if (!id) return undefined;
 if (found && found !== id) return undefined;
 found = id;
 }
 return found;
}
```

```
function hrefFor(id: string, lite: boolean): string {
 return lite ? `?lite=1&topic=${encodeURIComponent(id)}` :
}
```

```
/**
 * Строковая версия – для текстовой (`?lite=1`) версии,
 * через `dangerouslySetInnerHTML`, а эффекты разметки
 * HTML режется на «тег» и «текст», меняются только тек
 * вне `pre/code/a`.
 */
```

```
export function linkifyTicketsHtml(html: string, opts: {
 let inCode = 0;
 let inLink = 0;
 return html
 .split(/(<[>]*>)/g)
 .map((part) => {
 if (!part) return part;
 if (part.startsWith("<")) {
 const closing = part.startsWith("</");
```

```

 },
 });

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
}

let created = 0;
for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 TICKET_RE.lastIndex = 0;
 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;
 while ((m = TICKET_RE.exec(text)) !== null) {
 const from = Number(m[1]);
 const to = m[2] ? Number(m[2]) : from;
 const id = chapterForRange(from, to);
 if (!id || id === opts.selfId) continue;
 if (m.index > last) frag.appendChild(document.createTextNode(
 const a = document.createElement("a");
 a.className = "ticket-link";
 a.dataset.goto = id;
 a.setAttribute("href", hrefFor(id, opts.lite ?? false));
 a.textContent = m[0];
 frag.appendChild(a);
 last = m.index + m[0].length;
 created += 1;
 }
 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createTextNode(
 textNode.parentNode?.replaceChild(frag, textNode);
}
return created;
}

```

```

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")]
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\\s{2

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^(\d+)\.\/)?.[1]
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha

```



```

/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossary.ts"],
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".mjs" },
 external: ["react", "react-dom", "motion", "lucide-react"],
});

const { chapters } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));

const esc = (s) => s.replace(/[\.*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
 `(?<![\\p{L}\\p{N}_-])(?=${GLOSSARY_LABELS.map((l) => esc(l.label))})`
 "giu"
);

const map = new Map(GLOSSARY_LABELS.map((l) => [l.label, l.term]));

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подств. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/[\s\S]*?>/gi, "")
 .replace(/<[^\>]+>/g, " ");

```

```
*
* текст всех тем (без служебной разметки)
* + снимки визуализаций, если они отрендерены (tmp/viz-
* capture-viz-shots.mjs в CI через headless-браузер)
* + иллюстрации страниц (jpg из src/assets)
* + текстовое описание каждой демонстрации и переменных
* Python-компилятор (работает и без браузера – локально)
* – БЕЗ кода панели компилятора и без навигации приложения
*
* Запуск: npm run export:guide
* Цель: 40–60 страниц (жёсткий предел – 100).
*
* Вёрстка целиком на собственном курсоре (cy): pdfkit
* сам, длинные таблицы/код/боксы режутся на фрагменты
*/
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, ensureDir, humanSize } from "../utils";
import { htmlToBlocks } from "../html-to-blocks.mjs";

const GUIDE_PDF_PATH = path.join(OUT_DIR, "guide_lite.pdf");
const SHOTS_DIR = path.join(ROOT, "tmp", "viz-shots");
const ENTRY = path.join(ROOT, "tmp", "guide-entry.ts");

/** Картинки-иллюстрации страниц (те же, что показывает
const CHAPTER_IMAGES = {
 dijkstra: "src/assets/dijkstra-kind.jpg",
 "bellman-ford": "src/assets/bellman-ford-truck.jpg",
 floyd: "src/assets/floyd-network.jpg",
};

/** A4 в пунктах. */
const PAGE = { width: 595.28, height: 841.89, margin: 40 };
const CONTENT_W = PAGE.width - PAGE.margin * 2;
const TOP = PAGE.margin;
const BOTTOM = PAGE.height - PAGE.margin - 16;
```

```

/** Русское склонение счётных существительных: 1 страница
const plural = (n, one, few, many) => {
 const mod10 = Math.abs(n) % 10;
 const mod100 = Math.abs(n) % 100;
 if (mod10 === 1 && mod100 !== 11) return one;
 if (mod10 >= 2 && mod10 <= 4 && (mod100 < 10 || mod100 > 100))
 return many;
};

const plain = (text) =>
 String(text ?? "")
 .replace(UNSUPPORTED, "")
 .replace(/\s{2,}/g, " ")
 .replace(/\u0000/g, "")
 .trim();

/* ----- данные приложения (через esbuild)

async function loadAppData() {
 ensureDir(path.dirname(ENTRY));
 fs.writeFileSync(
 ENTRY,
 [
 'export { chapters, chapterTopics } from "../src/data/chapters";',
 'export { PAGE_SYNC } from "../src/data/vizSync";',
 'export { VIZ_REGISTRY } from "../src/components/vizRegistry";',
 'export { quizzes } from "../src/data/quizzes";',
].join("\n")
);
};
const res = await build({
 entryPoints: [ENTRY],
 bundle: true,
 write: false,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 logLevel: "silent",
 loader: { ".ts": "ts", ".tsx": "tsx", ".jpg": "dataurl" }
```

```

 const y = PAGE.height - PAGE.margin + 4;
 this.doc.font(this.fonts.regular).fontSize(7.6).fill(
 this.doc.text("Универсальное пособие • алгоритмы и
 width: CONTENT_W * 0.62,
 align: "left",
 lineBreak: false,
 ellipsis: true,
));
 this.doc.text(`${this.footerTitle} • ${this.pages}`
 width: CONTENT_W * 0.38,
 align: "right",
 lineBreak: false,
 ellipsis: true,
));
 this.doc.page.margins.bottom = prevBottom;
 this.doc.y = this.cy;
}

newPage(footerTitle) {
 this.stampFooter();
 if (footerTitle !== undefined) this.footerTitle = footerTitle;
 this.doc.addPage({ size: [PAGE.width, PAGE.height],
 this.pages += 1;
 this.cy = TOP;
}

firstPage(footerTitle) {
 if (footerTitle !== undefined) this.footerTitle = footerTitle;
 this.doc.addPage({ size: [PAGE.width, PAGE.height],
 this.pages += 1;
 this.cy = TOP;
}

/** Сколько места нужно, чтобы элемент не оторвался от
ensure(h, extra = 0) {
 if (this.cy + h + extra > BOTTOM) this.newPage();
}

```

```

 current = word;
 }
}
flush();
return chunks.length ? chunks : [source];
}

rule(color = "#c7d2fe", width = 1) {
 const y = this.cy;
 this.doc.moveTo(PAGE.margin, y).lineTo(PAGE.margin + width, y);
 this.cy = y + 8;
}

h1(rawText, { size = 20, color = "#111827", rule = true } = {}) {
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + (rule ? 14 : 6) + gapBefore, 22);
 this.cy += gapBefore;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(color).text(text);
 this.cy = Math.max(this.cy, this.doc.y) + 2;
 if (rule) this.rule();
}

h2(rawText, { size = 13.5, color = "#1e293b" } = {}) {
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + 6, 20);
 this.cy += 6;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(color).text(text);
 this.cy = Math.max(this.cy, this.doc.y) + 3;
}

h3(rawText, { size = 11.2, color = "#334155" } = {}) {
 const text = plain(rawText);
 if (!text) return;

```

```

 if (index === 0) {
 this.doc.font(this.fonts.regular).fontSize(size)
 .text(right, { align: "right", lineGap: 1 });
 }
 this.doc.font(this.fonts.regular).fontSize(size)
 .text(right, { align: "right", lineGap: 1 });
 this.cy = Math.max(y0 + h, this.doc.y) + 2.5;
 if (this.cy > BOTTOM) this.newPage();
 });
}

```

```

/** Код: режим на фрагменты по высоте страницы, каждый
code(rawText, { size = 8.0 } = {}) {
 const font = this.fonts.mono;
 const inner = CONTENT_W - 22;
 const lines = String(rawText ?? "").replace(UNSUPPORTED, "");
 // собираем фрагменты, которые влезают в одну рамку
 let i = 0;
 while (i < lines.length) {
 // Если у низа страницы осталось меньше минимальной высоты
 // СРАЗУ: иначе рамка уедет за BOTTOM, и pdfkit сработает
 // в обход рендерера (без колонтитула и нумерации)
 if (BOTTOM - this.cy - 16 < 60) this.newPage();
 const avail = BOTTOM - this.cy - 16;
 const maxH = avail;
 let chunk = [];
 let h = 0;
 while (i < lines.length) {
 const candidate = chunk.concat(lines[i]).join("\n");
 const ch = this.height(candidate, { font, size });
 if (chunk.length && ch > maxH) break;
 chunk.push(lines[i]);
 h = ch;
 i += 1;
 }
 const body = chunk.join("\n");
 const boxH = h + 14;
 this.ensure(boxH);
 const y0 = this.cy;

```

```

 };

 this.ensure(rowHeight(header) + 8, 14);
 let y = this.cy;
 rows.forEach((cells, idx) => {
 const row = [];
 for (let i = 0; i < cols; i++) row.push(cells[i]);
 const h = rowHeight(row);
 if (y + h > BOTTOM) {
 this.cy = y;
 this.newPage();
 y = this.cy;
 if (idx > 0) y += drawRow(header, y, true, false);
 }
 y += drawRow(row, y, idx === 0, idx % 2 === 1);
 });
 this.cy = y + 8;
}

quote(rawText) {
 const text = plain(rawText);
 if (!text) return;
 const font = this.fonts.oblique ?? this.fonts.regular;
 const width = CONTENT_W - 22;
 const opts = { font, size: 9.5, width, lineGap: 1 };
 // Длинная цитата режется по страницам заранее: рис
 // текст уйдёт за BOTTOM и pdfkit добавит страницу
 const chunks = this.splitToFit(text, opts, BOTTOM);
 chunks.forEach((chunk) => {
 const h = this.height(chunk, opts);
 this.ensure(h + 10);
 const y0 = this.cy;
 this.doc.rect(PAGE.margin, y0, 3, h + 4).fill("#a
 this.doc.font(font).fontSize(9.5).fillColor("#374
 this.cy = y0 + h + 10;
 if (this.cy > BOTTOM) this.newPage();
 });
}

```

```

 const y0 = this.cy;
 // сколько строк влезет на эту страницу
 const avail = BOTTOM - y0 - headH - 16;
 const taken = [];
 let h = 0;
 while (rest.length) {
 const ch = this.height(rest[0], { font: this.fonts.regular });
 if (taken.length && h + ch > avail) break;
 taken.push(rest.shift());
 h += ch;
 }
 const boxH = headH + h + 12;
 this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, boxH, 10);
 this.doc.font(this.fonts.bold).fontSize(9.2).fill('black');
 let y = y0 + 7 + headH;
 for (const l of taken) {
 this.doc.font(this.fonts.regular).fontSize(size);
 y += this.height(l, { font: this.fonts.regular });
 }
 this.cy = y0 + boxH + 7;
 if (rest.length) this.newPage();
}
}
}

```

/\* \_\_\_\_\_ сборка \_\_\_\_\_ \*/

```

async function main() {
 const fonts = {
 regular: pick(CANDIDATES.regular),
 bold: pick(CANDIDATES.bold),
 oblique: pick(CANDIDATES.oblique),
 mono: pick(CANDIDATES.mono),
 };
 if (!fonts.regular) throw new Error("Не найден TTF-шрифт");
 fonts.bold = fonts.bold ?? fonts.regular;
 fonts.mono = fonts.mono ?? fonts.regular;
}

```



```

});
R.para(`Дата сборки: ${today}`, { size: 9, color: "#666666" });

const missingViz = chapters.filter((c) => !VIZ_REGISTERED[c.id]);
R.box("Как устроено пособие", [
 "• Страница = один или несколько билетов: текст, анимация, видео",
 "• У каждой страницы есть интерактивная демонстрация, которую можно просматривать, кликая по ней, или просматривать её напрямую.",
 "• В этом PDF демонстрации показаны снимками (когда вы нажимаете на кнопку), а не видео (когда вы управляете.",
 missingViz.length
 ? `• Страницы без демонстрации: ${missingViz.map((c) => c.id).join(", ")}`
 : "• Демонстрация и синхронизация с компилятором"
]);

/* — содержание — */
R.newPage("Содержание");
R.h1("Содержание", { size: 18 });
chapters.forEach((c, i) => {
 const topics = chapterTopics[c.id] ?? [];
 R.para(
 `${String(i + 1).padStart(2, "0")}. ${c.title}${topics.length > 0 ? " — " : ""} ${topics.map((t) => t).join(", ")}`,
 { size: 9.3, gapAfter: 2 }
);
});

/* — темы — */
let shotsTotal = 0;
for (const chapter of chapters) {
 const topics = chapterTopics[chapter.id] ?? [];
 const shortTitle = chapter.title.replace(/^\\d+\\.--/, "");
 R.newPage(shortTitle);

 if (topics.length) {
 R.para(`Билет${topics.length > 1 ? "ы" : ""} ${topics.map((t) => t).join(", ")}`, {
 size: 2 });
 }
}

```

```

if (viz || syncKeys.length) {
 R.h2("Демонстрация и связь с компилятором");
 if (viz) {
 R.para(viz.title, { size: 10.2, color: "#1e293b" });
 if (viz.hint) R.para(viz.hint, { size: 9.1, color: "#1e293b" });
 }

 const shots = shotsFor(chapter.id);
 shotsTotal += shots.length;
 for (const shot of shots) {
 R.image(shot.file, {
 maxHeight: 300,
 caption: shot.name ? `${viz?.title ?? chapter.title} ${shot.name}` : shot.name,
 });
 }

 for (const key of syncKeys) {
 const sync = PAGE_SYNC[key];
 const demo = key.includes("#") ? key.split("#")[0] : "";
 const title = demo ? `Вкладка «${demo}» ${sync.variables.length} переменных` : sync.variables.length > 0 ? `Вкладка с ${sync.variables.length} переменными` : sync.stepNote;
 const lines = [];
 if (sync.stepNote) lines.push(`Шаг демонстрации: ${sync.stepNote}`);
 sync.variables.forEach((v) => lines.push(`• ${v.name} = ${v.value}`));
 lines.push("• Референсный код демонстрации в этой вкладке");
 if (lines.length) R.box(title, lines);
 }

 if (!shots.length) {
 R.para(
 "Снимки демонстрации добавляются автоматически. Если вы открываете эту страницу без браузера здесь остаётся текстовое описание.",
 { size: 8.5, color: "#6b7280", italic: true }
);
 }
}
}

```

```

/* блиц: те же вопросы, что в приложении, но сразу
const quiz = (quizzes && quizzes[chapter.id]) || []

```

```

 console.error("Ошибка сборки PDF пособия:", err.message);
 process.exit(1);
});

```

ФАЙЛ 93/111: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМ:

```

/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в
 * в единый документ public/export/full_code_all_pages.pdf
 */
import fs from "node:fs";
import path from "node:path";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir, } from

// A4 в пунктах PDF
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();

 if (pages.length === 0) throw new Error("Не найдено ни

 ensureDir(OUT_DIR);

```

ФАЙЛ 94/111: scripts/export/capture-viz-shots.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 122 | РАЗМЕР: 1000

```
/**
 * Снимки визуализаций для PDF-конспекта: tmp/viz-shots.
 *
 * Headless-браузер открывает каждую страницу с демонстрацией
 * блок #chapter-viz. У страниц с вкладками (sparse-table)
 * снимается каждая вкладка — файл получает имя `chapter-viz-sparse-table`
 * и PDF подписывает его как «режим «вкладка»».
 *
 * npm run export:shots # сервер уже поднят
 * SHOTS_BASE_URL=http://127.0.0.1:4173 npm run export:shots
 *
 * В CI это делает rebuild-pdf.yml: сборка → vite preview →
 * npm run export:guide. Локально без браузера скрипт просто
 * честно пишет, что снимки добавляются при сборке в CI.
 */
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import puppeteer from "puppeteer";
import { ROOT } from "../common.mjs";

const OUT_DIR = path.join(ROOT, "tmp", "viz-shots");
const BASE_URL = process.env.SHOTS_BASE_URL || "http://127.0.0.1:4173";

/** Вкладки, которые стоит снять отдельно: глава → подглава
const TABS = {
 "sparse-table": ["1D", "2D: построение", "2D: запрос"],
 mst: ["Краскал", "Прим", "Борувка"],
 treap: ["Собрать", "Split", "Merge"],
 "splay-tree": ["Обход", "Повороты", "Прогулка"],
};

const slug = (s) =>
 s
```

```

 await page.goto(`${BASE_URL}/?topic=${id}`, { waitU
 await page.waitForSelector("#chapter-viz", { timeou
 // демо-анимации: даём кадру собраться до осмысленн
 await new Promise((r) => setTimeout(r, 1600));
 const viz = await page.$("#chapter-viz");
 if (!viz) throw new Error("нет #chapter-viz");
 await viz.screenshot({ path: path.join(OUT_DIR, `${
 done += 1;

 for (const tab of TABS[id] ?? []) {
 try {
 const clicked = await page.evaluate((label) =>
 const root = document.querySelector("#chapter
 if (!root) return false;
 const btn = Array.from(root.querySelectorAll(
 (b.textContent ?? "").trim().startsWith(lab
));
 if (!btn) return false;
 btn.dispatchEvent(new MouseEvent("click", { b
 return true;
 }, tab);
 if (!clicked) continue;
 await new Promise((r) => setTimeout(r, 1200));
 const el = await page.$("#chapter-viz");
 await el.screenshot({ path: path.join(OUT_DIR,
 done += 1;
 } catch (error) {
 console.log(`[export:shots] вкладка «${tab}»
 }
 }
 console.log(`[export:shots] ok ${id}`);
} catch (error) {
 failed += 1;
 console.log(`[export:shots] FAIL ${id}: ${String(er
}
}

const manifest = fs

```

## Универсальное пособие • полный исходный код

```

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человечитаемые категории для оглавления. */
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
 [/^src\/data\/$/, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/$/, "Визуализаторы (в.
 [/^src\/components\/$/, "Интерактивные компоненты и сим
 [/^src\/layout\/$/, "HTML-разметка (layout)"],
 [/^src\/utils\/$/, "Утилиты"],
 [/^src\/$/, "Ядро приложения"],
 [/^scripts\/$/, "Скрипты сборки и экспорта"],
 [/^\.github\/$/, "CI / автоматизация"],
 [/^[^\/]+\$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
```

```

 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
}
return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
 return na - nb;
}));
});
}

```

```

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti
 const totalBytes = files.reduce((s, f) => s + fs.stat

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объём кода: ${humanSize(totalByt
 push(`Глав/билетов в пособии:        ${chapters.length}`)
 push();
}

```

```

 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
 });

 push(HR);
 push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сген
 push(HR);

 const txt = out.join("\n") + "\n";
 fs.writeFileSync(TXT_PATH, txt, "utf8");

 console.log("[1/3] код -> txt");
 console.log(`      файлов:  ${files.length}`);
 console.log(`      глав:    ${chapters.length}`);
 console.log(`      строк:   ${txt.split("\n").length}`);
 console.log(` выход: ${path.relative(ROOT, TXT
}

main();

```

ФАЙЛ 96/111: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ

```

/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.m
 * txt -> page-XXXX.png (render-pages.m
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
import fs from "node:fs";

```



```
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore" });
 return cmd;
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
);
}
```

---

ФАЙЛ 97/111: scripts/export/html-to-blocks.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 266 | РАЗМЕР: 1.2 KB

---

```
/**
 * HTML страницы пособия → плоский список блоков для парсинга
 *
 * Задача: вытащить из Tailwind-разметки САМ ТЕКСТ (как в ридере), сохранив структуру и ПОРЯДОК слов: заголовки,
 * таблицы, код, спойлеры <details>. Служебная разметка (svg, кнопки) отбрасывается, содержимое спойлеров — нет.
 *
 * Никаких внешних зависимостей: собственный мини-парсер.
 * куски хранятся отдельными узлами #text, поэтому инлайны
 * (, <code>,) не «уезжает» в конец абзаца.
 */
```

```

 stack.length = i;
 break;
 }
}
continue;
}

if (openTag) {
 const tag = openTag.toLowerCase();
 const attrs = {};
 const attrRe = /([a-zA-Z-_:]+)\s*=\s*"([^"]*)"|'
 let a;
 while ((a = attrRe.exec(attrsStr || "")) !== null)
 const node = { tag, attrs, children: [] };
 stack[stack.length - 1].children.push(node);
 if (!VOID.has(tag) && !selfClose) stack.push(node)
 continue;
}

if (textChunk !== undefined && textChunk.length) {
 stack[stack.length - 1].children.push({ tag: "#text",
}
}
return root;
}

const decodeEntities = (s) =>
 s
 .replace(/</g, "<")
 .replace(/>/g, ">")
 .replace(/"/g, '"')
 .replace(/'/g, "'")
 .replace(/ /g, "\u00a0")
 .replace(/—/g, "—")
 .replace(/–/g, "—")
 .replace(/«/g, "«")
 .replace(/»/g, "»")
 .replace(/…/g, "...")

```

```
function collectText(node) {
 if (node.tag === "#text") return clean(node.text);
 if (SKIP.has(node.tag)) return "";
 if (node.tag === "table") return tableRows(node).map(
 return clean(node.children.map(collectText).join(" "))
 }
}
```

/\*\* Сырой текст внутри <pre> – переносы строк важны. \*/

```
function collectRawPre(node) {
 if (node.tag === "#text") return node.text;
 if (node.tag === "br") return "\n";
 if (SKIP.has(node.tag) && node.tag !== "img") return
 return node.children.map(collectRawPre).join("");
}
```

/\*\* Таблица → массив строк из массивов ячеек. \*/

```
function tableRows(node) {
 const rows = [];
 const visit = (n) => {
 if (n.tag === "tr") {
 const cells = n.children.filter((c) => c.tag ===
 if (cells.some((c) => c.length)) rows.push(cells)
 return;
 }
 n.children.forEach(visit);
 };
 node.children.forEach(visit);
 return rows;
}
```

/\*\*

\* Обход дерева: собираем блоки в порядке документа.

\* @returns {Array<{type:string,text?:string,rows?:string}

\*/

```
export function htmlToBlocks(html) {
 const root = parseHtml(html);
 const out = [];
```

```
// есть блочные дети: идём по порядку, инлайн-пробе
if (hasBlockChild(node)) {
 let buf = "";
 const flush = () => {
 const t = clean(buf);
 if (t) push({ type: "p", text: t });
 buf = "";
 };
 for (const c of node.children) {
 if (c.tag === "#text") {
 buf += c.text;
 continue;
 }
 if (SKIP.has(c.tag)) continue;
 if (!BLOCK.has(c.tag)) {
 buf += inlineText(c) + " ";
 continue;
 }
 if (c.tag === "ul" || c.tag === "ol") {
 flush();
 emitList(c, listDepth);
 continue;
 }
 flush();
 walk(c, listDepth);
 }
 flush();
 return;
}
```

```
// листовой блок: один блок по типу тега
const text = inlineText(node);
if (!text) return;
if (HEADING.has(node.tag)) push({ type: "heading",
else if (node.tag === "li") emitListItem(node, null
else if (node.tag === "summary") push({ type: "summ
else if (node.tag === "blockquote") push({ type: "q
else if (node.tag === "figcaption") push({ type: "c
```

```
* Результат: tmp/export-pages/page-0001.png, page-0002
*/
```

```
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";
```

```
const execFileAsync = promisify(execFile);
```

```
/** Раскрываем табы и жёстко переносим длинные строки, */
```

```
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min((expanded.match(/^
 let rest = expanded;
 let first = true;
 while (rest.length > 0) {
 const width = first ? PAGE.cols : PAGE.cols - indent;
 parts.push((first ? "" : indent) + rest.slice(0, width));
 rest = rest.slice(width);
 first = false;
 }
 return parts;
}
```

```
function paginate(txt) {
 const source = txt.replace(/\r\n/g, "\n").split("\n");
 const pages = [];
 for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
}
```

```

];

 const concurrency = Math.max(2, Math.min(os.cpus().length, 10));
 let done = 0;
 let cursor = 0;

 async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(` отрендерено ${done}
 `);
 }
 }
 }

 await Promise.all(Array.from({ length: concurrency }, () => worker()));

 process.stdout.write("\n");

 const missing = jobs.filter((j) => !fs.existsSync(j.png));
 if (missing.length) throw new Error(`Не отрендерены ${missing.length} страниц`);

 const bytes = jobs.reduce((s, j) => s + fs.statSync(j.png).size, 0);
 console.log("[2/3] txt -> png");
 console.log(`      страниц: ${total}`);
 console.log(`      выход:   ${path.relative(ROOT, PAGE_PATH)}`);
 }

 main().catch((err) => {
 console.error("Ошибка рендера страниц:", err.message);
 process.exit(1);
 });
}

```

```

const sleep = (ms: number) => new Promise((r) => setTim

async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
 });
}

function bodyText(): string {
 return (document.body.textContent ?? "").replace(/\s+/g, " ");
}

function allButtons(): string[] {
 return Array.from(document.querySelectorAll("button"))
 .filter((b) => (b.getAttribute("aria-label") || b.getAttribute("title")));
}

/** Кнопки без доступного имени: ни aria-label, ни title */
function namelessButtons(scope: ParentNode = document): HTMLButtonElement[] {
 return Array.from(scope.querySelectorAll("button"))
 .filter((b) => {
 const name = [b.getAttribute("aria-label"), b.getAttribute("title")]
 .filter(Boolean)
 .join(" ")
 .replace(/\s+/g, " ")
 .trim();
 return name.length === 0;
 })
 .map((b) => `<button class="${(b.className || "").replace(/\s+/g, " ")}">`);
}

function findButton(match: string | RegExp): HTMLButtonElement | null {
 const list = Array.from(document.querySelectorAll("button"));
 return list.find((b) => {
 const hay = [b.getAttribute("aria-label"), b.getAttribute("title")]
 .filter(Boolean)
 .join(" ")
 .replace(/\s+/g, " ")
 .trim();
 return typeof match === "string" ? hay.includes(match) : match.test(hay);
 });
}

```

```
}

/** Ждём появления признака в DOM (текст/предикат), пер
async function waitFor(pred: () => boolean, timeoutMs =
 const deadline = Date.now() + timeoutMs;
 while (Date.now() < deadline) {
 if (pred()) return true;
 await settle(step);
 }
 return pred();
}

const RUN_DONE = [
 "Трасса готова автоматически",
 "Нет шагов трассы",
 "Пустой код",
 "Код не выполнялся",
 "Выполнение остановлено после",
 "Не удалось выполнить код",
 "Ошибка:",
];

let root: Root | null = null;
let host: HTMLElement | null = null;

function freshHost(): HTMLElement {
 const div = document.createElement("div");
 document.body.appendChild(div);
 return div;
}

async function mountApp(topic: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
}
```



```
const BAD_DOM = [
 { re: /\bNaN\b/, why: "NaN в разметке" },
 { re: /\bundefined\b/, why: "undefined в разметке" },
 { re: /\[object Object\]/, why: "[object Object] в ра
];

/**
 * Битые подстановки видны в КОРОТКИХ текстовых узлах (
 * вершины): «undefined», «NaN», «[object Object]». В д
 * псевдокода те же слова законны — их не трогаем.
 */
function domDefects(scope: HTMLElement | null): string[] {
 if (!scope) return [];
 const found: string[] = [];
 const walker = document.createTreeWalker(scope, 4 /*
 let node = walker.nextNode() as Text | null;
 while (node) {
 const text = (node.nodeValue ?? "").replace(/\s+/g,
 if (text.length > 0 && text.length < 60) {
 for (const bad of BAD_DOM) {
 if (bad.re.test(text)) found.push(`${bad.why}:
 }
 }
 node = walker.nextNode() as Text | null;
 }
 return [...new Set(found)];
}

async function test(group: string, name: string, fn: ()
 if (skipGroups.has(group[0])) return true;
 consoleErrors = [];
 const t0 = Date.now();
 let ok = true;
 let detail = "";
 try {
 const d = await fn();
 detail = d ?? "";
 }
```

```

const group = "А. страницы";
await test(group, `${chapter.id}: монтируется, бейдж
 await mountApp(chapter.id);
 const txt = bodyText();
 if (!txt.includes(chapter.title.slice(0, 18))) th
 if (!VIZ_REGISTRY[chapter.id]) return "визуализац
 if (!/встроенное демо|данные из компилятора/.test
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в
 const nameless = namelessButtons(host!);
 if (nameless.length) throw new Error(`кнопок без
 return `бейдж «встроенное демо» на месте, кнопок
});

```

```

await test(group, `${chapter.id}: панель открывается
 await mountApp(chapter.id);
 const toggle = findButton("Боковая панель Python")
 await click(toggle, "кнопка панели Python в навба
 const ta = textarea();
 if (!ta) throw new Error("редактор (textarea) не
 if (!ta.value.includes("# «")) return `редактор е
 const eye = findButton(/эта?лонный код/i);
 if (!eye) throw new Error("нет кнопки-«глаза» (вс
 const run = findButton(/Запустить код|автопроход/
 if (!run) throw new Error("нет кнопки запуска ►")
 return `кнопок в панели: ${allButtons().length}`;
});

```

```

await test(group, `${chapter.id}: автозапуск инициа
 await mountApp(chapter.id);
 await click(findButton("Боковая панель Python"),
 const done = await waitFor(() => RUN_DONE.some((m
 if (!done) throw new Error(`запуск не завершился
 const msg = RUN_DONE.find((m) => bodyText().includ
 const sync = PAGE_SYNC[chapter.id];
 if (!sync?.code) return `кода синхронизации нет:
 if (/Ошибка|Не удалось/.test(msg)) throw new Erro
 // после настоящего прогона визуализация обязана

```

```

 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в
 return `бейдж: ${txt.match(/данные из компилятор
 }));
}

/* — 3. Префиксные суммы 2D и разреженная таблица —
const gridCases: { viz: string; chapter: string; name
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
 9, 10, 11, 12]], S: [[1, 3, 6, 10], [6, 14, 24, 3
 тора/ },
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
 pref: [[1, 3], [4, 10]], r1: 0, c1: 0, r2: 1, c2
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
 9]], S: [[1, 3, 6], [5, 12, 21], [12, 27, 45]],
 { viz: "sparse-table", chapter: "sparse-table", nam
 [3, 3, 1, 1, 0], [1, 1, 0, 0, 0]] }, expect: /не
 { viz: "sparse-table", chapter: "sparse-table#2d-bu
 0, i: 2, j: 3, st: Array.from({ length: 4 }, () :
 pa/ },
 { viz: "sparse-table", chapter: "sparse-table#2d-qu
 30, j2: 40, res: 2 }, expect: /не совпадает|данны
];
for (const c of gridCases) {
 await test("С. матричные страницы", `${c.viz}: ${c.n
 await mountViz(c.viz);
 await act(async () => {
 emitVizState(c.viz, { line: 2, func: "<module>"
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\s
 if (!c.expect.test(txt)) throw new Error(`ожидали
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в
 return /размерность не совпадает/.test(txt) ? "пр
 }));
}

```

```

 await act(async () => {
 emitVizState(viz, { line: 5, func: "<module>",
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\\s
 if (!/данные из компилятора/.test(txt)) throw new
 if (!/ваши расстояния|dist/i.test(txt)) throw new
 const step = findButton(/^\\$|\\$ /) as HTMLButt
 if (step && !step.disabled) throw new Error("кноп
 if (domDefects(host).length) throw new Error(`апт
 await act(async () => {
 clearVizState(viz);
 await sleep(30);
 });
 const step2 = findButton(/^\\$|\\$ /) as HTMLBut
 if (step2 && step2.disabled) throw new Error("кноп
 return `кнопок до/после: ${before}/${allButtons()}
 });
}

```

```

/* — 5. Все референсные коды: настоящий Python → нас
const syncKeys = Object.keys(PAGE_SYNC).filter((k) =>
for (const key of syncKeys) {
 const sync = PAGE_SYNC[key];
 const chapterId = key.split("#")[0];
 await test("Е. эталонный код", `${key}: выполняется
 const raw = await py.runPythonAsync(buildDebugRun
 const result = JSON.parse(String(raw)) as DebugRe
 if (result.error) throw new Error(`Python-ошибка:
 if (result.steps.length === 0) throw new Error("н
 // объявленные переменные синхронизации реально п
 const seen = new Set<string>();
 for (const s of result.steps) for (const n of Obj
 for (const n of Object.keys(result.finalValues ??
 // в документации имя может быть видом «хк / х»,
 // проверяем, что хотя бы один python-идентификат
 const declared = sync.variables.map((v) => v.name
 const ident = (t: string) => /^[A-Za-z]+[A-Za-z0-

```

```

 return "редактор на месте, бейдж «встроенное демо»"
 });

```

```

await test("F. редактор", "синтаксическая ошибка: ред
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "def broken(:\n pass\n");
 const done = await waitFor(() => /Код не выполнен
 if (!done) throw new Error(`ошибка синтаксиса не по
 if (!textarea()) throw new Error("редактор исчез на
 if (!/invalid syntax|SyntaxError/i.test(bodyText()))
 return "ошибка показана, редактор остался";
});

```

```

await test("F. редактор", "гигантский print обрезаетс
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, 'dist = {0: 0}\nprint("x" * 200000)\np
 const ready = await waitFor(() => bodyText().includ
 if (!ready) throw new Error(`программа с огромным p
 const block = document.querySelector('[data-testid=
 if (!block) throw new Error("блок «Результат» не от
 const out = block.textContent ?? "";
 if (!/вывод обрезан/.test(out)) throw new Error(`па
 if (out.includes("ХВОСТ_ВЫВОДА")) throw new Error("
 if (out.length > 40000) throw new Error(`в панели $
 if (!textarea()) throw new Error("редактор исчез по
 if (domDefects(host).length) throw new Error(`артеф
 return `вывод обрезан до ${out.length} знаков, хвос
});

```

```

await test("F. редактор", "полная очистка: ничего не
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п

```

```

 { what: "заккрыть панель", re: /Заккрыть компилятор/i },
 { what: "шаг вперёд", re: /Шаг вперёд|вперёд/i },
 { what: "шаг назад", re: /Шаг назад|назад/i },
 { what: "свернуть листинг", re: /Свернуть листинг/i }
];
const btns = allButtons().join(" | ");
for (const m of mustBe) {
 if (!m.re.test(btns)) throw new Error(`в панели не ${m.what}`);
}
// На первом шаге переменных ещё нет (отладчик встал)
// поэтому связь с демонстрацией проверяем, дойдя до кнопки
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/i.test(bodyText()));
if (!linked) throw new Error(`эталон выполнен до кнопки "В конец трассы"`);

// — глаз ВЫКЛ: кода нет вообще, ничего не выполнялось
await click(findButton("Убрать эталонный код"), "глаз ВЫКЛ");
const cleared = await waitFor(() => bodyText().includes("глаз ВЫКЛ"));
if (!cleared) throw new Error(`после выключения глаза эталон не исчез`);
const ta3 = textarea();
if (!ta3) throw new Error("редактор пропал после выключения глаза");
if (ta3.value.trim() !== "") throw new Error(`глаз ВЫКЛ, но редактор не пуст`);
if (!/встроенное демо/.test(bodyText())) throw new Error("демо не включено");
if (/данные из компилятора/.test(bodyText())) throw new Error("демо не включено");
const afterButtons = allButtons();
const lost = editorButtons.filter((b) => b && !afterButtons.includes(b));
if (lost.length > 2) throw new Error(`после снятия с панели пропали кнопки: ${lost.map(b => b.what).join(", ")}`);

// ↵ возвращает инициализацию страницы
await click(findButton("Сбросить к шаблону страницы"), "↵");
const back = await waitFor(() => textarea()?.value === "");
if (!back) throw new Error(`↵ не вернул инициализацию`);
return "эталон вставляется в редактор, ведёт демонстрацию";
});

await test("F. редактор", "код не перетекает между страницами");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "п");

```

```

// Esc: прогон остановлен, трасса убрана, редактор
await key("Escape");
await settle(150);
if (!textarea()) throw new Error("Esc убрал редактор");
if (findButton(/Остановить прогон трассы/)) throw new Error("Остановить прогон трассы");
if (!/встроенное демо/.test(bodyText())) throw new Error("встроенное демо");
return "редактор всегда на месте: трасса рядом, сборщик мусора";
});

```

```

await test("F. редактор", "заккрытие панели во время прогона");
await mountApp("floyd"); // 869 шагов: прогон долгий
await click(findButton("Боковая панель Python"), "панель Python");
await waitFor(() => panel() !== null, 4000);
await click(findButton("Заккрыть компилятор"), "кнопка закрыть компилятор");
await settle(150);
if (panel()) throw new Error("панель не закрылась");
const backToDemo = await waitFor(() => /встроенное демо/);
if (!backToDemo) throw new Error("`после закрытия панели не появилось встроенное демо`");
await settle(3000); // ждём, пока догоняющий прогон
if (/данные из компилятора/.test(bodyText())) {
 throw new Error("заккрытая панель всё-таки опубликована");
}
if (!/встроенное демо/.test(bodyText())) throw new Error("встроенное демо не появилось");
return "панель закрыта во время выполнения – демонстрация";
});

```

```

await test("H. доступность", "панель: все кнопки подпаны");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "панель Python");
const namelessEditor = namelessButtons(panel() ?? document.querySelector("#editor"));
const editorBtns = allButtons().length;
const listing = await waitFor(() => !!findButton(/Отобразить трассу/));
if (!listing) throw new Error("панель не показала трассу");
const namelessListing = namelessButtons(panel() ?? document.querySelector("#listing"));
const listingBtns = allButtons().length;
if (namelessEditor.length) throw new Error("`в редакторе не все кнопки подпаны`");
if (namelessListing.length) throw new Error("`в листинге не все кнопки подпаны`");

```

/\* — 9. Ссылки между билетами, разделы содержания, б.

```
await test("Н. доступность", "содержание сжимается в
 await mountApp("bridges-code");
 const root = () => document.querySelector<HTMLElemen
 const toggle = () => findButton(/Сжать содержание|С
 if (!root()) throw new Error("нет сайдбара");
 if (!/lg:w-56/.test(root()!.className)) throw new E
 await click(toggle(), "сжать");
 if (!/lg:w-44/.test(root()!.className)) throw new E
 const items = Array.from(root()!.querySelectorAll("
 if (items.length < chapters.length) throw new Error
 if (root()!.querySelector("input")) throw new Error
 await click(toggle(), "скрыть");
 if (!/lg:w-0/.test(root()!.className)) throw new Er
 await click(toggle(), "развернуть");
 if (!/lg:w-56/.test(root()!.className)) throw new E
 return "широкое → узкое (темы на месте, поиска нет)
));
```

```
await test("J. шапка", "кнопки загрузки не прячутся и
 await mountApp("segment-trees");
 const menu = document.querySelector("#download-menu
 if (!menu) throw new Error("нет меню загрузки в шап
 const want = [
 { id: "download-guide-pdf-btn", file: "export/gui
 { id: "download-pdf-btn", file: "export/full_code
 { id: "download-txt-btn", file: "export/full_code
];
 for (const w of want) {
 const el = menu.querySelector<HTMLAnchorElement>{
 if (!el) throw new Error(`в шапке нет кнопки «${w
 const href = el.getAttribute("href") ?? "";
 if (!href.endsWith(w.file)) throw new Error(`«${w
 const name = [el.getAttribute("aria-label"), el.g
 if (name.trim().length < 5) throw new Error(`у кн
 }
 if (!menu.querySelector("#download-html-btn")) throw
```



```

// 11 «Мосты» и 12 «Точки сочленения» читаются парой
const items = Array.from(sidebar.querySelectorAll("b"));
const iBridges = items.findIndex((t) => t.includes("Мосты"));
const iArtic = items.findIndex((t) => t.includes("Точки сочленения"));
if (iBridges < 0 || iArtic < 0) throw new Error(`в содержании нет подсказки, в каком порядке читать`);
if (Math.abs(iArtic - iBridges) !== 1) throw new Error(`в содержании нет подсказки, в каком порядке читать`);
// 2 «разреженная» и 5 «префиксные» – в одном разделе
if (sectionOf("sparse-table").id !== sectionOf("prefixes").id)
 throw new Error("билеты 2 и 5 попали в разные разделы");
if (sectionOf("bridges-code").id !== sectionOf("graphs-code").id)
 throw new Error("билеты 11 и 12 попали в разные разделы");
// порядок чтения: номера билетов сверху вниз не убывающий
const nums = Array.from(sidebar.querySelectorAll("b"))
 .map((b) => (b.querySelector("span")?.textContent || ""))
 .filter((t) => /\d/.test(t))
 .map((t) => Number.parseInt(t, 10));
const ticketed = chapters.filter((c) => (chapterTopic(c) === "Билеты"));
if (nums.length !== ticketed) {
 throw new Error(`номера билетов видны у ${nums.length} билетов`);
}
for (let i = 1; i < nums.length; i += 1) {
 if (nums[i] < nums[i - 1]) throw new Error(`порядок чтения не убывающий`);
}
if (!sidebar.textContent?.includes("Порядок чтения"))
 throw new Error("в содержании нет подсказки, в каком порядке читать");
return `разделов ${headers.length}, 11+12 соседствуют`;
});

```

```

await test("I. ссылки", "введение: базовая база, обзор");
await mountApp("intro");
const txt = bodyText();
for (const must of ["Дерево как массив", "Куча", "Очередь"])
 if (!txt.includes(must)) throw new Error(`во введении не должно быть`);
if (!txt.includes("2n") || !txt.includes("2n+1")) throw new Error(`во введении не должно быть`);

```

```

 if (!shown().includes(list[1].question.slice(0, 24))
 // верный ответ
 await click(ask(list[1].options[list[1].correctIndex]);
 await settle(80);
 if (!/Верно/.test(shown())) throw new Error("на вер
 if (!/вопрос 2\\//.test(shown())) throw new Error(`с
 return `блиц из ${list.length} вопросов отвечает, р
 });

await test("I. факты", "память: разреженная жирная, п
const memoryRow = async (topic: string) => {
 await mountApp(topic);
 const rows = Array.from(document.querySelectorAll
 const row = rows.find((r) => /^\\s*Память/.test((r
 if (!row) throw new Error(`на странице «${topic}»
 return Array.from(row.querySelectorAll("td")).map
};
const sparse = await memoryRow("sparse-table");
const prefix = await memoryRow("prefix-sums-2d");
// билет 2: у префиксной N × M, у разреженной – с л
if (!/log/.test(sparse.join(" "))) throw new Error(
 ;
const prefixCell = sparse.find((c) => !/log/.test(c
if (!prefixCell) throw new Error(`в сравнении билет
// билет 5: зеркально – у префиксной без логарифмов
if (!/log/.test(prefix.join(" "))) throw new Error(
 ;
if (/log/.test(prefix[1] ?? "")) throw new Error(`в
const txt = bodyText();
if (/префиксн[^.] {0,40}(жирн|больше памяти)/i.test(
 throw new Error("на странице билета 5 префиксная
}
await mountApp("sparse-table");
const full = bodyText();
if (!/100 (коржей|копий|матриц)|сто (коржей|матриц)/
 throw new Error("на странице билета 2 не объяснен
}
if (!/не конкуренты|разные задачи/i.test(full)) {

```

```

 "d = [[0, 3, INF, 7], [8, 0, 2, INF], [5, INF, 1, 6], [4, 6, 3, 0]]",
 "k = 0",
 "i = 0",
 "j = 0",
 "for k in range(n):",
 " for i in range(n):",
 " for j in range(n):",
 " if d[i][k] + d[k][j] < d[i][j]:",
 " d[i][j] = d[i][k] + d[k][j]",
 "print(d[0], d[1][0])",
].join("\n")
);
const ready = await waitFor(() => bodyText().includes("набрал 4x4 с INF=999"));
if (!ready) throw new Error(`своя программа не выполнена`);
if (!bodyText().includes("[0, 3, 5, 6]")) {
 throw new Error(`Флойд на своей матрице 4x4 посчитал`);
}
if (!textarea()) throw new Error("редактор исчез, попробуйте еще раз");
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("демонстрация не принята");
const page = bodyText();
if (!/4x4/.test(page)) throw new Error(`демонстрация не принята`);
const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error(`артефакты в демонстрации`);
return "набрал 4x4 с INF=999 → d[0] = [0, 3, 5, 6]";
});

await test("К. свой код", "своя разреженная таблица на Python");
await mountApp("sparse-table");
await click(findButton("Боковая панель Python"), "поиск");
const ta = textarea();
if (!ta) throw new Error("нет редактора");
typeInto(
 ta,
 [
 "a = [5, 1, 9, 3, 7, 2]",
 "n = len(a)",
]
);

```

```

const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error(`артефакты в дефектах`);
return "эталон Косарайю: order считается DFS-ом, на DFS-е";
});

await test("G. lite-режим", "?lite=1 отдаёт текст без дефектов");
if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
}
if (host) host.remove();
host = freshHost();
window.history.replaceState({}, "", `/?lite=1&topic=${topic}`);
root = createRoot(host);
await act(async () => {
 root!.render(<App />);
});
await settle(120);
const txt = bodyText();
if (txt.length < 2000) throw new Error(`lite-режим: текст слишком короткий`);
if (/main\.py|Pyodide/.test(txt)) throw new Error("в lite-режиме не должно быть кода");
if (!/разрежен/i.test(txt)) throw new Error("в lite-режиме не должно быть сплошного текста");
if (findButton("Боковая панель Python")) throw new Error("в lite-режиме не должно быть боковой панели");
// текстовая версия: ссылки на другие билеты ведут к #, а не к /
// а блиц напечатан списком с ответами
const liteLinks = Array.from(document.querySelectorAll("a"));
const bad = liteLinks.filter((a) => !a.getAttribute("href"));
if (liteLinks.length === 0) throw new Error("в lite-режиме не должно быть ссылок");
if (bad.length) throw new Error(`в lite-версии ${bad.length} ссылок`);
const unknown = liteLinks.filter((a) => !chapters.some((c) => c.href === a.href));
if (unknown.length) throw new Error(`в lite-версии ${unknown.length} неизвестных ссылок`);
if (!/Блиц: вопросы по теме/.test(txt)) throw new Error("в lite-режиме не должно быть блиц-вопросов");
if (!/Верно:/.test(txt)) throw new Error("в lite-режиме не должно быть верных ответов");
return `${txt.length} знаков, ссылок в lite ${liteLinks.length}`;
});

```

```
/**
 * Аудит внутренних ссылок пособия: ничего не должно ве
 *
 * npm run verify:links
 *
 * Проверяет (по настоящим модулям приложения, собранным)
 * 1. билеты 1–24 закрыты страницами, id в chapterTopics
 * 2. разделы содержания: каждая страница в своём разд
 * 3. упоминания «билет N» в тексте ведут на существующи
 * 4. все data-goto / href="?topic=..." указывают на суще
 * 5. якоря href="#..." есть в той же главе;
 * 6. glossary.simulator существует в реестре и (кроме
 * имеет свою страницу – иначе модалка не получит
 * 7. glossary.demo – работающая мини-демо;
 * 8. у каждого ключа PAGE_SYNC есть запись в реестре
 * 9. у каждой страницы есть визуализация;
 * 10. страницы связаны перекрёстными ссылками (темы не
 */
import * as esbuild from "esbuild";
import { mkdirSync, writeFileSync } from "fs";

const OUT = "tmp/verify/links-entry.mjs";
const FAILS = [];
const NOTES = [];
const ok = (cond, message) => (cond ? true : (FAILS.push(message), false));

mkdirSync("tmp/verify", { recursive: true });
writeFileSync(
 "tmp/verify/links-entry.ts",
 [
 'export { chapters, chapterTopics, SECTIONS, section',
 'export { GLOSSARY } from "../../src/data/glossary"',
 'export { VIZ_REGISTRY } from "../../src/components',
 'export { PAGE_SYNC } from "../../src/data/vizSync"',
 'export { quizzes } from "../../src/data/quizzes";',
 'export { MINI_DEMO_KINDS } from "../../src/componen',
].join("\n")
);
```

```

/* 2. разделы содержания */
for (const chapter of chapters) {
 const sec = sectionOf(chapter.id);
 ok(!sec, `страница «${chapter.id}» не попала ни в од
 const tickets = chapterTopics[chapter.id] ?? [];
 ok(
 tickets.every((n) => n >= sec.from && n <= sec.to),
 `билеты страницы «${chapter.id}» (${tickets.join(",
 });
}
{
 const bySection = SECTIONS.map((s) => ({
 s,
 nums: chapters.flatMap((c) => (sectionOf(c.id).id =
 }));
 for (let i = 1; i < bySection.length; i += 1) {
 const prevMax = Math.max(...bySection[i - 1].nums);
 const curMin = Math.min(...bySection[i].nums);
 ok(curMin > prevMax, `разделы пересекаются: «${bySe
 }
 const all = bySection.flatMap((x) => x.nums);
 for (let n = all[0]; n <= all[all.length - 1]; n += 1
 ok(all.includes(n), `в содержании дыра: билет ${n}
 }
}

/* 3–5. ссылки внутри текста страниц */
const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{
let ticketMentions = 0;
let explicitLinks = 0;
const withoutLinks = [];
for (const chapter of chapters) {
 const html = chapter.content ?? "";
 let mentionsHere = 0;

 // 3. упоминания «билет N» – только внутри текста, не
 const textOnly = html

```

```

 `гlossарий «${term.id}»: simulator "${term.simula
 ые компилятора`
 });
}
if (term.demo) ok(demoKinds.has(term.demo), `гlossарий
}

/* 8–9. реестр и синхронизация */
for (const key of Object.keys(PAGE_SYNC)) {
 const base = key.split("#")[0];
 ok(registryIds.includes(base), `PAGE_SYNC["${key}"]:
}
for (const id of registryIds) {
 if (!chapterIds.has(id) && !EXTRA_DEMOS.has(id)) {
 FAILS.push(`реестр: визуализация "${id}" не является
 }
}
for (const chapter of chapters) {
 ok(registryIds.includes(chapter.id), `страница «${cha
}

/* 96. файлы, на которые ведут кнопки шапки, действител
{
 const { existsSync, statSync } = await import("fs");
 for (const file of ["public/export/guide_lite.pdf", "
 ok(existsSync(file), `шапка ссылается на ${file}, н
 if (existsSync(file) && statSync(file).size < 1000)
 }
}

/* 10. связанные темы + блиц */
for (const id of withoutLinks) NOTES.push(`нет перекрёс
const noQuiz = chapters.filter((c) => !(quizzes[c.id] ?

console.log("————— ссылки и структура содержания —
console.log(` страниц: ${chapters.length} • разделов:
console.log(` упоминаний «билет N» в тексте: ${ticketM
console.log(` визуализаций в реестре: ${registryIds.le

```

```

mkdirSync(OUT_DIR, { recursive: true });

writeFileSync(
 `${OUT_DIR}/contract-entry.ts`,
 `import { buildDebugRunner } from "../../src/data/deb
import { PAGE_SYNC } from "../../src/data/vizSync";
import { chapters } from "../../src/data/content";
export const runnerOf = (src: string) => buildDebugRunn
export const sync = Object.fromEntries(Object.entries(P
export const declared = Object.fromEntries(Object.entri
export const chapterIds = chapters.map((c) => c.id);
`
);

const built = await esbuild.build({
 entryPoints: [`${OUT_DIR}/contract-entry.ts`],
 bundle: true,
 outfile: `${OUT_DIR}/contract-bundle.mjs`,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 sourcemap: false,
 loader: { ".css": "empty", ".png": "dataurl", ".jpg":
 logLevel: "warning",
});
void built;
const mod = await import(`${process.cwd()}/${OUT_DIR}/c

const py = await loadPyodide({ indexURL: "node_modules/
const printed = [];
py.setStdout({ batched: (m) => printed.push(m) });
py.setStderr({ batched: (m) => printed.push("[stderr] "

const only = process.argv.slice(2);
const ident = (t) => /^[A-Za-z_][A-Za-z0-9_]*$/.test(t)
/** «xk / x», «dist[v]», «(i-1)//2», «π» → набор python
const alts = (raw) => raw.split(/[\/[\],]|\\s+или\\s+/).m
```



```

const neverFilled = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .map((a) => a.find((x) => ident(x) && seen.has(x)))
 .filter(Boolean)
 .filter((name) => {
 let sawValue = false;
 let sawNonEmpty = false;
 for (const step of [...result.steps, { values: result }]) {
 const v = step.values?.[name];
 if (v === undefined || v === null) continue;
 sawValue = true;
 if (!isEmpty(v)) sawNonEmpty = true;
 }
 return sawValue && !sawNonEmpty;
 });
if (neverFilled.length) issues.push(`объявлены, но ни

const stderr = printed.filter((p) => p.startsWith("[s
if (stderr.length) issues.push(`предупреждения: ${std

if (issues.length) problems++;
rows.push({ key, steps: result.steps.length, vars: seen
}

rows.sort((a, b) => (a.status === "ок") - (b.status ===
console.log("ЭТАЛОННЫЕ КОДЫ СТРАНИЦ (настоящий CPython
console.log("=".repeat(96));
for (const r of rows) {
 const mark = r.status === "ок" ? " ok " : " x ";
 console.log(`${mark}${r.key.padEnd(34)} шагов ${String
 === "ок" ? "" : r.status}`);
}
console.log("=".repeat(96));
console.log(`всего: ${rows.length} | с проблемами: ${
writeFileSync(`${OUT_DIR}/reference-code.json`, JSON.st
process.exit(problems > 0 ? 1 : 0);

```

```

const { window } = dom;

window.ResizeObserver = window.ResizeObserver ?? OBSERVER
window.IntersectionObserver = window.IntersectionObserver ?? OBSERVER
window.matchMedia =
 window.matchMedia ??
 ((media) => ({
 matches: false,
 media,
 onchange: null,
 addListener() {},
 removeListener() {},
 addEventListener() {},
 removeEventListener() {},
 dispatchEvent: () => false,
 }));
window.scrollTo = () => {};
window.HTMLElement.prototype.scrollIntoView = () => {};
window.HTMLElement.prototype.focus = window.HTMLElement.prototype.focus;
window.URL.createObjectURL = window.URL.createObjectURL;
window.URL.revokeObjectURL = window.URL.revokeObjectURL;
Object.defineProperty(window, "innerWidth", { value: window.innerWidth });
Object.defineProperty(window, "innerHeight", { value: window.innerHeight });
Object.defineProperty(window, "devicePixelRatio", { value: window.devicePixelRatio });
window.Element.prototype.getBoundingClientRect = function() {
 return { x: 0, y: 0, top: 0, left: 0, right: 400, bottom: 400 };
};

const copy = [
 "window",
 "document",
 "navigator",
 "HTMLElement",
 "HTMLInputElement",
 "HTMLTextAreaElement",
 "HTMLButtonElement",
 "HTMLAnchorElement",
 "HTMLDivElement",

```

ФАЙЛ 103/111: scripts/verify/live-matrix.test.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 103 | РАЗМЕР: 1000

```
/**
 * Модульные проверки разбора «живых» данных компилятора.
 *
 * npm run verify:unit
 *
 * liveMatrix() — то, благодаря чему визуализации матрицы
 * суммы 2D, разреженная таблица) принимают данные ЛЮБОГО
 * пользователя вместо захардкоженной демо-сетки.
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
writeFileSync(
 `${OUT_DIR}/unit-entry.ts`,
 `export { liveMatrix, pickLiveVar, clampIndex } from
`);
await esbuild.build({
 entryPoints: [`${OUT_DIR}/unit-entry.ts`],
 bundle: true,
 outfile: `${OUT_DIR}/unit-bundle.mjs`,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 loader: { ".css": "empty" },
 logLevel: "warning",
});
const { liveMatrix, pickLiveVar, clampIndex } = await i

let passed = 0;
const failures = [];
function check(name, actual, expected) {
```

```

check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m(["a", "b"], ["c"]));

/* — liveMatrix: особые значения —————
check('"inf" → +Infinity', cells(["inf", 1], [2, "-inf"]));
check('числа-строки "42" → 42', cells(["1", "2"], ["3", "4"]));
check("булевы значения ячейкой не считаются", m([[true, false], [false, true]]));
check("обрывок до 64×64", shape(Array.from({ length: 8096 }, () => 0), { numbers: true }));
check("hasNumbers=false невозможна для живой матрицы", m({ hasNumbers: false }));

/* — pickLiveVar —————
check("первое найденное имя по приоритету", pickLiveVar({ a: 1, b: 2 }, { a: 3, b: 4 }, { a: 5, b: 6 }));
check("берёт следующее имя, если первого нет", pickLiveVar({ a: 1, b: 2 }, { a: 3, b: 4 }, { a: 5, b: 6 }));
check("ничего не найдено → null", pickLiveVar({ x: 1 }, { x: 2 }, { x: 3 }));
check("нет снимка → null", pickLiveVar(undefined, ["d"], ["e"]));
check("ключ со значением undefined всё равно считается", pickLiveVar({ a: undefined }, { a: 1 }, { a: 2 }));

/* — clampIndex —————
check("null остаётся null", clampIndex(null, 5), null);
check("размер 0 → null", clampIndex(2, 0), null);
check("обрезка сверху", clampIndex(99, 5), 4);
check("обрезка снизу", clampIndex(-7, 5), 0);
check("дробный индекс усекается", clampIndex(2.9, 5), 2);
check("NaN → null", clampIndex(Number.NaN, 5), null);
check("Infinity → null", clampIndex(Number.POSITIVE_INFINITY, 5), null);

console.log("=".repeat(72));
console.log(`liveMatrix / pickLiveVar / clampIndex: протестировано`);
for (const f of failures) console.log(`  ✗ ${f}`);
console.log("=".repeat(72));
process.exit(failures.length ? 1 : 0);

```

```

function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window, 'value').set;
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubble: true }));
 });
}

async function click(el: Element | null | undefined, label: string) {
 if (!el) throw new Error(`нет кнопки: ${label}`);
 await act(async () => {
 el.dispatchEvent(new window.MouseEvent("click", { bubbles: true }));
 await sleep(20);
 });
}

function findButton(match: RegExp): HTMLButtonElement | null {
 return Array.from(document.querySelectorAll("button")).find(
 b => match.test([b.getAttribute("aria-label"), b.textContent].join(" "))
) as HTMLButtonElement | undefined;
}

export async function probe() {
 const host = document.createElement("div");
 document.body.appendChild(host);
 window.history.replaceState({}, "", "/?topic=dijkstra");
 const root = createRoot(host);
 await act(async () => {
 root.render(<App />);
 });
 await settle(80);

 process.stdout.write("\n=== 1. открыли панель, ждём асинхронно...");
 await click(findButton(/Боковая панель Python/), "панель Python");
 state("t=0");
 for (const t of [300, 1000, 3000, 8000]) {
 await settle(t - (t === 300 ? 0 : 0));
 state(`t+${t}`);
 if (/Трасса готова|Нет шагов|Пустой код|Ошибка/.test(state)) {
 return;
 }
 }
}

```

```
 } else {
 process.stdout.write(" редактора нет (листинг) – ждём редактора\n");
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("keydown", {
 key: "Esc",
 }));
 await sleep(30);
 });
 state("после Esc");
 }
 }
}
```

---

ФАЙЛ 105/111: scripts/verify/README.md

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

---

# Проверки приложения (`npm run verify`)

Три уровня проверок, которые можно гонять локально и в CI. Для локального запуска нужен: DOM даёт `jsdom`, Python – `pyodide` из `node\_modules`. Браузер нужен: 3.12, что грузится в браузере пользователя, только без чужих скриптов.

```
```bash
```

```
npm run verify          # всё сразу: unit → эталонные коды
npm run verify:unit     # 38 проверок разбора живых данных
npm run verify:code     # 46 эталонных кодов страниц по DOM
npm run verify:dom      # 139 сценариев: страницы, панели, кнопки
npm run verify:probe    # диагностика панели Python во время загрузки
```

Отчёты кладутся в `tmp/verify/` (каталог в .gitignore):
помазочно по DOM-сценариям, `reference-code.json` – по кодам.

Что именно проверяется

`verify:unit` – `live-matrix.test.mjs`

Разбор данных, которые присылает компилятор, в матрицу. Матрица имеет размерности: список списков, рваные строки, словарь слов.


```
-----  
    jsdomErrors.push(`unhandledRejection: ${reason instance  
});
```

```
log("прогоняю сценарии...\n");  
const results = await mod.runSuite({ py, window, document  
const sum = mod.summary();
```

```
writeFileSync(`${OUT_DIR}/report.json`, JSON.stringify(  

```

```
const failed = results.filter((r) => !r.ok);  
console.log("\n" + "=".repeat(96));  
console.log("СВОДКА");  
console.log("=".repeat(96));  
for (const [group, g] of Object.entries(sum.byGroup)) {  
    console.log(`  ${group.padEnd(26)} ok: ${String(g.ok)  
}
```

```
console.log(`\n  ВСЕГО ${sum.total}:    прошло ${sum.pass  
if (failed.length) {  
    console.log("\nПРОВАЛЫ:");  
    for (const f of failed) console.log(`  x [${f.group}]  
}
```

```
if (jsdomErrors.length) {  
    console.log("\nОШИБКИ ОКРУЖЕНИЯ (jsdom/неперехваченны  
    for (const e of [...new Set(jsdomErrors)].slice(0, 10  
}
```

```
const stray = consoleErrors.filter((e) => !/not wrapped  
if (stray.length) {  
    console.log(`\nCONSOLE.ERROR вне тестов (${stray.leng  
    for (const e of [...new Set(stray)].slice(0, 10)) con  
}
```

```
console.log(`\nотчёт: ${OUT_DIR}/report.json | время:  
process.exit(sum.failed > 0 || jsdomErrors.length > 0 ?
```

```
function log(message) {  
    process.stdout.write(`[verify:dom] ${message}\n`);  
}
```



```
-----  
    "import.meta.env.BASE_URL": '"/',  
    "import.meta.env.DEV": "true",  
    "import.meta.env.PROD": "false",  
  },  
  loader: { ".css": "empty", ".png": "dataurl", ".jpg":  
    logLevel: "warning",  
  });  
const mod = await import(`${process.cwd()}/${OUT_DIR}/p  
log("зондирую панель...");  
await mod.probe();  
process.exit(0);  
=====
```

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

=====

```
-----  
ФАЙЛ 108/111: .github/workflows/deploy-pages.yml  
КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9  
-----
```

name: Deploy app to GitHub Pages

```
# ДВЕ настройки репозитория нужно сделать один раз ВРУЧН  
# (у бота-интеграции нет admin-прав на них – API отвеча  
#  
# 1. Settings → Pages → Build and deployment → Source  
#    Если там «Deploy from a branch» (legacy) – шаг D  
# 2. Settings → Environments → github-pages → Deploy  
#    добавьте паттерн «arena/**» и ветку «main» (или  
#    иначе деплой из новой рабочей ветки отклоняется
```

on:

push:

branches:

- main

Позволяет проверить Pages прямо из рабочей ветки

```
uses: actions/upload-pages-artifact@v5
with:
  path: ./dist
```

```
deploy:
  name: Publish to GitHub Pages
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  steps:
    # Ранняя и понятная ошибка вместо невнятного «Fail»
    # когда в репозитории Pages не переключён на сбор
    - name: Preflight – Pages собирается через GitHub
      env:
        GH_TOKEN: ${ secrets.GITHUB_TOKEN }
      run: |
        bt=$(gh api "repos/${ github.repository }/pages/build_type")
        echo "Pages build_type: ${bt}"
        if [ "${bt}" != "workflow" ]; then
          echo "::error title=Pages не настроен::Сеттинг не настроен. См. комментарий в начале деплоя"
          exit 1
        fi

    - name: Deploy
      id: deployment
      uses: actions/deploy-pages@v5
```

ФАЙЛ 109/111: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРOK: 155 | РАЗМЕР: 6.1 КБ

name: Rebuild code PDF

```
if: "!contains(github.event.head_commit.message, '[
```

```
steps:
```

- name: Checkout
uses: actions/checkout@v6
with:
 fetch-depth: 0
 persist-credentials: true
- name: Setup Node.js
uses: actions/setup-node@v7
with:
 node-version: "24"
 cache: npm
- name: Install ImageMagick + DejaVu fonts
run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommen
 # На Ubuntu политика ImageMagick иногда запре
 # разрешаем то, что нужно пайплайну (text -> p
 for policy in /etc/ImageMagick-6/policy.xml /
 if [-f "\$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
done
convert -version
- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.densi
run: npm run export:png

```
    echo "| Артефакт | Размер |"
    echo "| --- | --- |"
    echo "| \`public/export/full_code_all_pages"
    echo "| \`public/export/full_code_all_pages"
    echo "| \`public/export/guide_lite.pdf\` |"
    echo "| Снимков визуализаций | ${ls tmp/viz"
    echo "| PNG-страниц | ${ls tmp/export-pages."
} >> "$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-export
    path: |
      public/export/full_code_all_pages.pdf
      public/export/full_code_all_pages.txt
    if-no-files-found: error
    retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action"
    git add -- public/export/full_code_all_pages.
      tmp/viz-shots
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full"
```

```
verify:
  name: unit + эталонный код + DOM
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-node@v4
      with:
        node-version: 22
        cache: npm

    # puppeteer здесь не нужен (он для растеризации Р
    # тянет Chromium – пропускаем, чтобы установка не
    - name: Зависимости
      run: npm ci
      env:
        PUPPETEER_SKIP_DOWNLOAD: 1
        PUPPETEER_SKIP_CHROMIUM_DOWNLOAD: 1

    - name: Типы и сборка
      run: |
        npx tsc --noEmit -p tsconfig.json
        npm run build

    - name: Модульные проверки (liveMatrix)
      run: npm run verify:unit

    - name: Эталонные коды страниц (настоящий CPython)
      run: npm run verify:code

    - name: DOM-проверки приложения (jsdom)
      run: npm run verify:dom

    - name: Отчёт проверок
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: verify-report
```

```
};  
``,`
```

Правила:

- ``id`` — kebab-case, уникальный, совпадает с ключом ``VI``.
подставляется автоматически, вручную ``div#slot-*`` вставляются
такие заглушки остались в старых главах и считаются мусором
- ``title`` начинается с номера билета: сортировка оглавления по
цифре заголовка. Одна страница может закрывать несколько глав
(``mst`` → 18, 19, 20) — тогда номер в заголовке один, а
объявляется в ``chapterTopics`` (``src/data/content.ts``)
- ``content`` — шаблонная строка. Внутри **никаких** `` `` и
экранирования: это сломает сборку.

```
---
```

2. Каркас страницы

```
` ``html  
<section id="dijkstra-content" class="mb-12 scroll-mt-12">  
  <!-- шапка: бейдж билета + заголовок -->  
  <div class="flex items-center mb-6 flex-wrap gap-3">  
    <span class="bg-blue-600 text-white px-4 py-1 rounded">  
      <h2 class="text-2xl sm:text-3xl font-bold text-white">  
    </div>  
  
    <div class="space-y-8">  
      <!-- смысловой блок (один или несколько) -->  
      <div class="bg-slate-700/50 p-6 rounded-xl border-l">  
        <h3 class="text-xl font-bold text-emerald-400 mb-4">  
        ...определение, абзацы, стат-сетка, аналогии, споры  
      </div>  
    </div>  
  </div>  
</section>  
` ``
```

Цвет левой полосы и заголовка блока — семантика раздела

```
```html
<div class="grid grid-cols-1 sm:grid-cols-3 gap-3 my-6">
 <div class="bg-slate-900 border border-slate-700 rounded">
 <p class="text-[10px] uppercase tracking-wider text">
 <p class="font-mono text-emerald-300 text-sm">0{(V+H
 </div>
 <!-- 2-я и 3-я ячейки – тот же markup -->
</div>
```
```

3.4 Сетка аналогий – ****обязательна**** на каждой странице

Слева – неформальная аналогия, справа – техническая альбом

```
```html
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
 <div class="absolute -top-3 left-4 bg-slate-700 text-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">...</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2 border-slate-800">
 <div class="absolute -top-3 left-4 bg-rose-900 text-emerald-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">...</p>
 </div>
</div>
```
```

Второй карточкой может быть «Аналогия 2», «Ограничения»
«Когда не работает» – markup тот же (пунктирная rose-pattern)

- Вложенные списки внутри спойлера: `
или `

3.7 Ловушки / частые ошибки

```
```html
<div class="bg-rose-950/30 p-5 rounded-lg border border-
 <p class="text-sm font-bold text-rose-300 mb-3">△ Гд
 <ul class="list-disc pl-5 space-y-2 text-sm text-slate
 Отрицательные веса. Дейкстра их не перев
 Переполнение: <span class="font-mono text-emera

</div>
```
```

4. Порядок разделов на странице

1. Шапка: бейдж билета + `h2`.
2. «Одной фразой» / «Суть алгоритма» + центрированное о
3. 1–3 абзаца объяснения (3.2), затем сетка ключевых чи
4. Аналогии (3.4) – обязательный блок.
5. Механика: пошаговое описание, таблица состояний/сравн
6. Спойлеры (3.6): референсный код, доказательство, форм
7. Ловушки (3.7) + границы применимости.
8. Связь с другими билетами («тот же find/union – сердце

Демонстрацию вставлять не нужно: `ChapterView` монтирует
по `id` главы после текста.

5. Контракт «демонстрация ↔ компилятор»

Каждая страница обязана иметь и визуализацию, и синхрон

Универсальное пособие • полный исходный код

Цель по объёму: 40–60 страниц, жёсткий предел – 100.

=====

КОНЕЦ ДОКУМЕНТА • 111 файлов • сгенерировано 15.09.2026

=====